

Ассемблер и программная модель процессоров x86/64

Ассемблеры i86/i386

Netwide Assembler (nasm)

Система команд i80x86/64

32-битный защищенный режим

64-битные режимы



Андрей Жуков



Материалы
на www.bhv.ru

Андрей Жуков

Ассемблер и программная модель процессоров x86/64

Санкт-Петербург
«БХВ-Петербург»

2024

УДК 004.43
ББК 32.973.26-018.2
Ж86

Жуков А. В.

Ж86 Ассемблер и программная модель процессоров x86/64. — СПб.:
БХВ-Петербург, 2024. — 400 с.: ил. — (Профессиональное программирование)
ISBN 978-5-9775-1761-4

Книга является практическим пособием по программной модели процессоров i80x86/64. Простейшие элементы этой модели (переменные, константы, методы адресации и система команд) изучаются с помощью ассемблера a86, отладчика d86. 32-битные возможности i80x86, включая защищенный режим, вентили, исключения и прерывания, привилегии, страничное преобразование, исключения, LDT и TSS, а также 64-битные режимы процессора x64 с исключениями и прерываниями в long mode изучаются с использованием ассемблера nasm.

Электронный архив на сайте издательства содержит исходные тексты примеров и необходимые для работы файлы.

Для программистов

УДК 004.43
ББК 32.973.26-018.2

Группа подготовки издания:

Руководитель проекта	<i>Павел Шалин</i>
Зав. редакцией	<i>Людмила Гауль</i>
Компьютерная верстка	<i>Ольги Сергиенко</i>
Дизайн обложки	<i>Зои Канторович</i>

"БХВ-Петербург", 191036, Санкт-Петербург, Гончарная ул., 20

Оглавление

Предисловие	9
Примечания	11
ЧАСТЬ I. РЕАЛЬНЫЙ РЕЖИМ.....	13
Глава 1. Установка программ	15
Установка и настройка <i>Bochs</i>	15
Формирование образа диска	16
Настройка отладочного варианта <i>Bochs</i>	18
Примечания	18
Глава 2. Программирование данных	21
Вызов <i>a86</i>	21
Программирование последовательностей	22
Программирование <i>bin</i> -файла	26
Примечания	29
Глава 3. Данные, имена и типы.....	31
Структура программы	31
Директивы определения данных	32
Обозначение чисел	37
Символические обозначения чисел, выражения	38
Переменные и метки	40
Типы имен	41
Типы и выражения	43
Алгоритм трансляции	46
Повторное определение имен	50
Локальные имена	51
Предопределенные имена	52
Имя <i>end</i>	52
Примечания	54
Глава 4. Способы адресации	57
Данные процессора	57
Обозначения операндов машинных команд	59

Способы адресации операндов	60
Регистровая и непосредственная адресация	60
Адресация данных в памяти.....	61
Прямая адресация	61
Косвенная адресация	63
Ограничение на адресацию операндов в памяти	67
Примечания	68
Глава 5. Система команд i8086	69
Способы адресации операндов	69
Регистровая, непосредственная и прямая адресация	70
Косвенная адресация	71
Косвенная адресация по значению одного регистра	71
Косвенная адресация по сумме значений двух регистров.....	71
Обзор системы команд.....	71
Команды пересылки	72
Арифметические команды	73
Логические команды	74
Команды сдвигов и вращений	74
Команды передачи управления.....	75
Адресация в командах передачи управления	76
Команды условных переходов.....	77
Воздействие команд на флаги.....	79
Строковые команды.....	82
Примечания	83
Глава 6. Программирование циклов.....	85
Поиск в массиве байтов	85
Поиск в массиве слов	87
Поиск байта со значением больше заданного	88
Подсчет байтов в заданном диапазоне значений	89
Алгоритмическое решение.....	89
Табличное решение	90
Примечания	92
Глава 7. Примеры программ	93
Обработка данных на уровне битов	93
Программирование ввода-вывода	95
Опережающие ссылки	98
Упаковка четырехбитовых кодов.....	100
Задания на составление программ	102
Задания первого уровня сложности	102
Варианты заданий	102
Задания второго уровня сложности.....	104
Примечания	108
Глава 8. Математический сопроцессор	111
Окно FPU в d86.....	111
Загрузка и выгрузка данных	112

Порядок двуместных операций	114
Организация ветвлений	117
Признаки в слове состояния	118
Настройки FPU	120
Форматы действительных чисел	121
Внутренний формат данных FPU	122
Стандартные форматы вещественных чисел.....	124
Операции FPU	125
Пересылки	126
Загрузка данных	126
Команды выгрузки.....	128
Команда обмена	130
Арифметические операции	130
Основные арифметические операции	131
Операции над знаковым битом.....	132
Округление до целого.....	133
Получение остатка от деления.....	134
Извлечение корня	135
Масштабирование.....	135
Операции сравнения и тестирования	135
Тригонометрические операции	137
Возведение в степень.....	137
Возведение числа 2 в целую степень	138
Возведение числа 2 в дробную степень	138
Вычисление целой и дробной частей значения степени	139
Возведение числа 2 в произвольную степень.....	139
Вычисление логарифмов.....	139
Команды управления	141
Сохранение и восстановление состояния	142
Задачи	143
Примечания	145
Глава 9. Сегменты.....	149
Эффективный адрес.....	150
Базовый адрес и сегментные регистры	151
Перепрограммирование сегментных регистров.....	154
Регистр <i>es</i>	154
Регистр <i>ss</i>	156
Регистр <i>cs</i>	157
Регистр <i>ds</i>	159
Повторный запуск резидентной программы	160
Программные секции.....	162
Префикс переназначения сегмента	167
Задачи	172
Примечания	173
Глава 10. Исключения.....	177
Таблица векторов.....	177
Векторные вызовы.....	178

Исключения и прерывания	185
Ассемблер <i>nasmt</i>	189
Отладочные исключения	193
Примечания	196
Глава 11. Внешние прерывания	197
Системный таймер	197
Клавиатура	201
Часы реального времени	203
Примечания	203
Глава 12. 32-битовые данные и адреса	205
Префиксы размерности операнда и адреса	205
Косвенная адресация через 32-битовые регистры	211
Новые команды	213
Примечания	215
ЧАСТЬ II. ЗАЩИЩЕННЫЙ РЕЖИМ	217
Глава 13. Код в защищенном режиме	219
Опыты с дескрипторами и защитой памяти	225
Первые опыты с привилегиями	226
Переключение сегментов кода и вентили вызова	227
Обратное переключение режима	231
Примечания	232
Глава 14. Данные и стек	235
Дескрипторы данных	235
Дескриптор стека	238
Режим <i>unreal</i>	241
Адресная линия A20	242
Привилегии сегментов данных	244
Примечания	250
Глава 15. Исключения и прерывания	253
Дескрипторы прерываний и исключений	253
Коды ошибок для исключений	258
Внешние прерывания	259
Поле <i>IOPL</i> в регистре флагов	266
Примечания	267
Глава 16. LDT и TSS	269
Дескрипторы LDT	269
Программное переключение контекста	272
Дескрипторы TSS	276
Примечания	285
Глава 17. Преобразование адресов	287
Первый вариант трансляции адресов	288
Вариант <i>PSE</i>	292
Вариант <i>PAE</i>	295

Плоская модель памяти	301
Примечания	305
Глава 18. Привилегии	307
Изменение уровня привилегий	307
Вызов привилегированной процедуры	311
Обращение к портам ввода-вывода	317
Привилегии при страничном отображении	318
Примечания	320
ЧАСТЬ III. 64-БИТОВЫЕ РЕЖИМЫ	321
Глава 19. Переход в режим совместимости.....	323
Переход в режимы <i>long</i>	323
Особенности режима совместимости	328
Примечания	330
Глава 20. Переход в 64-битовый режим.....	331
Загрузчик <i>ld0</i>	331
Применимость 64-битовых данных	339
Примечания	339
Глава 21. Особенности 64-битового режима	341
64-битовые операнды	342
Относительная адресация и перемещаемость	345
Селекторы <i>fs</i> и <i>gs</i>	349
Системные вызовы	352
Сегмент задачи в 64-битовом режиме	357
Обработка прерываний в 64-битовом режиме	359
Примечания	370
Послесловие.....	373
ПРИЛОЖЕНИЯ	375
Приложение 1. Компиляция Bochs	377
Подготовка к компиляции для Windows и Linux	377
Компиляция для Windows	378
Компиляция для Linux	379
Примечания	380
Приложение 2. Инструментальные программы FreeDOS	381
Файловый менеджер	381
Текстовый редактор <i>edit</i>	382
Редактор памяти <i>e32</i>	382
Форматы отображения данных в <i>d86</i>	383
Приложение 3. Дополнительные опыты с FPU.....	384
Команда <i>fisttp</i>	384
Формат <i>BCD</i>	385

Прерывания от i80x87	385
Синхронизация процессора и сопроцессора	388
Расширение MMX	389
Примечания	392
Приложение 4. Ошибки в a86/a386	393
Ошибки в a86	393
Ошибки в a386	394
Приложение 5. Описание электронного архива.....	395
Список источников	397
Предметный указатель	398

Предисловие

Вышел на Невский из кабака,
На окне мальчишка лохматый
Показал мне три языка,
Потом четвертый, потом пятый...
(Олег Григорьев. Люди)

У меня не создалось впечатления, что Холлоханы так уж часто выходили на ней в море. Позже выяснилось, что они вообще на ней не плавали и разделяли общее убеждение всех обитателей Грязной Ямы, что любая подобная попытка почти наверное окажется фатальной.
(Фарли Моуэт. Шхуна, которая не хотела плавать)

В этой книге используется язык ассемблера, даже три его разновидности: a86, a386 и `asm`¹. Но речь вовсе не о применении этих языков в программной индустрии, что (при современной тенденции развития программного обеспечения в сторону большей ресурсоемкости) даже не смешно². Ассемблер здесь — это лишь средство изучения программной модели процессоров x86-64, от реального режима и через 32-битовый защищенный режим к 64-битовому режиму. Что характерно, мы сами включаем 32- и 64-битовые режимы, в отличие от современных учебников ассемблера [5, 7–8, 11, 17], где практика проходит в готовой среде 64-разрядных операционных систем.

У того, кто уже ознакомился с оглавлением, может возникнуть вопрос: нужно ли столь тщательно изучать 32-битовый защищенный режим, если, по слухам, сегментная модель памяти в x64 упразднена? Действительно, в x64 принята уже не сегментная, а плоская модель памяти (хотя для селекторов `fs` и `gs` базовый адрес может быть ненулевым). Но тем не менее дескрипторы "упраздненных" сегментов остались. В дескрипторах кода по-прежнему присутствуют поля `R`, `DPL`, `C` и новый признак `L`, управляющий включением 64-битового режима. По-прежнему есть таблицы `GDT` и `IDT` — и даже россыпь `LDT`, если угодно. Привилегии также остаются в силе. Хотя от аппаратной многозадачности отказались, один экземпляр `TSS` не помещает³. А без знания схем страничного преобразования 64-битовый режим останется для вас "темным лесом". Не знать 32-битовый защищенный режим и при этом разбираться в 64-битовом — это из области фантастики.

¹ Здесь и далее ссылки на разд. "Примечания" в конце глав.

Среди публикаций на тему 32-битового защищенного режима можно отметить [12] и [15–16]. В [12] изложен опыт разработки ядра операционной системы⁴. Здесь много кода на ассемблере, но цель все-таки не исследование режимов, а скорейшая разработка ОС.

В [16] описана программная модель i80x86 в изложении одного из ее разработчиков, что ценно. Но, увы, нет ни одного примера программ. В [15] есть один пример, но в него втиснули все: и переход в защищенный режим, и аппаратную многозадачность, и страничное преобразование, переключаемое вместе с задачей... Первый пример — и сразу так сложно?

На мой взгляд, изучать в XXI веке вычислительную технику без многочисленных простых примеров — это анахронизм. Компьютеры теперь везде, и за любым теоретическим утверждением должна *сразу* следовать небольшая работающая программа-пример. Я обнаружил что-то подобное на курсах SiTrain по PLC Simatic⁵ (Siemens) и в самоучителе по химии [13].

На курсах SiTrain теория чередуется с упражнениями на стендах (PLC + миниатюрная модель конвейера). Изложение материала на курсах само по себе несколько хаотичное, в отличие от [6], но впечатляет то, что вычислительную науку можно преподавать таким необычным способом. А в серии самоучителей по химии, созданной авторами из Восточной Германии, теория изложена фрагментами не больше чем на полстраницы и чередуется с вопросами, ответы на которые (правильные или нет) направляют вас по той или иной ветви алгоритма в пределах главы.

Книга, предлагаемая вашему вниманию, состоит из трех частей, в которых используется симулятор PC под названием Bochs⁶ и операционная система FreeDOS, по минимуму (в первой и второй частях книги — как инструментальная среда, а в третьей — только для запуска программ из примеров).

- *Часть I "Реальный режим" (главы 1–12)* — это учебник для начинающих на основе ассемблера a86 и отладчика d86⁷. Они выбраны потому, что сделаны с толком, надежны и удобны. Например, сообщения об ошибках a86 вставляет прямо в исходный текст по месту, а при повторной трансляции, если программа исправлена, сообщения автоматически удаляются. Отладчик d86 благодаря уникальному режиму непосредственного выполнения позволяет быстро проверить все режимы адресации и систему команд i8086. В практической работе это занимает около шести часов.
- *Часть II "Защищенный режим" (главы 13–18)* — исследование 32-битовых возможностей i80x86, включая защищенный режим и большинство его аспектов: вентили, исключения и прерывания, привилегии, страничное преобразование во всех вариантах. Здесь на смену a86/d86 приходит ассемблер nasm⁸ (в варианте для DOS или для Windows⁹) и встроенный графический отладчик в составе Bochs. После первых опытов в среде Bochs-FreeDOS добавляем в программы вывод на экран, чтобы их выполнение можно было проследить также на PC, без симулятора.
- *Часть III "64-битовые режимы" (главы 19–21)* — погружение в 64-битовые режимы x64. Здесь уже без вариантов используется версия nasm для Windows, т. к. только она поддерживает 64-битовые возможности.

Сопровождающий книгу электронный архив содержит материалы, необходимые для практической работы, с примерами (см. *приложение 5*). Архив доступен для загрузки с сервера издательства "БХВ" по ссылке <https://zip.bhv.ru/9785977517614.zip>, ссылка на него также ведет со страницы книги на сайте <https://bhv.ru>.

Инструментальные программы не моего авторства, в том числе свободно распространяемые, не прилагаются. Установка этих программ подробно рассмотрена в *главе 1*. Заметим, что в книге на 99% используются программы категории freeware и shareware.

Примечания

1. Ассемблер a386 — это коммерческий продукт, он здесь встречается только пару раз, при переходе от a86 к nasm. Он вам не понадобится, поскольку исполняемые файлы, полученные в результате трансляции a386, приведены в электронном архиве. Ассемблер nasm существует в двух вариантах — для DOS и Windows, хотя можно сразу воспользоваться версией для Windows. Итого четыре варианта ассемблера.
2. Технологические аспекты применения ассемблера, как, например, интерфейс с языками высокого уровня, здесь не обсуждаются. Этот вопрос, особенно для 64-разрядных операционных систем, несложный, и все необходимое можно найти в среде программирования по клавише <F1>.
3. Кто хорошо знает 32-битовый защищенный режим, легко ответит почему.
4. Кстати, на сайте этого автора нашелся работающий (!) пример опроса дескриптора низкоскоростного устройства USB 1.0. С этого примера я когда-то начал разработку лабораторной по USB, которая состоялась и интересна тем, что выполняется в среде FreeDOS вручную — при помощи резидентного редактора портов gport и 32-битового редактора памяти e32.
5. Programmable Logic Controller, или PLC — основное средство автоматизации производства. Simatic (название семейства PLC) и SiTrain (учебные курсы и стенды) — это продукты корпорации Siemens.
6. Произносится так же, как "box", отсюда и пиктограмма в виде открытого картонного ящика.
7. В свое время я протестировал две версии a86, и по этим результатам (около десятка ошибок, причем в разных версиях разные) вышла версия 4.05, которую последующие лет двадцать успешно эксплуатировали студенты.
8. К сожалению, a386/d386 (32-битовый вариант a86/d86) выходит из игры, т. к. его поддержка прекращена (хотя ошибки есть). И дальше 32-битового защищенного режима на нем не уедешь.
9. В варианте для DOS мы переносим в образ DOS-диска исходные файлы и компилируем их в сеансе Bochs-FreeDOS. В варианте для Windows компилируем исходные файлы под Windows, а в образ DOS-диска переносим только исполняемые модули. Для создания 64-битовых приложений подходит только кросс-компилятор, работающий под Windows.

```

DB6_BUFFER:
010B MOV SI,011F
010E INT 3

AX 070A   i   c   1: =e1,ax,, 10101010
BX 0000   IP 0100  2: =d1,ax,, 1962
CX 0000   CS 6309  3:
DX 6309   SS 6309  4:
SI 011F   DS 6309  5: r5,b_1,, AB ab
DI 0000   ES 6309  6: r5,si,, AB ab
BP 0000   SP FFFE  0:

lea si,b_1

```

ЧАСТЬ I

Реальный режим

- Глава 1. Установка программ
- Глава 2. Программирование данных
- Глава 3. Данные, имена и типы
- Глава 4. Способы адресации
- Глава 5. Система команд i8086
- Глава 6. Программирование циклов
- Глава 7. Примеры программ
- Глава 8. Математический сопроцессор
- Глава 9. Сегменты
- Глава 10. Исключения
- Глава 11. Внешние прерывания
- Глава 12. 32-битовые данные и адреса

ГЛАВА 1

```
d86_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i  c  1: =e1,ax,, 10101010
BX 0000  IP 0100  2: =d1,ax,, 1962
CX 0000  CS 6309  3:
DX 6309  SS 6309  4:
SI 011F  DS 6309  5: r5,b_1,, AB ab
DI 0000  ES 6309  6: r5,si,, AB ab
BP 0000  SP FFFE  0:

lea si,b_1
```

Установка программ

Сборка японского велосипеда требует величайшего спокойствия духа.
(Роберт Персиг. Дзен и искусство ухода за мотоциклом)

Часть программ предназначена для работы в Windows, с них мы и начнем. Другая часть — для работы в FreeDOS (в сеансе симулятора Bochs).

Установка и настройка Bochs

Со страницы <https://sourceforge.net/projects/bochs/files/bochs/2.7/> скачайте установочную программу Bochs-win64-2.7.exe и выполните ее.

Конфигурирование, т. е. настройка возможностей, утилиты Bochs происходит при его компиляции из исходных файлов (*приложение 1*). Разработчики Bochs предлагают два готовых варианта исполняемого файла, скомпилированных с разной конфигурацией — рабочей (bochs.exe) и отладочной (bochsdbg.exe). В отладочном варианте Bochs включена поддержка встроенного отладчика, который нам поможет в опытах с защищенным и 64-битовым режимами. В первой части книги применяется исключительно рабочий вариант (bochs.exe) с расчетом на "внешний" отладчик d86 в составе ассемблера a86.

При каждом запуске любого из этих вариантов Bochs берет параметры из текстового файла, заданного в командной строке¹ с ключом `-f`.

Создайте свой каталог для запуска Bochs и скопируйте туда папку arch/01 из прилагающегося архива. В дальнейшем этот каталог называется стартовым. Сейчас он содержит следующие файлы:

- ☐ `__debug.bat` — командный файл для вызова отладочного варианта (bochsdbg.exe);
- ☐ `__work.bat` — командный файл для вызова рабочего варианта (bochs.exe);
- ☐ `bochsrc` — файл с параметрами Bochs, общий для обоих вариантов;
- ☐ `bx_enh_dbg.ini` — необязательный файл с параметрами отображения встроенного отладчика;
- ☐ `debug_on.txt` — дополнительный параметр для запуска отладочного варианта.

В файле `__work.bat` — пример командной строки для запуска Bochs. Опция `-f` задает имя файла с параметрами, а `-q` — пропуск диалога для редактирования параметров².

Сейчас вызов командного файла `__work.bat` ни к чему не приведет, т. к. для запуска Bochs в стартовом каталоге не хватает нескольких файлов. Их нужно скопировать из установочного каталога Bochs-2.7 (вероятно, он где-то в каталоге `C:\Program Files`):

- ❑ BIOS-bochs-latest — двоичный образ BIOS;
- ❑ VGABIOS-lgpl-latest — двоичный образ VGA BIOS;
- ❑ bochs.exe — рабочий вариант Bochs;
- ❑ bochsdbg.exe — отладочный вариант Bochs.

Добавив эти файлы в стартовый каталог, выполните командный файл `__work.bat`. В ответ появится сообщение "ata0-0: could not open hard drive image file 'disk_c.img'" об отсутствии образа загрузочного диска `disk_c.img`. Этот образ указан в `bochsrc`, но его еще предстоит создать.

Формирование образа диска

Зайдя на страницу sourceforge.net/projects/bochs/files/, выберите папку **Disk Images**, затем опцию **FreeDOS** и скачайте архив `fdos-10meg.tar.gz`. Файл `fdosmini.img` из архива поместите в стартовый каталог и переименуйте его в `disk_c.img`. Файл с параметрами у нас свой, из архива не берем.

Повторите вызов файла `__work.bat`. Должно появиться окно консоли Bochs, в которой отображается ход загрузки. Загрузка должна пройти успешно, появится приглашение `C:\>`. (Возможно, ему будут предшествовать запросы "Enter new date" и "Enter new time" на ввод даты и времени — в ответ нажимайте <Enter>.) Завершите сеанс Bochs, нажав кнопку **Power** в правом верхнем углу окна Bochs или просто закрыв окно.

Выясним, что находится в образе диска, чтобы убрать лишнее и добавить файлы, необходимые для работы. Для этого воспользуемся программой WinImage (winimage.com/download.htm).

Установите WinImage и нажмите на имя файла `disk_c.img`. В окне **WinImage** видна папка **dos** и три файла в корневом каталоге. Файл `fdconfig.sys` лишний, и вы можете его удалить, выбрав команду **Delete file** из контекстного меню.

В каталоге **dos** ценность представляет только подкаталог **bin** с системными утилитами. Но будьте осторожны: при помощи WinImage удалять *группы* файлов и непустые каталоги нельзя³. Эту работу можно безопасно выполнить в сеансе Bochs-FreeDOS. Завершите WinImage, вызовите файл `__work.bat` и, когда появится приглашение `C:\>`, выполните следующие команды:

```
c:\dos\bin\xcopy dos\bin fdos /d
c:\dos\bin\deltree dos /y
```


Первая команда копирует каталог C:\dos\bin в C:\fdos. Вторая удаляет каталог C:\dos со всеми подкаталогами.

Теперь добавим недостающие файлы в образ disk_c.img.

План действий следующий:

1. Извлекаем из прилагаемого архива папку disk_C, затем копируем в подкаталог disk_C\tools инструментальные программы для FreeDOS.
2. При помощи WinImage записываем содержимое папки disk_C в образ disk_c.img; в результате там образуются каталоги tools и work, а в корневой каталог добавляется файл autoexec.bat.

Сначала скачайте следующие файлы:

- ☐ a86.zip (eji.com/a86/);
- ☐ из коллекции old-dos.ru:
 - Volkov Commander 4.01;
 - bitmap.exe;
 - unzip 5.1;
- ☐ со страницы nasm.us/pub/nasm/stable/:
 - из каталога dos — nasm-x.xx.xx-dos.zip;
 - из каталога win64 — nasm-x.xx.xx-win64.zip.⁴

Поместите в каталог disk_C/tools следующие файлы:

- ☐ a86.com и d86.com из a86.zip (руководства a86manu.txt и d86manu.txt из a86.zip сохраните в рабочем каталоге, они вам могут пригодиться);
- ☐ bitmap.exe;
- ☐ vc.com из архива Volkov Commander v 4.01;
- ☐ nasm.exe и cwsdpmi.exe из архива nasm-x.xx.xx-dos.zip.

Теперь откройте образ disk_c.img в WinImage и перенесите в его корневой каталог *содержимое* папки disk_C, т. е. подкаталоги tools, work и файл autoexec.bat. (Перед копированием WinImage выводит диалоговое окно для подтверждения действия. Внимание: этот диалог чаще всего не виден, т. к. он *прячется* под текущим окном.)

Закройте WinImage, сделайте резервную копию disk_c.img и запустите сеанс Bochs-FreeDOS. После загрузки FreeDOS должен автоматически запуститься командный файл autoexec.bat, в результате чего на экране появится синее окно файлового менеджера Volkov Commander (VC).

Находясь в этом окне, нажмите клавишу <F9> и вызовите опцию **Right/Name**, чтобы файлы и каталоги в правой панели отображались в алфавитном порядке. Самая удобная конфигурация VC — когда левая панель убрана, из-за чего видна левая половина *консоли* с выводом запускаемых программ. Левую панель можно включать-выключать нажатием <Ctrl>+<F1> — проверьте. Уберите левую панель и со-

храните сделанные изменения, нажав клавишу <F9> и выполнив команду **Options/Save setup**.

Затем вызовите через командную строку три программы: `a86`, `nasm` и `bitmap`. Во всех случаях должна быть какая-то реакция, отличная от сообщения "Bad command or filename".

На этом сборка и проверка `disk_c.img` закончена⁵. На всякий случай сразу создайте резервную копию файла `disk_c.img`.

Файлы, которые находятся сейчас в образе `disk_c.img`, можно записать на загрузочную USB-Flash, чтобы запускать FreeDOS и проверять примеры из книги на PC⁶, а не только в симуляторе Bochs. Подключите USB-Flash и сделайте резервную копию ее данных. Скачайте из Сети и запустите программу `rufus`, в появившемся диалоге выберите из списка вариант FreeDOS и запустите форматирование. Выйдя из `rufus`, скопируйте на USB-Flash все файлы, которые находятся сейчас на `disk_c.img`, с сохранением структуры каталогов. Затем можно восстановить на USB-Flash ее исходные данные из резервной копии.

Настройка отладочного варианта *Bochs*

Осталось проверить вариант Bochs со встроенным графическим отладчиком⁷. Чтобы воспользоваться им, необходимо вызвать `bochsdbg.exe` (вместо `bochs.exe`), а в файле параметров в строке `display_library` должна быть опция "`gui_debug`". Проще всего было бы раз и навсегда включить эту опцию в файл `bochsrc`⁸, но `bochs.exe` отказывается с ней работать. Поэтому оставляем в `bochsrc` параметры для рабочего варианта, а файл параметров для отладочного варианта формируем как объединение `bochsrc` с файлом `debug_on.txt`, где задана та самая строка с опцией "`gui_debug`". Это объединение строится заново при каждом запуске файла `__debug.bat`. Так что в основе обоих вариантов Bochs — один файл параметров `bochsrc`.

Запустите командный файл `__debug.bat`. На экране появится окно встроенного графического отладчика Bochs, а окно консоли, которое мы видели прежде при запуске `__work.bat`, будет минимизировано. Останов произошел по адресу `ffffff0`, с которого запускается процессор `i80x86/64`. Нажмите кнопку **Continue**, и загрузка продолжится как обычно. Переключитесь на окно консоли, выбрав его из перечня минимизированных окон по изображению раскрытой коробки.

Примечания

1. По умолчанию параметры считываются из файла `.bochsrc` в установочном каталоге `bochs-<version>`, но мы будем явно задавать файл с параметрами.
2. Во-первых, в нем нет необходимости, поскольку параметры уже определены в файле `bochsrc`. Во-вторых, после этого диалога следует аварийное завершение Bochs (проверено в версиях 2.6.11 и 2.7). В версии 2.6.11 этот сбой происходит даже при немедленном закрытии диалога (без редактирования параметров).

3. Оценочная версия WinImage 10.0 при удалении нескольких файлов "промахивается", стирая часть неотмеченных файлов и оставляя часть отмеченных. Версия 11.0 в этой ситуации тоже сбивает, хотя лишнего вроде бы не стирает.
4. Это кросс-компилятор: он выполняется в Windows, может генерировать com-файл для FreeDOS и поддерживает 64-разрядный код. Эту версию нужно скачать и сохранить на будущее, а в каталог `disk_C\tools` включать ее не надо. Напротив, версия, скачанная из каталога `dos`, работает именно в FreeDOS. Ею можно пользоваться во второй части книги, т. к. она поддерживает 32-разрядный код, и ее можно сразу включить в каталог `disk_C\tools`. А можно вообще не пользоваться версией `pasn` для DOS, а сразу, начиная со второй части книги, работать в Windows с кросс-компилятором, копируя в образ диска только результаты трансляции.
5. Для сведения, программа `bximage.exe` (в установочном каталоге `bochs`) позволяет создать образ диска большего размера. Его можно подключить как дополнительный, для чего имя `img`-файла нужно указать в строке `ata1` файла параметров. В сеансе `Bochs-FreeDOS` этот "диск" потребуется один раз отформатировать командой `format /u d:.` (Но для работы с примерами из нашей книги дополнительный диск не понадобится.)
6. Запуск FreeDOS на PC возможен, если BIOS не переводит машину в защищенный или 64-битовый режим, а оставляет ее в реальном режиме.
7. Возможен вариант с текстовым встроенным отладчиком, но он неудобен, поскольку каждое действие (будь то выполнение по шагам или отображение регистров или памяти) требует ввода команды.
8. В надежде, что этот файл сгодится и для рабочего, и для отладочного вариантов `Bochs`.
9. То, что в рабочем и отладочном вариантах используется один и тот же файл параметров, удобно при экспериментах с параметрами, если вам их захочется.

ГЛАВА 2

```
DB6_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100 2: =d1,ax,, 1962
CX 0000  CS 6309 3:
DX 6309  SS 6309 4:
SI 011F  DS 6309 5: r5,b_1,, AB ab
DI 0000  ES 6309 6: r5,si,, AB ab
BP 0000  SP FFFE 0:

lea si,b_1
```

Программирование данных

Пробежав взглядом по списку, содержавшему свыше пятидесяти размеров и указаний, Слободьд понял, что у дамы, которой предназначалось платье, на животе три груди, причем каждая своей величины и формы. Помимо того, на спине у нее несколько больших горбов. На талию отводилось всего восемь дюймов, зато четыре руки, судя по проймам рукавов, по толщине не уступят стволу молодого дуба. О ягодицах не упоминалось вообще, однако величина клеши подразумевала чудовищные вещи.

(Роберт Шекли. Заказ)

Начнем с программ, целиком состоящих из данных, без единой машинной команды. При помощи простых и немногочисленных директив определения данных и констант¹ можно сразу запрограммировать что-то нетривиальное, например графические данные в стандартном формате bitmap.

Вызов a86

Транслятор a86 вызывается просто: в командной строке набираем a86 и имя исходного файла². Если ошибок нет, то a86 создаст одноименные файлы с расширениями com и sum. Файл com — это исполняемый модуль, а sum-файл содержит символьную информацию для отладчика d86: таблицу имен и макроопределения.

Если a86 обнаружит ошибку в исходном тексте, он копирует входной файл в файл с расширением old, а затем вставляет сообщения прямо в исходный текст по месту. (Одна из уникальных особенностей ассемблера a86.) Каждое сообщение занимает две строки: первая отмечает позицию ошибки, вторая объясняет причину.

Рассмотрим пример. Программа в файле test1.8 содержит ошибку³. Вызовите a86 test1.8, и a86 вставит в test1.8 два сообщения, как показано в листинге 2.1.

Вначале говорится о том, что все сообщения об ошибках будут удалены автоматически, если оставить первые две строки. Действительно, ничего не надо удалять, a86 сделает это сам при повторной трансляции.

Во втором сообщении значок ^ указывает позицию ошибки: начало незакрытой строки. Строка "плоха" потому, что в конце ее нет двойной кавычки.

Листинг 2.1. Сообщения об ошибках в исходном тексте (test1.8)

```

~^
#ERROR messages will be removed if you leave these first two lines in
z1:
dozen    equ    12
d_var    dd     dozen
a86      db     "I'm A86, fast & tiny.
~                ^
#ERROR 32: Bad String

```

В файле test1.8 исправьте строку⁴:

```
a86 db "I'm A86, fast & tiny."
```

Выполните повторную трансляцию. Сообщения об ошибках исчезли из исходного текста, old-файл тоже исчез, и появились результаты: test1.com и test1.sym. Вызывать на выполнение test1.com не следует. Процессор начнет отрабатывать данные test1.com как машинные команды — скорее всего, с плачевным исходом.

В листинге 2.2 показано содержимое test1.com по 16 байтов в строке. Заметим, что в файле оказалось именно то, что задано в исходном тексте: 32-битовое слово со значением 12 (4 байта в начале) и строка литер. Ни больше ни меньше, никаких заголовков, никакой служебной информации.

Листинг 2.2. Содержимое test1.com

```

0c 00 00 00 49 27 6d 20 41 38 36 2c 20 66 61 73
74 20 26 20 74 69 6e 79 2e

```

А сейчас спровоцируем ошибку другого рода. Отредактируйте третью строку test1.8:

```
a86 db fast, tiny
```

Повторите трансляцию. Имена `fast` и `tiny` в программе нигде не определены, но `a86` не вставляет сообщение в исходный текст⁵. Вместо этого он записывает имена в отдельный файл с расширением `und` (`undefined`). Проверьте эту ситуацию, потом замените неопределенные имена любыми числами и повторите трансляцию. Исчез ли файл test1.und?

Программирование последовательностей

Начнем практику с простейших программ, которые содержат только данные, без единой машинной инструкции на ассемблере, как в листинге 2.3. Это повторяющаяся последовательность байтов со значениями 0, 1, 0, 1 и т. д. общим объемом 1/8 Кбайт.

Листинг 2.3. Простая последовательность (var_0a.8)

```
sz      equ 1k/8                ; (1)
seq     db  sz/2 dup (0, 1)     ; (2)
sz      equ $ - seq             ; (3)
```

При трансляции имя `sz` получает значение 128 и тип `abs` (число). Далее следует директива `db` (define bytes), которая задает 64 повтора байтов с величинами 0 и 1 — всего 128 байт. Знак доллара в (3) — это текущее значение счетчика адресов при распределении памяти. В начале трансляции он равен 256. Имя `seq` в (2) получает текущее значение счетчика адресов, т. е. 256, и тип `byte`, поскольку за именем следует директива `db`. Счетчик адресов `$` после трансляции (2) равен $256 + 128 = 384$. В строке (3) — повторное определение имени `sz`, оно равно $284 - 256 = 128$.

Имя `sz` определено дважды. В отличие от многих языков программирования (и большинства ассемблеров), `a86` допускает повторное определение, если значение и тип имени не меняются. (Еще одно уникальное свойство `a86`.) В рассмотренном примере атрибуты `sz` при обработке (3) остались прежними: тип — `abs`, значение — 128. Но какой смысл в повторном определении?

Второе определение вводится для *проверки*: размер, заданный в (1), сравнивается с размером данных, определенных между (2) и (3). Если они расходятся, `a86` выдает ошибку "Conflicting multiple definition". Проверьте: добавьте в конец (2) запятую и число и повторите трансляцию.

В листинге 2.4 показан чуть более сложный вариант этой последовательности: в начале вместо пары байтов 0 и 1 идет слово, где записан размер `com`-файла⁶. Для задания слов используется директива `dw` (define words).

Листинг 2.4. Дополнение к простой последовательности (var_0b.8)

```
sz      equ 1k/8                ; (1)
seq     dw  end - $              ; (2)
        db  (sz/2 - 1) dup (0, 1) ; (3)
sz      equ $ - seq             ; (4)
;end:   ; (5)
```

Размер программы вычисляется в (2) как разность между адресом метки (5), которую `a86` ставит в *конце* исходного текста, и текущим значением счетчика адресов⁷. Счетчик адресов в (2) имеет то же значение, что и в начале программы, т. к. (1) не распределяет память, а только вводит новое имя⁸.

Прежде чем приступить к программированию последовательностей, проверим `a86` на предмет возможных слабостей.

Многие ассемблеры не могут обрабатывать висячую строку в конце исходного текста, она вызывает сбой. Для проверки отредактируйте программу `var_0b.8`: перейдите в конец строки (5) и, "наступив" на клавишу `<Del>`, сотрите все, что было в файле после строки (5). Сохраните файл, просмотрите его в кодах (`<F3>`) и сразу,

не выходя, <F4>) — в конце файла не должно быть кодов 0d и 0a (признак конца строки)⁹. Повторите трансляцию. Как видите, а86 справляется с висячей строкой.

Еще одна возможная неприятность — переполнение com-файла. Его размер ограничен величиной примерно 64 Кбайт¹⁰, и возникает вопрос: что если определить в программе данные большего объема? Проверить, что будет, нетрудно: достаточно перепутать местами операнды при определении размера массива, как в листинге 2.5. (Довольно частая ошибка у начинающих.)

Листинг 2.5. Переполнение com-файла (test2.8)

```
arr1    db      12 dup 1                ; (1)
tail:   ; (2)
        dw      ?                      ; (3)
sz       equ    offset arr1 - tail      ; (4) ?!
arr2     db      sz dup 3                ; (5) ?!
```

Хотели сделать массив (5) такой же длины, как (1), но в (4) перепутали местами уменьшаемое и вычитаемое и получили *sz* близкое к максимуму. При трансляции (5) должна появиться ошибка "Object Overflow".

В некоторых ситуациях, довольно редко, *исходный* файл при вставке сообщений об ошибках дополняется случайными данными, причем общий объем файла возрастает до десятков килобайтов. Тогда нужно срочно восстановить исходный файл из old-файла, куда а86 предусмотрительно скопировал первоначальный исходный текст.

Перейдем к задачам на программирование последовательностей данных:

1. Повторение байтов со значениями 1, 2, 4, 8, пока размер последовательности не достигнет 1 Кбайт, а затем еще один байт со значением -1.
2. Чередование байта со значением 1 и слова со значением 2, общий размер — 300 байт.
3. Повторяется последовательность: 8 байтов со значением -1, 4 слова со значением 0, 8 двойных слов со значением 256. Общий размер данных — 240 слов.
4. Два байта со значениями 27 и 'X' повторяются, пока размер данных не достигнет 1 Кбайт¹¹.
5. Три последовательности в одном файле, каждая размером 64 байта:
 - байты со значением 1;
 - слова со значением 2;
 - двойные слова со значением 4.
6. Троекратный повтор структуры данных:
 - слова со значениями 5 и 7;
 - два повтора последовательности байтов 0, 1, 2, 4;
 - двойное слово со значением 65 539.

7. Четырехкратный повтор структуры данных¹²:

- два байта со значениями 5 и 7;
- слово со значением 260;
- двойное слово со значением 1.

8. Пятикратный повтор структуры данных¹³:

- строка 'az';
- слово со значением -1;
- двойное слово со значением 012345678 (см. листинг 2.7).

Для проверки результата сравните полученный com-файл с одноименным dat-файлом:

```
fc /b var_<n>.com var_<n>.dat
```

Обратите внимание, что в вариантах 2–3 и 6–8 повторяются структуры данных с полями *разного* размера. Если преобразовать структуру в *массив* байтов или слов, то данные можно определить одной директивой `db` или `dw` с использованием конструкции `dup`. В листинге 2.6 приведено решение второй задачи, где чередуются байт со значением 1 и слово со значением 2.

Листинг 2.6. Решение второго варианта (var_2.8)

```
sz      equ      300
area    db        sz/3 dup (1, 2, 0)
sz      equ      $ - area
```

Слово здесь представлено парой байтов: 2 и 0. Почему не наоборот — 0 и 2? Потому что в младшем байте слова находится наименее значимая часть числа, а в старшем — наиболее значимая¹⁴. Иными словами, вес младшего байта — 1, а вес старшего — 256. Для примера в листинге 2.7 приведены варианты определения слова со значением 256 и двойного слова со значением 65 539.

Листинг 2.7. Определение слов и двойных слов байтами и словами (test3.8)

```
(1)      dw      0100      ; 256
(2)      db      0, 1

(3)      dd      010003      ; 65539
(4)      dw      3, 1
(5)      db      3, 0, 1, 0
```

При помощи `lst`-файла проверьте совпадение данных (1) и (2), а также совпадение данных (3)–(5).

Программирование bmp-файла

Мы выяснили, как при помощи ассемблера задавать простые последовательности данных; при этом мы пользовались константными выражениями для проверки размера сгенерированных данных. Теперь рассмотрим нетривиальный пример, где структуры данных сложнее, а константные выражения более изощренные. В листинге 2.8 показана программа, результат трансляции которой соответствует стандартному графическому формату bitmap. В FreeDOS его можно проверить при помощи утилиты `bitmap`¹⁵.

Для получения bmp-файла выполните следующие команды:

```
a86 bmp_0.8 TO bmp_0.bmp
bitmap bmp_0.bmp
```

После вызова утилиты `bitmap` нажмите клавишу <F5>, чтобы увидеть изображение в цвете: две вертикальные красные линии разной толщины по краям картинки с синим фоном¹⁶.

Листинг 2.8. Данные формата bitmap на ассемблере (bmp_0.8)

```
start:                                     ; (1)

;      Parameters                           ; (2)

iWidth      equ    64                      ; (3)
iHeight     equ    32                      ; (4)
bitsPerPixel equ    1                      ; (5)

;      Calculated constants                 ; (6)

Colors      equ    bit bitsPerPixel        ; (7)
pixelsPerByte equ    8 / bitsPerPixel      ; (8)
bytesPerLine equ    iWidth / pixelsPerByte ; (9)
byteCount   equ    bytesPerLine * iHeight  ; (10)

;      Bmp-file header                     ; (11)

      db    'BM'                          ; (12)
      dd    end - start                    ; (13)
      dd    0                             ; (14)
      dd    offset Pixels - start          ; (15)

;      Data header                         ; (16)

      dd    Palette - $                   ; (17)
      dd    iWidth                        ; (18)
      dd    iHeight                       ; (19)
```

```

        dw      1                                ; (20)
        dw      bitsPerPixel                    ; (21)
        dd      0                                ; (22)
        dd      byteCount                      ; (23)
        dd      4 dup 0                        ; (24)

Palette db      255, 0, 0,    ?                ; RGB ; (25)
        db      0,   0, 255, ?                ; (26)

Colors  equ     ($ - Palette) / 4              ; (27)
zero    equ     0                              ; (28)
zero    equ     ($ - Palette) mod 4             ; (29)

;                                               ; (30)
;   | x x                                x      ; (31)
;   | x x                                x      ; (32)
;   | 7 6 5 4 3 2 1 0 ... 7 6 5 4 3 2 1 0    ; (33)
;   +-----+                               ; (34)

b1      equ     bytesPerLine                    ; (35)

Pixels  db      iHeight dup (11000000xb, (b1 - 2) dup 0, bit 0) ; (36)

byteCount equ    $ - Pixels                      ; (37)

```

В этом примере широко используются константные выражения или выражения, вычисляемые при трансляции. С их помощью по значениям параметров (3–5) оцениваются:

- количество цветов (7)¹⁷;
- число точек, представленных одним байтом (8);
- число байтов, необходимое для начертания одной горизонтальной линии (9);
- общий размер данных растра (10).

Собственно данные начинаются с (12), а до этого шли определения констант. Мы можем редактировать параметры картинки (3–5), числовые значения данных палитры (25–26) и растра (36).

В примере изображение монохромное, в (25–26) заданы два цвета: краска номер 0 и краска номер 1. Одному биту в (36) соответствует одна точка. Номер краски выбирается по значению бита. Экран заполняется точками слева направо, строка за строкой снизу вверх. Соответственно в примере заданы две *вертикальные* линии по краям изображения. Биты из очередного байта (36) обрабатываются в обратном порядке, от старших к младшим, — поэтому вертикальные линии прижаты к *краям* картинки¹⁸.

Проверим, устойчива ли эта программа к изменению размеров картинки. Удвойте параметры (3–4); новая картинка должна выглядеть так же¹⁹.

Исправьте значения составляющих цвета в палитре²⁰ (25–26) и проверьте результат. Если вы не меняли число байтов в палитре, то все должно быть в порядке.

Добавьте один цвет (т. е. 4 байта) между (25) и (26) и выполните трансляцию. Ошибка возникает при повторном определении в (27). Первое определение в (7) — это заданное, ожидаемое число цветов в палитре, а второе, в (27) — фактическое значение, вычисленное исходя из распределения памяти. Мы задали три цвета вместо двух и получили ошибку при трансляции, которая уберегла нас от создания дефектного bitmap-файла²¹.

В лишней строке, которую вы добавили к палитре, удалите пока что только один байт. Теперь размер палитры не делится нацело на 4, что вызовет ошибку при повторном определении (29).

Удалите лишнюю строку в палитре и убедитесь, что картинка успешно транслируется и выглядит как прежде.

Теперь попытаемся нарушить размер данных раstra (36). Добавьте байт между (36) и (37). При трансляции возникнет знакомая нам ошибка, т. к. предполагаемый размер раstra, подсчитанный в (10), отличается от действительного (37).

Во всех заданиях требуется запрограммировать картинку размером 64 на 64 точки. По умолчанию для задания раstra (36) достаточно одной директивы `db`. Не удаляйте проверку (36), она уберегает вас от создания дефектного bitmap-файла.

Изображения следующие:

1. Две горизонтальные линии — сверху и снизу. Используйте три директивы `db`: для нижней линии, для промежутка и для верхней линии.
2. "Тельняшка", по две линии на полосу. Поменяйте цвет номер 1 в палитре с красного на белый (значения составляющих — 255, 255, 255). Нулевой цвет по-прежнему синий.
3. "Полосатая футболка" с чередованием одной красной линии и трех синих.
4. "Забор" с толщиной вертикальных реек 4 линии и таким же промежутком.
5. Шахматная доска с клеткой 8×8. Начните с клетки 1×1, это проще.
6. Рамка по краям толщиной в одну линию. Задайте три директивы `db`: для нижней линии, для пары вертикальных линий в промежутке и для верхней линии.
7. Квадрат в центре, расстояние от краев — 8 точек. Используйте три директивы `db`: для нижнего промежутка, для квадрата и для верхнего промежутка.
8. Покрасьте верхнюю половину картинки зеленым цветом на синем фоне.
9. Покрасьте левую половину картинки синей краской на зеленом фоне.

Во всех задачах ищите повторяющуюся последовательность байтов. Например, в задаче 4 надо повторить каждую горизонтальную линию, в задаче 2 — каждую пару полос (8 линий), а в задаче 5 — каждые два ряда клеток (16 линий).

Теперь увеличим число красок в палитре. Если сделать константу `bitsPerPixel` равной 4 или 8, то число цветов возрастет до $2^{4}=16$ или $2^{8}=256$. При этом при-

дется заново определить данные раstra (36). Он был монохромным, по одному биту на точку, а теперь каждая точка должна быть задана полубайтом или целым байтом.

Сначала задайте `bitsPerPixel = 4`. Число красок теперь 16, оно вычисляется автоматически в (7). Сейчас в палитре только два цвета, и нужно добавить 14 — сразу после (26). Эти дополнительные цвета нам не понадобятся, и можно задать их нулями в количестве 4×14 байт. Затем необходимо заново определить данные раstra (36). Теперь точка задана не битом, а четырьмя битами, и каждый байт в (36) представляет две точки. Вместо двоичных чисел вроде `11000000xb` имеет смысл воспользоваться шестнадцатеричными, т. к. каждая шестнадцатеричная цифра дает цвет одной точки. Например, шестнадцатеричное число `011` задает две точки первого цвета, а `010` — точку первого цвета и точку нулевого цвета²².

Если сделать `bitsPerPixel = 8`, то число цветов станет 256. Каждый байт раstra будет задавать одну точку, и для воспроизведения первоначальной двухцветной картинки байты в `Pixels` должны быть только двух значений: 0 и 1.

Примечания

1. Машинных команд — десятки и сотни, а этих директив всего четыре: три для определения данных и одна для констант.
2. В файловом менеджере VC достаточно нажать <Enter> на имени файла с расширением '8'. В зависимости от расширения VC вызывает команды, определенные в файле `vc.ext`.
3. Чтобы просмотреть файл в VC, установите курсор на этот файл и нажмите <F3>, а для редактирования — <F4>. Или можно воспользоваться внешним редактором `edit` (Приложение 2).
4. Выход из редактора `edit` — <Alt+X>, а из встроенного редактора VC — <Esc>. В любом случае вам предложат сохранить исправления.
5. То, что имя не определено, выяснится лишь к концу трансляции, и можно было бы вставлять сообщение внизу исходного текста. Но автор решил по-своему.
6. Размер `com`-файла в точности равен объему данных и кода, сгенерированных транслятором, поскольку в этом файле вовсе нет служебной информации — только то, что задано в исходном тексте.
7. В `a86` не следует явно задавать имя `end`.
8. Имя помещается во внутреннюю таблицу имен транслятора, никак не претендуя на место в выходном `com`-файле.
9. Редактор `edit`, в отличие от редактора, встроенного в VC, автоматически добавляет перевод строки в конце файла.
10. Точнее, 64 Кбайт (общий размер выделенного блока памяти) минус 256 байтов под PSP (Program Segment Prefix).

11. 'X' — это ASCII-код литеры X. Чтобы задать его в программе, не надо искать его числовое значение — ассемблер его знает. Просто записать как есть: 'X'.
12. Можно запрограммировать при помощи директивы `dw`.
13. Можно запрограммировать при помощи директивы `dw`, т. к. она допускает задание *двух* литер в *одинарных* кавычках (но их расположение в слове обратное).
14. Так принято в Intel. В Motorola наоборот — самый весомый элемент идет в начале.
15. К сожалению, `lcris` — лучшая утилита для этой цели, в Bochs не работает.
16. При использовании `lcris` экран за пределами картинки черный, а здесь все залито цветом фона.
17. Оператор `bit` (уникальный для `a86`) задает единицу в указанном бите и ноль во всех остальных.
18. Если бы вывод шел от младших к старшим битам, то между краями картинки и вертикальными линиями был бы просвет — 6 точек слева и 7 точек справа.
19. В программе `bitmap` она выглядит *так же*, а размер `bmp`-файла, конечно, должен увеличиться.
20. Символ `?` в директивах `db`, `dw` и `dd` означает неопределенное значение, обычно 0.
21. Поэтому не следует удалять эти проверки. Особенно много ошибок возникнет в (36) при программировании картинки, отличающейся от образца.
22. В `a86` числа, начинающиеся с нуля, считаются шестнадцатеричными, так что 10 — это десять, а 010 — шестнадцать.

ГЛАВА 3

```
DB6 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100 2: =d1,ax,, 1962
CX 0000  CS 6309 3:
DX 6309  SS 6309 4:
SI 011F  DS 6309 5: r5,b_1,, AB ab
DI 0000  ES 6309 6: r5,si,, AB ab
BP 0000  SP FFFE 0:

lea si,b_1
```

Данные, имена и типы

Я вас научу, с какого конца редьку есть!
(из к/ф "Сказ про то, как царь Петр арапа женил")

Остановимся на теории, которая стоит за примерами из предыдущей главы. Интерес представляют в том числе следующие вопросы:

- определение *данных*;
- определение и использование *имен*;
- понятие *типа* имени.

Эти объекты и понятия составляют существенную часть языка и в то же время самую простую, т. к. не зависят от системы команд.

Структура программы

Программа представляет собой последовательность операторов, по одному на строке. В листинге 3.1 показана структура исходного текста программы на языке ассемблера а86 для получения исполняемой программы com-формата.

Листинг 3.1. Структура исходного текста

```
jmp start
...           ; db/dw/dd
...

start:
...
...
int 020      ; ret
```

Вначале задана инструкция `jmp` для обхода области данных, которые начинаются от этой команды и идут до метки `start`. От этой метки начинаются машинные команды в символическом виде. Последняя инструкция выполняет возврат в FreeDOS.

Перед инструкцией (в той же строке) может стоять метка, т. е. имя с двоеточием, которое обозначает в дальнейшем адрес инструкции. Аналогично перед директивой определения данных можно поставить имя, которое в дальнейшем обозначает адрес данных в символическом виде. Литера `;` открывает комментарий, который продолжается до конца строки.

В редких случаях программа состоит только из инструкций или, еще реже (как в главе 1), из одних данных. В этих случаях ставить в начале команду обхода данных — лишнее.

Директивы определения данных

Директивы `db`, `dw` и `dd` резервируют соответственно байты, двухбайтовые слова и четырехбайтовые двойные слова. После оператора `db` (или `dw`, или `dd`) записывается значение элемента данных; знаком вопроса обозначается произвольная величина. Пример определения данных приведен в листинге 3.2.

Листинг 3.2. Определение данных (exa1.8)

```
dw    0123             ; (1)
dd    0abcd1234        ; (2)
dd    bit 16           ; (3)
db    '?'              ; (4)
db    '8'               ; (5)
db    1                 ; (6)
db    1                 ; (7)
db    1                 ; (8)
db    ?                 ; (9)
```

Числа в (1–2) начинаются с нуля, в а86 они считаются шестнадцатеричными. Это еще одна из особенностей а86, в других языках 16-ричное число (2) записали бы как `0xabcd1234` или `0abcd1234h`.

Итак, в (1) задано двухбайтовое слово со значением 291, в (2) — двойное слово, в (3) — двойное слово со значением $010000 = 65536$, в (4–5) — байты со значениями ASCII-кодов знака вопроса и восьмерки, в (6–8) — по одному байту со значением 1, а в (9) — байт с произвольным значением.

Запустите сеанс Bochs-FreeDOS, перейдите в каталог `c:/work/01` и выполните¹ трансляцию этого примера: `a86 exa1.8`. Результат — файлы `exa1.com` и `exa1.sym`. Нас интересует файл `exa1.com` с данными, только не вызывайте его на выполнение.

Можно просмотреть `exa1.com` по байтам в шестнадцатеричном формате²: сначала нажать `<F3>` (View), а потом переключиться на шестнадцатеричный формат — `<F4>`. А можно воспользоваться файлом-листингом. Чтобы получить его, повторите трансляцию с ключом `+L` в любом месте командной строки после `a86`. Отчет о ходе трансляции в `lst`-файле показан в листинге 3.3.

Листинг 3.3. Результат трансляции `exa1.8` (`exa1.lst`)

1 0100	23 01	dw	0123
2 0102	34 12 CD AB	dd	0abcd1234
3 0106	00 00 00 00	dd	bit 16
4 010A	3F	db	'?'
5 010B	38	db	'8'
6 010C	01	db	1
7 010D	01	db	1
8 010E	01	db	1
9 010F	00	db	?

В первом столбце показан номер строки, во втором — значение счетчика адресов, а дальше — байты в шестнадцатеричном коде. Видно, что счетчик адресов увеличивается от 256 (0100) на 1, или 2, или 4 в зависимости от числа распределяемых байтов.

Первые два байта — это слово (1): сначала идет младший байт 023, за ним старший — 1. Затем четыре байта, составляющие двойное слово (2). Порядок байтов — от наименее значимой части 034 в сторону наиболее весомой 0ab, как принято у Intel. В строке (3) задано двойное слово с установленным битом номер 16, но `lst`-файл показывает 0 во всех четырех байтах. (Это ошибка в `a86 v4.05`: значения `bit 0`, ..., `bit 15` соответствуют описанию, однако `bit 16`, ..., `bit 31` дают ноль³.) Следующие два байта — ASCII-коды⁴ вопроса и цифры 8. Далее идут три единичных байта (6–8). В конце — байт с произвольным значением, как правило, это 0 (по крайней мере, в программах из одного исходного модуля).

Исправьте третью строку в `exa1.8`:

```
dd 010000
```

Повторите трансляцию с ключом `+L` и убедитесь, что в третьей строке `lst`-файла появился ненулевой байт. Скопируйте полученный файл `exa1.com` в `exa1.dat`, откройте `exa1.8` и соберите (4–9) в один оператор:

```
db '?', '8', 1, 1, 1, ?
```

Повторите трансляцию и сравните результаты: `fc /b exa1.com exa1.dat`.

Теперь сгруппируем литеры в кавычках и три повтора единицы:

```
db '?8', 3 dup (1), ?
```

Проверьте результат.

Конструкция `dup` уже встречалась в предыдущей работе. Число перед `dup` задает счетчик повторов, а после `dup` идет список значений. Если в нем только одно значение, скобки можно не ставить:

```
db '?8', 3 dup 1, ?
```

Вот пример вложенной конструкции `dup`:

```
db 2 dup (1, 3 dup 0)
```

В нем задано двукратное повторение последовательности (1, 3 dup 0):

```
db 1, 3 dup 0, 1, 3 dup 0
```

Что в итоге означает:

```
db 1, 0, 0, 0, 1, 0, 0, 0
```

Следующий вопрос: какими могут быть числовые значения в директивах `db`, `dw` и `dd`? В табл. 3.1 приведены диапазоны чисел в зависимости от размерности данных. Диапазоны *знаковых* и *беззнаковых* чисел в табл. 3.1 объединены, так что результирующие диапазоны несимметричны. Например, байт может представлять знаковое число от -128 до $+127$ или беззнаковое число от 0 до 255. Вот и получается, что для байта можно задать число от -128 до 255.

Таблица 3.1. Диапазоны данных

Размерность	Диапазон
Байт	$-128...+255$
Слово	$-32\,768...+65\,535$
Двойное слово	$-2\,147\,483\,648...+4\,294\,967\,295$

Один и тот же набор битов может представлять разные значения в зависимости от того, считается ли число знаковым или беззнаковым. Так, например, последовательность битов 11111111 задает беззнаковое число 255 и в то же время знаковое число -1 . Различие проявляется, когда мы выбираем команду *ветвления* после сложения, вычитания или проверки числа. Например, переход, когда одно число больше другого, обозначается `ja` (jump if above) для беззнаковых чисел и `jg` (jump if greater) для чисел со знаком. Это разные команды. Команды умножения и деления — тоже разные для знаковых (`imul` и `idiv`) и беззнаковых (`mul` и `div`) величин.

Обратимся к примеру, приведенному в листинге 3.4.

Листинг 3.4. Диапазоны чисел (hex2.8)

```
(1) db -128, 127, 255, ?
(2) dw -32768, 32767, 65535, ?
(3) dd -2147483648, 2147483647, 4294967295, ?
(4) dw 055aa
```

Выполните трансляцию и проверьте результат при помощи файла-листинга или счет просмотра `com`-файла. Обратите внимание на первые два байта — это самое отрицательное (080) и самое положительное (07f) знаковые числа. Третий байт — максимальное беззнаковое число (0ff), а если рассматривать его как знаковое, то это -1 . Аналогично — для второй строки, только в диапазоне 16-битовых слов: 08000, 07fff и 0ffff. В третьей строке — для 32-битовых: 080000000, 07fffffff и 0fffffff.

Прежде чем приступить к опытам с предельными значениями, проверим еще один вариант просмотра двоичных данных: при помощи отладчика d86. Установите курсор на файл `exa2.sym`⁵ и нажмите клавишу `<Enter>` или выполните команду `d86 exa2.com`. (На всякий случай, выход из d86 — `<q>+<Enter>`.)

В обоих случаях d86 показывает содержимое файла `exa2.com`, загруженное в память с адреса `0100` (256). В левой верхней части экрана — команды, которые процессор "увидел" бы на месте данных, определенных в `exa2.8`, если бы ему пришлось выполнять `exa2.com`. Туда лучше не смотреть.

Для наблюдения за данными есть однострочные "окна" с номерами 1–6. Нажмите, например, клавишу `<2>` — и красный курсор перейдет в начало строки 2. Там мы сначала задаем формат, а потом начальный адрес: `b, 0100 <Enter>`. В строке 2 теперь отображаются байты в шестнадцатеричном формате (`b`) от адреса `0100` и далее, насколько хватает ширины окна.

Пока что неясно, где кончаются байты (1) и начинаются слова (2). Сделаем так, чтобы в первой строке отладчика были видны ровно четыре байта (1), а в строке 2 — следующие за ними четыре слова. Для этого нажмите клавишу `<1>` и введите `b4, 0100 <Enter>`, потом нажмите `<2>` и введите `w4, 0104 <Enter>`, потом нажмите `<3>` и введите `w8, 010c <Enter>`. Для отображения слов используем формат `w`, и его же — для отображения двойных слов по половинкам, т. к. специального формата для двойных слов нет. Выглядит получше, но все-таки неудобно то, что приходится задавать адреса в числовом виде (`0100`, `0104`, `010c`).

Нажмите клавишу `<3>` и введите символ двойной кавычки. Теперь в строке 3 отладчика мы видим продолжение вывода предыдущей строки. Начальный адрес данных для строки 3 отладчик подсчитал сам. Нажмите клавишу `<4>` и введите символ `"`. Чтобы увидеть завершающее слово `055aa`, продолжите вывод на пятую строку отладчика. Можно также продолжить вывод в верхнюю правую часть экрана: сотрите вывод в строке 5 (нажмите `<5>` и `<пробел>`), а потом нажмите `<F10>`.

Прежде чем выйти из d86, сохраним нажатия клавиш: `k s alta <Enter>`. Снова зайдя в d86 с тем же файлом данных, нажмите `<Alt>+<A>`. Ввод с клавиатуры, сохраненный в файле `alta.d8k`, будет воспроизведен и в результате восстановится отображение данных.

Чтобы совсем избавиться от адресов в числовом виде, необходимо определить *имена* в программе и указывать их в строках отображения данных d86. В листинге 3.5 показано, как можно определить имена последовательностей данных в рассматриваемом примере.

Листинг 3.5. Символические адреса (exa3.8)

```
data0 db -128, 127, 255, ?
data1 dw -32768, 32767, 65535, ?
data2 dd -2147483648, 2147483647, 4294967295, ?
caption db 'One-string "window", ', "it's d86."
```

Выполните трансляцию сразу с двумя ключами: `+L` и `+H5`. Ключ `+H` увеличивает в `lst`-файле число строк, на которых выводятся сгенерированные коды. Ключи можно объединить: `+LH5`. Имя `data0` — это символический адрес первого байта, определенного в строке (1) листинга 3.4. Числовые значения имен можно увидеть в списке в конце `lst`-файла.

Зайдите в `d86` с файлом `exa3.com`, нажмите `<Alt>+<A>` и заново настройте отображение данных: в строке 1 отладчика задайте `b4,data0`, в строке 2 — `w4,data1`, в строке 3 — `w8,data2`, в строке 4 — двойную кавычку для остатка данных `data2`, а в строке 5 — `r,caption`. (Формат `r` — наиболее удобный для отображения ASCII.) Вывод строки 5 продолжится вверху справа.

В кодовой части экрана (слева наверху) не видно ни одного из имен, определенных в программе. Но директив определения данных там тоже не видать, т. к. это область отображения кода, а не данных. В этой области выводятся только метки, и мы можем увидеть имена из листинга 3.5, если добавим к ним двоеточие. Вообще-то это плохая идея: метки нужны для обозначения адресов перехода в *коде*. Но любопытства ради проверьте: добавьте в текст программы `exa3.8` двоеточия после имен и заново выполните трансляцию и вызов `d86`.

Вернемся к вопросу о диапазонах. В `exa3.8` (см. листинг 3.5) задайте `-129` вместо `-128` и `256` взамен `255`. (Если вместо `127` задать `128`, это число хотя и не попадает в диапазон знакового байта, но как беззнаковое вполне допустимо.) В ответ `a86` выдает ошибку, но только для `256`. Если вернуть `255`, оставив `-129`, то в `com`-файле первый байт окажется равным `07f` — `a86` "подтянул" число `-129` к границе диапазона. Конечно, это недостаток `a86`.

Не блестяще, но бывает и хуже. Так, выход из диапазона при трансляции с языка `C` вообще не считается ошибкой. Старшая часть числа отбрасывается, байт `256` превратится в ноль, а `257` — в единицу (листинг 3.6). В лучшем случае транслятор выдает предупреждение, но их редко кто читает.

Листинг 3.6. Статическое переполнение в языке `C` (`exa4.c`)

```
#include <stdio.h>

void main ()
{
    char c = 257;
    unsigned int x = 0x1abcd;
    unsigned long y;

    y = x;
    printf("%x %lx\n", c, y);
}
```

В 16-битовом компиляторе тип `int` задает 16-разрядное слово, а `long` — 32-разрядное. Тип `char` — это всегда байт, для любого компилятора. Если воспользовать-

ся 16-битовым Watcom-C (wcl exa4.c), то полученная программа выведет сначала 1 (то, что осталось от 257), а затем abcd — то, что осталось⁶ от 0x1abcd.

Обозначение чисел

Числа могут быть заданы в двоичной, восьмеричной, десятичной и шестнадцатеричной системах счисления, а также кодами ASCII (литеры в одинарных кавычках).

По умолчанию действует десятичная система счисления. Если первая цифра числа — ноль, оно считается шестнадцатеричным. Суффиксы `b`, `q`, `d`, `h` позволяют изменить систему счисления для одного (текущего) числа на двоичную, восьмеричную, десятичную и шестнадцатеричную соответственно⁷. Рекомендуется ставить перед ними `x` — тогда суффикс ни в какой ситуации не будет воспринят как шестнадцатеричная цифра.

Примеры задания чисел приведены в листинге 3.7.

Листинг 3.7. Числа в разных системах счисления (exa5.8)

```
dw 18, 012, 12h           ; (1)
dw 11b, 011b, 011xb, 011q ; (2)
dw 011b, 012b             ; (3)
dw 12, 12d, 012d          ; (4)
dw 0a, 0ah, 0axh, ah      ; (5)
```

В строке (1) задано число 18 — сначала в десятичном формате, а потом два раза в шестнадцатеричном. В (2) задано число 3 — трижды в двоичном формате, потом в 8-ричном. (Интересно, что `011b`, которое а86 должен бы трактовать как шестнадцатеричное, он воспринимает как двоичное. Действительно, если перед `b` идут только нули и единицы, то а86 считает число двоичным, за счет чего улучшается его совместимость с `asm86`.) В строке (3) задано двоичное число 3 и шестнадцатеричное число 299. В (4) — два раза десятичное число 12, затем шестнадцатеричное число 301. В (5) — три раза шестнадцатеричное число со значением 10, а в конце, ошибочно, идентификатор, обозначающий младший байт регистра-аккумулятора.

Проверьте полученные значения по `lst`-файлу и в отладчике, используя формат `d` для отображения 16-разрядных беззнаковых слов⁸.

Числа, кратные 1024, в а86 можно обозначать с использованием суффикса `k`. Этот суффикс задает коэффициент 1024, перед ним должно быть десятичное число⁹. В листинге 3.8 приведены варианты определения 2 Кбайт данных.

Листинг 3.8. Множитель 'k' (exa6.8)

```
db 2048 dup 1
db 2k dup 2
dw 1k dup 3
dd 1k/2 dup 4
```

Символические обозначения чисел, выражения

Смысл введения символов для обозначения чисел такой же, как в языках высокого уровня: упростить корректировку параметров программы, сделать исходный текст понятнее.

Символ для обозначения числа вводится директивой `equ`:

```
name equ const_expr
```

Здесь `name` — имя, составленное по общим правилам (начинается с буквы или подчеркивания, а дальше — любое количество букв, подчеркиваний и цифр), а `const_expr` — число или константное *выражение*, составленное из чисел и/или символов, обозначающих числа, как в последней строке примера в листинге 3.9.

Листинг 3.9. Директива 'equ' (сх7.8)

```
dozen      equ 12
_sign_bit  equ 10000000xb
max_byte   equ 255
bakers_dozen equ dozen + 1
```

Выполните трансляцию с ключом `+L` и найдите в конце файла-листинга таблицу имен. Обратите внимание, что `com`-файл имеет нулевой размер — в программе нет ни данных, ни кода, одни определения имен.

Операции, допустимые в константных выражениях, приведены в табл. 3.2. Звездочкой отмечены варианты, уникальные для `a86`.

Таблица 3.2. Операции в выражениях

Операция	Действие или результат	Очередность
<code>()</code>	Изменение очередности операций	1
<code>*bit n</code>	Число, в котором <i>n</i> -й бит установлен в 1, а остальные сброшены (биты нумеруются от нуля)	
<code>high n</code> <code>low n</code>	Значение старшего байта числа <i>n</i> Значение младшего байта числа <i>n</i>	2
<code>*k by n</code>	Число, старший байт которого равен <i>k</i> , а младший <i>n</i>	
<code>k / n</code> <code>k * n</code>	Целая часть результата деления <i>k</i> на <i>n</i> Произведение чисел <i>k</i> и <i>n</i>	3
<code>k mod n</code>	Остаток от деления <i>k</i> на <i>n</i>	
<code>k shl n</code> <code>k shr n</code>	Сдвиг двоичного кода числа <i>k</i> на <i>n</i> бит влево Сдвиг значений разрядов числа <i>k</i> вправо на <i>n</i> бит	
<code>k + n</code> <code>k - n</code>	Сумма чисел <i>k</i> и <i>n</i> Разность чисел <i>k</i> и <i>n</i>	4

Таблица 3.2 (окончание)

Операция	Действие или результат	Очередность
not k	Поразрядная инверсия числа k	5
k or n	Поразрядное логическое сложение чисел k и n	6
k and n	Поразрядное логическое умножение чисел k и n	
k xor n	Поразрядное исключающее "или" чисел k и n	7

Проверьте с использованием lst-файла следующий пример (листинг 3.10).

Листинг 3.10. Константные выражения в директиве 'equ' (exa8.8)

```
alfa    equ    10 / 4                ; 2                ; (1)
beta    equ    low (alfa * 083)      ; 6                ; (2)
gamma    equ    low alfa * 083        ; 0106             ; (3)
right    equ    -4*(-2)               ; 8                ; (4)
wrong    equ    -4*-2                ; -2 (?)           ; (5)
         db     1 shl 5               ; 100000xb         ; (6)
bit5     db     bit 5                 ; 100000xb         ; (7)
         db     not bit 0              ; 11111110xb       ; (8)
         db     bit 1 + bit 3          ; 1010xb           ; (9)
array    dw     beta + 1 dup 'A' by 10 ; (10)
```

Обратите внимание: в строке (5) получен неправильный результат. После знака операции +, −, / или * отрицательное число следует задавать в скобках, как в (4). Считать это ошибкой а86 или нет¹⁰?

В строке (10) константные выражения использованы и до, и после ключевого слова dup. Задано семь слов со значением ASCII-кода литеры "A" в старшем байте и числом 10 в младшем байте.

Выполните трансляцию с ключом +L и проверьте таблицу имен в конце lst-файла. Обозначения mb, mw и md говорят о том, что имя означает адрес байта, слова, или двойного слова в памяти (тип byte, word или dword¹¹). Напротив, двоеточие говорит о том, что это не адрес данных, а константа или метка, т. е. адрес перехода в коде.

Несколько заданий по директивам db, dw, dd и equ с использованием выражений.

1. Запишите директиву определения 10 Кслов с ASCII-кодом '0' в младшем байте и '1' в старшем байте слова¹².
2. Запишите число, в котором установлен пятый разряд, двумя способами: при помощи суффикса xb и оператора bit n.
3. Запишите десятичные числа, соответствующие обозначениям: 015, 15, 11b, 17q, 0a, bit 3, 'A'.
4. Определите константу hours, равную произведению числа дней в году на число часов в сутках. Используя имя hours, задайте слово со значением числа часов

в году, двойное слово со значением количества минут в году и двойное слово со значением числа секунд в году.

5. Определите константу `len` со значением 100, а затем определите слово со значением площади квадрата со стороной `len`.
6. Определите константу `hip` со значением 200, затем определите слово со значением площади прямоугольного равнобедренного треугольника со сторонами `hip`.
7. При помощи операторов `bit` и `+` (или `or` вместо `+`) определите константу, содержащую единицы в третьем и седьмом разрядах; определите константу с другим именем, но с тем же значением, заменив оператор `bit` на `shl`.
8. При помощи операторов `bit`, `not` и `+` (или `or` вместо `+`) определите байт, в котором сброшены нулевой и третий разряды.

Проверьте решения, пользуясь `lst`-файлами. Размер полученного `com`-файла можно увидеть в `VC` внизу, если установить курсор на этот файл (еще есть команда `dir /w *.com`).

Переменные и метки

Пользовательские имена — это имена, определенные в тексте программы. Они заменяют числа при обозначении адресов и констант. Рассмотренные в предыдущем разделе символические обозначения чисел — это один из вариантов определения и применения пользовательских имен.

Адреса в символьной форме обозначаются именами переменных и меток.

Переменная — это символическое обозначение адреса первого байта (или слова, или двойного слова) данных, определенных директивой `db/dw/dd`. Имя переменной задается перед директивой определения данных и в дальнейшем обозначает адрес данных.

Метка — это символическое обозначение адреса инструкции. Метку задают в начале строки как имя с двоеточием, ею можно обозначить адрес перехода в командах передачи управления (листинг 3.11).

Листинг 3.11. Переменные и метки (exa9.8)

```

        jmp  start                      ; (1)

area1   db   bit 2, 055, 7             ; (2)
        dw   -2                        ; (3)
area2   dw   2, 0aa55, 7               ; (4)

start:                                     ; (5)
        not  area1                      ; (6)
        not  area2+2                    ; (7)
        not  area2-2                    ; (8)
        int  020                        ; (9)

```


Необходимые пояснения, прежде чем приступить к проверке примера:

- Имя `areal` обозначает адрес первого из трех байтов (2). Эти байты можно адресовать как `areal`, `areal+1`, `areal+2`. Аналогично слова (4) можно обозначать как `area2`, `area2+2`, `area2+4` (смещение удваивается, т. к. каждое слово занимает по два байта).
- Мнемоника `not` обозначает машинную инструкцию инвертирования битов (все биты операнда меняют значение на противоположное). Первая из них команда (6) задает инверсию *байта* со значением `00000100xb` по адресу `areal`; после ее выполнения значение байта равно `11111011xb`. Команда (7) задает инверсию *слова*, исходное значение которого равно `0aa55`, а результат будет `055aa`. Выполнение (8) инвертирует *слово* с начальным значением `-2` (`11111110xb`), которое определено в (3).

Этапы проверки:

- Выполните трансляцию, вызовите `d86` и настройте окна отображения данных; байты можно отображать как в формате `b` (hex), так и в формате `e` (binary).
- Пролистайте окна в правой верхней четверти экрана `d86`, нажимая `<F10>`. Это главный набор вспомогательных окон. Чтобы переключиться на альтернативный набор, нажмите `<Alt>+<F10>`. Теперь справа наверху показаны клавиши для выполнения кода. Нам пока достаточно пошаговой отладки (single step) по клавише `<F1>`. Нажимайте `<F1>`, наблюдая за изменением данных, пока не остановитесь на (9).
- Начните ввод инструкции `not areal`. После нажатия первой буквы ('n') курсор перейдет в нижнюю строку отладчика, а вы продолжайте ввод. После нажатия `<Enter>` команда будет выполнена. Ее можно повторить, не вводя заново, нажатиями `<F3>`.
- Выражения `areal+3` и `area2-2` обозначают один и тот же адрес в памяти, но они разного типа (`byte` и `word`). Поэтому размерность операции `not` для этих операндов разная, что заметно при выполнении команд `not areal+3` и `not area2-2`.

Итак, с именем переменной связан *тип*: переменная означает либо адрес байта, либо адрес слова, либо адрес двойного слова — в зависимости от директивы `db`, `dw` или `dd`, в начале которой задано имя. Одна и та же мнемоника инструкции может задавать операцию над байтом, словом или двойным словом — в зависимости от типа операнда¹³.

Типы имен

Тип символьных констант и меток — `abs`. Типы переменных — `byte ptr` (сокращенно `byte`), `word ptr` (сокращенно `word`), `dword ptr` (сокращенно `dword`). В `a86` можно еще короче: `b`, `w` и `d`.

Тип переменной позволяет обозначать операцию над байтом, словом или двойным словом одинаково. Так, например, `not` задает операцию инвертирования для всех битов байта, слова или двойного слова (выбор определяется типом операнда).

Тип назначается автоматически при определении имени. Явное указание типа при помощи `abs`, `b`, `w`, `d` используется в опережающих описаниях, а обозначения `b`, `w`, `d` можно применять для временного (в пределах одной команды) изменения типа переменной. Временное преобразование к типу `abs` выполняет оператор `offset`.

В листинге 3.12 приведен пример с теми же именами и данными, что в листинге 3.11.

Листинг 3.12. Явное указание типа операнда (exa10.8)

```
extrn item:w                                ; (1)
jmp start                                  ; (2)

areal db bit 2, 055, 7                     ; (3)
      dw -2                                ; (4)
area2 dw 2, 0aa, 7                         ; (5)

start: not word ptr areal                  ; word      ; (6)
      not word areal                      ; word      ; (7)
      not w areal                         ; word      ; (8)
      not b area2+2                       ; byte      ; (9)
      not start                          ; abs (?)    ; (10)
      not w start                         ; word      ; (11)
      mov ax, area2                       ; word      ; (12)
      mov ax, offset area2                ; adr       ; (13)
      mov item, ax                        ; word      ; (14)
      int 020                             ;           ; (15)

item dw ?                                 ; (16)
```

Инструкция (10) даст ошибку при трансляции, поскольку тип операнда — `abs`. Этот тип может обозначать либо константу, либо адрес инструкции. Константу нельзя изменить по определению, а модификация машинной команды — это либо ошибка, либо сомнительный программистский трюк. В строке (11) это ограничение обошли за счет временного преобразования типа.

Закомментируйте строку (10), повторите трансляцию и выполните команду `d86 exa10.com`. При входе в отладчик строки отображения данных (1–3) уже настроены на показ переменных (3–5) и (16). Каким образом? Задав отображение данных в `d86`, я сохранил последовательность нажатия клавиш командой `k s exa10` в файле `exa10.d8k`. При повторном запуске `d86` воспроизвел их автоматически, обнаружив в текущем каталоге одноименный `d8k`-файл.

Проверьте работу программы по шагам.

В строках (6–8) задано логическое инвертирование *слова* по адресу `areal`, несмотря на то, что имя `areal` обозначает адрес байта. Аналогично в (9) и (11) задана инверсия байта и слова, независимо от типа имени. Проверяя работу программы в отлад-

чике, обратите внимание, что выполнение (11) приведет к искажению кода инструкции (6) по адресу `start`. Инструкция (12) скопирует *содержимое* слова по адресу `area2` в регистр `ax` процессора (значение `ax` в результате станет равно двум). Напротив, инструкция (13) запишет в `ax` значение *адреса* `area2`.

Директива `extrn` в (1) задает тип для имени `item`, не резервируя памяти¹⁴. Память под этот байт распределяется в конце исходного текста (16), но имя используется раньше — в (14). Тип имени `item` должен быть известен транслятору при обработке (14), для этого тип указан заранее в (1). Необходимости в таком опережающем описании нет, если придерживаться типовой структуры программы, приведенной в листинге 3.1.

Типы и выражения

Рассмотрим правила совместимости типов в выражениях, а также правило для определения типа результата выражения.

Правила совместимости:

- ❑ Над именами типа `abs` выполнимы все операции, предусмотренные в выражениях (напоминаем, что типом `abs` характеризуются числа и метки).
- ❑ Имя переменной (тип `byte`, `word`, `dword`) в выражении допускается только справа и/или слева от знаков сложения и вычитания.

Правило определения типа результата:

- ❑ Результат сложения/вычитания принимает тип первого операнда.
- ❑ Как исключение, результат сложения с одной переменной всегда принимает тип переменной.

В листинге 3.13 для определения типа результата используется оператор `type name`, который возвращает число байтов, занимаемых переменной `name`:

- ❑ 1, если переменная типа `byte`;
- ❑ 2, если `word`;
- ❑ 4, если `dword`;
- ❑ 0, если это константа или метка (тип `abs`).

Листинг 3.13. Тип результата константного выражения (exa11.8)

```

byt1 db 4 dup 10
word1 dw 0200

obj1 equ type (byt1 - 1)           ; 1 - "byte"
obj2 equ type (word1 - 1)          ; 2 - "word"
obj3 equ type (1 - byt1)           ; 0 - "abs"
obj4 equ type (1 - word1)          ; 0 - "abs"
```

```

obj5    equ    type (word1 - byte1)           ; 2 - "word"
obj6    equ    type (byte1 - word1)           ; 1 - "byte"
obj7    equ    type (2 - word1)               ; 0 - "abs"

_obj1   equ    type (byte1 + 1)               ; 1 - "byte"
_obj2   equ    type (word1 + 1)               ; 2 - "word"
_obj3   equ    type (1 + byte1)               ; 1 - "byte"
_obj4   equ    type (1 + word1)               ; 2 - "word"
_obj5   equ    type (word1 + byte1)           ; 2 - "word"
_obj6   equ    type (byte1 + word1)           ; 1 - "byte"
_obj7   equ    type (2 + word1)               ; 2 - "word"

tail    equ    byte1 + 3
var      dw     offset word1, word1

```

Проверьте результат трансляции по `lst`-файлу. Обратите внимание на данные последней директивы. Там заданы два одинаковых значения — *адрес* слова `word1` (0104). Заметим, что *значение* данных, например, слова `word1` (0200) ни на какие директивы ассемблера не влияет. От значения переменной зависит только результат инструкций, где она используется.

Директива `equ` применяется не только для задания символических констант. В общем случае директива `name equ expr` вводит новое имя `name` с тем же значением и типом, что у константного выражения `expr`. В примере имя `tail` определено выражением `byte1 + 3`. Тип этого выражения образуется сочетанием двух составляющих его типов: `byte` и `abs`. Согласно правилам определения типа константного выражения `byte + abs` дает `byte`. Численное значение этого выражения на три больше значения `byte1`. В программе с адреса `byte1` заданы четыре байта со значениями 10, и выражение `byte1 + 3` адресует последний из них. В итоге переменная `tail` адресует последний байт, заданный в определении переменной `byte1`. Конкретные адреса `byte1` и `tail` в данном случае несущественны — важно лишь то, что `tail` задает отступ от начала последовательности байтов `byte1`.

В листинге 3.14 показаны примеры ошибок и неточностей при определении данных.

Листинг 3.14. Некорректные определения данных (exa12.8)

```

var5    db     (word1 - byte1) dup ?          ; (1)
var6    db     word1                          ; (2)
var7    db     offset byte1                   ; (3)

```

В строке (1) — ошибка: счетчик повторов в конструкции `dup` должен быть типа `abs`. В строках (2–3) допущена неточность: старший байт 16-разрядного адреса переменной будет отброшен (вероятно, с потерей значащих разрядов).

И в завершение темы об именах, данных и типах — ряд вопросов и заданий.

1. Перечислите имена, определенные в следующем фрагменте, и их типы:

```
noodle equ '0' by 0
needle db 40k / 7 dup 0
al dw offset needle - 0100
```

2. Найдите ошибки в следующем фрагменте:

```
src db "You're genious
dest dw bit 16 + offset src
size equ (dest - src) / 2
```

3. Данные определены следующим образом:

```
byte1 db 1, 2
word1 dw 03040, 05060
```

Какие значения они получают после выполнения последовательности инструкций:

```
not byte1
not byte1+1
not byte1+2
not word1
not word1+1
```

4. Данные определены следующим образом:

```
word1 dw 03040, 3 dup 0, 0ffff
```

Запишите инструкцию, выполнение которой проинвертирует слово со значением -1 .

5. Данные определены следующим образом:

```
word1 db 3k dup 0, 0ff
byte1 dw ?
```

Запишите инструкцию, которая проинвертирует последний байт, заданный директивой `db`, а также инструкцию, которая проинвертирует последнее слово из последовательности, заданной директивой `db`. (Попробуйте адресовать эти данные, используя имя `byte1`.)

6. Данные заданы следующим образом:

```
abcd db 'ABCD'
digit db '0123456789'
```

Какими станут их значения после выполнения инструкций:

```
not abcd
not abcd+6
not digit-3
```

7. Данные заданы следующими объявлениями:

```
byte1 db -2, 2, 3
```

Запишите команду для инвертирования слова со значением 02fe, и команду для инвертирования слова со значением 0302.

8. Данные заданы следующими объявлениями:

```
dword1 dd 0102ff04
```

Запишите команду, которая проинвертирует слово со значением 02ff, входящее в двойное слово, и команду, которая проинвертирует байт со значением 4 в двойном слове.

9. Данные заданы следующими объявлениями:

```
area1 db (080 shr 3) dup ?
area2 dw (offset area2 - offset area1) dup ?
dummy db ?
size equ offset dummy - offset area1
```

Чему равна константа size?

Фрагменты программ из вышеприведенных заданий собраны в файле quiz1.txt. Создайте на их основе программы и проверьте результат трансляции, используя lst-файл и отладчик.

Теперь остановимся подробнее на именах: какую информацию они несут и в каких случаях (и зачем) допускается их переопределение. Чтобы ответить на первый вопрос, начнем со схемы работы транслятора.

Алгоритм трансляции

Транслятор считывает исходный текст по строкам, от начала до конца, дважды. При первом просмотре транслятор обнаруживает *имена*, заданные пользователем (т. е. константы, метки, переменные), для каждого имени определяет атрибуты (числовое значение и тип) и записывает их в таблицу имен. При втором просмотре транслятор определяет *значения* данных и кодов инструкций и записывает их в выходной файл.

Для резервирования памяти предусмотрен счетчик адресов. Его начальное значение при трансляции com-программ — 0100 (256) — это размер PSP (Program Segment Prefix), к которому пристыковывается содержимое com-файла при загрузке¹⁵. При разборе очередного оператора ассемблера счетчик адресов увеличивается на число байтов, занимаемое машинным представлением инструкции или данных.

На первом просмотре каждой метке или переменной присваивается текущее значение счетчика адресов. Оно обозначается значком \$ (или ключевым именем this) и допускается в выражениях наряду с именами и числами (считается, что оно имеет тип abs).

Рассмотрим алгоритм первого прохода на примере программы bmp_0.8 из главы 2 (для этого понадобится калькулятор, настроенный на шестнадцатеричные числа¹⁶). Далее эта программа приводится по частям, и каждый раз мы показываем новый фрагмент таблицы имен.

```
(1) start:
(2) iWidth      equ 64
(3) iHeight     equ 32
(4) bitsPerPixel equ 1
```

В начале программы определена метка `start`. (Она служит в (12) для вычисления смещения данных растра от начала файла.) Имя `start` записывается в таблицу имен с числовым значением `0100` и типом `abs`. После обработки операторов (1–4) значение `$` не изменилось, т. к. не было ни данных, ни инструкций. Содержимое таблицы имен после трансляции (1–4) приведено в табл. 3.3. Собственно к таблице имен относятся только имя, значение и тип, а размер распределяемой памяти и значение счетчика адресов *после* трансляции оператора приведены для сведения.

Таблица 3.3. Таблица имени после трансляции строк 1–4

Имя	Значение	Тип	Размер в байтах	\$
start	0100	abs	0	0100
iWidth	64	abs	0	0100
iHeight	32	abs	0	0100
bitsPerPixel	1	abs	0	0100

Следующий фрагмент исходного текста:

```
(5) Colors      equ bit bitsPerPixel
(6) pixelsPerByte equ 8 / bitsPerPixel
(7) bytesPerLine equ iWidth / pixelsPerByte
(8) byteCount   equ bytesPerLine * iHeight
```

Здесь определены еще четыре имени типа `abs`, поскольку справа от `equ` все приведенные выражения имеют тип `abs`. То, что добавилось к таблице имен, показано в табл. 3.4.

Таблица 3.4. Фрагмент таблицы имен после трансляции (5–8)

Имя	Значение	Тип	Размер в байтах	\$
Colors	2	abs	0	0100
pixelsPerByte	8	abs	0	0100
bytesPerLine	8	abs	0	0100
byteCount	256	abs	0	0100

Дальше идет фрагмент, где начинается распределение памяти:

```
(9)      db      'BM'
(10)     dd      end - start
(11)     dd      0
(12)     dd      offset Pixels - start
```

В таблицу имен не попадает ничего, поскольку в начале директив (9–12) имена не заданы. Но счетчик адресов увеличивается: в (9) на два (две байтовые литеры), в (10) — на четыре (одно двойное слово), в (11–12) — еще два раза по четыре. В итоге $\$ = 0100 + 2 + 3 \cdot 4 = 010e$.

В строках (13–20) аналогично: идет распределение памяти, а новых имен нет. Задано девять двойных слов и два обычных. В итоге $\$ = 010e + 9 \cdot 4 + 2 \cdot 2 = 010e + 028 = 0136$

Наконец появляется директива распределения памяти, одна с именем:

```
(21) Palette db    255, 0, 0,    ?
(22)         db    0,    0, 255, ?
```

Транслятор заносит имя `Palette` в таблицу имен, еще не зная, что за ним последует. А последует директива `db`, и это значит, что тип имени — `byte`, а значение равно текущему счетчику адресов, т. е. `0136` (табл. 3.5).

Таблица 3.5. Фрагмент таблицы имен после трансляции (21)

Имя	Значение	Тип	Размер в байтах	\$
Palette	0136	byte	4	013a

После разбора (22) счетчик адресов станет $013a + 4 = 013e$.

В следующем фрагменте все имена имеют тип `abs`, память не резервируется, и впервые в константных выражениях участвует счетчик адресов.

```
(23) Colors equ ($ - Palette) / 4
(24) zero   equ 0
(25) zero   equ ($ - Palette) mod 4
(26) b1     equ bytesPerLine
```

При трансляции (23) выясняется, что имя `Colors` уже записано в таблицу имен (см. табл. 3.4). В `a86` это допустимо, если дубль имеет такие же атрибуты. Вычислим выражение в (23): $(013e - 0136) / 4 = 2$. Конфликта нет: значение и тип остались прежними — такими же, как у первого экземпляра `Colors`. В (24–25) — опять повторное определение, которое используется для проверки делимости размера палитры на 4. Проверим: $(013e - 0136) \bmod 4 = 0$. Действительно, у имени `zero` по-прежнему тип `abs` и нулевое значение (табл. 3.6).

Таблица 3.6. Фрагмент таблицы имен после трансляции (23–26)

Имя	Значение	Тип	Размер в байтах	\$
zero	0	abs	0	013e
b1	8	abs	0	013e

Завершающий фрагмент содержит данные раstra и повторное определение `byteCount` для проверки истинного размера этих данных.


```
(27) Pixels      db    iHeight dup (11000000xb, (b1 - 2) dup 0, bit 0)
(28) byteCount equ    $ - Pixels
```

При трансляции (27) в таблицу имен попадает новое имя `Pixels`. Как выясняется при считывании ключевого слова `db` тип этого имени — `byte`, а значение равно величине счетчика адресов в начале трансляции (27). Счетчик адресов при трансляции (27) увеличивается на $32 \times 8 = 256 = 0100$ и становится равным $013e + 0100 = 023e$ (табл. 3.7). Определение (28) не конфликтует с (8): имя `byteCount` имеет все то же значение `0100` — это предполагаемый и истинный размер данных раstra.

Таблица 3.7. Завершение таблицы имен

Имя	Значение	Тип	Размер в байтах	\$
Pixels	013e	byte	128	023e

Как следует из рассмотренного примера, на первом просмотре достаточно знать количество байтов, занимаемых машинным представлением инструкций и данных. Конкретные значения данных пока несущественны, главное — знать, *сколько* их. Из этого следует, что счетчик повторов в конструкции `dup` должен быть известен уже при первом просмотре.

По умолчанию `a86` создает выходной `com`-файл, содержащий только те коды, кото-рые были заданы в исходном тексте. И никакой служебной информации. Выходной файл имеет размер на `0100` меньше финального значения счетчика адресов. В рас-смотренном примере выходной файл генерируется в размере $318 = 013e$ байт.

Используя рассмотренный пример в качестве образца, выполните следующие зада-ния. (Соответствующие фрагменты на языке ассемблера собраны в файле `quiz2.txt`.)

1. Постройте таблицу имен:

```
bob      dd    5 dup end - bob, ?
top      dw    ($ - bob) dup (end - top)
what     equ   offset top - bob
let      db    what dup ?
that     equ   bob + 1
zz       db
```

2. Постройте таблицу имен:

```
szl      equ   (bit 2) * 7
neck     db    szl dup (1, 4 dup ?), ?
          dw    szl dup (1, 4 dup ?), ?
          dd    szl dup (1, 4 dup ?), ?
total    equ   end - szl
bottle   equ   neck + 8
```

3. Постройте таблицу имен:

```
alfa     dw    1
          db    13 dup (offset beta - end)
```

```

beta:
cold    dw    (1k + 1 - offset alfa) dup ?
gold    equ    cold - 2
zz      equ    end - alfa

```

4. Постройте таблицу имен:

```

        db    'Wow!'
        dw    end

dumb:
pump:
bump    equ    $
        dd    11 dup 11

lamp    dw    ($ - dumb) * 2 dup ?
stamp   equ    lamp - 1
zz      equ    $ - end

```

5. Укажите ошибку, возникающую на первом просмотре:

```

start:
msg      db    'Funny ha-ha...'
        db    $ - msg dup ?
        dw    1, 12, xx - start, (xx - start) dup ?

xx:

```

6. Укажите ошибку, возникающую на первом просмотре:

```

        dw    (offset sally - silly) dup ?

silly:
        dd    12 dup offset end

sally    dw    (offset sally - silly) * 2 dup ?

```

7. Укажите ошибку:

```

rob      db    (gamma - rob) * 2
        dw    rob, mob, 4 dup gamma

mob      equ    $ - rob

```

Проверьте решения с помощью а86, lst-файлов и отладчика d86.

Повторное определение имен

Повторное определение имени является ошибкой в том случае, если изменяются атрибуты имени — его числовое значение или тип (листинг 3.15). Это правило справедливо только для а86. В других ассемблерах (и в языках высокого уровня) повторное определение имен недопустимо в принципе.

Повторное определение имени `alfa` в (6) не противоречит первому определению в (1) — значение 12 и тип `abs` остаются прежними. Также атрибуты не меняются и при повторном определении `gamma` в (3), т. к. значение счетчика адресов в (2) и (3) одинаково, а тип по-прежнему `abs`. Аналогично второе определение `beta` в (5) не вносит ничего нового ни в значение, ни в тип имени. Напротив, третье определение `beta` в (7) конфликтует с предшествующими, и здесь а86 выдаст ошибку.

Листинг 3.15. Повторное определение имен (еха13.8)

```
alfa    equ    12                ; (1)
gamma:  ; (2)
gamma:  ; (3)
beta    db                ; (4)
beta    db    1, 10            ; (5)
alfa    equ    49 shr 2        ; (6)
beta:   ; (7)
```

Локальные имена

В а86 имеется особая категория имен — локальные, или переопределяемые. К ним относятся все имена, начинающиеся с буквы, за которой следует одна или более цифр, — так просто. Эти имена допускают многократные переопределения с изменением значения и/или типа (листинг 3.16).

Листинг 3.16. Локальные, или переопределяемые имена (еха14.8)

```
11    db    1                ; (1)
      dw    offset 11        ; (2)
      dw    offset >11      ; (3)
12    equ    1                ; (4)
11    dw    12               ; (5)
```

Имя 11 переопределено в (5) с изменением значения и типа. Символ > в строке (3) предписывает транслятору использовать *последующее* определение имени, а нам — искать это имя *ниже* в тексте.

После >11 имя 11 из таблицы имен удаляется. И пока оно не определено заново в (5), допустимы ссылки только вперед, как в (3). Листинг 3.17 иллюстрирует недопустимую ссылку на локальное имя.

Листинг 3.17. Недопустимая ссылка на локальное имя (еха15.8)

```
11    db    1                ; (1)
      dw    offset 11        ; (2)
      dw    offset >11      ; (3)
      dw    offset 11        ; (4) ?!
11    dw    12               ; (5)
```

В этом примере после >11 в (3) ссылка на 11 в (4) считается ошибкой. И вправду непонятно, какое из определений 11 имеется в виду в (4). Если нижнее, то нужно было поставить >11 в (4), а если верхнее, тут уж ничем не поможешь, придется вводить еще одно локальное имя — например, 12.

Предопределенные имена

Как во всех языках программирования, в языке ассемблера имеются встроенные, или предопределенные имена (также их называют ключевыми). Определение этих имен в программе недопустимо, а86 в таких ситуациях выдает ошибку "Misplaced Built-In Symbol" (листинг 3.18).

Листинг 3.18. Попытка переопределить встроенные имена (exa16.8)

```
ax      equ 12
mov     db 10, 20
b       dw 5
w       dw ?
d       dw ?
byte    db ?
st      db "Hello, world"
```

Все определения в листинге 3.18 ошибочны:

- ❑ `ax` — имя, обозначающее один из регистров процессора i80x86;
- ❑ `mov` — обозначение мнемоники инструкции;
- ❑ `b, w, d` — принятые в а86 сокращения ключевых слов `byte`, `word`, `dword`;
- ❑ `st` — обозначение стека сопроцессора i80x87.

Имя *end*

Во многих ассемблерах директива `end` обозначает конец исходного текста и является обязательной. Но в а86 имя `end` определяет сам транслятор в конце первого просмотра, так что значение `end` — это адрес первого байта за пределами программы.

С этим именем необходима осторожность: не следует его *определять*, тем более в начале какого-либо оператора. Если поставить `end` перед директивой `equ`, `db/dw/dd` или задать его в качестве метки, то а86 пропустит (проигнорирует) все, что задано в этом операторе, что может привести к искажению результата трансляции (листинг 3.19)¹⁷.

Листинг 3.19. Попытка определить имя *end* (exa17.8)

```
area    db 2 dup (1, 2)          ; (1)
end    db                ; ?!    ; (2)
sz      equ end - area          ; (3)
        db ?                  ; (4)
;end:   ;                    ; (5)
```

В этом примере транслятор прекратит обработку (2) при считывании `end`. Это имя он "мысленно" ставит в конце исходного текста (5). В результате константа `sz` —

уже не число байтов, заданных оператором (1), как предполагалось, а расстояние от начала (1) до конца программы. Из 1st-файла видно, что `sz` равно пяти, а не четырем.

Листинг 3.20 иллюстрирует наиболее распространенную грубую ошибку, связанную с именем `end`.

Листинг 3.20. Попытка перейти на метку `end` (exa18.8)

```

        jmp      end                ; (1)
msg      db      'In the middle of nowhere...' ; (2)
end:     int      020               ; (3)
;end:    ; (4)
```

Инструкция `int 020` даже не будет записана в `com`-файл. Мы хотели перейти к ней, а выскочили за пределы кода, определенного в программе¹⁸, — на неявно заданную метку `end`.

Найдите ошибки в следующих фрагментах (они собраны в файле `quiz3.txt`).

1.


```

11      dd      5 dup end - 11, ?
12      dw      ($ - 11) dup (end - >11)
what    equ     offset >11 - 12
this    equ     11 + 3
11      db
```
2.


```

start:
msg      db      'Ha-ha...'
          db      this - msg dup ?
          dw      1, 12, zz - start, (zz - start) dup ?

zz:
```
3.


```

          dw      (offset lamp - dumb) dup ?

dumb:

          dd      12 dup offset end

lamp     dw      (offset lamp - dumb) * 2 dup ?
```
4.


```

alfa     db      (gamma - alfa) * 2
          dw      alfa, beta, 4 dup gamma

gamma    equ     $ - alfa
```
5.


```

start    dd      1
zz       equ     this
msg1     db      1k dup (1, 7 dup 'A', ?)
          db      $ - msg2 dup ?
          dw      zz - start dup ?

msg2     db      1k dup (1, 7 dup 'A', ?)
```

Примечания

1. Тому, кто использует файловый менеджер VC, достаточно нажать клавишу <Enter> на имени `ex1.8`.
2. Для тех, кто не пользуется файловым менеджером, в коллекции **old-dos.ru** есть программы для просмотра данных в hex-формате, например `fed` (FileEdit v1.53).
3. Автор ассемблера объясняет это тем, что `a86` — 16-разрядный транслятор. Но ведь подобные трансляторы с языка C прекрасно справляются с "длинными" константами вроде `0x10000L`.
4. Таблица кодов ASCII — https://en.wikipedia.org/wiki/ASCII#Character_set.
5. Это не опечатка — посмотрите на строку "sym:" в файле `vc.ext`. Нажатие <Enter> на файле с расширением `sym` приводит к вызову отладчика `d86` с одноименным `com`-файлом.
6. На ассемблере мы ставим команду ветвления `jo` (после операций над знаковыми данными) или `jc` (для беззнаковых), и переполнение будет обнаружено. В языке релейно-контактных схем [6] (LD, или Ladder Diagram) проверка переполнения встроена в блоки арифметических операций, и оно приведет к разрыву цепи. А вот в языках высокого уровня эта проверка если и предусмотрена, то обычно за счет настроек компилятора, а не средствами языка. Разве что в Ada [13] значения данных типа "диапазон" контролируются всегда — как при трансляции, так и при выполнении.
7. Если первая цифра `0`, то суффикс `d` воспринимается как шестнадцатеричная цифра! Если же в начале `0`, а в конце `b`, то число воспринимается как двоичное (если в промежутке ничего кроме `0` и `1`) или как шестнадцатеричное (если в промежутке есть цифры `2–9`, `a–f`), причем финальное `b` считается шестнадцатеричной цифрой. Столь сложные правила введены для совместимости `a86` с `asm86` (и производными от него `masm/tasm`), где двоичное число задается суффиксом `b`.
8. Десятичное отображение беззнаковых байтов и слов задается форматами `n` и `d` соответственно. Отображение знаковых значений `d86` не поддерживает. (Форматы отображения данных в `d86` приведены в *приложении 2*.)
9. Например, `010k` для `a86` — это не `16k`, а именно `10k`. Это можно счесть ошибкой, но безвредной — вряд ли кому-то в этом контексте придет в голову употреблять недесятичные числа.
10. Этот вариант придет в голову только человеку, совсем не знакомому с программированием. Интересно, как поведет себя в этой ситуации транслятор с языка высокого уровня?
11. Понятие *типа* мы рассмотрим чуть позже.
12. Последовательности литер обычно встречаются в директиве `db`, но *пару* литер, заключенную в одинарные кавычки, можно задать и в `dw`, только их порядок должен быть обратным.

13. Типы появились в первом ассемблере для i8086 (asm86), а до этого было принято указывать размерность операции в мнемонике команды. Например, для мини-ЭВМ PDP-11 — `inc r0` и `incb r0`.
14. Название `extrn` неточное. Имена, перечисленные в `extrn`, не обязательно внешние (т. е. определены в других модулях), они просто *где-то* определены — возможно, даже в этом тексте, только ниже.
15. Загрузчик запрашивает у системы блок памяти 64 Кбайт + 256 байт. Эти 256 байтов в начале блока зарезервированы под системную информацию (в частности, со смещения 080 загрузчик помещает копию параметров командной строки при вызове программы). Загрузчик копирует `com`-файл в блок памяти, начиная со смещения 256, и передает управление по этому смещению.
16. В Windows 10 после вызова **Calculator** нужно выполнить команду **View/Programmer/Hex**.
17. Это плата за попытку a86 быть на "99% совместимым" с ассемблерами, производными от asm86: `masm`, `tasm` и `wasm`. Когда-то имя `end` обозначало конец исходного текста на перфоленте.
18. Эту программу следует проверять в отладчике по шагам, т. к. она может, пройдя по командам `por` (пустая операция) до максимального 16-битового адреса, перейти по кольцу на адрес 0, где специально для таких случаев закодирован выход в DOS.

ГЛАВА 4

```
DB6_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA i c 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:

lea si,b_1
```

Способы адресации

Холлоханы никак не нарекли шхуну и называли ее просто "она", а иногда — "эта сука".
(Фарли Моуэт. Шхуна, которая не хотела плавать)

В этой главе рассмотрены способы адресации операндов машинных инструкций применительно к микропроцессорам¹ семейства i80x86/64.

Данные процессора

Кроме данных в памяти, определяемых директивами db/dw/dd, программа на ассемблере имеет доступ к данным самого процессора.

Программно доступные данные процессора в дальнейшем называются регистрами. Из них чаще всего используются восемь 16-разрядных регистров общего назначения, которые в программе обозначаются как ax, bx, cx, dx, sp, bp, si, di.

Обозначения в табл. 4.1 отражают специальное назначение регистров в некоторых инструкциях. Например, регистр cx часто используется как счетчик, регистр ax — как источник входных данных и для хранения результатов команды².

Таблица 4.1. Обозначения регистров процессора i8086

Обозначение 16-разрядного регистра	Обозначение старшего байта	Обозначение младшего байта	Расшифровка обозначения
ax	ah	al	Accumulator
bx	bh	bl	Base
cx	ch	cl	Counter
dx	dh	dl	—
sp	—	—	Stack Pointer
bp	—	—	Base Pointer
si	—	—	Source Index
di	—	—	Destination Index

Байты регистров `ax`, `bx`, `cx`, `dx` доступны по отдельности. На ассемблере они обозначаются `ah`, `al` (старший и младший байты регистра `ax`), `bh`, `bl` (старший и младший байты `bx`), `ch`, `cl` (старший и младший байты `cx`) и `dh`, `dl` (старший и младший байты `dx`). Примеры иллюстрирует листинг 4.1.

Листинг 4.1. Доступ к составляющим 16-битового регистра (`exa1.8`)

```
mov    dx, 0           ; (1)
inc     dh              ; (2)
mov     al, dh          ; (3)
```

Выполните эту программу в отладчике по шагам (нажатием клавиши <F1>). Команда (1) записывает в `dx` ноль, затем (2) инкрементирует старший байт `dx`. В результате в `dx` сформировано число 0100 (256). Команда (3) копирует содержимое `dh` в `al`, так что `al` теперь содержит единицу. Полностью содержимое `ax` не известно, поскольку значение `ah` неопределенное — в `ah` мы ничего не записывали.

Пройдя по шагам следующие последовательности команд³, выясните, какие значения примут указанные регистры. (Перечисленные далее фрагменты собраны в файле `quiz1.txt`.)

1. `si`, `di`

```
mov     ah, bit 0
mov     dh, 0
mov     cl, ah
mov     ch, dh
mov     si, cx
mov     di, ax
```

2. `ax`

```
mov     ax, 0ff by (bit 1 + bit 4)
inc     ah
```

3. `ax`

```
mov     ax, 01ff
inc     ax
inc     al
```

4. `ax`

```
mov     ax, 01ff
inc     al
inc     ax
not     ax
```

5. `dx`

```
mov     ah, -2
mov     dh, -1
mov     dl, ah
inc     dh
```

6. ch

```
mov    cx, (bit 1) by -1
inc    cl
inc    cx
inc    ch
```

Обозначения операндов машинных команд

Операнды — это данные, которые обрабатываются машинной инструкцией. Например, команда `add` суммирует два операнда, а результат помещает в память на место первого операнда.

В обозначении машинной инструкции на ассемблере операнды следуют за мнемоникой операции. Если операндов два, то результат помещается на место первого операнда. В команде с двумя операндами их размерность (байт, слово или двойное слово) должна быть одинаковой⁴. Примеры иллюстрирует листинг 4.2.

Листинг 4.2. Примеры задания операндов (exa2.8)

```
nop                ; (1)
inc    ax          ; (2)
mov    ax, 10      ; (3)
add    al, bl      ; (4)
shl    ax, 1       ; (5)
shl    ax, cl      ; (6)
```

Комментарии:

- ❑ Команда (1) — без операндов, а по смыслу это пустая операция (реализована как обмен содержимого регистра `ax` с содержимым того же регистра `ax`).
- ❑ Команда (2) — инкремент, операнд у нее один (в данном случае содержимое регистра `ax`). Операнд считывается из регистра `ax`, а результат операции (увеличение на единицу) записывается в регистр `ax`.
- ❑ Команда (3) — копирование, считывает второй операнд (число 10) и записывает прочитанное в первый операнд — в регистр `ax`.
- ❑ Команда (4) — сложение, считывает операнды из регистров `al` и `bl`, а результат помещает на место первого операнда — в регистр `al`.
- ❑ Команда (5) считывает первый операнд, сдвигает его значение влево на число разрядов, заданных вторым операндом, записывает результат на место первого операнда.
- ❑ Команда сдвига (6) — исключение из правила: число сдвигов задано *байтовым* регистром `cl`, а сдвигаемая величина — *слово* (содержимое регистра `ax`).

Способы адресации операндов

Адресация операндов — это задание их местонахождения (где находится число для операции и куда поступает результат). В ассемблерах для i80x86 местоположение операндов обозначается только в том случае, если оно не фиксировано.

```
mul    cx        ; cx * ax -> dx, ax
add    ax, dx    ; ax + dx -> ax
```

Команда `mul` всегда считывает один сомножитель из регистра `ax`, а результат неизменно помещает в пару регистров `dx` и `ax`. Поэтому в инструкции `mul` указывается местонахождение только второго сомножителя. Напротив, команда сложения `add` может работать с любыми регистрами, поэтому операнды указываются явно.

Регистровая и непосредственная адресация

Операнд машинной команды находится или в регистре процессора, или непосредственно в инструкции, или в памяти данных.

Первый вариант адресации обозначается названием регистра, адресация называется *регистровой* (листинг 4.3).

Листинг 4.3. Примеры регистровой адресации (hex3.8)

```
mul    ax
inc    si
mov    dh, cl
```

Второй вариант: операнд задан в самой инструкции, адресация называется *непосредственной* (листинг 4.4). Непосредственное значение находится в конце машинного кода команды (а где-то в начале кода есть признак, что второй операнд непосредственный).

Листинг 4.4. Примеры непосредственной адресации (hex4.8)

```
add    al, 012        ; (1)
add    ax, 03456       ; (2)
add    ax, 1           ; (3)
shl    cx, 4           ; (4)
inc     7               ; (5) ?!
add     3, ax           ; (6) ?!
```

Выполните трансляцию. На (5–6) следуют сообщения об ошибках, поскольку непосредственный операнд — это константа, модифицировать ее нельзя. Для команд в формате⁵ `inc imm` и `add imm, r/m` даже нет машинного кода, в Intel они полностью исключены⁶.

Закомментируйте строки (5–6), повторите трансляцию с ключом `+L` и посмотрите в `lst`-файле, где находятся коды непосредственных операндов. Обратите внимание, число 1 в (3) кодируется двумя байтами⁷.

Адресация данных в памяти

Третий вариант адресации — операнд находится в памяти данных по некоторому адресу. В зависимости от способа задания адреса различают *прямую* и *косвенную* адресацию.

Прямая адресация

При прямой адресации значение адреса задано в конце кода команды (в коде команды также указан признак прямой адресации). При выполнении команды организуется доступ к памяти по адресу, значение которого предварительно считывается процессором из машинной инструкции. На языке ассемблера прямая адресация обозначается именем *переменной* или числом (константой типа *abs*) в квадратных скобках. В последнем случае иногда приходится уточнять размерность операнда (листинг 4.5).

Листинг 4.5. Примеры прямой адресации (exa5.8)

```
extrn  b_1:byte, w_1:word, c_1:abs      ; (1)

not     b_1                               ; (2) byte
not     w_1                               ; (3) word
not     [082]                             ; (4) ?!
not     byte ptr [082]                    ; (5) byte
not     byte [082]                        ; (6) byte
not     b [082]                           ; (7) byte
not     w [c_1]                           ; (8) word
mov     word ptr [082], al                 ; (9) ?!
mov     [082], al                         ; (10) byte
mov     [082], ax                         ; (11) word
mov     [082], 1                          ; (12) ?!
mov     [082], 0100                       ; (13) word
int     020                              ; (14)

c_1     equ     080                        ; (15)
b_1     db      0f                         ; (16)
w_1     dw      05555                      ; (17)
```

Оператор (1) говорит о том, что имена *b_1*, *w_1* и *c_1* где-то определены, и задает их тип. Действительно, эти имена определены в конце программы⁸, а их типы в (15–17) и (1) совпадают.

Выполните трансляцию. В строках (2–3) размерность операции (инкремент байта или инкремент слова) определяется по *типу* имени. В строке (4) не хватает информации о размерности, и *а86* выдаст сообщение "Is it Byte or Word?" В строках (5–9) эта информация задана операторами преобразования типа *byte ptr* и *word ptr* (или короче: *byte* и *word*; или еще короче: *b* и *w*.).

В строке (9) — разные размерности операндов (первый — word, второй — byte), что недопустимо в большинстве инструкций с двумя операндами. В таких случаях выдается сообщение "Byte/Word Combination Not Allowed". Исправьте (9), заменив word на byte или al на ax. В строках (10–11) размерность данных по адресу [082] приравнивается к размерности второго операнда, как того требует команда mov. В строке (12) вновь не хватает информации о размерности, поскольку непосредственное значение 1 может быть представлено как байтом, так и словом. Напротив, в (13) значение 0100 не может быть представлено байтом, отсюда вывод о размерности операции — слово⁹.

Исправив (4), (9) и (12), повторите трансляцию. При вызове отладчика задайте параметры командой строки отлаживаемой программы, например: d86 exa5.com 123 abc. Загрузчик скопирует строку "123 abc" по смещению 082. По смещению 080 он запишет длину этой строки, включая пробел в начале¹⁰. После входа в d86 задайте отображение переменных b_1, w_1 (форматы отображения — b и w), а также байтов со смещения 080 (на одной строке — формате r, а на другой — b) и сохраните нажатия клавиш: k s alta <Enter> — они еще пригодятся. Выполните программу по шагам, наблюдая за изменениями переменных.

Ответьте на следующие вопросы. (Операторы ассемблера собраны в файле quiz2.txt.)

1. Назовите способы адресации, которые использованы в следующих командах:

```
mov    ah, bh
inc    w [0]
add    byte [0fffe], type (w [0])
```

2. Укажите ошибки и объясните их причину:

```
mov    1 shl 3, ax
add    bx
mul    ax, cx
```

3. Укажите ошибки и объясните их причину:

```
add    [-2], 256
add    [10], 255
mov     [3], ax
inc     [0]
```

4. Какие способы адресации использует *машинная* команда, которая на ассемблере записывается таким образом:

```
mul    w [0]
```

5. Какова размерность следующих операций:

```
add    ax, 1
inc    byte ptr [0]
mov    ax, [0]
```

6. Какие способы адресации заданы в следующих командах и какие команды неправильные?

```
add    5, 9
add    ax, 1
mov    al, cx
```

Косвенная адресация

Адрес данных в памяти может быть задан косвенно. Это означает, что в команде указывается местоположение элемента памяти, где хранится *адрес* операнда (листинг 4.6). В i8086/286 таким элементом может быть только 16-разрядный регистр¹¹ (за исключением инструкций передачи управления).

Листинг 4.6. Пример косвенной адресации (exa6.8)

```
mov    si, 082    ; (1)
inc    b[si]      ; (2)
inc    si         ; (3)
```

Команда (1) записывает в регистр `si` начальный адрес параметров командной строки. В команде (2) адрес байта в памяти задан содержимым регистра `si`. В (3) `si` увеличивается на единицу и содержит 083 — адрес следующей литеры. Повторяя (2–3), мы двигаемся по памяти в порядке возрастания адресов.

Выполните трансляцию `exa6.8` и вызовите `d86` как в предыдущем примере: `d86 exa6.com 1234`. Нажмите <Alt>+<A>, чтобы отработать нажатия клавиш, сохраненные в файле `alta.d8k`. Должно появиться отображение данных с адреса 082. Задайте еще одну строку отображения данных, нажав, например, клавишу <6> и набрав `r,si` <Enter>, — это вывод от адреса, хранящегося в регистре `si`. Изначально `si` содержит случайное значение, и в окне 6 отображаются незнакомые нам данные. Но после выполнения (1) в нем видны уже параметры командной строки. Клавишей <F1> пройдите по шагам (2–3), оцените изменения данных и сдвиг отображения в окне 6 в результате возрастания величины в регистре `si`. Клавишей-стрелкой вернитесь на (2) и снова пройдите (2–3) по шагам¹².

Можно добавить в конце команду перехода, как в листинге 4.7. Тогда не придется вручную возвращать курсор на команду (2).

Листинг 4.7. Цикл с косвенной адресацией (exa7.8)

```
mov    si, 082    ; (1)
l1:    inc    b[si] ; (2)
        inc    si   ; (3)
        jmp   l1    ; (4)
```

В листинге 4.8 вместо командной строки используется массив байтов, заданный в программе.

Листинг 4.8. Обработка массива байтов, заданного в программе (exa8.8)

```

        jmp      start                                ; (1)

area    db      1, 3, 9, 56, -2, 8, 9 dup -4, 1k dup -1 ; (2)

start:  mov     si, offset area                        ; (3)
l1:     inc     b[si]                                  ; (4)
        inc     si                                    ; (5)
        jmp     l1                                    ; (6)

```

Оператор `offset` в (3) задает преобразование типа `area` из `byte` в `abs`, в результате чего в `si` запишется именно адрес, а не значение, хранящееся по этому адресу.

Процессор i8086/286 может использовать для косвенной адресации только четыре 16-разрядных регистра: `si`, `di`, `bx`, `bp`. Следовательно, команда `inc b[bx]` возможна, а `inc b[ax]` — нет. Начиная с i80386, для косвенной адресации годится любой из 32-разрядных регистров (в дополнение к перечисленным 16-битовым), например `eax`.

При косвенной адресации можно задать константу *смещения*, которая будет прибавляться к адресу операнда. Такая адресация называется *косвенной со смещением*. Код в листинге 4.9 выполняет ту же задачу, что код в листинге 4.8, но используя другую адресацию элементов массива.

Листинг 4.9. Косвенная адресация со смещением (exa9.8)

```

...
(3) start:  mov     si, 0
(4) l1:     inc     [si+area]
(5)         inc     si
(6)         jmp     l1

```

Теперь регистр `si` не указатель, а индекс в массиве. (Точнее было бы сказать "смещение", но пока массив байтовый, смещение равно индексу.) Обратите внимание, что в (4) не требуется уточнять тип операнда ('b' перед квадратными скобками убрano), поскольку в нем упомянуто имя `area`.

Напрашивается аналогия из области С-подобных языков: там тоже можно двигаться по массиву, используя либо указатель, либо индекс, как показано в листингах 4.10–4.11.

Листинг 4.10. Работа с массивом в языке С с использованием указателя (exa10.c)

```

...
char c, *p = area;

for (;;) {
    c = *p;

```



```

    if (c == 0) break;
    *p++ = ++c;
}
...

```

Листинг 4.11. Работа с массивом в языке C с использованием индекса (exa11.c)

```

...
int c, i = 0;

for (;;) {
    c = area[i];
    if (c == 0) break;
    area[i++] = ++c;
}
...

```

В ассемблере возможны разные сочетания обозначений смещения и регистра, но в любом случае обозначение *регистра* должно быть внутри квадратных скобок (листинг 4.12).

Листинг 4.12. Разнообразие обозначений косвенной адресации со смещением (exa12.8)

```

inc     area[si]
inc     [si]area
inc     [area+si]
inc     b -4[si] (2 * bit 1)

```

Попробуйте транслировать exa12.8, задав в начале программы массив `area`. Обратите внимание, что в `lst`-файле первые три команды занимают по 4 байта и выглядят одинаково, зато последняя занимает лишь пару байтов. Это потому, что смещение, заданное в ней, равно нулю, и `as86` применил косвенную адресацию без смещения¹³.

В рассмотренных примерах мы увеличиваем `si` на единицу, подготавливая доступ к следующему байту. Если требуется обрабатывать массив по словам, то значение регистра `si` в каждом цикле должно увеличиваться на два.

Скорректируйте программы `exa8.8` и `exa9.8` так, чтобы они обрабатывали массив 16-битовых данных:

- ☐ исправьте директиву определения данных — `dw` вместо `db`;
- ☐ измените размерность операций — `w[si]` вместо `b[si]`;
- ☐ увеличивайте `si` на 2 — `add si, 2` вместо `inc si`.

В рассмотренных вариантах косвенной адресации используется один регистр из набора: `si`, `di`, `bx`, `bp`. Наиболее развитый вариант косвенной адресации в `i8086/286` — по сумме значений двух регистров (постоянное смещение здесь также допустимо).

Такой способ адресации ограничен четырьмя сочетаниями регистров из множества `si`, `di`, `bp`, `bx`. Эти регистры образуют две группы: базовые (`bx/bp`) и индексные (`si/di`). Допускается сочетание регистров из разных групп, отсюда и название адресации — *базово-индексная* (листинг 4.13).

Листинг 4.13. Пример базово-индексной адресации (exa13.8)

```

mov     si, 1           ; (1)
mov     di, 2           ; (2)
watch:  mov     bx, bp, 080 ; (3)

inc     b[si+bx]        ; (4)
inc     w[di][bx]       ; (5)
inc     w[bx+si]        ; (6)
inc     b[bp+di+2]      ; (7)
inc     b[si+di]        ; (8) ?!
```

В строках (1–3) идет настройка базовых и индексных регистров: в качестве базы пишем (3) начальный адрес копии параметров командной строки `080`, а индексы задаем (1–2) небольшими, чтобы строку параметров при вызове `d86` не пришлось писать слишком длинной. В (3) указана встроенная макрокоманда `mov`; во что она преобразуется, вы увидите в отладчике по метке `watch`.

Базово-индексная адресация обычно используется для доступа к элементам двумерных массивов или массивов структур. Заметим, что, начиная с `i80386`, косвенная адресация возможна по сумме двух произвольных 32-разрядных регистров.

Задания:

1. Напишите команды, составляющие тело цикла для инкремента *слов*, начиная с адреса `area`, с использованием косвенной адресации со смещением.
2. Напишите команды, составляющие тело цикла для инкремента массива *слов*, применив косвенную адресацию без смещения.
3. Напишите команды, составляющие тело цикла для увеличения на 20 каждого *второго слова* последовательности данных с адреса `area`, с использованием косвенной адресации со смещением.

4. Данные определены следующим образом:

```
buf  dw  1k dup (1, 3, -1), -3
```

Напишите команды, образующие тело цикла, для инкремента *слов* массива `buf`, задав косвенную адресацию со смещением.

5. Данные определены следующим образом:

```
buf    dw    1k dup (1, 3, -1), -3
```

Напишите команды, образующие тело цикла для инкремента *байта*, начиная от адреса `buf`, применив косвенную адресацию со смещением.

6. Напишите команды, образующие тело цикла для инкремента каждого *четвертого байта* последовательности, с использованием косвенной адресации со смещением и без смещения.
7. Напишите команды, составляющие тело цикла для инкремента байтов, но в обратном порядке — от последнего байта массива к его началу. Уменьшение `si` реализуйте с помощью команды декремента `dec`.

Ограничение на адресацию операндов в памяти

В тех командах, где необходимо *явно* обозначить два операнда, в памяти данных может находиться только один из них (листинг 4.14).

Листинг 4.14. Команды с двумя операндами, один из них в памяти (exa14.8)

```
mov     w [30], 200
add     bx, w [40]
add     b[0103], [si]           ; ?!
mov     w[si], [di]             ; ?!
```

Это ограничение связано с возможностями машинного представления команд в i80x86: в *коде* команды нет места для задания второго операнда в памяти. Это не означает, что машинным командам вообще запрещено адресовать больше одного операнда в памяти. Например, команда `movsw` считывает слово по адресу `[si]` и записывает результат по адресу `[di]`. Однако в коде этой команды нет сведений об адресах операндов, т. к. источник *всегда* адресуется регистром `si`, а приемник — регистром `di`.

Объясните, почему все команды в листинге 4.15 неправильные.

Листинг 4.15. Команды с ошибками (exa15.8)

```
add     [si+bx], w[0]
inc     [si+bx]
inc     w [bl]
inc     w[di+ax]
add     1, b[si+di]
mov     al, w [0]
mov     ax
add     [si], b[di]
inc     b [bh]
```

Примечания

1. Методы адресации операндов в i80x86 далеко не столь разнообразны и сложны, как, например, в процессорах Digital Equipment Corporation (DEC), появившихся гораздо раньше. Но их оказалось достаточно.
2. Например, один из сомножителей команды `mul` всегда находится в `ax`, и туда же записывается младшая часть результата.
3. Команда `inc` прибавляет единицу, команда `not` инвертирует биты.
4. Это простое правило справедливо только для i8086/286. В i80386 появились команды `movzx` и `movsx` для копирования байта в слово или слова в двойное слово, но о них чуть позже. Но команды, впервые возникшие в 8086, по-прежнему требуют одинаковой размерности операндов.
5. В справочниках (например, в [3, 2.5.1]) `imm` (Immediate) означает непосредственный операнд, `r` — регистр, `m` — ячейку памяти, а `r/m` — либо регистр, либо ячейку памяти.
6. Напротив, в процессорах DEC допускались любые методы адресации для обоих операндов, в том числе непосредственная для операнда-приемника. Выполнение (5) привело бы к увеличению непосредственного операнда в конце команды, в результате чего (5) превратилась бы в `inc 8`.
7. Это, так сказать, 16-битовая единица. А в 64-битовой команде `add rax, 1` это уже 64-битовая единица!
8. Структура программы в *листинге 3.1* проще, т. к. не приходится дважды задавать тип имени (сначала в опережающем описании `extrn`, а потом еще в определении переменной или константы). Рекомендую переделать этот пример по образцу *листинга 3.1*, перенеся (15–17) на место (1) и добавив команду обхода данных.
9. Напомню, что мы сейчас обсуждаем 16-разрядный компилятор, который поддерживает операции только с байтами и 16-битовыми словами. Будь он 32-разрядным, оператор (12) вызвал бы ошибку, т. к. число 0100 может быть представлено как словом, так и двойным словом.
10. Напомню, что код `com`-программы начинается со смещения 0100, а до него идет PSP (Program Segment Prefix), зарезервированный загрузчиком, куда и копируются параметры командной строки.
11. Начиная с i80386, адрес может храниться также в 32-разрядном регистре.
12. Как ни удивительно, возможность свободно перемещаться по отлаживаемому коду и продолжать выполнение от текущего положения курсора — это уникальная особенность d86. Другие известные мне отладчики "намертво" привязаны к потоку выполнения, не позволяя ни обойти, ни повторить отработку фрагментов программы.
13. Если добавить в начале `exa12.8` одну строку `area db ?`, то в d86 мы увидим совсем не те команды. Дело в том, что, дизассемблируя `com`-файл, d86 "не знает", что в коде есть вкрапления данных. И сбивается из-за лишнего байта в начале. Чтобы увидеть команды из `exa12.8`, следует сдвинуть начальную точку дизассемблирования "вниз" на один байт — `<Ctrl>+<D>`.

ГЛАВА 5

```
d86_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100  2: =d1,ax,, 1962
CX 0000  CS 6309  3:
DX 6309  SS 6309  4:
SI 011F  DS 6309  5: r5,b_1,, AB ab
DI 0000  ES 6309  6: r5,si,, AB ab
BP 0000  SP FFFE  0:

lea si,b_1
```

Система команд i8086

Агабек выхватил из пояса тыквенный кувшинчик с волшебным составом.
— Лимчезу! Пуцугу! Замнихоз — грозно возопил он, брызгая из кувшин-
чика на стражников. — Каламай, дочимоза, чимоза, суф, кабахас!
— Держи его! Держи! Хватай! Вяжи! Тащи! Не пускай!..
<...> Их заклинания, как и следовало ожидать, оказались неизмеримо
могущественнее: через минуту Агабек был повергнут и связан по рукам
и ногам.
(Л. В. Соловьев. Повесть о Ходже Насреддине)

Тема главы — изучение способов адресации операндов и системы инструкций i8086/286 с помощью режима непосредственного выполнения в d86.

Отладчик различает вводимые вручную команды по первой *букве*: команды отлад-
чика (например, *k*, *q*) и команды ассемблера начинаются с *разных* букв¹. Если пер-
вая буква относится к ассемблеру, красный курсор уходит в левый нижний угол,
отображает там введенную букву и ожидает продолжения ввода до нажатия
<Enter>. Введенная строка передается на трансляцию a86, и если в ней нет ошибок,
то результирующий машинный код помещается в буфер трансляции и выполняется.
Выполнение ранее введенной команды ассемблера можно повторить нажатием
<F3> (RepeatCmd).

Способы адресации операндов

Воспользуемся данными, которые определены в программе exa1.8 (листинг 5.1).

Листинг 5.1. Данные для изучения способов адресации и системы команд (exa1.8)

```
(1)  d86_buffer:
(2)      db          20 dup 090          ; nop
(3)  b_1    db          'AB ab'
(4)  c_1    equ       $ - b_1
(5)  b_2    db          "12 ab"
(6)  b_3    db          c_1 dup '&'
(7)  w_1    dw          1, -1
```

В строке (3) заданы пять байтов по адресам `b_1`, `b_1+1`, ... `b_1+4`. В (7) определены слова с адресами `w_1`, `w_1+2`. Метка (1) явно задает положение буфера трансляции², а в строке (2) резервируется место для этого буфера: двадцать однобайтовых команд `nop` (пустая операция).

Скомпилируйте `exa1.8` с ключом `+L`; из `lst`-файла выпишите значения имен и запустите отладчик: `d86 exa1.com`. Нажатия клавиш, записанные в `exa1.d8k`, задают отображение значения регистра `al` в двоичном и десятичном форматах и отдельно вывод значения регистра `ax` в десятичном формате.

Регистровая, непосредственная и прямая адресация

Выполните команду:

```
mov ax, 7
```

Результат выполнения — число 7 в регистре `ax`. Местоположение результата (*приемника*) задано при помощи регистровой адресации, а входное значение (*источник*) — непосредственным операндом 7. Обратите внимание на результат трансляции по метке `d86_buffer`. Сколько байтов занимает только что выполненная машинная команда? Команда `int 3` в буфере трансляции — это точка останова, которую отладчик записывает перед тем, как передать управление по адресу `d86_buffer`.

Выполните команду `mov ah, c_1`. Поскольку имя `c_1` определено константой, значение `c_1` считается непосредственным.

Ассемблер `a86` позволяет задавать в команде `mov` более двух аргументов. Выполните:

```
mov al, dl, cl, 8
```

По результату трансляции объясните смысл этой встроенной *макрокоманды*.

Выполните команды:

```
mov bx, d86_buffer
mov cx, [d86_buffer]
mov dx, offset d86_buffer
```

Значение `d86_buffer` считается непосредственным, т. к. имя `d86_buffer` определено как метка. Напротив, `[d86_buffer]` обозначает операнд в памяти, адресация прямая. Выражение `offset name` имеет тип константы — `abs`, а его значение равно значению `name`. Какие значения записаны в регистры `bx`, `cx`, `dx` и почему?

Выполните команды:

```
mov si, w_1
mov di, [w_1]
mov bp, offset w_1
```

Какие значения копируются в регистры `si`, `di`, `bp` и почему?

Для записи в регистр значения *адреса* данных в памяти рекомендуется использовать команду `lea` (Load Effective Address). Выполните команду:

```
lea ax, w_1
```

Команда `lea` вычисляет адрес данных в памяти и записывает его в регистр, при этом обращения к данным не происходит. Какую адресацию задает обозначение `w_1`? Запишите в `bx` адрес `b_1` двумя способами: при помощи `mov` и при помощи `lea`.

Косвенная адресация

Процессор i80x86 поддерживает косвенную адресацию данных в памяти по значениям одного или двух регистров, в том числе с дополнительным смещением. Для косвенной адресации используются регистры `bx`, `si`, `di`, `bp`. Адрес образуется в общем случае суммой содержимого регистров и смещения.

Косвенная адресация по значению одного регистра

Сейчас в регистре `bx` записан адрес `b_1`. В свободном окне отображения памяти задайте `b5, bx` — вывод данных от адреса, записанного в регистр `bx`.

Выполните команды:

```
inc b[bx]
inc b[bx+1]
```

Первая команда увеличивает на единицу значение байта по адресу `[bx]`. Во второй команде введено смещение, равное единице. В результате значения 'А' и 'В' изменились на 'В' и 'С'. Выполните аналогичные операции над байтами со значениями 'а' и 'б'.

Выполните следующие две команды и объясните результат:

```
mov bx, 2
mov [bx+b_1], '+'
```

Используя регистр `di` и смещение, заданное именем `w_1`, увеличьте на единицу слово со значением `-1` из строки (7) листинга 5.1. Какое значение должно быть в регистре `di`?

Косвенная адресация по сумме значений двух регистров

При таком способе адресации адрес данных в памяти образуется суммой содержимого двух регистров и, возможно, дополнительного смещения. Допустимые сочетания регистров: `[bx+si]`, `[bx+di]`, `[bp+si]`, `[bp+di]`.

Запишите в `bx` число 2. Выполните следующие команды и объясните результат:

```
lea si, b_1
inc b[bx+si]
```

Обзор системы команд

В системе команд процессора i80x86 выделяются следующие основные группы:

- ☐ команды пересылки;
- ☐ арифметические команды;

- ☐ логические команды;
- ☐ команды сдвигов и вращений;
- ☐ команды передачи управления;
- ☐ строковые команды.

Команды пересылки

В группу команд пересылки, кроме рассмотренных команд `mov` и `lea`, входят:

- ☐ команда обмена `xchg`;
- ☐ команда чтения из таблицы (команда *перекодировки*) `xlat`;
- ☐ команда записи в стек `push`;
- ☐ команда чтения из стека `pop`.

Запишите в `ax` число `011ff` и выполните команды:

```
xchg al, ah
xchg ax, dx
```

Команда `xlat` считывает байт из таблицы, адрес которой задан в `bx`, по индексу, указанному в `al`. Результат записывается в `al`. Запишите в `bx` адрес `b_1`, в `al` — число 2, выполните `xlat` и объясните результат. С использованием `xlat` прочитайте значение третьего элемента таблицы `b_1`.

Команда `push` копирует заданный ей операнд размерностью *слово* в память по адресу `[sp-2]`, после чего уменьшает `sp` на два³. Наблюдая за нижним окном отображения памяти (справа от `sp`), выполните команды:

```
push 012
push -1
push dx
```

В нижнем окне видны три слова — от адреса, содержащегося в `sp`, до адреса, первоначально записанного в `sp`. Число перед двоеточием — это счетчик записей. Регистр `sp` содержит адрес последней записи.

Команда `pop` выполняет обратное действие: считывает слово из памяти по адресу `[sp]` и увеличивает `sp` на два. Выполните команды:

```
pop bx
pop cx
pop dx
```

Первая команда `pop` считывает данные, записанные последней командой `push`; последняя команда `pop` считывает данные, записанные первой командой `push`. То есть чтение данных происходит в очередности, обратной порядку записи, по принципу *стека*. Вершина стека — это данные по адресу `sp` (указатель стека).

Пользуясь стеком, обменяйте значения регистров `si` и `di`.

Арифметические команды

К арифметическим командам относятся:

- ☐ команда сложения `add`;
- ☐ команда вычитания `sub`;
- ☐ команда инкремента (увеличение на 1) `inc`;
- ☐ команда декремента (уменьшение на 1) `dec`;
- ☐ команда арифметической инверсии `neg`;
- ☐ команды умножения `mul/imul`;
- ☐ команды деления `div/ldiv`;
- ☐ команды расширения знака `cbw/cwd`.

Запишите в регистр `ax` число 7. Выполните команды:

```
sub  al, 6
neg  al
add  al, 3
inc  al
dec  al
```

Запишите в `bl` число 3 и выполните команду `mul bl`. Эта команда перемножает беззнаковые числа, заданные регистрами `bl` и `al`, результат помещается в `ax`. Команда `imul bl` выполняет аналогичные действия, рассматривая сомножители как знаковые.

Проверим разницу между `mul` и `imul`. Байт со значением `11111111xb` задает беззнаковое число 255 и знаковое число -1. Запишите это значение в `al` и `bl` и выполните `mul bl`. С теми же исходными значениями выполните команду `imul bl`.

Задайте в `bh` число 3, в `ax` — число 22. Выполните команду `div bh`. Эта команда делит беззнаковые числа, содержащиеся в `ax` (делимое) и `bh` (делитель). Целая часть частного помещается в `al`, остаток — в `ah`. Проверьте разницу между беззнаковым (`div`) и знаковым (`ldiv`) делением, задав делимое 22 и делитель -3. Какое беззнаковое число соответствует знаковому числу -3 в диапазоне байта? Какой знак у остатка — такой, как у делимого или как у делителя?

Если в команде умножения/деления операнд задан *словом*, то в операции участвует не только `ax`, но и `dx`. Запишите в `ax` и `bx` число 2. Выполните команду `mul bx`. Эта команда перемножает значения `bx` и `ax` и помещает результат в пару `dx, ax`: старшую часть результата — в `dx`, младшую — в `ax`.

Выполните команду `div bx`. Эта команда делит 32-разрядное число, заданное парой регистров `dx` (старшая часть) и `ax` (младшая часть), на значение регистра `bx`. Частное помещается в `ax`, а остаток — в `dx`.

Команда `cbw` (Convert Byte to Word) предназначена для копирования значения `al` в `ax` с сохранением знака: знаковый бит числа в регистре `al` распространяется на все биты регистра `ah`. Запишите в `al` число -2 и выполните команду `cbw`. Выполните

`neg ax` и вновь — команду `cbw`. Объясните результаты и выясните, как работает команда `cwd` (Convert Word to Doubleword).

Логические команды

Команда логического отрицания `not` инвертирует все разряды операнда. Выполните команду `not al`.

Команды `and`, `or`, `xor` реализуют поразрядные логические операции "и", "или", "исключающее или". Операция выполняется параллельно над всеми парами одноименных разрядов источника и приемника.

Команда `and` используется как *выключатель* — для сброса в ноль тех разрядов приемника, которые сброшены в источнике (значение в источнике команды `and` называется *маской*). Запишите в регистр `al` число `11011xb`. Выполните команду `and al, 1100xb`. Запишите в регистр `al` число `-1` и выполните команду `and al, not (bit 1)`.

Команда `or` применяется как *включатель* — для установки в единицу тех разрядов приемника, которые установлены в источнике. Выполните команду `or al, bit 0`.

Команда `xor` действует как *переключатель* — инвертирует те разряды приемника, которые установлены в источнике. Выполните несколько раз команду `xor al, bit 7` (для повтора используйте клавишу <F3>). Бит 7 меняет значение, а остальные — нет. Составьте и выполните команду для одновременного переключения разрядов 1, 3 и 4 регистра `al`. Составьте и выполните команду `xor`, которая инвертирует все разряды регистра `al`.

Если команде `xor` в качестве приемника и источника задан один и тот же регистр, то результат выполнения — ноль. Составьте и выполните команду `xor` для обнуления⁴ регистра `ax`.

Команды сдвигов и вращений

Запишите в `al` число `bit 0`. Выполните команду `shl al, 1` (логический сдвиг влево). Младший бит результата сбрасывается в ноль, а старший бит исходного значения теряется. Повторяйте эту команду до обнуления регистра `al`.

Задайте в `al` число `201`. Выполните восемь раз команду `shr al, 1` (логический сдвиг вправо). Чем определяется значение младшего бита результата? Что записывается в старший бит?

Арифметический сдвиг на один разряд вправо/влево аналогичен делению/умножению *знаковых* чисел на 2. Запишите в `al` отрицательное число `-40` и выполните команду `sar al, 1`. Чтобы выяснить абсолютное значение результата, воспользуйтесь командой `neg al`. Установите `al` равным `20` и еще раз выполните `sar al, 1`. Чем определяется значение старшего бита результата?

Команда арифметического сдвига влево `sal` не отличается от команды `shl` (разные обозначения одной и той же операции). Выполните команду `sal al, 1` и проверьте содержимое буфера трансляции.

Установите в `al` значение 080. Выполните по восемь раз команды циклических сдвигов `ror al, 1` и `rol al, 1`. Как действуют эти команды?

Число сдвигов может быть задано либо константой, либо значением регистра `cl`. Запишите в `cl` число 4 и выполните команды:

```
rol  al, cl
sar  al, 2
```

Первая команда меняет местами тетрады регистра `al`. Какое действие выполняет вторая команда?

Команды передачи управления

При выполнении программы регистр `ip` (Instruction Pointer) содержит адрес текущей машинной команды. При отработке текущей команды значение `ip` автоматически увеличивается на длину команды, и это новое значение становится адресом следующей команды. В результате команды обрабатываются по последовательно возрастающим адресам. Переход от одной последовательности команд к другой выполняется командами *передачи управления*, которые принудительно устанавливают значение `ip`.

Если выполнить такую команду в непосредственном режиме `d86`, управление будет передано в обход точки останова `int 3` в буфере трансляции, т. е. в никуда. Чтобы программа остановилась по адресу перехода, нужно по этому адресу поставить точку останова. Для исследования переходов воспользуемся программой `exa2.8`, в которой заранее запрограммированы семь точек останова по меткам `m1–m7` (листинг 5.2).

Листинг 5.2. Программа для исследования команд переходов (`exa2.8`)

```
_labels macro          ; (1)
#rx17                  ; (2)
m#nx:   int 3          ; (3)
#em                  ; (4)
        _labels      ; (5)

_w      dw      m4      ; (6)
```

`d86_buffer:`

В пределах этой программы выполнять команды переходов по адресам `m1–m7` безопасно. (Слово `_w` со значением адреса `m4` понадобится нам для изучения *косвенных переходов*.)

И пояснения для тех, кому интересны подробности⁵ листинга 5.2. Внутри макроопределения (1) задан цикл (2) по первым семи параметрам макровывоза (5). Но в вызове (5) параметры вообще не указаны, и переменная `x` в теле цикла (3) принимает значение пустой строки. Зато *номер* параметра `#nx` меняется от 1 до 7, в ре-

зультате чего генерируется семь меток от `m1` до `m7`. Обозначение конца цикла опущено, т. к. оно совпадает с концом (4) макроопределения.

Выполните трансляцию и вызовите отладчик: `d86 exa2.com`. Нажатия клавиш, сохраненные в файле `exa2.d8k`, задают отображение адреса `_w` и данных по этому адресу.

Адресация в командах передачи управления

В группу команд передачи управления входят:

- ☐ `jmp r16/m16/imm` — безусловный переход;
- ☐ `call r16/m16/imm` — вызов подпрограммы;
- ☐ `ret` — возврат из подпрограммы;
- ☐ `ret imm` — возврат из подпрограммы с очисткой стека;
- ☐ `j<x> imm` — переход по условию `x`;
- ☐ `loop<x> imm` — переход по условию `(--cx != 0) && (x)`.

Обозначения `r16`, `m16`, `imm` — это соответственно 16-разрядный регистр, слово в памяти и непосредственное значение (immediate). Заметим, что для адресации источника в командах `jmp` и `call` может быть использован *любой* из ранее рассмотренных методов адресации. Напротив, команды перехода по условию допускают только непосредственную адресацию.

Рассмотрим применение *регистровой* адресации в команде `jmp`. Запишите в `ax` значение `m3` и выполните команду `jmp ax`. В результате в `ip` записано значение из регистра `ax`.

Проверим выполнение `jmp` при адресации данных в *памяти*. В переменной `_w` задано значение метки `m4`. Выполните команду `jmp _w`. В результате в `ip` записано значение, заданное в переменной `_w`.

Наиболее часто в командах перехода используется *непосредственная* адресация. Выполните команду `jmp m5`. В `ip` записано значение `m5`.

В общем случае, чтобы выяснить, куда направлен переход, надо мысленно заменить команду `jmp <adr>` на несуществующую команду `mov ip, <adr>`. Тогда, например, команда `jmp ax`, которая задает нечто странное: "прыжок на регистр" — становится просто-напросто "командой" `mov ip, ax` — записью содержимого `ax` в `ip`. Путаница возникает из-за того, что адресация здесь формально *регистровая*, а переход получается *косвенным*, поскольку адрес перехода задан содержимым регистра.

Команда `call` — это команда `jmp`, с той лишь разницей, что она сохраняет в стеке адрес продолжения последовательности команд, откуда было передано управление. Команда `ret` действует так, как действовала бы несуществующая инструкция `pop ip` — считывает значение с вершины стека в `ip`. В результате (если между выполнением команд `call` и `ret` значение `sp` не изменилось) по команде `ret` возобновится выполнение последовательности, прерванной при отработке `call`.

Выполните команду `call m2`. Значение `ip` стало равным `m2`, а в стек записан адрес команды `int 3`, которая в буфере трансляции следует за командой `call`. Чтобы убедиться в этом, переместитесь клавишей <Page Down> к метке `d86_buffer`.

Выполните встроенную макрокоманду⁶ `push m7, m6, m5`. Дважды выполните `ret`, а затем `ret 2`. Необязательный непосредственный операнд команды `ret` задает приращение `sp` после чтения из стека. Как изменялись значения `ip` и `sp`?

Команды условных переходов

Команда условного перехода `j<х>`, или *ветвление*, записывает в `ip` непосредственный операнд, если выполнено условие `<х>`. Если условие не выполняется, то перехода нет и команда равносильна пустой операции.

Запишите в `cx` ноль и выполните команду `jcxz m1`. Условие перехода `<cxz>` читается как CX Zero, т. е. `cx = 0`. Запишите в `cx` число 2, выполните `jcxz m3`.

Условия для большинства ветвлений основаны на значениях *флагов* процессора. Значения флагов устанавливаются в результате выполнения арифметических и логических команд, команд сдвига и вращения. Значения флагов характеризуют результат *в целом*, например единичное значение флага `z` (Zero) говорит о том, что результат равен нулю.

Значения флагов в `d86` видны над вторым столбцом регистров. Флаги, установленные в единицу, отображаются буквами, как показано в табл. 5.1. Флаги, сброшенные в ноль, `d86` не показывает⁷.

Таблица 5.1. Флаги в слове состояния процессора i8086

Флаг	Название	Значение
o	Overflow	Переполнение
s	Sign	Знак результата
z	Zero	Нулевой результат
a	Auxiliary carry	Вспомогательный перенос (между тетрадами)
e	parity Even	Четное число единиц
c	Carry	Перенос/заем
d	Downward direction	Направление вниз (в сторону уменьшения адресов) ⁸
i	Interrupt enable	Разрешение внешних прерываний

Запишите в `al` число 1. Выполните команду `dec al`. Результат нулевой, и в строке флагов появилась буква `z`. Выполните команду `jz m1`. Условие перехода — Zero: `z = 1`. Еще раз выполните команду `dec al`. Результат равен `0ff`, или `-1`. Поэтому `z = 0` (результат ненулевой), `s = 1` (результат — отрицательное число). Вычитание единицы из нуля потребовало займа, поэтому `c = 1`.

С каждым из флагов `z`, `o`, `s`, `c` связаны *одноименные* условия, которые могут быть заданы в командах ветвления. Для каждого из этих условий есть обратное — `nz`, `no`, `ns`, `nc`. Например, команда `jc` задает переход при `c = 1`, а команда `jno` — при `o = 0`. Отдельная группа условий отражает результат сравнения двух чисел. Для обозначения отношений `=`, `<>`, `>`, `<`, `>=`, `<=` используются буквы из табл. 5.2.

Таблица 5.2. Обозначения арифметических отношений

Обозначение	Расшифровка	Значение
b	Below — ниже	Беззнаковое "меньше"
a	Above — выше	Беззнаковое "больше"
e	Equal — равно	Любое "равно"
g	Greater — больше	Знаковое "меньше"
l	Less — меньше	Знаковое "больше"
n	Not — не	Префикс "не"

Выполните команду `cmp al, 1`. Эта команда сравнивает значения пары чисел, оценивая разность `al - 1`. В результате устанавливаются флаги, на которых основаны условия `=`, `<>`, `>`, `<`, `>=`, `<=`. В данном случае команда `cmp` сравнила числа `0ff` (`-1`) и `1`. Эти числа не равны, поэтому условие `e` (`Equal`) не выполняется, а обратное условие `ne` (`Not Equal`) выполняется. Условие `e` эквивалентно условию `z` (равенство чисел означает, что их разность равна нулю). Условие `a` (`Above`) выполняется, т. к. `0ff > 1`. Также выполняются условия `nb` (`Not Below`), `ae` (`Above or Equal`), `nbe` (`Not Below-or-Equal`). (Заметим, что условия `a` и `nbe` эквивалентны.) Условие `g` (`Greater`) не выполняется, т. к. `-1 < 1`. Выполняются условия `l` (`Less`), `le` (`Less or Equal`), `ng` (`Not Greater`), `nge` (`Not Greater-or-Equal`).

За командой `cmp` обычно следует команда, которая использует полученную установку флагов. Проверьте выполнение команд:

```
ja m2
jl m3
jne m4
jna m5
```

Выполните команду `test al` (обратите внимание на буфер трансляции). Ассемблер `i86` допускает сокращенную форму записи команды `test al, al`, которая оценивает поразрядное логическое произведение операндов. Если `al` равно нулю, то и результат операции `"и"` равен нулю, и флаг `z = 1`. Если `al` не равно нулю, то и результат логического умножения `al на al` не равен нулю, и `z = 0`. Выполните команду `test w_1`. Прокомментируйте результат трансляции.

К командам условной передачи управления также относятся команды *циклов* (точнее, они применяются для организации счетных циклов). Условие перехода для всех команд этой группы: `--сх <> 0`. Содержимое `сх` уменьшается на единицу,

и переход выполняется, если результирующее значение `cx` не ноль. Запишите в `cx` число 1, выполните команду `loop m6`. В `cx` ноль, и перехода не было. Повторите эту команду. В `cx` теперь 0ffff, переход произошел.

У команды `loopz` есть дополнительное условие `z = 1`, помимо (`--cx != 0`). Переход произойдет, если `cx` после уменьшения на единицу не обнулится и при этом `z = 1`. Если `--cx` обнулится или `z = 0`, то переход отменяется. (Команды `loopz` и `loope` эквивалентны.) Условие перехода для команд `loopnz` и `loopne` аналогичное, только переход происходит при `z = 0`. Сейчас как раз `z = 0` (в результате выполнения `test w_1`), а `cx = 0ffff`. Выполните команды `loopnz m2` и `loopz m3`. Прокомментируйте результаты.

В большинстве ветвлений условие связано с флагами процессора⁹. Для установки значений флагов имеются специальные команды `test` и `cmp`, но многие другие команды тоже устанавливают флаги как побочный результат выполнения основной операции. Заметим также, что некоторые команды используют флаг `c` в качестве операнда. К ним относятся `adc` (сложение с прибавлением переноса), `sbb` (разность с вычитанием займа), `rcl/rcr` (циклический сдвиг, или вращение влево/вправо через флаг `c`).

Воздействие команд на флаги

На флаги состояния `c`, `o`, `z`, `s`, `e` воздействует большинство арифметических операций:

- ☐ `c` — устанавливается, если *беззнаковый* результат вне диапазона;
- ☐ `o` — устанавливается, если *знаковый* результат окажется вне диапазона (знаковое переполнение);
- ☐ `z` — устанавливается при нулевом результате;
- ☐ `s` — устанавливается, если старший (знаковый) бит результата равен единице, т. е. результат — *отрицательное* число;
- ☐ `e` — устанавливается, если младший *байт* результата содержит четное количество битов, установленных в единицу (четный паритет).

Воздействие команд на флаги состояния, по группам команд, показано в табл. 5.3. (Условные обозначения, принятые в табл. 5.3, раскрыты в табл. 5.4.)

Таблица 5.3. Воздействие команд на флаги состояния

Флаги состояния	o	c	s	z	e
Сложение и вычитание					
ADD ADC SUB SBB					
CMP NEG CMPS SCAS	+	+	+	+	+
Инкремент и декремент					
INC DEC	+	–	+	+	+

Таблица 5.3 (окончание)

Флаги состояния	o	c	s	z	e
Умножение и деление					
MUL IMUL	+	+	?	?	?
DIV IDIV	?	?	?	?	?
Логические операции					
AND OR XOR TEST	0	0	+	+	+
Сдвиги и вращения					
SHL SHR (1)	+	+	+	+	+
SHL SHR (>1)	?	+	+	+	+
SAR	0	+	+	+	+
ROL ROR RCL RCR (1)	+	+	–	–	–
ROL ROR RCL RCR (>1)	?	+	–	–	–

Таблица 5.4. Обозначения, принятые в табл. 5.3

Обозначение	Значение флага
–	Не изменяется
+	Изменяется в соответствии с результатом
1	Устанавливается в единицу
0	Сбрасывается в ноль
?	Непредсказуемое или неопределенное

Инструкции сложения и вычитания воздействуют на все флаги состояния. При этом флаги переполнения `o` и переноса `c` показывают, что результат вышел за диапазон знаковых или беззнаковых чисел соответственно; флаги знака `s`, нуля `z`, паритета `e` показывают, что результат отрицательный, нулевой, содержит четное число единичных битов.

Вновь запустите отладчик с программой `ex1.com`.

Запишите в `ax` и `bx` число 511. Хотя эти числа можно было бы сложить одной командой `add ax, bx`, воспользуемся байтовым сложением, чтобы на примере двойных байтов продемонстрировать сложение с *многократной точностью*.

Выполните команду `add al, bl`. Сложение младших байтов `ax` и `bx` (255 + 255) вызвало перенос (`c` = 1), который надо учесть при сложении старших байтов. Для этого используется вариант команды сложения `adc`. Выполните команду `adc ah, bh`. Оцените результат в `ax`. Результат правильный, т. к. при сложении старших байтов переноса не возникло (`c` = 0). Выполните аналогично сложение чисел –1 и –2. Какой из результатов правильный — беззнаковый или знаковый?

Вычитание с многократной точностью выполняется по той же схеме, но с использованием команды `sbb`, флаг `c` устанавливается в случае займа.

Инструкции сравнения `cmp`, `cmps`, `scas` действуют подобно команде вычитания, но результат никуда не записывается¹⁰. Инструкции сравнения предназначены только для установки значений флагов.

Инструкции инкремента и декремента воздействуют на флаги состояния так же, как сложение и вычитание, но не изменяют флаг `c`. Запишите в `al` число `-1`, сбросьте флаг `c` командой `cld` и выполните `add al, 1`. Восстановите значение `al` и флага `c`, затем выполните `inc al`.

Переполнение в результате арифметической инверсии `neg <arg>` возникает, если операнд — наименьшее из отрицательных чисел в диапазоне `<arg>`. Запишите в `al` число `-128`, выполните `neg al`. Флаг `o` = 1, поскольку `+128` "не помещается" в `al`. Флаг `c` устанавливается, если результат `neg` отличается от исходного значения. Исследуйте поведение флага `c` при выполнении `neg al` с исходными значениями `al` = 0, 1 и `-128`.

Установка флагов `c` и `o` при выполнении `mul/imul` означает, что получен результат *двойной размерности*. Запишите в `al` число 40 и выполните команду `mul al`. Флаги `c` и `o` установлены в 1, т. к. в `ah` попали значащие цифры результата. Повторите опыт с `al` = 10. Теперь результат помещается в `al` целиком, поэтому `c` = `o` = 0.

Поскольку поразрядные логические команды никогда не приводят к переполнению, флаги `c` и `o` этими командами сбрасываются в ноль. Флаги `z`, `s`, `e` отражают характеристики результата по общим правилам.

Команды вращений и логических сдвигов устанавливают флаг `c` равным значению бита, выталкиваемого за пределы разрядной сетки. Задайте в `al` число 2, установите флаг `c` в единицу командой `stc` и выполните сдвиг `shr al, 1`. Флаг `c` отражает состояние младшего бита исходного значения. Повторите команду `shr al, 1`.

Проверьте, какой бит определяет значение флага `c` при числе сдвигов больше одного: тот, что выталкивается первым, или тот, что выталкивается последним? Для этого задайте в `al` значение `bit 6`, обнулите флаг `c` командой `cld` и выполните `shl al, 2`.

Команды арифметических сдвигов можно рассматривать как умножение или деление на степень двойки. Флаги состояния отражают результат по общим правилам, за одним исключением: значение флага `o` не определено, если число сдвигов больше единицы. Результат арифметического сдвига вправо (`sar`) — всегда в диапазоне, поэтому флаг `o` обнуляется.

Флаг `o` при выполнении вращений устанавливается по общим правилам, хотя не вполне понятно, что в этом случае означает арифметическое переполнение.

В завершение рассмотрим команды сохранения и восстановления флагов. Выполните команду `pushf`. Значения флагов в виде 16-разрядного слова записаны в стек. Инвертируйте флаг `c` командой `cmc`, затем выполните команду `popf`. Значения флагов на момент выполнения `pushf` восстановлены.

Команды прямой установки флагов реализованы только для флага `c`. Это команды `cld` (обнуление `c`), `stc` (установка `c` = 1) и `cmc` (инверсия `c`). Для флагов управления реализованы команды `cld` (обнуление `d`), `std` (установка `d` = 1), `cli` (обнуление `i`) и `sti` (установка `i` = 1).

Следующие команды не влияют на флаги:

- ☐ пересылки;
- ☐ передачи управления;
- ☐ логическая инверсия `not`.

Строковые команды

Строковые команды несколько упрощают поэлементную (последовательную) обработку массивов байтов и слов. Эти команды реализуют операции чтения (`lods`), записи (`stos` и `movs`) и сравнения (`scas` и `cmps`). Текущий элемент массива-источника адресуется регистром `si` (Source Index), а текущий элемент массива-приемника — регистром `di` (Destination Index). Если в операции участвует только один массив, то второй операнд — всегда или `al`, или `ax`. После выполнения операции значение индексного регистра автоматически изменяется на один или два в зависимости от размерности операции. Изменение положительное, если флаг `d` (Downward direction) равен нулю, и отрицательное, если `d` = 1. Размерность операции задает суффикс `b/w` мнемоники команды (без суффикса операция байтовая).

Выполните команду `lea si, b_1`. В свободных окнах отображения памяти задайте спецификации `r,b_1`; `r,b_2`; `r,si` и `r,di`. Сбросьте флаг `d` командой `cld`. Выполните команду `lodsb`. В `al` записан байт из памяти по адресу `[si]`, после чего значение `si` увеличилось на единицу. Повторите два раза команду `lodsb`, наблюдая за `al` и `si`, затем установите обратное направление командой `std` и еще пару раз выполните `lodsb`.

Запишите в `di` адрес `b_1`, а в `al` — ASCII-код '*', установите прямое направление и выполните команду `stosb`. Значение из `al` записано в байт по адресу `[di]`, сам же `di` увеличился на единицу. Запишите в `ax` число '1' by '2' и выполните команду `stosw`. Как изменились массив `b_1` и регистр `di`?

Запишите в `di` адрес `b_2`, а в `al` — ASCII-код пробела. Выполните команду `scasb`. Эта команда сравнивает значения `[di]` и `al`, устанавливая флаги подобно команде `cmp`. Повторите `scasb`, пока значения `[di]` и `al` не совпадут (флаг `z` = 1).

Восстановите в `di` адрес `b_2`, а в `si` запишите адрес `b_1`. Выполните один раз команду `movsb`. В чем заключается ее действие?

Выполните команду `cmpsb`. Эта команда сравнивает значения `[di]` и `[si]`, устанавливая флаги состояния. В строках `b_1` и `b_2` сейчас должны совпадать последние байты 'b'. Наблюдая за значением флага `z`, повторяйте `cmpsb` до совпадения значений `[di]` и `[si]`.

Рассмотренные команды зацикливаются за счет использования *префиксов повторения* `rep<x>`. (Префикс — это специальный байт перед кодом команды, который

изменяет ее поведение.) Префикс повторения `rep<x>` перед строковой командой блокирует автоматическое увеличение `ip`, пока выполняется условие, аналогичное условию перехода для команды `loop<x>`.

Запишите в `cx` число `c_1`. Это количество байтов в массивах `b_1`, `b_2` и `b_3`. В `si` задайте адрес `b_2`, а в `di` — адрес `b_3`. Выполните команду `rep movsb`. Команда с префиксом `rep` может быть представлена в виде:

```
11:  jcxz      12
      movsb
      loop   11
12:
```

Префикс `rep` имеет смысл с командами `stos` и `movs`. С командами `scas` и `cmps` уместны префиксы `repne` (`repnz`) и `repe` (`repz`), которые задают дополнительное условие выполнения цикла по значению флага `z`. Запишите в `al` код 'b', в `di` — адрес `b_3`, в `cx` — число `c_1`. Выполните `repne scasb`. Цикл поиска литеры 'b' завершился успешно, о чем свидетельствует значение флага `z`. Нулевое значение `cx` не свидетельствует ни за, ни против: выполнение условия `ne` могло произойти на *последней* итерации, как в нашем случае. Обратите внимание, что `di` содержит адрес *следующего* байта после найденного. Такой эффект "выноса" указателей характерен для всех строковых команд.

Составьте команды для подготовки и выполнения поиска в строке `b_1` байта с нулевым значением и байта со значением, отличным от нуля.

Заметим, что строковая команда без префикса действует как две простые команды, но проигрывает им в скорости — таков эффект параллельного выполнения "простых" команд, начиная с i80386. Напротив, строковая команда с префиксом развивает рекордную скорость¹¹, потому что она уже прочитана из памяти и декодирована, и процессор только повторяет малую часть проделанной работы.

Примечания

1. Блестящее решение — ведь нам не придется нажимать специальную клавишу, чтобы начать ввод инструкции ассемблера. А ведь пришлось бы, да при вводе *каждой* команды, не будь автор a86/d86 таким изобретательным.
2. Чтобы видеть результат трансляции в режиме непосредственного выполнения. Если метка `d86_buffer` не определена, d86 размещает буфер трансляции по своему усмотрению.
3. Команда `push` с байтовым операндом не работает, только со словами.
4. Команда `xor` для обнуления регистра короче, выполняется быстрее, чем `mov ax, 0`, и поэтому характерна для ассемблерного стиля прошлого века.
5. Уникальный язык макрокоманд a86 рассмотрен в `a86manu.txt`.
6. У машинной команды `push` — только один операнд.

7. Некоторые отладчики показывают *все* флаги: большими буквами — установленные, маленькими — сброшенные. Что лучше?
8. Используется в строковых командах, где регистр-указатель автоматически изменяется после выполнения операции.
9. В командах `loop` и `jcxz` — со значением регистра `cx`.
10. Команды `scas/cmps` относятся к строковым командам, которые будут рассмотрены чуть позже.
11. Если для вас важна скорость выполнения, рекомендуется выравнивать данные по их естественной границе, т. е. размещать 16-битовые слова по четным адресам, 32-битовые — по адресам, кратным 4, и т. д. Для этого в ассемблерах есть директива `even` или `align`.

ГЛАВА 6

```
DB6 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA i c 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FF0E 0:

lea si,b_1
```

Программирование циклов

Лингвист пояснил нам, что в розовой колбе — имена существительные, в зеленой — глаголы, в голубой — прочие компоненты ялмезианской речи. Затем он осторожно слил содержимое колб в мензурку; в ней образовалась густая мутно-бурая жижа. <...> Лексинен заявил, что лингвистический отвар остынет через 20 минут, после чего он введет каждому из нас нужную учебную дозу. <...> Через 43 минуты после шприцевания мы сможем объясняться на новом для нас языке, освоение же письменности зависит в дальнейшем от нас самих. Что касается болезненных явлений, то они длятся не более двух часов.

(Вадим Шефнер. Лачуга должника, гл. 27)

— Как? Разве я недостаточно упражняюсь? Прежде чем встать, я раз семь перевернусь с боку на бок. Неужели этого мало?

(Франсуа Рабле. Гаргантюа и Пантагрюэль)

В предыдущих главах мы рассмотрели элементы ассемблера: определение данных и имен, систему команд в объеме i8086/286, а также режимы адресации, необходимые для вычислений и обработки массивов. Объединив эти элементы, мы получим программы с исполняемым кодом.

В этой главе приведены примеры построения программ с *циклами* и *условным выполнением*. Все примеры на одну тему — поиск в массиве.

Поиск в массиве байтов

Задан массив байтов, требуется выяснить, есть ли среди них байт со значением 7. Решение приведено в листинге 6.1.

Листинг 6.1. Поиск в массиве байтов (find_1.8)

```
x equ 7 ; (1)

jmp start ; (2)

array db -128, 127, 0, 255 ; (3)
db x-1, x+1, x ; (4)

size equ $ - array ; (5)
```

```

start:  mov     cx, size           ; (6)
        lea     si, array - 1     ; (7)
l1:      ; (8)
        inc     si                ; (9)
        cmp     b[si], x          ; (10)
        loopne  l1                ; (11)
        jne     miss              ; (12)

found:  int     3                  ; (13)
miss:   int     020                ; (14)

```

Искомое значение указано в (1) под именем *x*. При определении массива (3) заданы предельные значения для байта (чтобы при тестировании проверить граничные ситуации), а в (4) — значения в окрестности *x* (с той же целью) и само *x*. Константа *size* (5) дает размер массива в байтах. Поскольку массив обрабатывается в цикле байт за байтом, число повторений должно быть равно длине массива¹. В качестве счетчика повторов используем регистр *cx*: в (6) он инициализируется, а в (11) уменьшается на единицу. В строке (11) цикл может завершиться по двум причинам: либо при обнулении *cx*, либо по условию *e*, которое отражает результат сравнения (10).

В тело цикла (9–10) включена команда (9) инкремента регистра-указателя для подготовки доступа к следующему байту массива. Поскольку *si* сначала увеличивается, а потом используется для доступа к данным, то в (7) перед входом в цикл *si* устанавливается на единицу меньше.

Итак, выход из цикла по команде (11) может произойти по условию *e* (когда искомое число найдено) и по условию *cx* = 0. С теми данными, что определены в (3–4), выход произойдет сразу по обоим условиям, т. к. искомое число задано в конце массива и обнаружится на *последней* итерации. Поэтому на выходе из цикла в (12) проверяется не *cx*, а условие *e/ne*.

Проверьте программу `find_1.8` в отладчике. Дойдя до (13), проверьте значения регистра *cx* и флага *z*. Нажмите клавишу <Home>, чтобы вернуться к началу, и в режиме непосредственного выполнения измените последний байт массива (по смещению 6). Выполните прогон командой `g` <Enter>; программа должна остановиться на (14), обойдя (13). Вновь нажмите <Home> и запишите *x* в начало массива. Теперь программа должна остановиться² на (13). В регистре *cx* теперь не ноль, а число оставшихся (невыполненных) повторов.

Что произойдет, если поменять местами (9) и (10), чтобы порядок действий был естественным, а заодно убрать вычитание единицы из (7), как показано в листинге 6.2?

Листинг 6.2. Неправильный поиск в массиве байт (`find_1b.8`)

```

        lea     si, array           ; (7)
l1:      ; (8)
        cmp     b[si], x            ; (9)

```

```
inc    si                ; (10) ?!
loopne ll               ; (11)
```

Проверьте в отладчике. Команда (10) будет каждый раз по-своему устанавливать флаг *z* (в примере — всегда обнулять его), отменяя результат сравнения (9). Посмотрите, как на последней итерации команда (9) устанавливает флаг *z*, а (10) его сбрасывает.

Поиск в массиве слов

Поменяем условия задачи: пусть задан массив слов и требуется выяснить, есть ли среди них *слово* со значением 7. Решение приведено в листинге 6.3.

Листинг 6.3. Поиск в массиве слов (find_2.8)

```
...
array  dw      08000, 07fff, 0, 0ffff ; (3)
       dw      x-1, x+1, x           ; (4)
size   equ     $ - array             ; (5)

start: mov     cx, size / 2           ; (6)
       lea     si, array - 2         ; (7)
ll:    ; (8)
       add     si, 2                 ; (9)
       cmp     w[si], x              ; (10)
       loopne  ll                   ; (11)
...
```

Изменились директивы (3–4), счетчик циклов теперь вдвое меньше (6) числа байтов в массиве, отступ при инициализации (7) регистра-указателя теперь равен двум, и на 2 увеличивается указатель в (9), а в команде сравнения (10) явно задан тип *word*.

В листинге 6.4 приведен другой вариант решения задачи о поиске (на примере поиска слова): с применением строковой команды *scas*.

Листинг 6.4. Поиск в массиве с применением строковой команды (find_3.8)

```
...
start: mov     cx, size / 2           ; (6)
       lea     di, array             ; (7)
       mov     ax, x                 ; (8)
       cld                                     ; (9)
ll:    ; (10)
       scasw                                     ; (11) ax <> w[di++] ?
       loopne  ll                   ; (12)
...
```

Команда (11) сравнивает слово по адресу `di` со значением в регистре `ax`, сохраняя результат сравнения во флагах, и продвигает указатель `di` на два, т. е. к следующему или предыдущему слову. Когда флаг направления сброшен (9), `di` увеличивается³.

Команда `scas` с префиксом повторения `repne` позволяет исключить даже команду `loopne`. Весь цикл, со всеми проверками и продвижением указателя, теперь выполняется одной командой. Напомним, что префикс — это байт с уникальным значением, который, будучи поставлен перед кодом команды, изменяет способ ее выполнения.

Префиксы *повторения* действуют лишь на *строковые* команды, зацикливая их выполнение подобно команде `loop<x>`. Решение задачи о поиске в массиве с использованием строковой команды и префикса повторения показано в листинге 6.5.

Листинг 6.5. Применение строковой команды с префиксом `rep` (find_4.8)

```
...
start:  mov     cx, size / 2           ; (6)
        lea     di, array             ; (7)
        mov     ax, x                 ; (8)
        cld                               ; (9)
        repne scasw                   ; (10)
        jne     miss                 ; (11)
        ...
```

Команда (10) содержит в себе и тело цикла, и управление циклом.

Поиск байта со значением больше заданного

Изменим условия исходной задачи: в массиве *знаковых* данных требуется найти байт со значением больше заданного. Решение приведено в листинге 6.6.

Листинг 6.6. Поиск байта со значением больше заданного (find_5.8)

```
...
start:  mov     cx, size               ; (6)
        lea     si, array - 1         ; (7)
l1:      ; (8)
        inc     si                    ; (9)
        cmp     b[si], x              ; (10)
        jg      found                 ; (11) !
        loop    l1                    ; (12) !
miss:    int     3                     ; (13) !
found:   int     020                   ; (14)
```

По сравнению с первым примером (см. листинг 6.1) изменилось условие поиска: вместо `e/ne` (равно/не равно) теперь используется `g` (Greater — знаковое "больше").

Поскольку в командах `loop<x>` условие `g` неприменимо, выход из цикла при успешном поиске выполняется отдельным ветвлением (11). Чтобы представить себе условие перехода при выполнении пары команд `cmp` и `j<x>`, мысленно поставьте между операндами `cmp` знак сравнения `=`, `<`, `>`, `<=`, `>=`, соответствующий условию `<x>`:

```
b[si] > 1 ; <x> = g (greater)
```

Обратите внимание: в (12) теперь обычная команда `loop`. При успешном поиске мы выходим из середины (11) цикла на (14), а при безуспешном — на следующую за (12) команду.

Тело цикла в листинге 6.6 можно сократить, заменив (9–10) строковой командой, как показано в листинге 6.7.

Листинг 6.7. Сокращенный вариант программы из листинга 6.6 (find_6.8)

```
...
lea    si, array                ; (7)
11:                                ; (8)
scasb                                ; (9)
jg      found                   ; (10)
loop    l1                      ; (11)
...
```

Дальнейшее упрощение цикла невозможно — как не существует команды `loopng`, так нет и префикса `repng`.

Подсчет байтов в заданном диапазоне значений

Сформулируем другой вариант исходной задачи поиска: подсчитать число байтов со значением в диапазоне 10...99, результат сформировать в регистре `ax`.

Алгоритмическое решение

В листинге 6.8 приведен первый вариант решения — за счет использования команд сравнения и условных переходов. Проверка диапазона закодирована в *инструкциях*.

Листинг 6.8. Подсчет байтов со значением в заданном диапазоне (cnt_1.8)

```
_low    equ    10                ; (1)
_high   equ    99                ; (2)

def_3    macro                    ; (3)
#rx1L                    ; (4)
    db    #x-1, #x, #x+1        ; (5)
```

```

#em                                ; (6)

        jmp      start              ; (7)

array   db      0, -128, +127, 255   ; (8)
        def_3    _low, _high        ; (9)
size    equ     $ - array            ; (10)

start:  lea      si, array            ; (11)
        mov     cx, size              ; (12)
        cld                                ; (13)
        mov     dx, 0                ; (14)
11:                                ; (15)
        lodsb                                ; (16)
        cmp     al, _low              ; (17)
        jb      >12                  ; (18)
        cmp     al, _high            ; (19)
        ja      >12                  ; (20)
        inc     dx                   ; (21)
12:     loop    11                   ; (22)
        mov     ax, dx               ; (23)
;      ...
        int     020

```

Строковая команда (16) копирует значение байта из [si] в al и продвигает указатель si (увеличивает, если флаг d сброшен). Поскольку регистр al теперь занят, для подсчета числа байт в заданном диапазоне временно используем свободный регистр dx (14, 21, 23).

В (9) задан вызов макрокоманды, определенной в (3–6). В результате резервируются шесть байтов со значениями 9, 10, 11 (числа в окрестности 10) и 98, 99, 100 (числа вокруг 99), что необходимо для тестирования программы. В макровывозе (9) заданы два параметра, а в макроопределении объявлен цикл (4) по параметрам макровывоза — от первого до последнего, который обозначается буквой L (last). Переменная x на первом заходе r-цикла примет значение _low, а на втором — _high, и каждый раз будет сгенерирована директива db с тремя значениями. Результаты макроподстановки можно увидеть в lst-файле, они обозначены буквой m.

Табличное решение

Программирование таблиц позволяет в ряде случаев полностью исключить команды ветвлений. Логика программы становится проще, а время выполнения сокращается.

В листинге 6.9 приведен вариант решения задачи о подсчете байтов в заданном диапазоне значений. Вместо команд сравнений и ветвлений используется таблица размером 256 байтов, определенная директивами db. Для каждого числа из диапазона 0–255 в таблице записано либо 1 (признак "число в диапазоне"), либо 0 (число

вне диапазона). Эта 1 или 0, прочитанная из таблицы, прибавляется к счетчику `dx`. Доступ к таблице выполняется при помощи инструкции `xlat`.

Листинг 6.9. Табличное решение задачи о подсчете байтов в диапазоне (cnt_2.8)

```
p1      equ      10                      ; (1)
p2      equ      99                      ; (2)
...

map      db      p1 dup 0                ; (11)
         db      (p2 - p1 + 1) dup 1     ; (12)
         db      256 - ($ - map) dup 0   ; (13)

start:   lea      si, array                ; (14)
         mov     cx, size                  ; (15)
         lea     bx, map                  ; (16)
         cld                                ; (17)
         mov     ax, dx, 0                ; (18)
11:      ; (19)
         lodsb                                ; (20)
         xlat                                ; (21)
         add     dx, ax                    ; (22)
         loop    11                       ; (23)
         mov     ax, dx                    ; (24)
;
         ...
int      020
```

Таблица (11–13) задает 256 байтов⁴, из них в единицу установлены только элементы с номерами от 10 до 99. Чтение таблицы выполняется командой (21), которая адресует данные в памяти по сумме `bx` и `al`; адрес таблицы записан (16) в `bx` заранее. На каждой итерации в `al` попадает (20) очередное число из массива `array`, и это значение команда (21) использует как смещение в таблице (11–13). После выполнения (21) в `al` оказывается 0 или 1 — признак принадлежности диапазону. Предварительно старшая часть `ax` обнуляется (18), чтобы можно было целиком⁵ прибавлять `ax` к счетчику `dx` (22).

Запрограммируйте свою таблицу для этой задачи, варианты приведены в табл. 6.1. В вариантах 1–5 значения расположены произвольно (нерегулярно), а в вариантах 6–9 они идут через равные промежутки.

Таблица 6.1. Варианты задания

Вариант	Значения
1	10–99, 101–122
2	'A'–'Z', 'a'–'z'
3	'A'–'Z', 'a'–'z', '0'–'9'

Таблица 6.1 (окончание)

Вариант	Значения
4	0, 5–255
5	0, '', 255
6	1–9, 11–19 и т. д. до 241–255 включительно
7	1–3, 11–13 и т. д. до 251–253 включительно
8	8–15, 18–25, ... 248–255
9	7, 17, ..., 247

Примечания

1. Если бы у нас был массив 16-битовых слов, то число повторений оказалось бы вдвое меньше размера массива.
2. Напомним, что команда `int 3` в отладчике равносильна *точке останова*.
3. Регистры `di` (destination index) и `si` (source index) так называются потому, что в строковых командах они адресуют *приемник* и *источник* соответственно. Хотя слово "index" тут неточное — в строковых командах эти регистры играют роль *указателей*.
4. Чтобы заполнить таблицу `map`, мы разбили диапазон 0–255 на три поддиапазона. В примере это `[0...10[` — нули, `[10...99]` — единицы, `]99...255]` — вновь нули. Длина поддиапазонов: $10 - 0 = 10$ (p_0), $99 - 10 + 1 = 90$ ($p_2 - p_1 + 1$), $255 - 99 = 156$ ($255 - p_2$). Добавка единицы для второго диапазона объясняется тем, что он закрытый (границы 10 и 99 входят в него); в остальных случаях диапазон полукоткрытый. Для полностью закрытого диапазона следует вычитать единицу. Представьте себе диапазон `]2...5[`, т. е. `[3...4]`. Длина первого диапазона вычисляется как $(5 - 2) - 1$, а длина второго — как $(4 - 3) + 1$.
5. Прибавить `al` к `dx` нельзя, они разной размерности.

ГЛАВА 7

```
DB6_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100 2: =d1,ax,, 1962
CX 0000  CS 6309 3:
DX 6309  SS 6309 4:
SI 011F  DS 6309 5: r5,b_1,, AB ab
DI 0000  ES 6309 6: r5,si,, AB ab
BP 0000  SP FFFE 0:

lea si,b_1
```

Примеры программ

— Конечно, хочу в Кремль! Даже очень!

Тогда мама улыбнулась:

— Ну вот, съешь всю кашу, и пойдем. А я пока посуду вымою. Только помни — ты должен съесть все до дна!

И мама ушла на кухню.

А я остался с кашей наедине. Я пошлепал ее ложкой. Потом посолил... Посыпал песку, попробовал... Еще хуже стало... <...> А она к тому же была очень густая. Если бы она была жидкая, тогда другое дело, я бы зажмурился и выпил ее.

(Виктор Драгунский. Денискины рассказы)

— У тебя не мама, а золото, — сказал Карлсон, — жаль только, что она такая жадная. Никогда не видел, чтобы варили так мало каши.

(Астрид Линдгрен. Мальчи и Карлсон)

В этой главе рассматриваются на примерах следующие темы:

- ☐ обработка данных на уровне битов;
- ☐ составление программ с вложенными циклами;
- ☐ ввод исходных данных и вывод результатов;
- ☐ опережающие ссылки.

Обработка данных на уровне битов

Задан байтовый массив. Для каждого байта требуется вычислить число битов, установленных в единицу, и записать это значение в начало массива.

В первом варианте решения в листинге 7.1 данные определены прямо в тексте программы — по известному адресу `buf`, а размер данных известен — `size`. Это *отладочный* вариант. На его основе мы в дальнейшем создадим программу, которая читает входные данные (заранее неизвестного объема) из стандартного входного потока, а результаты записывает в стандартный выходной поток.

Листинг 7.1. Отладочный вариант решения задачи (var_1.8)

```

        jmp      start                      ; (1)

buf     db      bit 0, bit 1 + bit 7, -1, 0 ; (2)
size    equ     $ - buf                    ; (3)

start:  lea      si, buf                    ; (4)
        mov     di, si                     ; (5)
        cld                                ; (6)
        mov     cx, size                    ; (7)

11:                                           ; (8)
        lodsb                                ; (9)
        mov     ah, 0                      ; (10)
        push    cx                          ; (11)
        mov     cx, 8                      ; (12)

12:                                           ; (13)
        shr     al, 1                      ; (14)
        adc     ah, 0                      ; (15)
        loop    12                         ; (16)
        pop     cx                          ; (17)

        mov     al, ah                     ; (18)
        stosb                                ; (19)
        loop    11                         ; (20)
        int     020                        ; (21)

```

Здесь использован цикл в цикле: внешний цикл — от (8) до (20), и вложенный — от (13) до (16). Оба цикла — счетные, с командой `loop` в конце. В них задействован один и тот же регистр-счетчик `cx`, поэтому перед началом вложенного цикла сохраняем (11) `cx` в стеке, а после завершения — восстанавливаем (17).

В начале программы настраиваем (4–5) указатели `si` и `di` на один и тот же адрес `buf`. В дальнейшем указатель `si` используется строковой командой чтения (9), а `di` — строковой командой записи (19). После чтения (9) очередного байта из массива (2) обнуляем (10) счетчик битов `ah` и входим (11–12) во вложенный цикл для обработки битов байта. В нем мы сдвигаем прочитанный байт и, поскольку вытолкнутый бит копируется во флаг `c`, прибавляем (15) его к счетчику `ah`. Выйдя из вложенного цикла, восстанавливаем (17) счетчик внешнего цикла и сохраняем (18–19) `ah` в массиве на месте прочитанного байта¹.

Заметим, что вместо отладки по шагам можно, установив курсор на строку (20), нажимать клавишу <F9> (GoTillHere), что равносильно заданию временной точки останова на (20) и команде Go (нажатие клавиши <g>, а затем <Enter>). При каждой нажатии <F9> выполняется одна итерация цикла.

В листинге 7.2 показано, как можно сократить число итераций вложенного цикла, если выходить из него сразу при обнулении `al`. Тогда инструкция `loop` не нужна, а с ней исчезают и команды сохранения, инициализации и восстановления `cx`.

Листинг 7.2. Еще один отладочный вариант решения (var_1b.8)

```

...
11:                                ; (8)
    lodsb                         ; (9)
    mov     ah, 0                 ; (10)
    cld                             ; (11)
12:                                ; (12)
    adc     ah, 0                 ; (13)
    shr     al, 1                 ; (14)
    jnz     l2                    ; (15)

    mov     al, ah                ; (16)
    adc     al, 0                 ; (17)
    stosb                             ; (18)
...

```

Вложенным циклом (12–15) теперь управляет команда `jnz`. Перед входом в цикл обнуляем (11) флаг `c`, чтобы при первом выполнении (13) не прибавить лишнего. Кажется, что (14–15) лучше поменять местами с (13), но тогда понадобилась бы дополнительная команда `jmp` в конце цикла. Мы сэкономили одну команду в цикле, заплатив за это двумя дополнительными командами: (11) на входе и (17) на выходе.

Программирование ввода-вывода

Добавим в программу `var_1b.8` операции *ввода-вывода*. Если воспользоваться *стандартными потоками* ввода-вывода, то можно при вызове программы связывать их с файлами:

```
prog <data.in >data.out
```

Литера `<` означает, что стандартный ввод программы `prog` связан с файлом `data.in`. Перенаправление выходного потока задается знаком `>`. Файл `data.out` создается операционной системой заново, а если файл с таким именем существует, то он урезается до нулевого размера.

По умолчанию стандартные потоки работают с консолью, т. е. с клавиатурой и дисплеем. Программа во всех вариантах одна и та же, а перенаправление берет на себя операционная система. В листинге 7.3 представлен фрагмент программы с чтением из стандартного входного потока.

Листинг 7.3. Чтение стандартного входного потока

```

dos      macro
        mov     ah, #1
        int     021
#em

```

```

mov     bx, 0           ; stdin
mov     cx, 1k
lea     dx, buf
dos     03f
jc      read_error
...

```

Чтение выполняется запросом к DOS (int 021) с номером функции в `ah`, равным 03f. Для обращения к DOS определена макрокоманда `dos` с одним параметром — номером функции DOS. Перед обращением к `dos 03f` следует задать:

- ❑ в `cx` — максимальное число байтов для чтения;
- ❑ в `dx` — адрес массива, куда будут записаны введенные данные;
- ❑ в `bx` — номер потока (ноль — номер стандартного входного потока).

Если при выполнении запроса произошла ошибка, флаг `c` = 1. При успешном выполнении функции 03f (на выходе `c` = 0) в регистр `ax` записан счетчик прочитанных байтов, если он окажется меньше заданного числа байтов, это означает, что мы дошли до конца потока.

Фрагмент программы для записи в стандартный выходной поток приведен в листинге 7.4.

Листинг 7.4. Запись в стандартный выходной поток

```

...
mov     bx, 1           ; stdout
mov     cx, 1k
lea     dx, buf
dos     040
jc      write_error
...

```

Параметры и выходные значения функции 040 (запись) аналогичны функции 03f (чтение):

- ❑ `cx` — число байтов для записи;
- ❑ `dx` — адрес массива-источника данных;
- ❑ `bx` — номер потока (единица — номер стандартного выходного потока);
- ❑ при завершении функции `c` = 1 означает ошибку;
- ❑ если при завершении `c` = 0, то в `ax` — число записанных байтов.

Разница лишь в номере функции DOS (040 — запись, 03f — чтение) и в номере потока (единица у стандартного выходного потока)².

В следующем варианте программы, в листинге 7.5, исходный массив считывается из стандартного входного потока, а результаты преобразований записываются в стандартный выходной поток; массив обрабатывается *in situ*³.

Листинг 7.5. Рабочий вариант решения задачи (var_2.8)

```

dos      macro                                ; (1)
        mov     ah, #1                        ; (2)
        int     021                          ; (3)
#em                                             ; (4)

        mov     bx, 0                        ; (5)
        mov     cx, 08000                    ; (6)
        lea     dx, buf                      ; (7)
        dos     03f                          ; (8)
        jc      >19                          ; (9)

        mov     cx, ax                      ; (10)
        jcxz    >19                          ; (11)

        call    make                        ; (12)

        mov     bx, 1                        ; (13)
        lea     dx, buf                      ; (14)
        dos     040                          ; (15)
        jc      >19                          ; (16)

        mov     al, 0                       ; (17)
        dos     04c                          ; (18)
19:                                             ; (19)
        mov     al, 1                       ; (20)
        dos     04c                          ; (21)

make:                                         ; (22)
        push    cx                          ; (23)
        lea     si, buf                      ; (24)
        mov     di, si                      ; (25)
        cld                                       ; (26)
11:                                             ; (27)
        lodsb                                ; (28)
        mov     ah, 0                       ; (29)
        clc                                       ; (30)
12:                                             ; (31)
        adc     ah, 0                       ; (32)
        shr     al, 1                       ; (33)
        jnz     12                          ; (34)

        mov     al, ah                      ; (35)
        adc     al, 0                       ; (36)
        stosb                                ; (37)
        loop    11                          ; (38)

```

```

    pop     cx                ; (39)
    ret                    ; (40)

buf     db                    ; (41)

```

Обратите внимание: память для массива в программе не резервируется⁴. Имя `buf` определено в конце программы, обозначая *начало* массива, а сам массив в `com-файл` не попадает. После загрузки `com-программы` между адресом (41) и стеком будет "просвет" не меньше 08000 байтов⁵ — это число и задано в (6) как ограничитель количества читаемых байтов.

Опережающие ссылки

Поскольку имя `buf` определено в конце текста, при трансляции (7), (14) и (24) его *тип* неизвестен. В этих командах ссылка на имя `buf` — *опережающая*, и транслятор вынужден строить догадки о типе имени. Но почему бы не отложить решение до второго просмотра? Ведь к концу первого просмотра все имена и типы выяснятся. Нельзя потому, что в задачу первого просмотра входит еще и *распределение* памяти, а для многих команд ассемблера размер машинной инструкции зависит от *типа* операнда⁶.

Почему в `var_2.8` опережающие ссылки не привели к конфликтам? Потому что для команды `lea` решение однозначное: `buf` — это *адрес* памяти, и неважно, ссылается он на байт или на слово, т. к. адрес в любом случае 16-битовый. В листинге 7.6 приведены еще несколько примеров, когда длина машинных команд не зависит от типа имен.

Листинг 7.6. Команды, длина которых не зависит от типа имени (ref_1.8)

```

    lea     si, x_1          ; (1)
    lea     si, x_2          ; (2)
    lea     di, x_3          ; (3)
    lea     di, x_4          ; (4)
;   mov     ax, x_3          ; (5)
    mov     x_3, 256         ; (6)
x_1     equ   ?              ; (7)
x_2     db                    ; (8)
x_3     dw                    ; (9)
x_4:                    ; (10)

```

Выполните трансляцию `ref_1.8` и проверьте по `lst-файлу`, что длина команд (1–4) с операндами разных типов одинакова. Проверьте в отладчике, как `a86` понял смысл команд (1) и (4). Незнакомое имя в строке (6) может быть только типа `word`, поскольку записать число 256 можно только в слово. Последующее определение (9) не противоречит этому предположению.

Если убрать комментарий в начале строки (5), мы получим ошибку. Незнакомое имя в (5) может быть или константой (`abs`), или словом в памяти (`word`). В этой ситуации `a86` выбирает `abs`, что противоречит последующим операторам (6) и (9).

Рассмотрим примеры, где длина команды зависит от типа имени.

Листинг 7.7. Зависимость длины команды от типа операнда

```
mov    days, 7           ; byte/word (5/6)
mov    al, daze          ; abs/byte (2/3)
jmp    smth              ; word/abs long/abs short (4/3/2)
add    xx, yy
```

В программе `ref_2.8` заданы всевозможные варианты операндов, упомянутых в листинге 7.7. Вы можете скомпилировать `ref_2.8` (операторы с ошибками "Byte-Sized Constant Required" закомментируйте) и проверить по `lst`-файлу следующие утверждения:

- ❑ Имя `days` на месте операнда-приемника может означать только переменную, хотя неизвестно какой размерности. Если это байт, то и число 7 будет представлено байтом, а длина машинной команды составит пять байтов. Если это слово, то число будет представлено словом и длина команды — шесть байтов.
- ❑ Тип `daze` может быть `abs` или `byte`. В первом случае число занимает один байт (столько же, сколько регистр `al`), а вся команда — два байта. Если выяснится, что число `daze` больше 255, `a86` сочтет это ошибкой.
- ❑ Тип `smth` тоже не известен наверняка. Это может быть метка (тип `abs`), а может быть `word` (в слове по адресу `smth` записан адрес перехода). Во втором случае длина команды — четыре байта. В первом же случае возможны варианты: если переход короткий, то расстояние до него кодируется знаковым байтом, а если длинный, то словом. В итоге число байтов в команде варьируется от трех до четырех.
- ❑ В последней команде возможны сочетания операндов `word-imm16`, `byte-imm8`, где `imm` — непосредственный операнд, представленный в первом случае 16, а во втором — 8 битами (длина варьируется). Сочетания `word-word`, `byte-byte` в системе команд `i8086` недопустимы: нельзя явно указать два операнда в памяти.

Итак, разбор `lst`-файла, полученного при трансляции `ref_2.8`, показывает, что длина команд может зависеть от типа операнда. Но в `ref_2.8` имена определены заранее, до их упоминания в командах.

Так что же делает ассемблер, встретив *незнакомое* имя, обозначающее операнд команды?

Для операнда команд перехода все ассемблеры выбирают наиболее вероятный тип `abs`, а диапазон перехода берется с запасом — `long`. Обратите внимание на переход `jmp smth3` в `ref_2.8`. Судя по полученному коду, он считается длинным. Но на практике часто достаточно коротких переходов — на дистанцию от -128 до $+127$ байтов. Так что в большинстве случаев переходы без указания атрибута `short` кодируются с избытком⁷.

В `a86` переход короткий, если имя *локальное*, т. е. состоит из буквы и цифры⁸. Проверьте это утверждение самостоятельно; переход вперед на локальную метку обозначается значком `>`, например `jmp >m7`.

В случаях, не связанных с переходами, а86 считает, что неизвестный операнд-источник имеет тип *abs*. Эта ситуация рассмотрена при разборе *листинга 7.6*. Но если неизвестный операнд-приемник может быть двух типов (*byte* или *word*), то а86 выдает ошибку, как в *листинге 7.8*.

Листинг 7.8. Неизвестный операнд-приемник (ref_3.8)

```

mov     days, 7      ; (1)
mov     daze, 7      ; (2)
...
days   db           ?      ; (3)
daze    dw           ?      ; (4)
```

Ассемблеры *masm/tasm* в этой ситуации резервируют место *по максимуму* — на тот случай, если операнд окажется типа *word* (что потребует 16-битового представления числа 7, а не 8-битового). После трансляции (3) выяснится, что команда (1) байтовая и один байт в ее коде лишний. На его место записывают⁹ однобайтовую инструкцию *nop*. Зато меньше сложностей при трансляции!

Правильно ли это? Автор а86 утверждает, что ассемблер не должен ничего делать "за нашей спиной". Вряд ли кто из программистов ожидал появления команд *nop*, которых он не писал, тем более что при отладке на уровне *исходного текста* они не видны. (Кстати, такая отладка, предлагаемая *masm/tasm*, опасна, т. к. маскирует ошибки трансляции. Напротив, *d86* показывает именно то, что есть в *com*-файле, не полагаясь на исходный текст.)

Можно сделать вывод, что опережающие ссылки в любом случае нежелательны. И если они неизбежны, то следует дать компилятору сведения об именах при помощи директивы *extrn* в начале текста:

```
extrn   buf:byte
```

или, сокращенно (в а86):

```
extrn   buf:b
```

Объявление *extrn* говорит о том, что имя *buf* имеет тип *byte*, а определение *buf* находится в каком-то из модулей программы, и не обязательно в другом — возможно, в этом же самом.

Упаковка четырехбитовых кодов

Рассмотрим еще один пример обработки битов. Результат выполнения программ *var_<n>.8*, рассмотренных в начале главы, — это массив байтов со значениями 0–8 (число битов в байте). Можно упаковать полученные значения по два в байт.

Предположим, что в соседних байтах содержатся следующие значения по битам:

```
x x x x 0 1 1 0      x x x x 1 0 0 0
```

Эти числа 6 и 8. Старшие тетрады, помеченные знаком *x*, интереса не представляют (там всегда нули). Результат упаковки должен выглядеть так:

```
0 1 1 0 1 0 0 0
```

Покажем этапы упаковки. Предположим, первый код (число 6) записан в регистр *dl*, а второй (число 8) — в *al*. Результат упаковки формируется в *al*.

Листинг 7.9. Упаковка тетрад байтовых регистров (*pack_1.8*)

```
;          dl                      al
; x x x x 0 1 1 0          x x x x 1 0 0 0

                                shl    al, 4
; x x x x 0 1 1 0          1 0 0 0 x x x x

    shr    dl, 1
; x x x x x 0 1 1 -> c(0)    1 0 0 0 x x x x

                                rcr    al, 1
; x x x x x 0 1 1          c -> 0 1 0 0 0 x x x

    shr    dl, 1
; x x x x x x 0 1 -> c(1)    0 1 0 0 0 x x x

                                rcr    al, 1
; x x x x x x 0 1          c -> 1 0 1 0 0 0 x x

    shr    dl, 1
; x x x x x x x 0 -> c(1)    1 0 1 0 0 0 x x

                                rcr    al, 1
; x x x x x x x 0          c -> 1 1 0 1 0 0 0 x

    shr    dl, 1
; x x x x x x x x -> c(0)    1 1 0 1 0 0 0 x

                                rcr    al, 1
; x x x x x x x x          c -> 0 1 1 0 1 0 0 0
```

Биты из *dl* передаются в *al* по одному через флаг *c*. (Конечно, четырехкратный повтор команд *shr* и *rcr* можно оформить в виде счетного цикла или макрокомандой, как в *pack_1b.8*.) Проверьте работу *pack_1.8* в отладчике. Для поразрядного отображения регистров *al* и *dl* используйте форматы *=el,ax* и *=el,dx*.

Если байты находятся в одном слове, проще сдвинуть все слово сразу на 4 разряда. Как в первом варианте упаковки, сначала нужно придвинуть второй код к левой границе байта. Пусть второй байт находится в *al*, а первый — в *ah*.

Листинг 7.10. Упаковка младших тетрад регистров *ah* и *al* (pack_2.8)

```

;          ah          al
; x x x x 0 1 1 0 x x x x 1 0 0 0

          shl    al, 4
; x x x x 0 1 1 0 1 0 0 0 x x x x

          shr    ax, 4
; x x x x x x x x 0 1 1 0 1 0 0 0

```

Составьте программу для сжатия 4-битовых данных, прочитанных из входного потока, взяв за основу программы `pack`.

Задания на составление программ

Далее приведены две группы заданий. В первой группе — тривиальные задачи на последовательную обработку одномерных массивов, во второй — чуть сложнее.

Задания первого уровня сложности

Выполните выбранное задание в двух вариантах.

- Данные исходного массива или массивов определены в самой программе. Это отладочный вариант. Ввод-вывод программировать не надо, достаточно того, что программа работает в отладчике и в окнах отображения памяти видны исходные данные и результаты. Формат отображения следует выбрать исходя из данных: массив кодов ASCII отображается в формате `r`; массив слов — в формате `w` или `d`; массив байтов — в формате `b` или `n`; а если речь о битовых операциях, то предпочтительнее формат `e`. После настройки отображения данных сохраните нажатия клавиш, выполнив команду: `k s alta <Enter>`. При каждом входе в `d86` вы можете, нажав `<Alt>+<A>`, вернуться к сохраненной настройке, а если дать файлу `alta.d8k` имя `com`-файла, то нажатия будут обрабатываться автоматически.
- Данные массивов считываются из файла через стандартный входной поток, а результаты записываются в другой файл через стандартный выходной поток. Если по условию задачи на входе задана пара массивов одинаковой длины, разделите прочитанное пополам, первая половина относится к первому массиву, вторая — ко второму. Размер прочитанного известен: он записан в регистр `ax` после выполнения `dos 03f`; остается разделить его надвое (сдвиг вправо¹⁰) и использовать как смещение от начала `buf`.

При отладке вызывайте `d86` так: `d86 x.com <test_1.bin >out.dat`.

Варианты заданий

- Найдите максимум в массиве слов, а затем замените элементы массива с нулевыми значениями на этот максимум. (В отладчике отобразите исходный массив, значение максимума и результирующий массив).

2. Задан массив ASCII-кодов. Подсчитайте число двойных символов (например, "tutti buzz" содержит две пары). (В отладчике отобразите исходный массив и результат подсчета; при выполнении второго варианта в выходной файл запишите слово, содержащее результат подсчета.)
3. Вычислите сумму (в формате *слова*) значений массива байтов. По сумме вычислите среднее значение в формате байта. (В отладчике отобразите исходный массив и значение среднего, при выполнении второго варианта в выходной файл запишите слово, содержащее результат подсчета среднего.)
4. Задан массив ASCII-кодов. Подсчитайте количество заглавных латинских букв. (В отладчике отобразите исходный массив и результат подсчета, при выполнении второго варианта в выходной файл запишите слово, содержащее результат подсчета.)
5. Задан массив, содержащий знаковые слова. Сформируйте на его месте массив с абсолютными значениями исходного массива. Предусмотрите проверку переполнения при выполнении команды `neg`. (В отладчике отобразите исходный и результирующий массивы, в выходной файл при выполнении второго варианта запишите результирующий массив.)
6. Задан массив ASCII-кодов. Подсчитайте количество латинских букв и отдельно количество литер десятичных цифр. (В отладчике отобразите исходный массив и результат подсчета, при выполнении второго варианта в выходной файл запишите полученные счетчики в формате слов.)
7. Заданы два массива байтов одинаковой длины. Сформируйте массив на месте первого, содержащий значения попарно совпадающих элементов исходных массивов. (В отладчике отобразите исходные массивы, результирующий массив и число совпадающих элементов, при выполнении второго варианта в выходной файл запишите результирующий массив.)
8. Задан массив ASCII-кодов. Сформируйте по его содержимому массив ASCII-кодов в обратной последовательности. (В отладчике отобразите исходный и результирующий массивы, при выполнении второго варианта в выходной файл запишите результирующий массив.)
9. Заданы два массива байтов одинаковой длины. Сформируйте массив на месте первого массива, содержащий индексы попарно совпадающих элементов исходных массивов (отсчет индексов — от нуля). (В отладчике отобразите исходные массивы, результирующий массив и число совпадающих элементов, при выполнении второго варианта в выходной файл запишите результирующий массив.)
10. Задан массив знаковых байтов. Сформируйте массив, содержащий индексы тех элементов исходного массива, значения которых находятся в заданном диапазоне (отсчет индексов — от нуля). (В отладчике отобразите исходный массив, результирующий массив и число элементов в нем, при выполнении второго варианта в выходной файл запишите результирующий массив.)
11. Задан массив ASCII-кодов. Сформируйте на его месте массив ASCII-кодов, в котором все латинские буквы из исходного массива преобразованы к верхне-

му регистру. (В отладчике отобразите исходный и результирующий массивы, при выполнении второго варианта в выходной файл запишите результирующий массив.)

12. Задан массив беззнаковых слов. Сформируйте на его месте массив, содержащий значения тех элементов исходного массива, которые находятся вне заданного диапазона. (В отладчике отобразите исходный массив, результирующий массив и число элементов в нем, при выполнении второго варианта в выходной файл запишите результирующий массив.)
13. Заданы два массива знаковых слов одинаковой длины. Запишите на месте первого массива результаты поэлементного умножения исходных массивов, предусмотрите проверку переполнения при умножении. (В отладчике отобразите исходные массивы, результирующий массив, при выполнении второго варианта в выходной файл запишите результирующий массив.)
14. Задан массив ASCII-кодов. Сформируйте на его месте массив ASCII-кодов, в котором все латинские буквы преобразованы к нижнему регистру. (В отладчике отобразите исходный и результирующий массивы, при выполнении второго варианта в выходной файл запишите результирующий массив.)
15. Заданы два массива беззнаковых слов одинаковой длины. Сформируйте на месте этих массивов результаты поэлементного деления исходных массивов (целую часть результатов сохраните на месте первого массива, а остатки — на месте второго). (В отладчике отобразите исходные и результирующие массивы, при выполнении второго варианта в выходной файл запишите массив остатков.)

ВНИМАНИЕ!

В задачах, где есть сложение (2–4 и 6), необходимо предусмотреть проверку переполнения. Там, где задан массив слов (1, 5, 12) или два массива байтов (7, 9), при чтении данных из входного потока нужно проверять счетчик прочитанных байтов на четность. Аналогично, если задано два массива слов (13, 15), значение счетчика байтов должно делиться нацело на четыре — для получения счетчика циклов.

Задания второго уровня сложности

1. Задан массив ASCII-кодов. Сформируйте гистограмму цифр в виде массива из десяти слов, так что значение его нулевого элемента равно числу литер '0' в исходном массиве, значение первого элемента — числу литер '1', и т. д. до '9'. Входные данные возьмите из файла test1.bin, получив его из test1.8. Результат должен быть следующим: 2 6 11 19 26 39 50 67 83 102.
2. Подсчитайте число битов с единичным значением в битовом массиве, счет ведите в двойном слове с использованием команд add и adc. Входные данные возьмите из файла test2.bin, получив его из test2.8. Результат счета должен быть 016000.
3. Получите из двухбитовых кодов Грея трехбитовые, из них — четырехбитовые и т. д. до восьмибитовых. Промежуточные результаты формируйте в области

данных размером 256 байтов (изначально первые четыре байта этой области содержат двухбитовые коды); там же в итоге окажется и окончательный результат.

Как получить коды Грея для $n+1$ бит исходя из кодировки для n бит? Обозначим $G(i)$ набор кодов для i бит. Первая половина $G(n+1)$ — это набор $G(n)$, каждый код которого дополнен слева битом 0. Вторая половина $G(n+1)$ — это коды из $G(n)$ в обратном порядке, дополненные слева битом 1. Пример получения $G(3)$ из $G(2)$:

00 01 11 10	$G(2)$
000 001 011 010	первая половина $G(3)$
10 11 01 00	$G(2)$ в обратном порядке
110 111 101 100	вторая половина $G(3)$
000 001 011 010 110 111 101 100	$G(3)$ целиком

Результат запишите в файл, его содержимое должно совпасть с файлом `test3.bin`.

4. Упаковка 6-битовых кодов.

Задан массив байтов, длина его кратна 4. В каждом байте имеют значение только младшие шесть битов. Требуется "сжать" последовательность битов так, чтобы ликвидировать незначащие старшие биты с сохранением исходного порядка битов.

Далее проиллюстрировано получение одной "упаковки", где в четырех строках показаны:

- номера байтов от 0 до 3;
- номера разрядов в байтах;
- четыре исходных байта (символами +, −, *, # отмечены значащие разряды, символом вопроса — незначащие);
- три байта результата.

;	0	1	2	3
;	0 1 2 3 4 5 6 7	0 1 2 3 4 5 6 7	0 1 2 3 4 5 6 7	0 1 2 3 4 5 6 7
;	+++++??	-----??	*****??	#####??
;	+++++--	-----*	* * # # # # #	

Обратите внимание, что здесь разряды нумеруются слева направо — не так, как обычно принято. Такая нумерация отражает смысл задачи, но создает путаницу: как теперь понимать сдвиги влево (`shl`) и вправо (`shr`)? Ответ такой: буква `l` (left) в названии команды сдвига или вращения всегда означает сдвиг в сторону *старших* битов, а буква `r` — в сторону *младших*.

Входные данные возьмите из файла `test4.in` (6-битовые коды Грея), результат запишите в файл и сравните его содержимое с `test4.out`.

5. Задана последовательность байтов. В каждом байте выделите биты с номерами от 0 до 5, а в шестом бите сформируйте дополнение до четности. Полученные семибитовые коды требуется упаковать так, чтобы порядок битов не изменился,

по аналогии с заданием 4. Восемь кодов упаковываются в семь байтов. Число байтов во входной последовательности должно быть кратно 8.

Входные данные возьмите из test5.in (6-битовые коды Грея), результат запишите в файл и сравните его содержимое с test5.out.

6. Задана последовательность ASCII-кодов. Представьте ее в коде RADIX-50¹¹. Три литеры исходной последовательности перекодируются по табл. 7.1. Тройка полученных кодов c_1, c_2, c_3 упаковывается в 16-битовое слово по схеме $(c_1 \times 50xq + c_2) \times 50xq + c_3$. Например, коды для литер 'ABC' — это 1, 2, 3. В результате упаковки получаем $(1 * 50xq + 2) * 50xq + 3 = 3223xq$. Процесс повторяется для следующих трех литер; и так до конца входной строки, длина которой должна быть кратна трем.

Таблица 7.1. Коды RADIX-50

Литера	Код RADIX-50
Пробел	0
A–Z	1 – 32xq
\$	33xq
.	34xq
(резерв)	35xq
0–9	36xq – 47xq

Литеры, не принадлежащие множеству 'A'–'Z', ' ', '\$', '0'–'9', кодируются числом 35. Например, коды для последовательности литер 'AB', 0 — это 1, 2, 35xq, а результат упаковки равен 3255xq. Еще один пример: код RADIX-50 для последовательности литер 'X2B' составляет 115402xq.

Входные данные возьмите из test6.in, результат запишите в файл и сравните его содержимое с test6.out.

7. Задана последовательность беззнаковых байтов **A**. Сформируйте сглаженную последовательность **B** по формуле

$$B(i) = A(i-1)/4 + A(i)/2 + A(i+1)/4.$$

Обратите внимание, что **B** на два элемента короче, чем **A**. Для уменьшения погрешностей округления и сокращения числа делений (или сдвигов вправо) используйте следующий вариант формулы:

$$B(i) = ((A(i-1) + A(i+1))/2 + A(i))/2.$$

Чтобы уменьшить вероятность переполнения при сложении, вычисления выполняйте над словами, но контролировать переполнение все равно необходимо.

Входные данные возьмите из файла test7.in, запишите результат в файл и сравните его содержимое с test7.out.

12. Дана последовательность ASCII-кодов. Требуется преобразовать литеры в четырехбитовые коды в соответствии с табл. 7.2. Литеры не из множества '0'–'9', '-', '.' считаются разделителями и кодируются значением 0с. Разделители, следующие подряд, объединяются — тогда вся их последовательность шифруется одним кодом 0с. Конец последовательности обозначается как 0f.

Таблица 7.2. Таблица перекодировки к заданию 12

ASCII-код	4-битовый код
'0'–'9'	0–9
'-'	0a
'.'	0b

Пример с данными из файла test12.bin: –3.14 ,! ? # , , , 16. Эта последовательность ASCII-кодов преобразуется в 4-битовые коды 0a 3 0b 1 4 0с 1 6 0f. Они, в свою очередь, упаковываются по два в байт (в младшей тетраде — первый код из пары, в старшей — второй). Результат упаковки: 03a 01b 0с4 061 0f.

Примечания

- 1. Процессоры от i80386 предоставляют дополнительные инструкции для доступа к биту слова или двойного слова. В них задается адрес данных и номер бита — либо константой, либо содержимым регистра. Это bt (Bit Test), btc (Bit Test & Complement), bts (Bit Test & Set) и btr (Bit Test & Reset). Также имеются инструкции поиска ближайшего бита, установленного в единицу: bsf (Bit Scan Forward) и bsr (Bit Scan Reverse).
- 2. Функции для открытия и создания файлов мы не рассматриваем, т. к. для выполнения заданий достаточно стандартных потоков.
- 3. Означает, что результат записывается на место исходных данных. (Излюбленная фраза Н. Вирта.)
- 4. В а86 директива определения данных может не задавать ни одного байта (список значений пустой).
- 5. Точнее, 0fffe – 0200 – buf (0fffe — начальное значение sp, а 0200 — место под стек, с запасом).
- 6. Понадобилось бы три просмотра: на первом определяются имена и их типы, на втором — числовые значения имен, а также распределяется память, а на третьем память заполняется конкретными значениями. Но в а86 используется классическая двухпроходная схема.
- 7. До появления i80386 избыточность могла возникнуть только при кодировании jmp, т. к. лишь эта команда допускала варианты short и long. Но в i80386 и выше все переходы, в том числе условные, могут быть и короткими, и длинными, так

что выиграш от применения локальных меток в а386 (32-битовая версия а86) еще больший.

8. Эффект от локальных имен а86 двойной. Во-первых, переход на локальное имя — короткий, даже без атрибута `short`. Во-вторых, поскольку эти имена допускают переопределение, их можно применять в макрокомандах с циклами. Объявление `local`, принятое в других макроассемблерах, в а86 упразднено.
9. При просмотре некоторых программ в отладчике эти многочисленные `por` видны. Они означают, что здесь поработал `masm/tasm`, а данные определены после кода.
10. Если в результате сдвига $c = 1$, значит, прочитано *нечетное* число байтов, и следует перейти к аварийному завершению программы, как в (9, 16) листинга 7.5. (Такой же исход будет, если на входе нечетное число байтов, а по заданию должен быть массив *слов*.)
11. Этот формат использовался для хранения имен файлов в операционной системе RT-11 для мини-ЭВМ PDP-11.

ГЛАВА 8

```
d86 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100  2: =d1,ax,, 1962
CX 0000  CS 6309  3:
DX 6309  SS 6309  4:
SI 011F  DS 6309  5: r5,b_1,, AB ab
DI 0000  ES 6309  6: r5,si,, AB ab
BP 0000  SP FFFE  0:

lea si,b_1
```

Математический сопроцессор

— Это совсем не то, что маленькие 10-амперные аппараты для невротиков. Это стационарная 25-амперная машина с большим запасом прочности, предназначенная для действительно тяжелых, застарелых случаев.
(Роберт Шекли. *Терапия*)

Однажды самогонный аппарат
Подумал так: "Я не персона грата,
Но все-таки, в известном смысле, брат
Фотографического аппарата.
Работать я могу день изо дня.
Я хорошо сработан, молод, емок,
Так почему ж не пользуют меня
Хотя бы для простых натурных съемок?"
(Николай Эрдман и Владимир Масс. *Басни*)

Для обозначения сопроцессора принята аббревиатура FPU (Floating Point Unit), Intel также использует название Numeric Processor eXtension, или NPX. Справочные сведения по FPU приведены в [1, глава 6].

Окно FPU в d86

Вызовите d86 и несколько раз нажмите клавишу <F10>. В правом верхнем углу экрана должна появиться область отображения **FPU**, его вид в исходном состоянии показан на рис. 8.1. В строках 0–7, а также в строке **Tags** — прочерки, это значит, что регистры 0–7 пусты.

Такой вид у окна **FPU** будет при запуске d86 сразу после загрузки FreeDOS на PC. Однако при запуске d86 в сеансе Bochs-FreeDOS во всех регистрах будет ноль, а в строке **Tags**, которая отражает состояние регистров, — буквы **z** (Zero). Для очистки регистров выполните команду инициализации **fini**, и окно будет выглядеть так, как на рис. 8.1. Назначение других полей, отражающих состояние FPU, выяснится в дальнейшем.

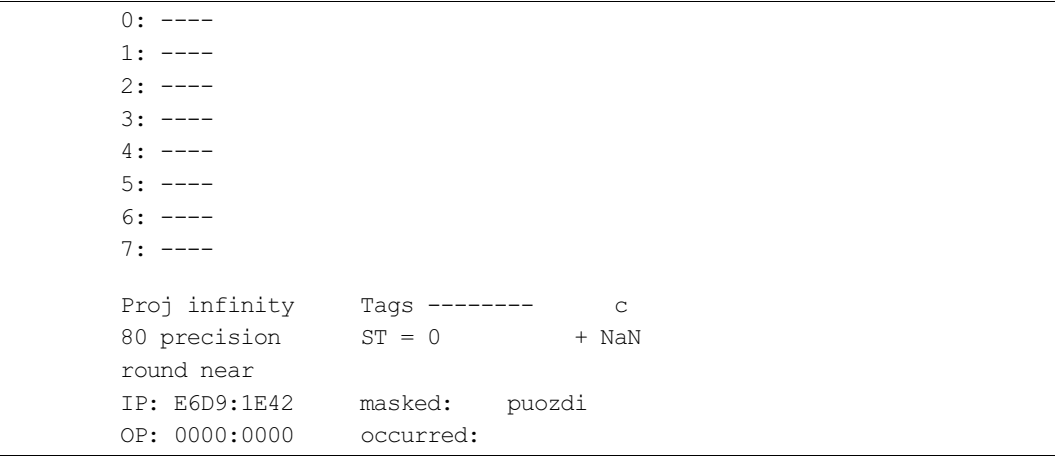


Рис. 8.1. Окно FPU в исходном состоянии

Загрузка и выгрузка данных

Выполните команды `fld 1.0` (или специальную команду `fld1` для загрузки 1.0) и `fld 2.0`. В результате наверху массива регистров оказалось число 2.0, под ним — 1.0, `ST` (указатель стека сопроцессора) изменил значение, а в строке **Tags** появились буквы `f`, которые обозначают нормализованные числа с плавающей точкой.

Выполните также команды `fld 3.0` и `fld 2` — без точки. Эти команды совершенно разные и обе уникальны для `a86`. Команда для загрузки произвольной константы вроде 3.0 не имеет машинного представления, это встроенная макрокоманда `a86`. А целое число `n = 0-7` в команде `fld` — это сокращение общепринятого обозначения `st(n)`. В результате выполнения команды `fld 2` мы получили на вершине стека FPU копию регистра 2.

Для дальнейших опытов выполните трансляцию программы `test1.8` из листинга 8.1 и зайдите с этой программой в отладчик.

Листинг 8.1. Данные для исследования команд загрузки-выгрузки (`test1.8`)

```
...
d86_buffer:                                ; (3)
        db      20 dup 090                 ; (4)

xq      dq      ?                          ; (5)
xd      dd      ?                          ; (6)
xt      dt      ?                          ; (7)

i16     dw                               ; (8)
i32     dd                               ; (9)
i64     dq      7                          ; (10)
```



```
bcd      dt      01962      ; (11)
```

```
...
```

В программе заданы: буфер трансляции (3–4) для режима непосредственного выполнения d86, три переменные (5–7) для хранения данных в форматах с плавающей точкой (8-байтовая, 4-байтовая и 10-байтовая), три переменные (8–10) для хранения данных целочисленных форматов разной размерности, переменная для хранения 10-байтовой BCD-переменной. Все эти форматы FPU поддерживает при обмене между системной памятью и своими регистрами.

Выполните команды `finit` и `fld 1.0`. По метке `d86_buffer` виден результат трансляции в режиме непосредственного выполнения, и команда `fld 1.0` выглядит как загрузка данных из памяти по адресу `0107`, а затем обход этих данных, чтобы прийти к точке останова по адресу `0111`. Данные при трансляции были определены как 10-байтовые (что соответствует надписи **80 precision** в окне **FPU**) директивой `dt` (`define ten bytes`). Их значение показано в строках отображения памяти — в формате с плавающей точкой, а также в виде последовательности байтов. Машинное представление мы рассмотрим чуть позже, а пока остановимся на особенностях загрузки данных и выполнения операций.

Выполните загрузку чисел 2.0 и 3.0. При каждой загрузке новое число оказывается на вершине стека FPU — в регистре 0, сдвигая прежде загруженные значения вниз, в регистры с большими номерами. Это иллюзия стека, поскольку все ранее загруженные значения остаются на своих местах, в тех же физических регистрах. Изменяется только число в регистре `ST` (по кольцу в пределах 0–7); от величины `ST` и отсчитываются номера регистров. Используя для повтора клавишу `<F3>`, выполните восемь раз команду `fdecstp`, потом `fincstp`. Меняя таким способом величину `ST`, мы прокрутили по кольцу данные FPU. При этом данные остались в тех же физических регистрах, строка **Tags** не изменялась.

В итоге на вершине стека FPU оказались числа 3.0, 2.0 и 1.0. Выполните команду `fchs` — это смена знака числа на вершине стека. Затем выполните двуместную операцию сложения: `fadd`. На вершине стека — результат сложения двух верхних значений (т. е. регистров 0 и 1). Было два значения, а осталось одно, и при этом нижние регистры, начиная с третьего, переместились вверх, примыкая к результату на вершине стека. Произошла, так сказать, "свертка" стека FPU.

Если повторить команду сложения, то следующим результатом будет ноль, а если выполнить ее еще раз, то получится специальное значение `NaN` (`Not a Number`), которое говорит о серьезной ошибке в данных FPU. В окне **FPU** появилась буква `i` (`Invalid`) — в строке **Tags** она означает неправильное число, а в строке **occured** — событие `Invalid operation`. Выполните команду `fchs`. Как видите, операции с `NaN` дают все тот же `NaN`; а то, что знак `NaN` может быть разным, несущественно.

Освободите регистр 0 командой `ffree 0`. Событие `i` (`Invalid operation`) сохраняется, пока мы не выполним полный сброс `finit` или не очистим события командой `fclex` (`FPU CLEAr EXceptions`); проверьте.

Вновь загрузите те же числа 1.0, 2.0 и 3.0 в FPU. Выясним, в каком порядке расположены операнды двуместной операции. Выполните вычитание: `fsub`. Отрицатель-

ное число на вершине стека говорит о том, что уменьшаемое (первый операнд) находилось в регистре 1, а в регистре 0 было вычитаемое (второй операнд). Так что двуместная операция `opr` выполняется по схеме `st(1) opr st(0) -> st(0)`. Можно считать иначе, как в [1, разд. 6], что операция выполняется по схеме `st(1) opr st(0) -> st(1)`, с последующим выталкиванием из стека (`pop`) регистра 0. Тогда схема выполнения такая же, как в центральном процессоре: есть операнд-приемник и операнд-источник с фиксированным положением, в FPU приемник — это `st(1)`, а источник — `st(0)`, вот только после выполнения операции источник, ненужный больше, исчезает.

Загрузите число Пифагора командой `fldpi` и выполните сначала умножение `fmul`, а потом деление `fdiv`. Результат дробный и отрицательный (делили -1 на π), а буква **p** (Precision lost), появившаяся в строке **occured** после деления, означает потерю значащих цифр при *сохранении* результата.

Как передать значение из FPU в основную память? Выполните вручную команды `fst xd`, `fst xq` и `fst xt`. (Аббревиатуры `fld` и `fst` означают соответственно FPU Load и FPU Store.) Последняя команда существует только в варианте `fstp`, где суффикс `p` означает "вытолкнуть с вершины стека". Переменные `xd`, `xq` и `xt`, определенные в `test1.8`, имеют разную размерность: двойное (32-битовое) слово для хранения *real*-числа одинарной точности (то, что в языке C обозначается как `float`); 8-байтовое (64-битовое) слово для чисел двойной точности (`double`) и нестандартное 10-байтовое (80-битовое) слово, которое используется только в FPU (и для хранения *промежуточных* результатов в основной памяти). Значения переменных `xd`, `xq` и `xt`, отображаемые в `d86`, могут не совпадать, т. к. точность представления дробной части у них разная.

Точно так же, указывая переменную `xd` или `xq` или `xt` в команде `fld`, мы можем загрузить числа в FPU из основной памяти. Проверьте загрузку и выясните разницу между этими значениями, применив команду `fsub` и, возможно, команду `fld 0` для дублирования значения на вершине стека¹.

Порядок двуместных операций

Предположим, требуется разделить 1 на 3. Что загружать сначала — 1 или 3? Первый операнд (число 1.0) должен быть в регистре 1, поэтому следует начать с него. Порядок действий таков: загрузка 1.0, загрузка 3.0 и деление. Требуемая арифметическая операция обозначается как $1 / 3$. Это привычная инфиксная нотация, знак операции стоит между операндами. Порядок действий в FPU задает другая нотация — постфиксная², в ней знак операции следует за операндами: $1\ 3\ /$.

Преобразовав вычисляемое выражение в обратную польскую запись, мы получим последовательность действий для классической стековой машины, каковой является FPU. Вот пример преобразования (в квадратных скобках записаны уже преобразованные фрагменты):

$((a * a) - (b * b)) * \pi$

$[[aa*] - [bb*]] * \pi$

$$[aa*bb*-]\pi$$

$$[aa*bb*-\pi*]$$

Буква или цифра означает загрузку (fld), а знаки арифметических операций (*, /, -, +) — это команды fmul, fdiv, fsub, fadd.

Проверим вычисления для значений $a = 3$ и $b = 2$; результат $5 \cdot \pi$ должен быть примерно равен 15,7. Но сначала, оставаясь в отладчике с программой test1.com, проверим обмен целочисленными данными между системной памятью и FPU.

Переменные (8–10) имеют разную размерность и отображены на одну и ту же память со значением 7 в младшем байте, т. к. в (8–9) список значений пустой. Выполните три команды fld (FPU Integer Load), указав ей эти переменные. Проинвертируйте число в регистре 0 (fchs), в свободной строке отображения данных задайте b8,i64 и выполните fist (FPU Integer Store), указав последовательно имена (8–10). (Для 64-битового целого есть только вариант fistp.)

Загрузите в FPU число 3.4 и сохраните его в памяти как целое — командой fist. По умолчанию установлен режим округления к ближайшему целому (в окне состояния — надпись **round near**). Поэтому в основной памяти получим 3. Повторите опыт с числами -3.4, 3.5, -3.5, -2.5, используя для сохранения команду fist i16 и, в случае отрицательного числа, инвертируя результат командой neg i16, чтобы выяснить абсолютное значение. Округление происходит в сторону ближайшего целого, но для половинки выбор делается в пользу ближайшего четного: 3,5 → 4.0, -3,5 → -4.0, -2,5 → -2.

В листинге 8.2 приведена программа вычисления $((a*a)-(b*b))*\pi$.

Листинг 8.2. Вычисление выражения $[aa*bb*-\pi*]$ (calc_1.8)

```

        jmp start                                ; (1)

a16     dw   3                                  ; (2)
b16     dw   2                                  ; (3)
c64     dq   ?                                  ; (4)

start:                                       ; (5)
        finit                                    ; (6)
        fld   a16                                ; a          ; (7)
        fld   a16                                ; a          ; (8)
        fmul                                     ; *          ; (9)
        fld   b16                                ; b          ; (10)
        fld   b16                                ; b          ; (11)
        fmul                                     ; *          ; (12)
        fsub                                     ; -          ; (13)
        fldpi                                    ; pi         ; (14)
        fmul                                     ; *          ; (15)
        fstp  c64                                ; (16)
        int   020                                ; (17)

```

Входные значения (2–3) определены в памяти как целые 16-битовые, а результат сохраняется в переменной (4) в формате с плавающей точкой³.

Интересно, что, кодируя выражение в постфиксной записи, нам даже не нужно знать, в каких регистрах окажутся входные данные и промежуточные результаты. Единственный вопрос — хватит ли восьми регистров FPU? В этом выражении пять значений, так что восьми регистров хватит с избытком. Вообще-то и пяти регистров не понадобится. Сколько их потребуется на каждом шаге, можно выяснить, открывая символ за символом выражение $aa*bb*-π^*$:

a (1), aa (2), aa^* ($2 - 1 = 1$), $aa*b$ ($3 - 1 = 2$), $aa*bb$ ($4 - 1 = 3$),

$aa*bb^*$ ($4 - 2 = 2$), $aa*bb^*-$ ($4 - 3 = 1$),

$aa*bb^*-π$ ($5 - 3 = 2$), $aa*bb^*-π^*$ ($5 - 4 = 1$).

Как вы, наверное, заметили, я вычитал число операций из числа букв. Максимум занятых регистров — три, после выполнения команды (11).

Проверьте программу из листинга 8.2 в отладчике. Вместо повторных загрузок (7–8) и (10, 11) одного и того же числа можно дублировать значение на вершине стека командой `fld 0`, проверьте этот вариант. Еще проще — воспользоваться командой возведения в квадрат `fsqr`, проверьте.

Для преобразования выражения в постфиксную запись воспользуемся программой (в сеансе Bochs-FreeDOS) `postfix.exe`. После приглашения `'>'` введите выражение без пробелов⁴ с *однобуквенными* именами и/или числами, состоящими из одной цифры, например:

```
(a*a-b*b)*p
```

Уже в процессе ввода будет выводиться фрагмент в обратной польской записи, а при завершении ввода по `<Enter>` — выражение целиком. Затем при нажатии любой клавиши экран очищается. Результат должен получиться следующим:

```
aa*bb*-p*
```

При помощи `postfix` напишите программу для вычисления сопротивления электрической цепи при параллельном соединении сопротивлений r_1 , r_2 , r_3 . Для загрузки единицы есть специальная команда `fld1`.

Проверьте сначала для значений $r_1 = 19$, $r_2 = 23$ и $r_3 = 28$ — должно получиться 8, после округления до целого. Затем задайте $r_1 = 0$ и пройдите программу по шагам. После деления на ноль флаг сопроцессора `z = 1` (by Zero division), а в регистре 0 — значение `+Infinity` (плюс бесконечность)⁵. При дальнейшем сложении конечных значений `c +Infinity` результат по-прежнему `+Infinity`, но после итогового деления `1.0/+Infinity` получается правильное значение `0.0`.

Выясните, каков результат сохранения `+Infinity` и `-Infinity` в 16-битовых целочисленных переменных.

Составьте программы для вычисления выражений⁶:

- 1. $(x - (12 + y))/100 + 2$, при $x = -125897, y = 084512$.
- 2. $(1/2 + (1/3 + 1/4)*7)/17$.
- 3. $1/(1+s/(1+s/(1+s)))$, где $s = \text{sqrt}(2)$.

Результаты для контроля:

- 1. $-2.102210\text{e}3$.
- 2. 0.26960784314 .
- 3. 0.52859547919 .

Организация ветвлений

Для организации ветвлений по результатам выполнения операций FPU используется такая последовательность действий:

- ❑ команда проверки `ftst` или сравнения `fcom` устанавливает битовые признаки `c3`, `c2` и `c0` в `sw` — слове состояния FPU;
- ❑ команда `fstsw ax` (FPU STore Status Word) копирует `sw` в регистр CPU `ax`;
- ❑ команда `sahf` (Store AH to Flags) помещает `ah` в регистр флагов CPU, причем перечисленные флаги FPU попадают во флаги `z` (Zero), `e` (Even parity) и `c` (Carry) CPU, как показано в табл. 8.1.

Таблица 8.1. Соответствие между `sw` (FPU) и `flags` (CPU)

sw	flags	Отображение в окне FPU
c0	c	C
c2	e	u (Unordered)
c3	z	Z

Флаг `c` определяет условие `b` (Below), флаг `z` — условие Zero (ноль или равенство). Необычно то, что флаг `e` (parity Even) в этом случае означает ошибку, когда числа в FPU несопоставимы⁷. В `d86` флаги FPU отображаются названиями соответствующих им флагов CPU в конце первой строки состояния, правее строки **Tags**.

Проверим. Вновь вызовите `d86` с программой `test1.com`. Выполните `finit`, а затем — команду проверки вершины стека на ноль — `ftst`. Поскольку стек пуст, то результат сравнения пустого регистра `0` с нулем окажется неопределенным и в окне FPU будет установлен флаг `u`. Выполните макрокоманду `flags` — в CPU будет установлен флаг `e`. Для переходов в начале кода заданы две метки с точками останова. Выполните `jpe m2` — переход на `m2` должен произойти, поскольку признак "parity even" установлен⁸.

Теперь выполните команду загрузки нуля `fldz` и повторите `ftst`. Справа от восьми признаков в строке **Tags** появилась буква `z` — флаг нулевого результата, а признак

ошибки `u` исчез. После вызова `flags` буква `z` появится среди флагов CPU, а буква `e` исчезнет.

Загрузите единицу командой `fild1` и вновь выполните `ftst` и `flags`. Результат должен быть ненулевой, флаг `z` = 0. С теми же значениями на вершине стека (1.0 и 0.0) выполните команду сравнения `fcom`. Никакие флаги не установлены, поскольку `st(0)` не меньше и не равно `st(1)`. Поменяйте местами эти значения командой `fxch` (FPU `eXCHange`) и повторите сравнение. В FPU установлен флаг `c`, т. к. `st(0) < st(1)`. При вызове `flags` флаг `c` = 1 будет скопирован из FPU в CPU, а дальше, видимо, должен последовать переход `jc`.

Напишите программу поиска заданного значения в массиве действительных чисел, используя команду `fcom` и макровывод `flags`. Суффикс `p` или `pp` в команде `fcom` означает, что после сравнения один или оба операнда выталкиваются из стека.

В процессорах P6 появилась команда `fcomi`, которая помещает результат сравнения сразу в регистр флагов CPU. Чтобы проверить ее, загрузите в FPU три величины, так чтобы `st(0)` = 0, `st(1)` = 3 и `st(2)` = 1. Сбросьте флаг переноса в CPU командой `cld`. Выполните `fcomi`, и в регистре флагов CPU получим `c` = 1. Результат верный, поскольку `0 < 3`. Затем загрузите ноль (`fildz`) и выполните `fcomip`. Результат `z` = 1, поскольку `0 = 0`. Стек остался без изменений, т. к. лишний ноль из него вытолкнут (действие суффикса `p` в команде).

Еще одна команда, появившаяся в процессорах P6, — `fcmov<c>` (FPU Conditional MOVe). Она копирует значение указанного регистра FPU в `st(0)` при условии `<c>`. Для ее проверки сначала выполните `stc` для установки флага переноса в CPU, а затем `fcmovc 2`. Попытка скопировать `st(2)` в `st(0)` не удалась, потому что, согласно [1, Table 6-13], вместо условия `c` здесь должно быть `b` (Below). Команда `fcmovb 2` действительно копирует 1.0 из `st(2)` на вершину стека.

Если теперь выполнить `fcomi 2` (сравнение `st(0)` с `st(2)`), то `z` = 1 и `c` = 0. Поскольку флаг нуля установлен, то вызов `fcmovz 1` скопирует регистр 1 в регистр 0.

Признаки в слове состояния

В слове состояния `sw` есть и другие признаки [1, Figure 6-3], которые можно анализировать после выполнения команды `fstsw ax` в регистре `ax`. Наибольший интерес представляют следующие:

- признаки ошибок, собранные под заголовком Exception Flags;
- суммарный признак Exception Status⁹, который отражает наличие хотя бы одной ошибки из перечня Exception Flags;
- признак ошибки стека Stack Fault, который всегда появляется вместе с флагом `IE` (Invalid-Operation Exception); при этом флаг `c1` = 1 означает переполнение регистрового стека FPU, а `c1` = 0 — антипереполнение.

Для проверки вызовите `d86` с программой `test2.com`, выполните инициализацию, а затем макрокоманду `fsw` (FPU Status Word), определенную в `test2.8`. Программа `test2.8` приведена в листинге 8.3.

Листинг 8.3. Данные и макросы для опытов с `sw` и `cw` (test2.8)

```

il6      dw      ?                      ; (1)
xd       dd      ?                      ; (2)
cw       dw      ?                      ; (3)

fsw      macro                                ; (4)
        push     ax                      ; (5)
        fstsw    ax                      ; (6)
        mov      bl, al                  ; bx - low sw    ; (7)
        mov      cl, ah                  ; cx - hi sw    ; (8)
        pop      ax                      ; (9)
#em                                             ; (10)

fcw      macro                                ; (11)
##if #1                                     ; (12)
        mov      cw, #1                  ; (13)
        fldcw    cw                      ; not "ax"!      ; (14)
##else                                       ; (15)
        fstcw    cw                      ; (16)
        mov      ax, cw                  ; ax - low cw    ; (17)
        mov      dl, ah                  ; dx - hi cw    ; (18)
##endif                                     ; (19)
#em                                             ; (20)

```

Макрокоманда (4) читает слово состояния FPU и помещает его младший байт в `bl`, а старший — в `cl`. Отображение данных в `d86` настроено так, что биты 0–7 слова `sw` видны в первой строке отображения памяти, а биты 8–15 — во второй¹⁰. Сейчас там нули.

Попытаемся получить все ошибки из [2, Figure 6-3]. Поделите в FPU 1.0 на 3.0. Из-за потери точности появится ошибка PE. Выполнив `fsw`, убеждаемся, что соответствующий бит 5 установлен в единицу. (Как ни странно, при этом `ES` = 0.) Поделив ненулевое число на ноль, получим результат `+Infinity`, а `fsw` покажет 1 в бите 2 — признак `ZE`.

События `DE` и `UE` можно получить, работая с очень малыми, на грани исчезновения (превращения в ноль), числами. Порядок 80-битовых чисел (64 бита для представления мантиссы), используемых в FPU для внутреннего представления данных, составляет ± 4932 [2, Table 6-5].

Получим сначала событие `DE`. Загрузите в FPU число 2.0, а потом $1.0\text{e-}4932$. Второе число уже денормализованное, поскольку оно меньше предела $3.37\text{e-}4932$. Но пока что `DE` = 0. Согласно [2, 6.3.3.2] `DE` := 1, если *операнд* денормализованный. Загрузите числа 0.01 и $1.0\text{e-}4931$, оба они нормализованные (второе — на грани). Выполните команды `fclex` (FPU `CLear` `EX`ceptions) и `fmul`. Хотя результат денормализованный, но сами операнды были в порядке — поэтому `DE` = 0, а признаки `PE` = `UE` = 1 свидетельствуют о потере точности.

Итак, в `st(0)` сейчас денормализованное число. Загрузите 100.0. Перед умножением выполните `fclex`, поскольку признаки ошибок автоматически не сбрасываются. Теперь, после умножения, ситуация обратная: результат нормализованный, а `DE = 1` говорит о том, что какой-то из операндов был денормализованным.

И еще один опыт по получению события UE. Будем уменьшать число на вершине стека, пока оно не превратится в ноль. Будем делить, причем командой `fdiv 1`, чтобы все регистры оставались на местах, и повторять деление клавишей <F3>. Событие UE наступит довольно скоро (`fsw` покажет единицу в бите 4), но задолго до обнуления результата.

Для получения события OE загрузите 2.0 и 1.0e4932 и выполните `fmul 1`. Результат сразу `+Infinity` с установкой признака `o`; вызов `fsw` показывает единицу в бите 3. (Как ни странно, но по-прежнему `ES = 0`.)

Чтобы получить ошибку IE, достаточно переполнить стек или выполнить операцию с пустым регистром 0 (антипереполнение стека). В этом случае также устанавливается признак ошибки стека¹¹ SE — либо переполнение (`C1 = 1`), либо антипереполнение (`C1 = 0`).

Начнем с антипереполнения. Выполните инициализацию и любую операцию над вершиной стека, например команду смены знака `fchs`. Поскольку `st(0)` пуст, то `IE = 1`, а `C1 = 0`. В окне **FPU** отладчика `d86` виден флаг `IE`, но значение `SF = 1` в нем никак не отображается. Значение `C1` отображается точкой там, где обычно видны флаги `z`, `u` и `c`. Сейчас точки нет, так что `C1 = 0`. Вызов `fsw` показывает единицы в битах 0 (`IE`), 6 (`SF`), а бит 9 (`C1`) сброшен.

Добьемся переполнения стека. Выполнив инициализацию FPU, повторяйте загрузку любого числа, пока в `st(0)` не появится NaN. В окне **FPU** виден флаг `IE = 1` и точка (`C1 = 1`). Вызов `fsw` показывает 1 в битах 0 (`IE`), 6 (`SF`) и 9 (`C1`).

Единица в бите `ES` (Exception Summary) так и не появилась. Скорее всего, признак `ES` уже не поддерживается, подобно признаку `B` (`FPU Busy`)¹².

Настройки FPU

Режимы FPU настраиваются записью в управляющее слово `cw` (Control Word) [2, Figure 6-4]. Например, чтобы разрешить прерывания по флагу `PE` (в слове состояния), нужно записать единицу в разряд `PM` управляющего слова¹³. Можно также задать режим округления `RC` (Rounding Control) и установить меньшую точность в поле `PC` (Precision Control).

Предварительно командой `fstcw` (FPU SToRe Control Word) читаем текущее значение `cw`, изменяем значения требуемых полей и записываем результат обратно командой `fldcw` (FPU LoAD Control Word).

Проверим сначала на примере поля `RC`. Зайдите в отладчик с программой `test2.com`. Выполните инициализацию и вызовите макрокоманды `fsw` и `fcw` (FPU Control

Word). Результат чтения `sw` — в первой паре строк отображения памяти, а `cw` показан в `ax` (037f) и по битам в строках 4–5. Единицы в битах 0–5 `cw` означают, что все прерывания замаскированы¹⁴. Код 11 в поле `pc` (биты 8–9) задает максимальную точность за счет использования 80-битового формата, а код 00 в поле `pc` (биты 10–11) — округление к ближайшему четному. Включим десятый бит, чтобы задать `RC = 01`, выполнив команды:

```
or ax, bit 10
fcw ax
```

Для проверки вызовите команду `fcw` без параметров.

В окне **FPU** надпись **round near** изменилась на **round -inf**. Округление теперь идет "вниз", и результат окажется меньше точного значения. Для проверки загрузите число 3.9 и сохраните его в основной памяти как целочисленное: `fist il6`. Результат виден в шестой строке отображения памяти.

В *приложении 3* приведена команда `fisttp` (FPU Integer Truncate-Store-Pop), которая записывает в память целое число, отбрасывая дробную часть независимо от текущего режима округления.

Уменьшать точность в поле `pc` (Precision Control) особого смысла не имеет и ничего не дает в смысле скорости вычислений. Эта настройка позволяет разве что оценить точность вычислительных устройств, работающих с *real*-данными меньшей размерности (32- и 64-битовыми)¹⁵.

Если уменьшить точность до 32 битов, то мы будем получать результат в FPU таким, каким он обычно становится после сохранения в 32-битовой переменной. Вызовите `d86` с программой `test2.com` и поделите 1 на 3. Выполните команду `fst xd`, тем самым сохранив результат в переменной `xd`, что отображается в третьей строке отладчика. Отметьте разницу между `xd` и числом на вершине стека. Выполните инициализацию и уменьшите точность, записав в поле `pc` (биты 8–9) код 00: `fcw 07f`. Строка **80 precision** в окне **FPU** сменилась надписью **32 precision**. Повторите деление и сравните значения `xd` и `st(0)`.

Итак, мы рассмотрели базовые арифметические операции в классической стековой машине, а также регистры состояния и управления FPU. Перед тем как приступить к более основательному изучению системы команд, остановимся на форматах данных с плавающей точкой и их машинном представлении.

Форматы действительных чисел

Регистр FPU вмещает 80-битовое число с плавающей точкой в так называемом *расширенном* формате, превышающем по точности и диапазону стандартные 32-битовые (тип `single` в языке C) и 64-битовые (`double`). Он применяется в FPU или во временных переменных программы, как `ext` в листинге 8.4.

Листинг 8.4. Действительные данные разного диапазона и точности (types.8)

```

        jmp      start                ; (1)

ext     dt      1.00001e-387         ; (2)
f64     dq      12.0                 ; (3)
f32     dd      12.0                 ; (4)

start:  finit                    ; (5)
        fld     ext                 ; (6)
        fst     f64                 ; (7)
        fclex                     ; (8)
        fst     f32                 ; (9)
        fstp    ext                 ; (10)
        int     020                 ; (11)

```

Для отображения 32-, 64- и 80-битовых данных с плавающей точкой в d86 есть форматы `fd`, `fq` и `ft`. Если после входа в отладчик нажать `<Alt>+<F10>`, то при дальнейшем нажатии клавиши от `<1>` до `<6>` появится перечень возможных форматов отображения. Выбрав опцию `f` (Float), получаем подсказку по форматам: `fd`, `fq` и `ft`.

Настройте в d86 отображение действительных переменных (2–4). Проверьте, во что превратится 80-битовое число (2) при его сохранении (7, 9) в переменных (3, 4) с меньшим диапазоном. Флаги FPU после выполнения (7) и (9) говорят и потере точности (`p`) и значащих разрядов (`u` — underflow)¹⁶, вследствие чего получен ноль.

Действительно, диапазон 64-битовых действительных чисел — примерно от $5e-324$ до $1.8e+308$, а 32-битовых — от $1.4e-45$ до $3.4e+38$ (для отрицательных значений то же, только с минусом в начале). Проверьте, задав в (1) число в 64-битовом диапазоне, но вне 32-битового, и пусть оно будет достаточно большим, чтобы при выполнении (9) получить событие `o` (overflow) и бесконечное значение `+Infinity`.

Внутренний формат данных FPU

Зайдите в отладчик с программой `inner.com`. В `inner.8` определены макрокоманды для наблюдения за данными FPU по битам. Макроопределения доступны отладчику d86 через `sym`-файл.

Выполните в непосредственном режиме команду `what 1.625` и нажмите `<Enter>`. По этой макрокоманде в FPU загружается указанное число, а затем оно копируется в расширенном формате в переменную. По значению ее битов формируется ASCII-строка с литерами '0' и '1'. Отображение настроено так, чтобы в окнах отладчика были видны фрагменты полученной ASCII-строки, соответствующие битовым полям числа в формате FPU. Вот, например, составные части числа 1.625:

```

r1,sign,, 0
r15,exp,, 011111111111111

```


превратится в ноль. Сравните минимальное денормализованное число (в третьей строке результатов) с пределом нормализованного числа в [1, Table 6-5]; разница ощутимая.

Ноль в бите 63 допустим лишь тогда, когда $\text{exp} = 0$. Если $\text{exp} <> 0$, то можно, уменьшая exp , сдвигать мантиссу влево, пытаясь нормализовать число. Поэтому числа с $\text{exp} <> 0$ и нулем в бите 63 названы *псевдо*-денормализованными. Сам FPU такие числа не порождает, но допускает их загрузку из внешней памяти. Выполните `bits 1, 63`. Это нормализованное число. А теперь попробуем его денормализовать, задав в виде `bits 2, 62`. Как видите, значение в стеке FPU то же, но оно помещено как `Unnormal (Unn\)`, и если выполнить команду `fadd`, получим уже `NaN`.

В массиве тегов все виды ненормализованных значений отмечены буквой `i` (`illegal`). Тем не менее операции с денормализованными значениями допустимы. Заметим, что ноль отмечается в тегах как `z` (`Zero`), хотя, по сути, это то же денормализованное число ($\text{exp} = 0$) с $\text{sgnd} = 0$.

Рассмотрим теперь специальные значения на противоположной, верхней границе диапазона:

```
bits 07fff, 63
bits 07fff, 62
```

Первое значение равно `+Infinity`, а второе — `NaN`. Видимо, значение $\text{exp} = 07fff$ зарезервировано для представления специальных величин. Вот пример числа, близкого к максимуму — с $\text{exp} = 07ffe$:

```
bits 07ffe, 63
```

Как выяснилось, величина exp для нормализованных значений находится в диапазоне `1...07ffe`. Соответственно значение порядка ограничено диапазоном `-03ffe...+03ffe`. Нулевое значение exp зарезервировано для задания близких к нулю денормализованных чисел, а $\text{exp} = 07fff$ представляет бесконечность.

Проверьте выполнение арифметических действий над денормализованными числами. После вычислений рекомендуется обновлять ASCII-отображение, вызывая `what` без параметров (в этом случае он показывает значение `st(0)`).

Выясните, что представляет собой по разрядам значение `NaN`.

Стандартные форматы вещественных чисел

Рассмотренное внутреннее представление данных в `i80x87` отличается от стандарта IEEE 7547, где допускаются только нормализованные числа (ноль — особый случай, и отмечается в тегах специальным образом). Единица перед двоичной точкой становится лишней, она уже не задается, а подразумевается. Поле `sgnd` содержит только биты *после* двоичной точки.

По стандарту данные в формате с плавающей точкой могут быть 32-битовыми (одинарная точность) или 64-битовыми (удвоенная точность) [1, Figure 6-8].

Данные этих типов в программе определяются при помощи директив `dd` и `dq`. Для загрузки в FPU и копирования из FPU используются инструкции `fld` и `fst(p)`.

Чтение из FPU в память сопровождается форматными преобразованиями: данные приводятся к диапазону 32- или 64-разрядных, с округлением.

Проведем опыты с 32-разрядным представлением вещественных чисел. Зайдите в отладчик с программой `float.com` и выполните:

```
what 1.625
```

Теперь в `sgnd` вместо 1101 записано значение 101, т. к. старшая единица, которая в расширенном формате стоит перед двоичной точкой, задана *неявно*. Еще выразительнее выглядит результат `what 1.0`: в поле `sgnd` одни нули — но это дробная часть, а в целой части подразумевается единица.

Выполните теперь:

```
what 1.0e-40
```

Результат выглядит как денормализованное число в расширенном формате. Но здесь все числа нормализованные, поэтому нулевое значение `exp` задает двоичный порядок `-07f`. Мантисса равна 1.0000001000101101100001. Значение в шестой строке отображения данных — это результат копирования (при выполнении `what`) значения из `st(0)` в 32-битовую переменную в памяти.

В стандартных форматах ноль, будучи ненормализованным числом, задается как специальное значение: его признаком является сочетание `exp = sgnd = 0`. Попросту ноль во всех форматах задается нулями во всех битах.

Максимальное значение `exp` в стандартных форматах по-прежнему зарезервировано для бесконечности. Выполните:

```
what 1.0e40
```

В поле `exp` все биты установлены в единицу — это бесконечность. В `st(0)` показано то же число 1.0e40, но при его сохранении в переменную `float` оно превратилось в `+Infinity`, т. к. оно слишком большое по меркам 32-битового представления. А что если загрузить его в `st(0)`? Проверьте: `fld float`. Как и следовало ожидать, бесконечность осталась бесконечностью.

Кодировка специальных значений приведена в [1, Table 6.8–10].

Операции FPU

Многие из операций нам уже знакомы, но сейчас мы рассмотрим их не в "художественном беспорядке", а по группам:

- ☐ пересылки, в том числе загрузка констант;
- ☐ арифметические операции;
- ☐ сравнения;
- ☐ трансцендентные операции;
- ☐ управляющие операции.

В классической стековой машине все действия выполняются над вершиной стека $st(0)$, и если операция двуместная, то в ней также участвует примыкающий к вершине операнд-приемник $st(1)$. Двуместная операция выполняется по схеме $st(1) := st(1) \text{ opr } st(0)$, затем $st(0)$ выталкивается из стека. То есть при выполнении операции приемник — это $st(1)$, источник — $st(0)$, но в конце $st(1)$ переходит на место $st(0)$.

Кроме этой схемы, характерной для классической стековой машины, в FPU имеются дополнительные возможности адресации операндов и размещения результатов:

- ❑ некоторые *одноместные* операции дают *два* результата — основной и побочный, в регистрах 0 и 1, сдвигая вниз остальные значения в стеке;
- ❑ во многих двуместных операциях возможно задание источника в памяти;
- ❑ многие двуместные операции (загрузка, выгрузка и четыре действия арифметики) позволяют адресовать данные в глубине стека;
- ❑ при выполнении некоторых двуместных операций операнды (и все остальные данные FPU) остаются на месте;
- ❑ введены операции с обратным порядком операндов, по схеме $st(0) \text{ opr } st(1)$; в их обозначении есть буква *r* (Reverse).
- ❑ Дополнительная информация по командам FPU есть в [1, 6.4], а сводка команд приведена в документации по *a86*, на стр. 45–48.

Пересылки

Инструкции пересылок включают в себя:

- ❑ загрузку данных $f(i/b)ld$;
- ❑ выгрузку данных $f(i/b)st(p)$;
- ❑ обмен данных в стеке $fxch$.

Загрузка данных

Загрузка данных выполняется командой:

$f(i/b)ld \text{ <src>}$

Источник <src> — это либо адрес переменной в памяти типа $w/d/q/t$, либо номер регистра FPU (число от 0 до 7). Заметим, что обозначение регистра FPU числом $0...7$ принято только в *a86/a386*, а в других ассемблерах он обозначается как $st(0)...st(7)$. Команда с опцией *i* ($fild$), предназначенная для загрузки целочисленного значения, допускает только данные в памяти типа $w/d/q$. Команда с опцией *b* ($fbld$) для загрузки BCD-числа допускает только данные в памяти типа *t*. Команда без опции ($fild$) для загрузки действительного числа допускает данные в памяти типа $d/q/t$ и номера регистров FPU.

Поскольку в директиве dd/dq может быть задано и действительное, и целое число, можно по ошибке загрузить целое число командой $fild$ (и в $st(0)$ будет число, близ-

кое к нулю), а действительное — командой `fild` (в `st(0)` окажется огромное число), как в листинге 8.5.

Листинг 8.5. Ошибочная загрузка чисел не того типа (`miss.8`)

```

        jmp     start

dozen_f dd    12.0
dozen_i dd    12

start:
        fild   dozen_f
        fld    dozen_i

        int    020
```

Проверьте программу `miss.com` в `d86`. Первая загрузка даст неожиданно большое число, а вторая — число, близкое к нулю.

Встроенная макрокоманда `fld` в `a86` позволяет задавать в качестве операнда непосредственное значение с плавающей точкой, например `fld 5.1`. Макровывоз транслируется примерно так²⁰:

```

        cs fld  t >m1
        jmp    >m2
m1      dt     5.1
m2:
```

Мы уже пользовались этой возможностью в режиме непосредственного выполнения `d86`. Но включать эти макросы в *программу* не рекомендуется. Они дают громоздкий код, где команды и данные перемешаны, что затрудняет отладку в `d86`. В работающей программе константу следует задавать значением переменной²¹.

Теперь продемонстрируем загрузку значения из регистра `FPU`, когда источник задан числом `0...7` (листинг 8.6). Обратите внимание, загрузка чисел `8.0` и `7.0` здесь задана именно так, как этого делать *не* следует. Почему — увидите, зайдя с этой программой в отладчик.

Листинг 8.6. Загрузка значений из регистров данных (`fld_1.8`)

```

fld     8.0      ; 8
fld     7.0      ; 7      8
           ;          |
           ; +-----+
           ; |
fld     1        ; 8      7      8
           ;          |
           ; +-----+
           ; |
```

```
fld    1      ; 7      8      7      8
        ; |
fld    0      ; 7      7      8      7      8
fld    5
```

При выполнении последней команды регистр 5 не содержит числа (свободен), что приводит к загрузке NaN.

Приемник во всех операциях загрузки — это регистр 7, который после уменьшения указателя стека становится регистром 0. Операнд-источник считывается перед уменьшением `st`, так что команда `fld 0` дублирует вершину стека, а две подряд команды `fld 1` дублируют пару значений на вершине стека.

Вариант программы с правильной загрузкой чисел приведен в листинге 8.7.

Листинг 8.7. Загрузка чисел из переменных в памяти (fld_2.8)

```
...
real    dd      8.0, 7.0
...
fld     real
fld     real+4
...
```

Для загрузки некоторых констант предусмотрены специальные команды:

- ❑ нуль — `fldz`;
- ❑ единица — `fld1`;
- ❑ число π — `fldpi`;
- ❑ $\log_2(10)$ — `fldl2t`;
- ❑ $\log_2(e)$ — `fldl2e`;
- ❑ $\log_{10}(2)$ — `fldlg2`;
- ❑ $\ln(2)$ — `fldln2`.

Команды выгрузки

Выгрузка выполняется командой:

```
f(i/b)st(p) <dst>
```

Источник — всегда вершина стека, регистр 0. Приемник `<dst>` — это либо адрес переменной в памяти (типа `w/d/q/t`), либо номер регистра 0...7.

Опция `p` (Pop) задает освобождение регистра 0 с последующим увеличением `st`, в результате значение "выталкивается" из стека. Выгрузка BCD или 64-битового целого возможна только с опцией `p` (причина, видимо, в экономии кодов команд)²².

Особенность выгрузки в том, что адрес `<dst>` вычисляется до увеличения `st`. Поэтому команды выгрузки `c <dst> = 0...7` дают интересные эффекты (листинг 8.8):

- ❑ инструкция `fstp 0` задает копирование вершины стека в регистр 0, заданный в команде, после чего вершина стека освобождается. В результате команда `fstp 0` просто *удаляет* верхний элемент стека, и ее можно использовать вместо несуществующей команды `fpop`;
- ❑ команда `fst 0` копирует `st(0)` в `st(0)`, т. е. не делает ничего²³;
- ❑ команда `fst <n>` при $n = 1...7$ копирует вершину стека в регистр n , даже если он занят, остальные данные остаются на своих местах, т. к. `st` не изменяется.

Листинг 8.8. Эффекты при выполнении команд `fst (p)` (`fst_1.8`)

```

(1)      finit
(2)      fldln2          ; ln2
(3)      fldl            ; 1      ln2
(4)      fst      3      ; 1      ln2      -      1
(5)      fst      0      ; 1      1      ln2      -      1
(6)      fstp     0      ; 1      ln2      -      1
(7)      fld      0      ; 1      1      ln2      -      1

```

Содержимое регистра 0 по команде (4) копируется в регистр 3, команда (5) пустая, оператор (6) удаляет число с вершины стека, а (7) дублирует `st(0)`.

Еще один интересный эффект от многократного выполнения команды `fstp <n>` при $n = 1...7$ — передача по кольцу значений между регистрами $0...(n-1)$. Полный оборот требует n -кратного выполнения этой команды. В листинге 8.9 приведен пример для $n = 3$.

Листинг 8.9. Передача значений по кольцу командой `fstp (fst_2.8)`

```

finit                                     ; (1)
fldl2t                                   ; (2)
fldl2t                                   ; (3)
fldl2e                                   ; (4)
fldl                                     ; (5)
fldz      ; 0      1      l2e      l2t      l2t      ; (6)

fstp     3      ; 0      1      l2e      0      l2t      ; (7)
          ; 1      l2e      0      l2t      ; (8)
fstp     3      ; 1      l2e      0      1      ; (9)
          ; l2e      0      1      ; (10)
fstp     3      ; l2e      0      1      l2e      ; (11)
          ; 0      1      l2e      ; (12)

fstp     0      ; 1      l2e      ; (13)
fucompp   ; -      ; (14)
...

```

После загрузки значений в порядке убывания получаем в стеке FPU возрастающие числа. (Числа в регистрах 3 и 4 в передаче по кольцу не участвуют, но интересно посмотреть, что с ними станет.) В комментариях к (7), (9) и (11) показаны два этапа выполнения команды `fstp 3`: копирование `st(0)` в регистр 3, а потом освобождение регистра 0. Вращение идет по кольцу влево, и хотя в нем участвуют регистры 0–2, значения в глубине стека постепенно перемещаются вверх и, попав в регистр 3, перезаписываются. Можно сделать вывод, что данные за пределами кольца не сохраняются.

Инструкция (13) выталкивает число из регистра 0. Для очистки сразу двух регистров на вершине стека применен вариант инструкции сравнения (14): с двойным суффиксом `p` (`Pop`)²⁴. (В примере результат сравнения интереса не представляет.)

В этой группе инструкций относительно недавно появилась команда `fisttp` (FPU Integer Truncate-Store-Pop) — запись в память целого числа с отбрасыванием дробной части²⁵. Пример приведен в *приложении 3*.

Команда обмена

Команда `fxch i` меняет местами значения в регистрах 0 и `i`. Команду `fxch 1` на языке ассемблера можно задавать без операндов — `fxch` (в машинном коде это одно и то же)²⁶.

Специальной команды для обмена местами регистров с произвольными ненулевыми номерами не предусмотрено. Хотя бы один операнд должен быть на вершине стека. Такой обмен несложно выполнить в три команды при помощи макроса `fswap` из листинга 8.10.

Листинг 8.10. Обмен местами значений произвольных регистров (`fxch.8`)

```
fswap    macro
          fxch    #1
          fxch    #2
          fxch    #1
#em
          ...
          fswap   2, 3
          ...
```

Добавьте в программу `fxch.8` команды загрузки данных и проверьте в отладчике работу макрокоманды `fswap`: сначала отработайте по шагам вызов, заданный в программе, а потом выполните макрос в непосредственном режиме, например `fswap 1, 3` <Enter>, повторяя вызовы клавишей <F3>.

Арифметические операции

Арифметические операции в FPU — это четыре действия арифметики и несколько дополнительных команд на их основе, а также извлечение квадратного корня.

Основные арифметические операции

Обозначения операций FPU похожи на названия операций центрального процессора (add, sub, mul, div), только добавлена буква f (FPU) в начале.

Среди арифметических операций имеются команды с *реверсированием* — это subr и divr. В них операнды заданы в обратном порядке²⁷: приемник меняется с источником, хотя результат по-прежнему записывается в *приемник*. Буква p (Pop) в конце задает удаление значения из st(0), и она может быть добавлена, если приемник не st(0).

Варианты арифметических команд, кроме команд с реверсированием, показаны в табл. 8.2, где <op> — операция из множества add, sub, mul, div.

Таблица 8.2. Форматы арифметических инструкций

Формат команды	Действие	Примеры
f<op>(p)	1 op 0 → 1, pop	fadd
f<op> i f<op> 0, i	i op 0 → 0	fadd 0 fadd 1 fadd 0, 7
f<op> i, 0	i op 0 → i	fdiv 2, 0
f<op>p i, 0	i op 0 → i, pop	fsubp 1, 0
f<op> mem	0 op mem → 0	fadd d[0200]
fi<op> mem	0 op int(mem) → 0	fiadd d[0180]

Примеры команд, в том числе с реверсированием, приведены в листинге 8.11, результаты выполнения в неочевидных случаях выделены полужирным шрифтом.

Листинг 8.11. Примеры арифметических команд (math_1.8)

```
        jmp      start                                ; (1)

two     dw      2                                    ; (2)
eight   dd      8.0                                  ; (3)
dozen   dw      12                                   ; (4)

start:  finit                                         ; (5)

        fld     eight                                ; (6)
        fild    two                                  ; (7)
        fdiv    ; 4                                  ; (8)
        fldl    ; 1                                  ; (9)
        fdivr   ; 0.25                               ; (10)
```

```
fld      eight      ; 8          0.25          ; (11)
fild     two        ; 2          8          0.25      ; (12)
fadd     0          ; 4          8          0.25      ; (13)
fsub     2          ; 3.75      8          0.25      ; (14)

fadd     1, 0       ; 3.75      11.75      0.25      ; (15)
fild     two        ; 2          3.75      11.75      0.25  ; (16)
fsub     2, 0       ; 2          3.75      9.75      0.25  ; (17)
fdivp    2, 0       ; 3.75      4.875      0.25      ; (18)
fisub    dozen      ; -8.25     4.875      0.25      ; (19)
fsub     eight      ; -16.25    4.875      0.25      ; (20)
fsubr    ; -21.125    0.25          ; (21)
...
```

Команда в строке (8) выполняет операцию $1/0 \rightarrow 0$, операнды удаляются из стека, затем загружается результат. Команда (10) работает по схеме $0/1 \rightarrow 0$ и тоже удаляет операнды из стека. Команда (13) выполняется по схеме $0*0 \rightarrow 0$; в результате число в `st(0)` удваивается. Команда (14) действует по схеме $0-2 \rightarrow 0$, при этом оставляя операнд-источник (и все остальные данные) на прежних местах. Команда (15) выполняет операцию $1+0 \rightarrow 1$, в (17) аналогично, только для вычитания: $2-0 \rightarrow 2$. Команда (18) с суффиксом `p`, выполнив операцию по схеме $2/0 \rightarrow 2$, удаляет операнд-источник с вершины стека, результат, в конце концов, оказывается в регистре `1`.

Заметим, что операции `fdivr` и `fdiv 0, 1` (равно как `fsubr` и `fsub 0, 1`) похожи, но не идентичны. Схема выполнения одна и та же: $0 \text{ op } 1 \rightarrow 0$, но в первом случае (команда с реверсированием без операндов) происходит свертка стека ("классический" исход), а во втором случае в регистре `1` остается операнд-источник. Пример — в листинге 8.12.

Листинг 8.12. Операции с обратным порядком операндов (math_2.8)

```
finit
fld     two      ; 2
fld     eight    ; 8      2
fdivr   ; 4

finit
fld     two      ; 2
fld     eight    ; 8      2
fdiv    0, 1     ; 4      2
```

Операции над знаковым битом

Корректировка бита `sign` выполняется командами:

- ❑ инвертирование или изменение знака — `fchs`;
- ❑ обнуление знака или получение абсолютного значения — `fabs`.

Эти команды не нуждаются в указании операндов. Источник и приемник — всегда содержимое регистра 0. Команда `fchs` вам знакома, проверьте работу `fabs` в `d86`.

Округление до целого

Команда `frndint` округляет значение в регистре 0 до целого согласно текущему режиму округления `rc` (Rounding Control) в управляющем слове `cw`.

Пример с установкой `rc` и проверкой команды `frndint` приведен в листинге 8.13.

Листинг 8.13. Округление с отбрасыванием дробной части (round.8)

```

        jmp      start                                ; (1)

include fpu.inc                                    ; (2)
...

cw      dw      ?                                    ; (7)

set_rc  macro                                        ; (8)
        fstcw   cw                                    ; (9)
        mov     ax, cw                                ; (10)
        and     ax, not round_ctrl                    ; (11)
        or      ax, round_#1                          ; (12)
        mov     cw, ax                                ; (13)
        fldcw   cw                                    ; (14)
#em                                           ; (15)

check   macro                                        ; (16)
#rx11                                       ; (17)
        fld     #x                                    ; (18)
        frndint                                     ; (19)
#em                                           ; (20)

start:                                       ; (21)
        finit                                        ; (22)
        set_rc  near                                ; (23)
        check   x, y, z, u                        ; (24)
        ...

        finit                                        ; (34)
split:                                       ; (35)
        fldpi   ; i.f                               ; (36)
        fld     0 ; i.f i.f                         ; (37)
        set_rc  chop                                ; (38)
        frndint ; i i.f                             ; (39)
        fsub    1, 0 ; i 0.f                         ; (40)
        int     020                                ; (41)

```

Для смены режима округления используется макрокоманда `set_rc` с параметром в символическом виде — `near/down/up/chop/zero` (`chop` и `zero` — синонимы). Эти параметры определены в подключаемом файле (2). Макрокоманда сохраняет (9) слово состояния в переменной (7) командой `fstcw` (Fpu STore Control Word), а затем, установив (10–13) в этой переменной заданный код, загружает ее обратно в слово состояния командой `fldcw` (Fpu LoaD Control Word).

В программе проверяются все режимы округления. В листинге 8.13 приведены первая проверка (23–24) и финальный фрагмент от метки (35), где округление с отбрасыванием дробной части (38) позволяет разделить действительное число `i.f` на целую `i` и дробную части `0.f`. Дробная часть получена в результате вычитания `i` из исходного значения `i.f`.

Получение остатка от деления

Команды `fprem` и `fprem1` (FPU Partial REMainder) формируют в `st(0)` остаток от деления регистра 0 на регистр 1 (обратный порядок операндов). Команды идентичны, но считается, что `fperm` не соответствует стандарту.

Команда `fprem(1)` применяется для уменьшения аргумента периодической функции, например `sin` или `cos`²⁸. Желательно привести аргумент к диапазону `[0...2π]`, поэтому сначала вызывают `fprem(1)` с делителем `2π`.

Если разница между порядком делимого и порядком делителя больше 63, то `c2 = 1` означает, что получен *частичный* остаток и требуется повторить выполнение `fprem(1)`. Для доступа к флагу `c2` необходимо вызвать макрокоманду `rd_flags` из `fpu.inc`, в результате чего `c2` FPU отобразится на флаг `e` (parity Even) CPU. Пример приведен в листинге 8.14.

Листинг 8.14. Вычисление остатка от деления (`fprem1.8`)

```

...
dvs    dt      0.1
dvd     dt      1.0e20 + 0.14

...
fld     dvs      ; 0.1
fld     dvd      ; (1.0e20 + 0.14)    0.1
11:     fprem1
        rd_flags
        jpe      11
...

```

Ожидаемый остаток от деления $(10^{20} + 0.14)$ на `0.1` равен `0.04`. Однако полученный результат `0.0447` заметно отличается от `0.04`. Возможно, причина в неточном делителе: все-таки `0.1` в двоичном представлении — это бесконечная периодическая дробь (в чем можно убедиться при помощи макрокоманды `what 0.1` в составе `inner.8`.)

Дальнейшие опыты показывают, что причина скорее в очень большой разнице между делимым и делителем. А установка флага `c2` — это, пожалуй, не приглашение к повтору команды, а сигнал катастрофической потери точности. К тому же опыты не обнаруживают никакой разницы в точности между `fperm1` и `fperm`. Коды команд, действительно, разные, но похоже, что их реализация стала одинаковой.

Извлечение корня

Команда `fsqrt` извлекает квадратный корень из значения в регистре 0, помещая результат в тот же регистр. Корень из отрицательного числа — NaN. Корень из отрицательного нуля дает -0 . (Минус ноль также может получиться из $+0$ в результате выполнения `fchs`.)

Масштабирование

Команда "масштабирования" `fscale` умножает значение `st(0)` на двойку в степени, заданной регистром 1. Результат помещается в `st(0)`. При возведении в степень используется только *целая* часть значения в регистре 1, дробная часть в расчет не принимается. Загрузите в FPU числа 3.9 и 5.0, выполните `fscale`. В `st(0)` должно получиться $5 * (2^3) = 40$, а под ним — 3.9.

Если в регистре 1 задано $-\text{Infinity}$, то результат `fscale` — ноль. Если в `st(1)` задано $+\text{Infinity}$, то при *ненулевом* значении `st(0)` результат `fscale` — тоже $+\text{Infinity}$. Проверьте, получив бесконечности в результате деления на $+0.0$ и -0.0 . Значение в регистре 1 в любой ситуации остается без изменений.

Команда `fextract` выполняет обратное действие: раскладывает число в регистре 0 на мантиссу и порядок²⁹. Мантисса записывается в `st(0)`. Значения `st(1...6)` сдвигаются вниз, освобождая место для записи порядка в `st(1)`.

Вызовите `d86` с программой `inner.com` и выполните `fldz`, `fldl`, `what 0.625` и `fextract`. Значение в регистре 0 станет равным 1.25 (мантисса), а в регистре 1 появится -1 (двоичный порядок *без смещения*). Ранее загруженные числа 0 и 1 после `fextract` не исчезли, а сместились в глубину стека.

Если исходное значение равно 0.0 , то выполнение `fextract` приводит к ошибке `z` (Zero divide), в `st(0)` остается 0, а в регистр 1 записывается $-\text{Infinity}$, проверьте.

Операции сравнения и тестирования

Команда `fcom` анализирует тег регистра 0 и устанавливает флаги `c3`, `c2`, `c0`, как показано в [1, Table 6-15]. Команда `ftst` сравнивает число в регистре 0 с нулем. Команды из группы `fcom` сравнивают регистр 0 с заданным операндом или с регистром 1, если операнд не задан.

Команда `ftst` и команды из группы `fcom` (FPU COMpare) устанавливают флаги `c3`, `c2`, `c0` по результатам сравнения, как в табл. 8.3.

Таблица 8.3. Установка c3, c2 и c0 в результате сравнений

c3 c2 c0	Значение
1 1 1	Числа несопоставимы
0 0 0	>
0 0 1	<
1 0 0	=

Эти флаги можно парой команд переписать в слово состояния CPU:

```
fstsw    ax        ; sw -> ax
sahf                                ; ah -> flags
```

Тогда они видны в CPU как: c0 -> c (Below), c2 -> e (Unordered) , c3 -> z (Zero).

Формат команд сравнения:

```
f(u/i)com(p/pp) <src>
```

- ❑ Опция *u* (Unordered) имеет значение только в особых ситуациях — при сравнении так называемых *QNaN* (Quiet NaN). Если же числа несопоставимы, то (независимо от опции *u*) флаги *c3*, *c2*, *c0* и признак *IE* (Invalid operation) устанавливаются в единицу. Если задана опция *u*, то при сравнении *QNaN* флаг *IE* не устанавливается.
- ❑ Опция *p* (Pop) задает удаление содержимого регистра *0* из стека после сравнения, опция *pp* удаляет содержимое двух регистров — *0* и *1*.
- ❑ Опция *i* означает, что источник *<src>* содержит адрес *целого* значения в памяти. Сравнение с 64-битовыми целыми в памяти (типа *q*) не поддерживается, допускается сравнение только с 16- и 32-битовыми (типа *w*, *d*).
- ❑ Без опции *i* в *<src>* допускается задание адреса действительного значения в основной памяти. Сравнение с данными расширенной точности в памяти (тип *t*) не реализовано, допускаются только типы *d* и *q*.

Примеры задания инструкций семейства *fcom*:

```
fcom
fcomp
fcompp
fcom    2
fcomp   2
fcomp   q[0120]
ficompp d[0200]
```

Инструкция *fcompp*, помимо основного назначения, используется для удаления из стека значений регистров *0* и *1*.

Тригонометрические операции

Команды для вычисления тригонометрических функций — `fsin`, `fcos`, `fsincos`; `fptan` (FPU Partial TANgent) и `fpatan` (FPU Partial ArcTANgent).

Команды `fsin`, `fcos`, `fsincos` вычисляют функции $\sin$ или/и $\cos$ от аргумента, заданного в регистре 0. Результат `fsin`, `fcos`, а также первый результат выполнения `fsincos` ($\sin$), помещается на место исходного значения в `st(0)`. Второй результат `fsincos` ($\cos$) загружается в стек, сдвигая вниз значение $\sin$. В итоге по команде `fsincos` косинус оказывается на вершине стека, а под ним — синус.

Вызовите `d86`, загрузите $\pi/3$ и выполните команду `fsincos`. Значение $\cos$ в `st(0)` равно 0.5, а $\sin$ в `st(1)` — 0.866.

Команда `fptan` помещает результат вычисления тангенса на место регистра 0, а затем выполняет команду загрузки единицы `fld1`. В итоге регистр 0 содержит единицу³⁰, а в регистре 1 сохранен результат.

Если операнд тригонометрической команды больше 2^{63} , результат не записывается (значение в регистре 0 остается без изменений), а `c2 = 1`. Необходимо командой `fprem1` привести абсолютное значение операнда к диапазону $[0...2\pi[$ и еще раз выполнить тригонометрическую команду³¹.

Команда `fpatan` дает значение $\arctan(y/x)$, где x находится в `st(0)`, а y — в `st(1)`. Таким образом, она сначала делит `st(1)` на `st(0)`, удаляет их из стека, вычисляет функцию арктангенса и помещает результат в `st(0)`. Диапазон операндов не ограничен.

Возведение в степень

Вычисление x^y для произвольных значений x и y выполняется как $2^{(y * \log_2(x))}$.

Предварительное вычисление показателя степени $z = y * \log_2(x)$ выполняется командой `fyl2x`. Для $x = 10$ или e вычислять логарифм не нужно — есть встроенные константы, которые загружаются командами `fldl2t`, `fldl2e`.

Команда `fyl2x` вычисляет значение $y * \log_2(x)$, где y задано в регистре 1, а x — в регистре 0. Величина x должна быть больше нуля. Результат (после удаления значений из регистров 0 и 1) загружается в регистр 0.

При значениях x , близких к нулю, точность `fyl2x` падает. Следует в такой ситуации использовать команду `fyl2xp1`. В отличие от `fyl2x`, для команды `fyl2xp1` регистр 0 должен содержать не x , а разность между x и 1 ($\epsilon = x - 1$). Абсолютная величина ϵ должна быть меньше $1 - (\sqrt{2})/2$ (~ 0.2929).

Вычислив z , надо затем возвести двойку в эту степень, что выполняется командами `f2xm1` (для дробной части z) и `fscale` (для целой части z). Команду `fscale` мы рассмотрели. Команда `f2xm1` возводит число 2 в степень, заданную `st(0)`, и сохраняет результат, уменьшенный на 1, в `st(0)`. Она выполняется только для дробных значений степени, иначе результат будет неопределенным (а признаки ошибок не устанавливаются).

Решение задачи в общем виде разбивается на вычисление:

- $z = y * \log_2(x)$;
- целой и дробной частей $z \rightarrow i.f$;
- $2^{(i.f)} = 2^i * 2^{0.f}$.

При $x = 2$ вычисление z отпадает за ненадобностью, т. к. $\log_2(2) = 1$ и $z = y$.

Рассмотрим подпрограммы для этапов вычисления x^y , все они входят в исходный модуль power.8.

Возведение числа 2 в целую степень

Для получения целой степени числа два используется команда масштабирования fscale. Подпрограмма p2i вычисляет 2^i , при входе в подпрограмму значение степени задано в st(0), туда же помещается результат (листинг 8.15).

Листинг 8.15. Подпрограмма для возведения 2 в целую степень (power.8)

```
p2i:                ; i
    fldl            ; 1          i
    fscale          ; 1*(2^i)    i
    fxch            ; i          2^i
    fstp    0       ; 2^i
    ret
```

Команды fxch и fstp 0 удаляют значение i из стека.

Для проверки вызовите отладчик с программой power.com и выполните команды finit, fldz, fldl, fld 5.1 и call pow2_i. Результат $2^5=32$ должен быть в st(0), число 5.1 из стека ушло, а 0 и 1, загруженные до вызова подпрограммы, остались.

Возведение числа 2 в дробную степень

Подпрограмма p2f вычисляет 2^f — для значений f в пределах открытого интервала]-1...+1[. Входное значение и результат — в регистре 0 (листинг 8.16).

Листинг 8.16. Подпрограмма для возведения 2 в дробную степень (power.8)

```
p2f:                ; f
    f2xm1           ; 2^f-1
    fldl            ; 1          2^f-1
    fadd            ; 2^f
    ret
```

Для проверки выполните в отладчике fld 0.5 и call p2f. Результат $2^5=32$ должен быть в st(0) — на месте входного значения 0.5.

Вычисление целой и дробной частей значения степени

Чтобы возвести число два в произвольную степень z , необходимо выделить целую и дробную части z . С этой целью временно устанавливается режим округления `zero`, и при помощи `frndint` извлекается целая часть — i . Дробная часть $f = z - i$. Подпрограмма `split` в листинге 8.17 формирует из числа, заданного в `st(0)`, два результата: целую часть в `st(0)` и дробную часть в `st(1)`.

Листинг 8.17. Подпрограмма расщепления числа (power.8)

```
split:                                ; z(i.f)
    fld     0                        ; z(i.f)  z(i.f)
    set_rc  chop
    frndint                                ; i      i.f
    fxch                                ; z(i.f)  i
    set_rc  near
    fsub    1                        ; f=z-i   i
    ret
```

Макрокоманда `set_rc` вызывается дважды: перед получением целой части и перед получением дробной части. Макрокоманда использует переменную `cw`, которая необходима для команд чтения-записи управляющего слова FPU и определена в начале `power.8`.

Возведение числа 2 в произвольную степень

Подпрограмма `p2z` вычисляет 2^z для произвольного значения z — отдельно для целой части i , отдельно для дробной части f . Окончательный результат есть произведение значений, полученных для целой и дробной частей. Входное и выходное значения — в регистре 0 (листинг 8.18).

Листинг 8.18. Подпрограмма для возведения 2 в произвольную степень (power.8)

```
p2z:                                ; z
    call    split                    ; f      i
    call    p2f                      ; 2^f    i
    fxch                                ; i      2^f
    call    p2i                      ; 2^i    2^f
    fmul                                ; 2^z
    ret
```

Вычисление логарифмов

Для вычисления x^y в общем виде необходимо еще подсчитать $z = y \log_2(x)$. Для $x = 10$ и $x = e$ предусмотрены константы `log210` и `log2e`, загружаемые командами `fldl2t` и `fldl2e`.

Вычисление z для оснований, отличных от 2, 10 или e , выполняется командой `fyl2x`, а при значениях x , близких к 1, — командой `fyl2xpl`. Для этих команд значе-

ние y должно быть в $st(1)$, а x — на вершине стека. При их выполнении происходит свертка стека и результат оказывается в $st(0)$. Разница между ними в том, что `fyl2x` считает $y * \log_2(x)$, а `fyl2xp1` вычисляет $y * \log_2(x+1)$, где $0 < |x| < (1 - 2^{1/2} / 2)$, т. е. примерно в пределах ± 0.37 .

Примеры применения этих команд приведены в листинге 8.19, везде $y = 1.0$.

Листинг 8.19. Вычисление логарифмов командой `fyl2x` (`log_1.8`)

```

        jmp      start                                ; (1)

x       dd      10.0, 3.9, 3.7, 1.4, 1.3, 1.2, 1.1    ; (2)
sz      equ     ($-x) / type x                        ; (3)

start:                                     ; (4)
        finit                                         ; (5)
        mov     cx, sz                               ; (6)
        lea     si, x                                ; (7)
11:                                           ; (8)
        fldl                                         ; y = 1      ; (9)
        fld     d[si]                                ; x           ; (10)
        fyl2x                                         ; (11)

        add     si, type x                            ; (12)
        loop    11                                   ; (13)

```

Проходим по массиву (2), очередное вычисленное значение находится в $st(0)$, а остальные сдвигаются вниз. Размер массива ограничен семью элементами.

В листинге 8.20 используется другая команда, а входные данные — те же.

Листинг 8.20. Вычисление логарифмов командой `fyl2xp1` (`log_2.8`)

```

        jmp      start                                ; (1)

x       dd      10.0, 3.9, 3.7, 1.4, 1.3, 1.2, 1.1    ; (2)
sz      equ     ($-x) / type x                        ; (3)

start:                                     ; (4)
        finit                                         ; (5)
        mov     cx, sz                               ; (6)
        lea     si, x                                ; (7)
11:                                           ; (8)
        fld     d[si]                                ; (9)
        fldl                                         ; (10)
        fsub                                         ; x-1          ; (11)
        fstp    d[si]                                ; (12)

```

```

    add    si, type x                ; (13)
    loop   ll                        ; (14)

    finit                                ; (15)
    mov     cx, sz                     ; (16)
    lea     si, x                      ; (17)
11:                                ; (18)
    fldl                                ; y ; (19)
    fld     d[si]                      ; x ; (20)
    fyl2xpl                                ; (21)
    add     si, type x                ; (22)
    loop    ll                        ; (23)
```

В первом цикле (8–14) уменьшаем значения в массиве, поскольку команда (21) будет прибавлять единицу к *x*. Затем вызываем цикл (18–23), который заново проходит по массиву, оставляя результаты вычислений в стеке FPU.

Как ни странно, результаты обеих программ идентичны — как при относительно больших *x*, так и при *x*, стремящихся к единице (табл. 8.4)³².

Таблица 8.4. Результаты вычислений $\log_2(x)$

x	log ₂ (x)
1.1	0.137503555019507454
1.2	0.263034463161342982
1.3	0.378511570335990113
1.4	0.485426802601291171
3.7	1.88752528933430633
3.9	1.96347415925337805
10.0	3.32192809488736235

Команды управления

Большинство команд управления рассмотрено в предшествующих разделах, поэтому здесь ограничимся кратким обзором.

- ❑ Команда `finit` выполняет общий сброс FPU. В результате режим округления `rc = near`, флаги особых ситуаций в `sw` сброшены, прерывания в `cw` запрещены, все регистры данных помечены как незанятые и `st = 7`.
- ❑ Команды `fdecstp`/`fincstp` увеличивают/уменьшают указатель стека FPU на единицу по кольцу, как бы передвигая значения между регистрами вниз/вверх.
- ❑ Команда `ffree i` помечает регистр *i* как незанятый.
- ❑ Команда `fclex` сбрасывает признаки особых ситуаций в `sw`.

- ❑ Команда `fstcw/fstsw` считывает значение из `cw/sw` в 16-битовую переменную в памяти, в команде `fstsw` также допускается операнд `ax`.
- ❑ Команда `fldcw` записывает в `cw` значение 16-битовой переменной из памяти.
- ❑ Команда `fnop` равносильна пустой операции³³.
- ❑ Команда `wait/fwait` останавливает CPU до завершения текущей операции в FPU. Тем самым процессор синхронизирует свою работу с сопроцессором. (Эта команда скорее относится не к FPU, а к CPU.) В i80387 такая синхронизация требуется только при обработке исключений в связи с ошибками FPU.

Сохранение и восстановление состояния

Сохранение контекста может понадобиться при переключении процессов между задачами. Также, поскольку регистры MMX объединены с регистрами FPU, сохраняя состояние FPU, мы сохраняем и состояние MMX. Вместе с тем при переходе от незавершенных операций FPU к MMX (и наоборот) тоже приходится сохранять и восстанавливать состояние.

Команда `fstenv <m14byte>` считывает в память данные о состоянии и настройках FPU, их структура приведена в [2, Figure 11-7] и в листинге 8.21. Побочный эффект этой команды — запрет в `cw` всех прерываний по особым ситуациям.

Листинг 8.21. Образ состояния и регистров FPU (fpu.inc)

```

struc
ctrl      dw      ?
state     dw      ?
tags      dw      ?
ip_ptr    dd      ?
opr_ptr   dd      ?
st07      dt      8 dup ?      ; st(0)..st(7)
fpu       ends

```

В листинге 8.22 приведен код для проверки команд сохранения и восстановления состояния FPU.

Листинг 8.22. Сохранение и восстановление состояния FPU (fsave.8)

```

include fpu.inc          ; (1)

        finit            ; (2)

        fldl             ; (3)
        fldpi            ; (4)
        fstenv [0]        ; (5)
        int 3            ; (6)
        fldenv [0]        ; (7)

```

```
fsave    [0]      ; (8)
int      3        ; (9)
frstor   [0]      ; (10)
...
```

Зайдите в отладчик с `fsave.com`. Дойдите до команды (6). Мы сохранили состояние по адресу 0 — в начале блока данных, выделенного `com`-программе. (Там находится область PSP размером 256 байтов, с информацией, которой мы практически не пользуемся.) Поскольку значения имен в листинге 8.22 — это смещения от начала *структуры*, которую мы загрузили по нулевому адресу, то эти имена напрямую адресуют нужные нам значения. Так, например, слово состояния доступно по имени `state`, управляющее слово — по имени `ctrl`, а слово тегов [1, Figure 6-5] — по имени `tags`. Битовые поля слова состояния и управляющего слова приведены в [1, Figure 6-4] и [1, Figure 6-3].

В полученном образе измените значение, соответствующее `sw`: запишите на место 037f код 077f, обойдите точку останова (6) — и тогда при выполнении (7) будет выбран режим округления `-inf`.

Выполните (8). Команда `fsave <m94byte>` тоже сохраняет состояние FPU, но уже полностью, включая регистры. При этом она выполняет общий сброс (в отличие от `fstenv`). Структура образа должна быть такой же, как после (5), но к ней теперь примыкают копии восьми регистров данных, от `st(0)` до `st(8)`, по 10 байтов каждый. При выполнении (10) регистры будут восстановлены.

Задачи

Все решения начинаются с команды `finit`, а первые действия в отладчике — это настройка строк отображения данных (для вещественных данных — форматы `fd/fq/ft`), открытие окна FPU и сохранение нажатий клавиш: `k s prg <Enter>`, где `prg` — имя `com`-файла без расширения.

1. Загрузите в FPU три разных числа. Выполните вычитания между следующими регистрами, без удаления данных из стека: `1 := 0 - 1`; `2 := 2 - 0`; `0 := 1 - 0`.
2. Поменяйте местами значения регистров 1 и 2 при помощи `fxch`. Скопируйте командами `fld`, `fst(p)` значение из регистра 2 в регистр 5. При помощи `fstp` прокрутите по кольцу регистры 0, 1 и 2. Удалите значения регистров 0 и 1 из стека при помощи команды сравнения.
3. Напишите программу, которая загружает в стек числа 1.0, 2.0, 3.0, затем формирует в регистре 5 число `-5.0` так, чтобы остальные регистры (кроме 0, 1, 2 и 5) были *в итоге* свободны. Затем программа в цикле вызывает подпрограмму, которая передает по кольцу значения между регистрами 0–2, в каждом цикле восстанавливая регистр 5 при помощи `fxch`.
4. Сформируйте на вершине стека число `-0.0`, потом `+Infinity`, `-Infinity`, `NaN` и денормализованное число.

5. Вычислите значения $199975 \bmod 3$, $\sin(\pi/4)$, $\operatorname{tg}(\pi/8)$ и корня третьей степени из 7.
6. Напишите программу для вычисления выражения $x * (x - 2) / \sqrt{x + 1}$. Значение x задано в 32-битовой переменной в формате действительного числа. Загрузка x из памяти должна быть только одна, в начале, потом загруженное значение дублируется в стеке командами `fld/fst(p)`.
7. Составьте два варианта программы для поиска максимума из пяти значений, загруженных в регистры 0–4. Варианты следующие:
 - поиск выполняется непосредственно в FPU, в цикле — за счет вращений стека или команд обменов: при помощи команд из набора `fdecstp`, `fincstp`, `fstp 5`, `fxch`;
 - поиск выполняется в образе FPU, полученном командой `fsave`; смещение данных от начала образа можно задать именем `reg`, которое определено в файле `fpu.inc` в составе следующей структуры³⁴:

```

struc
ctrl      dw      ?
state     dw      ?
tags      dw      ?
ip_ptr    dd      ?      ; ip + opcode
opr_ptr   dd      ?      ; linear addr of operand (20 bits)
reg      dt      8 dup ?  ; st(0)..st(7)
fpu       ends

```

Сравнивать числа в формате с плавающей точкой несложно: смещение порядка (добавление `03fff`) было введено для того, чтобы при сравнении не приходилось отдельно анализировать поле `exp`. И, поскольку `sign` находится в старшем бите, вещественные числа можно сравнивать как *знаковые целые* многократной точности. Сравнение идет от старших слов к младшим, цикл завершается при несовпадении значений. Если не совпали сразу (на первом слове), то сравнение завершается проверкой `ja/jl`. Если не совпали следующие пары слов, то сначала следует выполнить беззнаковую проверку `ja/jb`, чтобы выяснить, какая из пары мантисс оказалась больше по абсолютной величине, а затем проверить `sign` — если он установлен, то результат сравнения обратный.

8. Составьте программу генерирования гармонических колебаний по следующей рекуррентной схеме:

$$\begin{aligned}
 y(n+1) &= A * y(n) + B * x(n) \\
 x(n+1) &= C * y(n) + D * x(n)
 \end{aligned}$$

$A = D = \cos(b)$, $B = -C = \sin(b)$, $b = 2\pi/k$, где k — число точек на период. Начальные значения: $y(0) = 0$, $x(0) = 1$ (x — косинус, y — синус). Для вычисления коэффициентов A – D воспользуйтесь калькулятором. Рекомендуемое значение k — порядка 40.

9. Выполните задание из главы 7, используя вместо целых чисел вещественные данные 32- или 64-битового формата.

Примечания

1. Оценка разницы будет куда интереснее, если сохранять результат деления единицы на тройку.
2. Другое название — обратная польская запись в честь ее автора Яна Лукасевича.
3. Обмен данными между FPU и основной памятью также возможен в упакованном двоично-десятичном формате, который сейчас используется довольно редко. (Англоязычное название — Binary Coded Decimals, или BCD.) В FPU эти данные занимают по 10 байтов, определяются директивой `dt` наподобие 16-ричных чисел, но без цифр `a...f` (например, 01962 означает BCD-число 1962). Девять байтов задают 18 десятичных цифр, по одной в каждой тетраде, а знак числа определяется седьмым битом старшего байта. Вы можете проверить кодировку BCD, выполняя команду сохранения `fbstp bcd` (где `bcd` опеределено директивой `dt`) для чисел, например, +1.0 и -1.0.
4. Клавиши редактирования в `postfix.exe` не работают.
5. В окне **FPU** выводится **Proj infinity**, что означает проективную, т. е. беззнаковую бесконечность. На деле же используется аффинная бесконечность, знаковая. Причина в том, что i8087 поддерживал два варианта бесконечности, а в дальнейшем оставили только аффинную.
6. Для извлечения квадратного корня есть команда `fsqrt`.
7. Напомню, что в центральном процессоре назначение флага `e` другое: он равен единице, если в младшем байте результата — четное количество единиц. Просто в CPU не нашлось более подходящего флага для индикации ошибки, возникающей при сравнении данных в FPU.
8. Переход при `e = 0` — командой `jpo` (Jump if Parity Odd).
9. Его также называют Exception Summary.
10. На следующие строки пока не обращайте внимания, они понадобятся при изменении настроек FPU.
11. Еще раз: если `SF = IF = 1`, то это ошибка стека, и значение `C1` уточняет ситуацию (переполнение или антипереполнение).
12. В [2] о флагах `v` и `es` сказано, что они доступны на запись, но это не имеет смысла. Подозрительное заявление, которое еще больше убеждает в том, что `es` в каких-то моделях может не работать.
13. Прерывания от FPU рассмотрены в *приложении 3*.
14. Обратите внимание, что прерывание по ошибке стека (бит `SF` в `sw`) не маскируется, такого бита в `cw` нет.

15. Например, PLC Simatic S7-300/400 работает с действительными числами одинарной точности. Точность его вычислений можно оценить на РС, сравнив результаты i80x87 в обычном режиме (80-битовые данные с плавающей точкой) и в режиме пониженной точности.
16. В предыдущих опытах мы получили событие UE до обнуления результата. Оно говорит о потере значащих разрядов в *начале* числа, это куда серьезнее, чем потеря разрядов в конце (PE).
17. Чтобы изменить знак числа, например сделать -1.625 из $+1.625$, достаточно поставить 1 в бите sign. Вы можете это проверить, вызвав what -1.625 .
18. Справедливо только для *нестандартного* расширенного формата. В стандартных форматах целая часть вообще не представлена, считается, что она всегда равна единице, за исключением особых случаев: 0.0, Infinity, NaN и т. п.
19. В дробной части числа 1.101 есть половинка и осьмушка, а четвертушки нет.
20. Перед fld указан префикс переназначения сегмента, о них речь пойдет в *главе 9*.
21. Не следует путать: fld 2.0 — встроенная макрокоманда для загрузки числа 2.0, а fld 2 — команда для загрузки регистра номер 2. В любом случае число записывается в st(0), но сначала уменьшается по кольцу указатель стека ST, освобождая место на вершине стека.
22. Команд fist q[0] и fbst t[0] в машинном представлении нет, допустимы команды fistp q[0] и fbstp t[0].
23. В отличие от нее, команда fld 0 дублирует вершину стека.
24. Можно было использовать команду без буквы u. Команды из групп fcom и fucom различаются только тем, что по-разному обрабатывают QNaN и SNaN (варианты представления NaN [1, 6.3.3.6 и Table 6.7]). В эти тонкости мы вдаваться не будем.
25. Независимо от режима округления, заданного в поле rc управляющего слова cw.
26. Команда fxch — одна из самых быстрых. В Pentium предусмотрено выполнение fxch в параллель с другими инструкциями FPU, причем *только* для этой команды.
27. Операций faddr и fmulr нет, поскольку сложение и умножение коммутативны.
28. Максимально допустимое абсолютное значение аргумента, начиная с i80387, — 2^{63} .
29. Команда fextract применяется при возведении в степень произвольного числа, что мы рассмотрим чуть далее.
30. Сейчас эта 1 в st(0) ни к чему, но она немного упрощала вычисления sin/cos по значению tg — в i8087 команд fsin и fcos еще не было.
31. Опыты с командой fperm(1) говорят о том, что при большой разнице между делимым и делителем ее точность низкая.

32. Похоже на то, что реализация команд совершенствуется, а в документации на всякий случай остаются прежние рекомендации.
33. Некоторые команды управления, унаследованные от i8087 и i80287, недействительны в i80387 и воспринимаются как `fnop`.
34. Встретив ключевое слово `struc`, ассемблер запоминает текущее значение счетчика адресов и продолжает трансляцию с `$ = 0`, пока не встретит директиву `ends`. В результате разбора структуры в этом примере `ctrl = 0` (тип `word`), `state = 2` (тип `word`) и т. д. Предположим, в `bx` записан адрес этой структуры. Тогда к ее полям можно обратиться так: `mov ax, [bx+reg]` (чтение 16-битового слова из образа регистра 0).

ГЛАВА 9

Сегменты

```
0086 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100  2: =d1,ax,, 1962
CX 0000  CS 6309  3:
DX 6309  SS 6309  4:
SI 011F  DS 6309  5: r5,b_1,, AB ab
DI 0000  ES 6309  6: r5,si,, AB ab
BP 0000  SP FFFE  0:

lea si,b_1
```

— ...Он никогда в своей долгой жизни не видывал ступеней и лестниц!.. Мы должны показать ему нашу лестницу! Мы должны научить его ходить по ступеням! <...>

— Это только вначале страшновато, а потом ничего, — подала голос невестка Кулчемга.

— Я тебе покажу, как надо шагать, — наставительно произнес лекарь и начал спускаться, держа над собой факел; спускался он очень медленно и как-то странно занося ноги.

(Вадим Шефнер. Лачуга должника)

— Рыба целая? — в ужасе вскричал я, бросаясь к нему.

— Надеюсь, что да, — ответил Джордж, осторожно поднимаясь на ноги и осматриваясь.

Но он ошибся. Форель лежала на полу, разбитая на тысячу кусков, — я сказал тысячу, но, возможно, их было только девятьсот. Я не считал. <...> Форель была гипсовая.

(Джером К. Джером. Трое в лодке (не считая собаки))

Код исполняемых программ com-формата ограничен 16-битовым адресным пространством, т. е. должен помещаться в пределах 64 Кбайт минус 256 байтов PSP (Program Segment Prefix). В учебных примерах и задачах не только код, но также данные и стек помещаются внутри программного сегмента размером 64 Кбайт.

В дальнейшем, программный сегмент — это блок памяти, распределенный при загрузке программы, первые 256 байтов которого (PSP) заполнены системной информацией (в частности, параметрами командной строки со смещения 082). По умолчанию речь идет о программном сегменте com-программы, размер которого всегда 64 Кбайт, т. е. в пределах 16-битового адресного пространства.

Тут возникают вопросы:

- ☐ Каким образом программный сегмент встраивается в 20-битовое адресное пространство реального режима i80x86?
- ☐ Как обратиться к данным вне программного сегмента?

Прежде чем ответить на них, посмотрим, как адресуются данные, код и стек *внутри* программного сегмента.

Эффективный адрес

Адрес, заданный в машинном коде, отсчитывается от начала программного сегмента. Это так называемый *эффективный* адрес (effective address¹), в реальном режиме он 16-битовый. При обращении к памяти эффективный адрес складывается с 20-битовым линейным адресом начала программного сегмента². Полученный 20-битовый адрес выставляется на шину адреса (листинг 9.1)³.

Листинг 9.1. Эффективный адрес при обращении к данным и стеку (ea_1.8)

```
include seg_mem.inc                                ; (1)

s_top:                                             ; (2)
    _brk                                           ; (3)
    jmp      start                                ; (4)

x      dw      01155, 0                            ; (5)
s      db      "A bear, however hard he tries,", 0a ; (6)
      db      "Grows tubby without exercise.", 0   ; (7)
area   db      9,8,7,6,5,4,3,2,1,0                 ; (8)

start: mov      sp, s_top          ; then hit ^t    ; (9)
      mov      bx, offset area      ; (10)

loop1:                                             ; (11)
      mov      al, [bx]              ; (12)
      dec      al                    ; (13)
      js       exit                  ; (14)
      mov      [bx], al              ; (15)
      inc      bx                    ; (16)
      jmp      loop1                ; (17)

exit:  not      x                    ; (18)
      mov      bp, sp                ; (19)
      not      w [bp+x-s_top]        ; (20)
      mov      si, 2                 ; (21)
      lea      ax, [bp+x-s_top+si]   ; (22)
      not      w [bp+x-s_top+si]    ; (23)

      push     055aa                 ; (24)

      _brk                                           ; (25)
      int      020                       ; (26)
```

Откройте отладчик d86 с программой ea_1.com.

Вся адресная информация в машинных командах — 16-битовая. Эффективный адрес переменной в памяти может быть задан числом (прямая адресация), как в (18), или регистром (косвенная адресация), как в (12, 15, 20 и 22–23). В машинных

командах *перехода* применяется относительный адрес, отсчитываемый от текущей команды, дистанция перехода задана знаковым словом или байтом⁴. Эффективный адрес перехода рассчитывается как сумма относительного адреса и текущего значения `ip` (Instruction Pointer).

Макрокоманды в строках (3) и (25), так называемый *magic breakpoint*, предназначенны для работы с отладчиком Bochs, они нам пригодятся чуть позже.

После выполнения строки (3) мы находимся на команде перехода (4). Она занимает три байта, в этом можно убедиться, глянув на адрес следующей команды или в `lst`-файл. В строке (1) отображения данных `d86` показывает машинный код от адреса `ip`. Первый байт всегда содержит код команды, так что второй и третий байты в (4) — наверняка 16-битовое смещение (04b — младший байт, 0 — старший). Эффективный адрес перехода равен сумме 0105 (значение `ip` после считывания команды из памяти) и 04b, что составляет 0150 — это и есть адрес метки `start`.

Пройдите по шагам до начала цикла (11–17). Перенос (9) указателя стека на начало `PSP` сделан для того, чтобы до минимума сократить место, занимаемое программой, это понадобится в дальнейшем⁵.

Эффективный адрес следующей инструкции (при отсутствии перехода) считывается из `ip`. При доступе к данным — из константы в коде команды (при прямой адресации), либо из регистра `bx`, `bp`, `si`, `di` (при косвенной адресации), либо по сумме `bx/bp`, `si/di` и константы (базово-индексная адресация со смещением, наиболее сложный вариант 16-битовой косвенной адресации). Например, в строках (12) и (15) эффективный адрес данных берется из регистра `bx`. В строке (20) он вычисляется как сумма содержимого регистра `bp` и числа 3 (разница между адресом переменной `x` и числом 0100), а в (22) к этому еще прибавляется регистр `si`. В (22–23) адресация базово-индексная со смещением, причем эффективный адрес, полученный в (22), не используется для доступа к памяти, а только сохраняется в `ax`.

Эффективный адрес при доступе к стеку берется либо из регистра `sp` (команды `push`, `pop`, `call`, `int`), либо из регистра `bp` (возможно, в сочетании с `si/di` и константой)⁶. В команде (24) эффективный адрес равен текущему значению `sp`.

Что касается (20) и (22–23), нет ли здесь ошибки — ведь `bp` предназначен для адресации данных⁷ в глубине *стека*? Но в `com`-программе стек и данные совмещены (используют одинаковое базовое значение линейного адреса), поэтому различие между `bp` и `bx` при косвенной адресации неощутимо.

Выполните программу до конца: `g <Enter>`. Значения в массиве (8) уменьшились на единицу, значения в (5) проинвертированы, а команда (24) записала в стек (по адресу 0100 - 2 = 0fe) число 055aa.

Базовый адрес и сегментные регистры

Базовый адрес программного сегмента задан в сегментных регистрах. В `d86` они видны справа от регистров `ax ... bp`. Их несколько, но при загрузке `com`-программы в них записывается одно и то же значение.

Число в сегментном регистре означает номер 16-байтового параграфа. Отсчет параграфов идет от нулевого линейного адреса. Поскольку $16 = 010$, то линейный адрес параграфа равен $0S_4S_3S_2S_10$, где S_i — шестнадцатеричные цифры, из которых состоит номер параграфа. Если, например, сейчас в *cs*, *ss*, *ds*, *es* — число 06312, то 20-битовый линейный адрес программного сегмента равен 063120.

Итак, сегментные регистры содержат 16-битовый номер параграфа, при его сдвиге на 4 бита влево получается 20-битовый линейный адрес начала параграфа.

Номер параграфа часто называют сегментным адресом, т. к. этот 16-битовый номер задает местоположение областей памяти (сегментов) внутри 20-битового линейного адресного пространства. Но это все-таки номер с шагом 16 байтов, а не адрес байта.

Сейчас строка *s* находится в памяти по адресу, например, 6312:0107, записанному в формате "сегмент:смещение" (точнее, номер параграфа и смещение)⁸. Ему соответствует линейный адрес $063120 + 0107 = 063227$. Задайте в любой строке отображения данных: *r*, 06312, 0107 <Enter>. К сожалению, задать линейный адрес в *d86* не получится, но можно это сделать в отладчике *Bochs*.

Выйдите из *Bochs* и запустите его заново в отладочном варианте: `__debug.bat`. Отладчик *Bochs* остановится на адресе `ffff_fff0`, с которого запускается процессор *i80x86/64*. Нажмите кнопку **Continue** — произойдет загрузка *FreeDOS* и запуск файлового менеджера, как обычно. Окно отладчика теперь серого цвета, т. к. он отдал управление *FreeDOS*, а окно *FreeDOS* свернуто, его можно открыть, нажав на пиктограмму с изображением открытого желтого ящика. Из окна *FteeDOS* вызовите файл `ea_1.com`, напрямую, без *d86* — и отладчик *Bochs*, обрабатывая макрос (3), перехватит управление⁹ и остановится на следующей команде (4).

Прежде чем начать пошаговую отладку, найдем строку *s*. Положение программного сегмента наверняка теперь другое, и придется заново рассчитать его линейный адрес. Действительно, в сегментных регистрах — число 3633 (а было 3632). Эффективный адрес строки *s* — по-прежнему 0107. Получаем $036330 + 0107 = 036437$. В меню **View** вызываем опцию **Linear Dump** (<F7>) или **Physical Dump** (<Ctrl>+<F7>)¹⁰. В появившемся окне **4K Memory Dump** задаем, например, 036430. Строка должна быть хорошо различима в коде ASCII, поэтому расширьте поле **ASCII** в правом верхнем углу окна. Под строкой *s* виден массив `area: 09 08 ... 01 00`.

Обратите внимание, что отображение данных в *Bochs* не обновляется автоматически. То, что вы видите в окне с данными, — это информация на момент нажатия <F7> или <Ctrl>+<F7>. Чтобы увидеть текущие значения, придется заново выполнить <F7> или <Ctrl>+<F7>, а прочие команды *Bochs*, даже **Refresh**, бесполезны.

И, раз уж мы зашли в отладчик, отвлекуемся ненадолго от главной темы и рассмотрим возможности отладки в дополнение к самой простой — пошаговой (**Step**).

Мы можем поставить точки останова, которые сработают или при выполнении машинной команды, т. е. как обычно, или при обращении к данным (чего нет в *d86*). Проверим сначала второй вариант. Выберите элемент массива, например со значением 5, и выполните команду **Command/Watch Write**. (К сожалению, выбранный

элемент ничем не отмечен, пока мы не выполним **Watch**.) Запустите продолжение (**Continue**). При попытке записать 4 на место 5 произойдет останов. Теперь проверим первый вариант останова. Выберите строку (23) и выполните **Command/Breakpoint (ASM selected)**. (Команда **Break** в меню **Command** не имеет отношения к точкам останова, она позволяет вернуться в отладчик, если отлаживаемая программа заиклилась.)

Вы, наверное, заметили, что в отладчике Bochs косвенно адресуемый операнд, например (12), отображается как `ds:[bx]`. Это означает, что при обращении к данным в памяти смещение отсчитывается от параграфа, номер которого записан в регистр `ds` — Data Segment. Тот же регистр `ds` показан в команде (18) с прямой адресацией. А в (20), (22–23), где в косвенной адресации участвует регистр `bp`, значится сегментный регистр `ss` — Stack Segment.

Итак, в трех главных сегментных регистрах `cs` (Code Segment), `ds` (Data Segment) и `ss` (Stack Segment) заданы базовые линейные адреса кода, данных и стека в 20-битовом адресном пространстве. Если речь об адресе перехода или следующей машинной инструкции, то базовое значение находится в `cs`, а эффективный адрес — в `ip` (или образуется прибавлением к `ip` относительного адреса, заданного в команде перехода). При доступе к *данным* базовый адрес задан регистром `ds`, а эффективный — в коде команды (напрямую или значением регистра при косвенной адресации). При доступе к стеку базовый адрес лежит в `ss`, а эффективный — в `sp` или `bp`, в зависимости от команды. И есть еще дополнительный сегмент данных `es` (Extra Segment), который в паре `es:di` служит для адресации операнда-приемника строковых команд `stos`, `movs` и `cmps`.

Настройка регистров при запуске `com`-программы показана на рис. 9.1. Все сегментные регистры настроены на начало программного сегмента (т. е. блока данных, выделенного загрузчиком), данные `com`-файла записаны со смещения `0100`, и это же значение присваивается `ip`.

В первых `0100` байтах сформирован `PSP`, по адресу `0` в нем записан код команды `int 020`, по смещению `080` — длина параметров командной строки, а со смещения `082` — сами эти параметры в текстовом виде, а больше там нет ничего особо интересного. Что касается стека, при `sp = 0` загрузчик выполняет команду `push 0`, в результате чего `sp = 0ffff`, а по адресу `0ffff` записан ноль. Сделано это для того, чтобы из программы можно было выйти как из процедуры — командой `ret`. При условии, что в стек ничего не записано, эта команда передаст управление по нулевому смещению, т. е. в `PSP` на команду `int 020`.

Исполняемая программа, за счет свободной настройки базовых адресов, может быть размещена в любом участке памяти (с точностью до параграфа). Так, за счет сегментации памяти, при разработке `i8088/86` был решен вопрос о перемещаемости кода и данных в физической памяти, что необходимо для организации многозадачных систем¹¹.

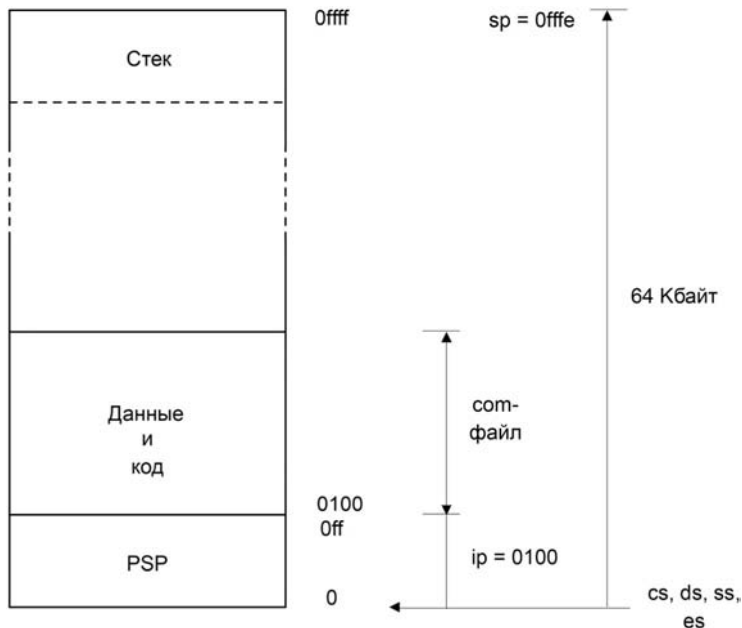


Рис. 9.1. Результат загрузки com-программы

Перепрограммирование сегментных регистров

Пока в сегментных регистрах одно и то же число, мы не можем проверить их назначение. Для такой проверки необходимо записать в какой-нибудь сегментный регистр что-то другое и убедиться, что доступ к памяти изменился. Опыты начнем с относительно свободного¹² регистра `es`.

Регистр `es`

Попытаемся с помощью регистра `es` адресовать данные за пределами программного сегмента. Например, с линейного адреса `0b8000` начинается видеобуфер дисплея в текстовом режиме. Если записать по этому адресу байт, соответствующий знаку *, а в следующий байт — код цвета (например, 5), как в листинге 9.2, то в левом верхнем углу экрана выскочит звездочка (фиолетовая на черном фоне). Каждая строка экрана в текстовом режиме занимает 80 позиций, т. е. 160 байтов, а всего строк на экране 25.

Листинг 9.2. Вывод прямым отображением в видеопамять (`vid_1.8`)

```

mov     ax, 5 by '*'                ; (1)
mov     es, 0b800                   ; laddr = 0b8000 ; (2)
mov     di, 160 * 20                ; (3)
mov     cx, 10                      ; (4)
cld                                       ; (5)
rep stows                           ; (6)
```

В команде (3) задано смещение по вертикали на 20 строк. (Если задать слишком маленькое смещение, то выведенный символ при завершении программы исчезнет — в результате прокрутки экрана.) Линейный адрес данных, адресуемых командой (6), начинается с $0b8000 + 160 * 20 = 0b8000 + 0c80 = 0b8c80$. Этот адрес можно задать несколькими способами, выбирая разные базовые адреса. Например, взять $0b8c80$ в качестве базового, и тогда эффективный адрес равен нулю. Исправьте команды (2–3), как показано далее, и проверьте.

```
mov     es, 0b8c8      ; laddr = 0b8c80      ; (2)
mov     di, 0          ; (3)
```

Аналогично найдите решения для базовых адресов $0b8c00$ и $0b0000$.

Если и дальше уменьшать базовый адрес, то 16-битового смещения однажды не хватит, чтобы "дотянуться" до линейного адреса $0b8c80$. Вот этот предел: $0b8c80 - 010000 = 0a8c80$, т. е. параграф номер $0a8c8$. Максимальное 16-битовое смещение $0ffff$ от границы этого параграфа уже не достигает линейного адреса $0b8c80$. А если сделать базовый адрес больше $0b8c80$, то адрес $0b8c80$ опять станет недоступным, потому что 16-битовый эффективный адрес — число неотрицательное. (Иными словами, линейный адрес не может быть меньше базового.)

Итак, мы обратились к видеобуферу, который находится над программным сегментом. С таким же успехом можно обратиться к памяти ниже программного сегмента. Например, по линейному адресу 0417 (в области данных BIOS) есть байт с признаками нажатия управляющих клавиш <Ctrl>, <Ins>, <Alt> и т. п. Зайдите в отладчик $d86$ и задайте в строке отображения данных $e1, 041, 7$ <Enter>. (Адрес задан двумя числами: номер параграфа 041 и смещение 7 .) Поймать момент нажатия клавиш без фиксации (<Ctrl>, <Ins>, <Alt>) в отладчике не удастся, но можно отследить состояние клавиш с фиксацией — <Caps Lock> и <Num Lock>.

Мы задали линейный адрес 0417 как $041:7$, базовый линейный адрес равен 0410 . Попробуйте другие значения, например 0 и 0400 . Потом задайте отображение $e1, 7$ <Enter>, а в ds (командой `mov`) запишите число 041 — номер параграфа, от которого идет отсчет при доступе к данным. В частности, при выводе данных согласно спецификации $e1, 7$ <Enter> отладчик $d86$ использует текущее значение ds и начальное смещение 7 .

Переделайте программу из листинга 9.2: замените es на ds и вместо строковой команды примените команду `mov` с косвенной адресацией через di (в этом случае базовый линейный адрес определяется регистром ds).

Настраивая по-своему ds и es , мы получаем доступ к данным вне программного сегмента. Чтобы вернуть исходные значения, проще всего восстановить их из регистра cs — в контексте `com`-программы он, как правило, не изменяется и указывает на начало программного сегмента.

Теперь попытаемся изменить базовые адреса стека и кода. Практического смысла в этом нет, но эти трюки помогут освоиться с сегментной моделью памяти и свободнее обращаться с настройками базового адреса.

Регистр ss

В файле `seg_mem.inc` определена макрокоманда `set_ss`, вызов которой смещает `ss` относительно `cs` на расстояние, заданное параметром (число параграфов). Пример ее использования приведен в листинге 9.3.

Листинг 9.3. Согласованное изменение пары `ss:sp` (`stack_1.8`)

```
include seg_mem.inc

    push    0x55aa        ; (1)
    cli     ; (2)
    mov     sp, 0fe-2     ; (3)
    set_ss  0ff0          ; (4)
    sti     ; (5)
    pop     ax            ; (6)
    ret     ; (7)
```

Здесь задано смещение базового адреса стека на `0ff0` параграфов, т. е. на `0ff00` байтов от начала программного сегмента. Увеличение базового адреса стека мы скомпенсировали уменьшением `sp` (было `0fffc`, стало `0fc`), так что вершина стека (по адресу `ss:sp`) осталась на прежнем месте. Проверить просто: число, записанное (1) в стек до его перенастройки, будет прочитано (6) в регистр `ax`, а (7) передаст управление по нулевому адресу, т. е. в начало PSP, куда загрузчик записал команду выхода в DOS.

Обратите внимание, что во время изменения `sp` и `ss` (3–4) мы запрещаем (2) внешние прерывания, а потом разрешаем их (5) вновь.

Хотя вершина стека осталась на месте, его емкость, которая определяется начальным значением `sp`, уменьшилась. После $0fe/2 = 127$ записей стек переполнится, и `sp` станет равным `0fffe`, адресуя память за пределами программного сегмента!

В листинге 9.4 приведен усовершенствованный вариант программы из листинга 9.3.

Листинг 9.4. Изменение `ss:sp` исходя из заданного размера стека (`stack_2.8`)

```
include seg_mem.inc ; (1)

st_sz equ 0100 ; (2)
zero equ 0 ; (3)
zero equ st_sz and 03f ; (4)

    push    055aa        ; (5)
    cli     ; (6)

    mov     dx, sp        ; (7)
    and     dx, not (st_sz-1) ; (8)
```

```

shr      dx, 4                ; (9)
set_ss   dx                  ; (10)
and      sp, st_sz-1         ; (11)

sti                      ; (12)
pop      ax                  ; (13)
ret                      ; (14)

```

Требуемый размер стека задан в строке (2) числом, в котором шесть младших битов должны быть обнулены, что проверяется в (3–4). Получается, что минимальный размер стека — 040, или 64 байта. (Казалось бы, `st_sz = 0` — совершенно недопустимо, но программа составлена так, что `ss` и `sp` в этом случае остаются прежними.) Далее текущее значение `sp` разделяется на две части. Старшая часть, формируемая в (7–8), — это смещение (в байтах) базового адреса стека, а после (9) — то же, но в параграфах, и это значение используется в (10). Младшая часть формируется в (11) — это новое значение `sp`.

Составьте программу, которая установит базовый адрес стека в конце программы (т. е. за неявной меткой `end`), а в `sp` запишет 0100. Проверьте в отладчике с использованием команд `push/pop`.

Регистр `cs`

Что произойдет, если согласованно изменить значение пары `cs:ip`, которая задает линейный адрес инструкции? Будет ли программа правильно выполняться после этого? Скорее всего, будет, потому что, кроме некоторых особых случаев¹³, процессор не записывает в `ip` прямой адрес. Обычно он *увеличивает* `ip` на длину текущей команды, а если встречается переход, то дополнительно на заданный в команде интервал (относительный адрес).

Попробуем одновременно увеличить `cs` на единицу, а `ip` уменьшить на 010. Добиться одновременности можно при помощи команды `jmp` с операндом типа `dword`¹⁴: в его старшем слове задан номер параграфа, а в младшем — эффективный адрес¹⁵. Для начала проверим сам способ перехода, не меняя `cs` и `ip`. В листинге 9.5 приведено определение макроса, который выполняет прыжок на месте, используя для этого 32-битовую переменную, хранящую адрес в формате "сегмент:смещение".

Листинг 9.5. Дальний переход без изменения `cs` (`seg_mem.inc`)

```

leap0    macro
mov      ax, cs                ; (1)
mov      w [m1+2], ax         ; (2)
mov      ax, offset m2        ; (3)
mov      w [m1], ax           ; (4)
jmp      d [m1]               ; (5)

```

```

m1      dd      ?                ; (6)
m2:                                ; (7)
#em

```

Двойное слово, в котором будет сформирован адрес перехода, определено в (6). Команды (1–2) запишут в его старшую часть копию `cs`, а (3–4) — смещение метки (7) в младшую часть (6). (Действие операторов (3–4) можно задать одной командой, без использования регистра, т. к. сочетание прямой и непосредственной адресации в i80x86 допустимо.) Дальний переход (5) записывает в `cs` и `ip` значения из двойного слова (6). В результате мы перейдем на (7).

Добавьте вызов `leap0` в начало программы `ea_1.8` (после вызова `_brk`) и убедитесь в отладчике, что переход из (5) попадает в (7), а `cs` при этом не изменяется.

В листинге 9.6 есть макроопределение, которое увеличивает `cs` на заданное число параграфов, одновременно уменьшая `ip` на соответствующее число байтов (в 16 раз больше числа параграфов, заданного параметром макровывода).

Листинг 9.6. Согласованное изменение `cs` и `ip` (`seg_mem.inc`)

```

leap1  macro
        mov     ax, cs            ; (1)
        add     ax, #1           ; (2)
        mov     w [m1+2], ax     ; (3)
        mov     ax, m2 - (#1*010) ; (4)
        mov     w [m1], ax       ; (5)
        jmp     d [m1]           ; (6)
m1      dd      ?                ; (7)
m2:                                ; (8)
#em

```

Первый параметр вызова `#1` — приращение `cs` в параграфах. Эта величина добавляется (2) к копии `cs`, и результат записывается (3) в старшее слово адреса (7). В младшее слово адреса пишем эффективный адрес метки (8), уменьшив его на число *байтов*, на которое увеличился базовый адрес кода. После выполнения строки (6) мы должны оказаться в (8), но при этом `cs` изменится!

Добавьте вызов `leap1 2` в начало программы `ea_1.8` (после вызова `_brk`) и проверьте в отладчике, что переход из (5) попадает в (7), а `cs` при этом возрастает на 2. Сразу после перехода отображение кода в отладчике `d86` изменится: метки или исчезнут, или появятся в неожиданных местах. Причина в том, что значения имен в `sym`-файле больше не соответствуют эффективным адресам. Например, судя по `lst`-файлу, метка `start` имеет значение `0166`, а `loop1` — `016c`. Поскольку мы принудительно изменили `ip`, отладчик показывает эти метки не там, где они определены в исходном тексте, а остальные не показывает вообще. Тем не менее программа работает.

Регистр *ds*

Скорректировать *ds* после запуска *com*-программы — значит нарушить доступ к данным внутри программного сегмента, потому что эффективный адрес данных зафиксирован при трансляции. Можно проверить в *d86*, вызвав его с программой из листинга 9.1 (убрав из нее недавно добавленные вызовы макросов *leap*). Прежде чем пройти программу по шагам, изменим *ds*. Если увеличить *ds*, то, записывая в массив (8), мы, вероятно, испортим какую-нибудь команду в теле цикла (11–17). Лучше уменьшить *ds* так, чтобы эффективные адреса *x*, *s* и *area* "целились" внутрь *PSP*, куда мы можем записывать что угодно.

Попробуем рассчитать смещение *ds*. Из *lst*-файла определим размер данных (5–8). Эффективный адрес метки *start*, где кончаются данные, — 0150. Если уменьшить базовый адрес хотя бы на 050, то все данные окажутся внутри *PSP*. Проверим. Войдите в отладчик и запишите в *ds* значение *cs*, уменьшенное на 5: `set_ds -5 <Enter>`. Но там сплошные нули, неинтересно. Попробуем уменьшить *ds* так, чтобы вместо строки *s* адресовать копию параметров командной строки с адреса 082. Для вызова *d86* воспользуйтесь командным файлом *ds.bat*¹⁶.

Эффективный адрес строки *s* равен 0109. Мы хотим уменьшить базовый адрес так, чтобы вместо 0109 попасть в 082. Разность равна 087 байтов. Можем уменьшить *ds* на 8 или 9 параграфов (080 или 090 байтов). Вызовите командный файл *ds.bat* и выполните в отладчике команду `set_ds -9`. В шестой строке отображения данных нажмите клавишу "<". Эта программа работает, но не с теми данными, которые планировались при ее составлении: фактические данные сместились в *PSP*, а предполагаемые оказались вне игры.

На практике *ds* можно изменять на время доступа к внешним данным, подобно *es*. В строковых командах пара *ds:si* адресует источник данных (*si* — Source Index), в отличие от пары *es:di*, которая адресует приемник (*di* — Destination Index). В листинге 9.7 приведен пример доступа к видеобufferу в текстовом режиме.

Листинг 9.7. Копирование видеобufferа (*vid_2.8*)

```

mov     ds, 0b800          ; ds:si          ; (1)
mov     si, 0              ;               ; (2)
lea     di, buf            ; es:di          ; (3)
mov     cx, end - buf      ;               ; (4)
cld                                ; (5)
rep movsw                  ; (6)
mov     ds, cs             ; !              ; (7)
int     020                ; (8)

even                                ; (9)
buf     db     "How sweet to be a cloud,", 0a ; (10)
        db     "Floating in the blue.", 0a   ; (11)
        db     "Every little cloud", 0a      ; (12)
        db     "Always sings aloud."        ; (13)
```

Похоже на программу из листинга 9.2, только копирование идет из видеобuffers в память (10–13), заданную в программе¹⁷.

Войдите в отладчик с `vid_2.com` и выполните программу целиком: `g <Enter>`. В строке отображения данных вместо стишка, заданного в (10–13), будут литеры верхней строки экрана `d86`, чередующиеся со значком треугольника — так отображается число, задающее цветовой атрибут (белое на синем фоне) символа в видеобufferе. Чему оно равно, кстати? Можно изменить формат отображения в первой строке на `rx40` или `rb20`, в первом случае каждый второй байт не будет выводиться (`x` — Skip), а во втором он будет отображаться в hex-коде.

Почему надо было пройти программу сразу до конца? Потому что в (1) мы изменили `ds`, в результате чего в строке данных вывелось бы неизвестно что с адреса `0b800:buf`. Поэтому до восстановления `ds = cs` (7) лучше не останавливаться.

При наблюдении *кода* в окне `d86` операторы (1) и (7) выглядят не так, как в программе. Дело в том, что прямая запись константы в сегментный регистр невозможна, так же как копирование между сегментными регистрами. Только через регистр общего назначения (например, `ax`), или стек — парой команд `push/pop`. В `a86` команды, подобные (1) и (7), реализованы как встроенные макросы, в `d86` мы видим результаты их вызова.

Итак, для `com`-программ мы проверили возможность изменения базовых адресов, управляющих доступом к коду, данным и стеку. Практический интерес представляет изменение доступа к данным, а прочие трюки — это лишь демонстрация гибкости сегментной модели памяти в реальном режиме. И под конец еще один пример: мы настроим сегментные регистры и передадим управление на программу, которая после завершения осталась в памяти (так называемая резидентная программа).

Повторный запуск резидентной программы

В листинге 9.8 приведена программа `vid_3.8` с двумя вариантами завершения: обычным, с освобождением памяти, которую распределил загрузчик, и так называемым TSR (Terminate and Stay Resident) с сохранением части программного сегмента в памяти.

Листинг 9.8. Программа для повторного запуска (`vid_3.8`)

```
extrn    rep_cnt:w,tsr:b                                ; (1)

mov      ax, 5 by '*'                                    ; (2)
mov      es, 0b800          ; laddr = 0b8000            ; (3)
mov      di, 160*20                                           ; (4)
mov      cx, rep_cnt                                           ; (5)
inc      rep_cnt                                              ; (6)
cld                                           ; (7)
rep stowsw                                           ; (8)
```



```

        test    tsr                        ; (9)
        jnz     >19                       ; (10)
        inc     tsr                       ; (11)

p_sz    equ     (end shr 4) + 1           ; (12)

        mov     ah, 031                   ; (13)
        mov     dx, p_sz                  ; (14)
        int     021                       ; (15)
19:                                           ; (16)
        mov     ah, 04c                   ; (17)
        int     021                       ; (18)

tsr      db      0                        ; (19)
rep_cnt  dw      3                        ; (20)

```

Команды (2–8) выполняют вывод в видеобuffer аналогично программе vid_1.8 из листинга 9.2. Разница только в том, что в счетчик повторов `cx` записывается не константа, а переменная (20), которая тут же инкрементируется (6). При первом вызове vid_1.com будет выведено три звездочки, при втором — четыре и т. д. После вывода проверяем (9) переменную в строке (19), изначально она равна нулю, поэтому первый раз мы проходим по ветке (11–15), где задана функция DOS (13–15) для завершения программы с сохранением ее в памяти. В следующий раз, поскольку `tsr = 1`, произойдет переход на (16–18) — обычный выход в DOS.

Чтобы оставить программу в памяти при завершении, используем функцию 031 DOS. Номер функции записывается (13) в `ah`, а в `dx` — размер сохраняемой памяти (в параграфах) от начала программного сегмента¹⁸, затем команда (18) выполняет программный вызов прерывания по вектору 021, обращаясь к сервису операционной системы¹⁹. Размер сохраняемой памяти (константа `p_sz`) виден в `lst-файле`.

Чтобы запустить эту программу повторно, надо выяснить, с какого параграфа она расположена в памяти, затем настроить регистры так, как показано на рис. 9.2: сегментные регистры — на начало PSP, `ip := 0100`, а что касается стека²⁰, то `sp := 0100`.

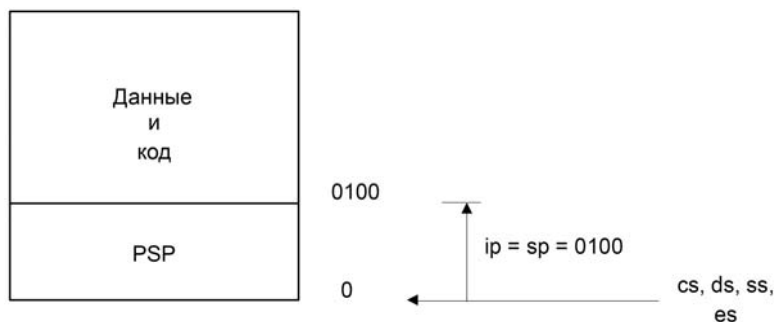


Рис. 9.2. Программа vid_3 при повторном запуске

Выполните команду DOS `mem /u`. В полученном списке резидентных модулей найдите `vid_3`. Его сегментный адрес — в левой колонке. Прибавив к нему единицу²¹, получите номер параграфа, с которого начинается программный сегмент `vid_3`. Это число надо будет записать во все сегментные регистры.

Передача управления резидентной программе закодирована в отдельной программе, показанной в листинге 9.9.

Листинг 9.9. Повторный запуск резидентной программы `vid_3` (`jump.8`)

```
base      equ      0???? + 1      ; (1)

          cli                      ; (2)
          mov      ax, base        ; (3)
          mov      ds, ax          ; (4)
          mov      es, ax          ; (5)
          mov      ss, ax          ; (6)
          mov      sp, 0100        ; (7)
          sti                      ; (8)

          jmp      base:0100        ; (9)
```

Подставьте в (1) значение сегментного адреса `vid_3`, прочитанное из списка модулей (не увеличенное на единицу). При запрещенных в строке (2) внешних прерываниях пишем (3–6) значение (1) в `ds`, `es` и `ss`, перемещаем (7) указатель стека на вершину `PSP` и, разрешив прерывания (8), выполняем дальний переход в точку входа `vid_3`, тем самым устанавливая `cs` и `ip`.

Программа `vid_3`, получив управление, выведет на экран четыре звездочки и завершится обычным способом, т. к. после первого выполнения `tsr = 1`. Строго говоря, самостоятельной программы `vid_3` нет, а есть только ее фрагмент кода и данных, выполняемый как часть программы `jump`. При запуске `jump` загрузчик распределил, как обычно, 64 Кбайт памяти, и этот блок должен быть освобожден при ее завершении. То, что завершение `jump` происходит в блоке `vid_3`, не имеет значения, главное, что `jump` заканчивается выходом в DOS с освобождением выделенной ей памяти.

При каждом выполнении `vid_3` увеличивается счетчик `rep_cnt`, так что при первом вызове `jump-vid_3` будет выведено четыре звездочки, при втором — пять и т. д.

Ненадолго отложим забавы на тему эффективных и линейных адресов и рассмотрим, каким образом сегментная модель памяти поддерживается в языке ассемблера.

Программные секции

В `com`-программе все базовые адреса настроены на начало программного сегмента, и если требуется изменить базовый адрес данных, то это, как правило, данные *за*

пределами программы, чаще по каким-то фиксированным адресам, как, например, видеобуфер. Но если собственных данных (в пределах программы) становится много, исходный текст разбивают на секции. Каждая секция транслируется с нулевого смещения и имеет свой базовый адрес, который при переключении секций записывается в сегментный регистр `ds`. Максимальный размер секции — 64 Кбайт.

Для примера рассмотрим вариант программы из листинга 9.10, но с двумя секциями данных, двумя секциями кода, а также с отдельной секцией стека²². Секции, или *логические* сегменты, обозначаются ключевым словом `segment`, перед которым задано имя секции.

Листинг 9.10. Программа `ea1.8`, разбитая на секции (`ea2.08`)

```

D2 segment                                ; (1)
    db      "(segment D2)", 0             ; (2)
area      db      9,8,7,6,5,4,3,2,1,0      ; (3)

D1 segment                                ; (4)
s         db      "(segment D1)", 0        ; (5)

_stack segment stack                      ; (6)
    db      "(stack segment)", 0          ; (7)
    db      40 dup 1                      ; (8)

C2 segment                                ; (9)
    db      "(segment C2)", 0             ; (10)
work_with_area:                          ; (11)
    mov     bx, offset area               ; (12)
loop1:                                       ; (13)
    mov     al, [bx]                     ; (14)
    dec     al                           ; (15)
    js      exit                         ; (16)
    mov     [bx], al                     ; (17)
    inc     bx                           ; (18)
    jmp     loop1                        ; (19)
exit:                                         ; (20)
    retf                                  ; (21)

D1 segment                                ; (22)
x         dw      01155, 0               ; (23)

C1 segment                                ; (24)
    db      "(segment C1)", 0             ; (25)
main:                                           ; (26)
    mov     ds, D2                       ; (27)
    call    far work_with_area           ; (28)

```

```
mov    ds, D1                ; (29)
not     x                    ; (30)
inc     s                    ; (31)

mov     ah, 04c              ; (32)
int     021                  ; (33)
```

Выполните трансляцию следующим образом: `a86 +O ea2.08 +HL5`. Ключ `+O` включает так называемый объектный режим `a86`: результат трансляции — `obj`-файл, из которого при помощи компоновщика получаем `exe`-файл.

В `lst`-файле обратите внимание на то, что эффективные адреса при трансляции каждой новой секции начинаются с нуля. Секция (22–23) не новая, она была начата раньше (4–5), поэтому трансляция возобновляется со смещения, достигнутого при обработке (4–5). Иными словами, секции с одинаковыми именами объединяются.

Секция со строки (6) с атрибутом `stack` резервирует место для стека, причем настройка `ss:sp` будет выполнена при загрузке автоматически. Точка входа в программу (26) определена меткой²³ `main`, для `a86` это ключевое слово. Изначально `ds` и `es` настроены на `PSP`, как показано на рис. 9.3. Поэтому перед обращением к данным секции `D2` мы записываем (27) ее сегментный адрес в `ds`. В строке (29) мы обращаемся к данным другой секции — и заново настраиваем `ds`.

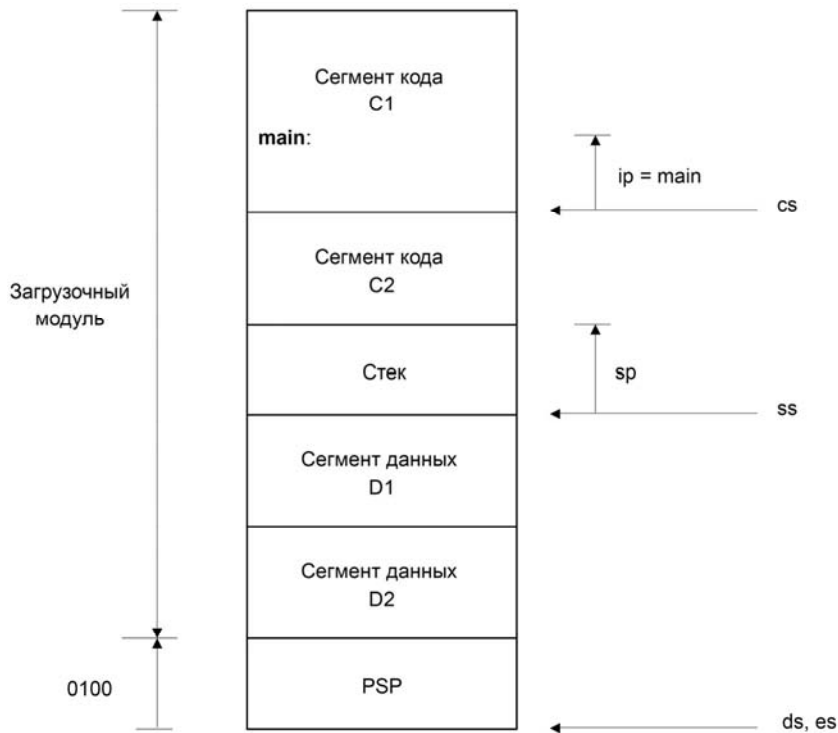


Рис. 9.3. Программа `ea2.exe` сразу после запуска

Код тоже поделен на секции, и поскольку у них разные имена, то и базовые адреса будут различными. Поэтому переходы (28) и возвраты из подпрограмм (21) должны быть дальними, т. к. они должны изменять значение `cs`.

В качестве компоновщика можно взять, например, утилиту `warplink` из коллекции **old-dos.ru**. Этот компоновщик отличается подробной диагностикой и даже дает советы при ошибках²⁴. Из всего архива `warplink` достаточно взять `warplink.exe`.

Выполните команду: `warplink /mx ea2.obj`

Ключ `/mx` задает формирование подробного `map`-файла, где показан не только результат объединения одноименных секций, но и его участники.

В полученном `map`-файле для каждой секции указано ее *смещение* (в параграфах) от начала загрузочного модуля. Именно эти относительные значения закодированы в (27–29), поскольку при подготовке программы неизвестно, по какому линейному адресу она будет загружена. Когда загрузчик определится с базовым адресом, он скорректирует код (27–29) так, что сегментные адреса там будут уже не относительные, а абсолютные.

Запустите `d86` с полученным `exe`-файлом. Эффективный адрес точки входа `main` случайно совпал с эффективным адресом `work_with_area` (эти имена относятся к разным секциям), и почему-то `d86` предпочел показать `work_with_area`. Далее отображаемые имена соответствуют²⁵ листингу 9.10.

Изначально данные, отображаемые в строках 1–3 отладчика, не соответствуют программе. Не удивительно, ведь `ds` сейчас указывает на начало `PSP`. Пройдя четыре команды, реализующие запись (27) `ds`, увидим, что данные секции `D2` отображаются правильно (строки 1), а секции `D1` — неправильно²⁶ (строки 2 и 3). Понятно почему, ведь мы настроили `ds` на `D2`, а не на `D1`.

При выполнении дальнего перехода (28) `cs` изменится, но после обработки (12–19) массива `area` и выполнения дальнего возврата (21) мы вернемся на (29), где перенастроим `ds` для доступа к секции `D1`. Как и следовало ожидать, после перенастройки `ds` в строках 2–3 отладчика все правильно, а строка 1, наоборот, показывает не то.

В листингах 9.11а и 9.11б приведена та же программа, разбитая на два исходных модуля. Чтобы компоновщик объединил одноименные секции разных модулей, необходимо в исходных текстах добавить к ключевому слову `segment` атрибут `public`.

Листинг 9.11а. Главный модуль, ссылающийся на имена из других модулей (ea2_1.08)

```
extrn    work_with_area:far, x:word           ; (1)

D2 segment                                     ; (2)
    db    "(segment D2)"                     ; (3)
area     db    9,8,7,6,5,4,3,2,1,0           ; (4)

D1 segment public                             ; (5)
s        db    "(segment D1)"               ; (6)
```

```

_stack segment stack                ; (7)
        db      "(stack segment)"  ; (8)

C1 segment                          ; (9)
        db      "(segment C2)"     ; (10)

main:                                  ; (11)
        mov     ds, D2              ; (12)
        call    work_with_area     ; (13)

        mov     ds, D1              ; (14)
        not     x                   ; (15)
        inc     s                   ; (16)

        mov     ah, 04c             ; (17)
        int     021                 ; (18)

```

Директива `extrn` необходима при трансляции, если имя определено в другом модуле, иначе `a86` выдаст ошибку.

Листинг 9.116. Модуль с открытым доступом к переменной и подпрограмме (ea2_2.08)

```

        public  work_with_area, x   ; (1)

_stack segment stack                ; (2)
        db      40 dup 1            ; (3)

D1 segment public                   ; (4)
x        dw      01155, 0           ; (5)

C2 segment                          ; (6)
        db      "(segment C1)"     ; (7)

work_with_area:                     ; (8)
        mov     bx, offset area     ; (9)

loop1:                               ; (10)
        mov     al, [bx]             ; (11)
        dec     al                   ; (12)
        js      exit                 ; (13)
        mov     [bx], al             ; (14)
        inc     bx                   ; (15)
        jmp     loop1               ; (16)

exit:                                ; (17)
        retf                          ; (18)

```

Директива `public` означает, что перечисленные имена будут записаны в `obj`-файл как глобальные, т. е. доступные другим модулям. Без этой директивы компоновщик не сможет собрать программу из объектных модулей. Вызов компоновщика для рассматриваемого случая: `warplink /mx ea2_1.obj ea2_2.obj`. Результат — `ea2_1.exe`.

Если посмотреть на шестнадцатеричный код ехе-файлов, полученных в рассмотренных примерах, видно, что данные, стек и код расположены довольно компактно в конце, а до смещения 0200 находится служебная информация. Прочесть ее можно при помощи утилиты `readexe` из коллекции `old-dos.ru` — частично. Описание ехе-формата: tuttle.github.io/2015/03/28/mz-exe-files.html.

Теперь вернемся к теме эффективных и линейных адресов и рассмотрим возможность однократного (в пределах одной команды) изменения базового адреса.

Префикс переназначения сегмента

Вспомним правила использования сегментных регистров.

- Для считывания очередной инструкции применяется пара `cs` и `ip`.
- Для обращения к стеку — `ss` и `sp/bp`.
- Для обращения к данным — сегментный регистр `ds` и смещение, заданное в команде.
- Если смещение задано косвенно через `bp`, то базовый адрес берется из `ss`.
- Операнд-приемник строковой команды адресуется с парой `es:di`.

Если по умолчанию используется `ds`, то можно назначить команде другой сегментный регистр, добавив префикс²⁷ *переназначения* сегмента. Можно также переназначить `ss`, но только при косвенной адресации через `bp`. Переназначить `es` нельзя²⁸.

Проверим эти утверждения. Следующая программа разбита на два исходных модуля: в листинге 9.12а начальный фрагмент для инициализации данных, а в листинге 9.12б — продолжение, для иллюстрации правил использования сегментных регистров и префиксов переназначения.

Листинг 9.12а. Фрагмент для инициализации данных (prep.8)

```
include seg_mem.inc                                ; (1)

struc                                              ; (2)
b0      db          ; ea = 0, byte                ; (3)
w0      dw  ?       ; ea = 0, word                 ; (4)
w2      dw  ?       ; ea = 2, word                 ; (5)
ends                                          ; (6)

fill    macro                                          ; (7)
        mov     bx, 0                                ; (8)
        mov     cx, #1                              ; (9)
m1:     cs mov   [bx], bl                             ; (10)
        inc     bl                                  ; (11)
        loop    m1                                  ; (12)
#em                                          ; (13)
```

```

fill    256                                ; (14)
cli                                           ; (15)
mov     bp, sp, 020                         ; (16)
set_ss  3                                    ; (17)
sti                                           ; (18)

set_ds  1                                    ; (19)
set_es  2                                    ; (20)

```

В макроопределении (7–13) задана запись возрастающей последовательности чисел от начала PSP, так чтобы байт по адресу 0 содержал 0, байт по адресу 1 — число 1 и т. д. до адреса, заданного параметром. В результате макровывоза (14) значение каждого байта PSP равно его смещению от начала программного сегмента, что поможет нам определить адрес элемента данных при доступе к нему. (Эта макрокоманда также используется в листинге 9.12б для повторной инициализации перед выполнением следующего теста.)

В строках (17) и (19–20) устанавливаем сегментные регистры: смещаем базу сегмента данных (19) на 010 байтов, базу дополнительного сегмента данных (20) — на 020 байтов, а базу стека (17) — на 030 байтов. В строке (16) устанавливаем `sp = 020`, так что вершина стека находится по смещению 050 от начала PSP.

В строках (2–6) задано определение *структуры*. Встретив его, транслятор запоминает текущее значение счетчика адресов и обнуляет его. В результате `b0` получит нулевое значение и тип `byte`, `w0` — 0 (поскольку в (3) ни одного байта не распределили) и тип `word`, а `w2` — 2 и тип `word`. После (6) транслятор восстанавливает счетчик адресов и продолжает обработку исходного текста. Таким неочевидным способом мы определили имена, которыми можно заменить громоздкие обозначения `b[0]`, `w[0]` и `w[2]`.

Текст `prep.8` входит в начало основной программы `steps.8`, он подключается директивой `include`. Программу `steps.8` мы разберем чуть позже, а пока просто выполните ее трансляцию и вызовите с отладчиком `d86`. В файле `steps.d8k` задана не только настройка строк отображения данных, но и команда `g <Enter>`. Вся подготовительная часть (`prep.8`) будет отработана до точки останова.

Несколько раз нажмите `<F10>`, чтобы в правой верхней части экрана появилось продолжение шестой строки отображения данных, как показано на рис. 9.4.

На рис. 9.4 обратите внимание, что значения сегментных регистров разные. Судя по заполнению памяти в строках 1–3 и 5, базовые адреса сегментов смещены относительно друг друга на 010 байтов. (Значение каждого байта в PSP после инициализации — это его смещение от начала PSP.) Пять слов от вершины стека вниз, видимо, были использованы²⁹ системными процедурами обработки прерываний, но остальные значения (в строке шесть и в дополнительном окне данных) идут в порядке возрастания.

Перейдем к экспериментальной части; исходный код показан в листинге 9.12б.

TEST1:	0020	5150	5352	5554	5756	5958	5B5A
# 0129 INT 3	002C	5D5C	5F5E	6160	6362	6564	6766
012A PUSH 1	0038	6968	6B6A	6D6C	6F6E	7170	7372
012C POP AX	0044	7574	7776	7978	7B7A	7D7C	7F7E
012D NOT B[0]	0050	8180	8382	8584	8786	8988	8B8A
...							
0143 MOVSB	00BC	EDEC	EFEE	F1F0	F3F2	F5F4	F7F6

AX 6314	i	e	1:	b16,cs,0,	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
BX 0000	IP	0129	2:	b16,ds,0,	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
CX 0000	CS	6312	3:	b16,es,0,	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F
DX 6312	SS	6315	4:	w4,ss,bp-2,	0206	5150	5352	5554												
SI 0000	DS	6313	5:	w8,ss,0,	3130	3332	3534	3736	3938	3B3A	3D3C	3F3E								
DI 0000	ES	6314	6:	"0010	4140	4342	4544	0000	6314	0129	6312	0206								
BP 0020	SP	0020	0:																	

Рис. 9.4. Экран d86 перед началом опытов с сегментными регистрами

Листинг 9.126. Программа для опытов с сегментными регистрами (steps.8)

```

include prep.8                                ; (1)

test1: int      3                              ; (2)

        push    1                            ; ss:sp      ; (3)
        pop     ax                            ; ss:sp      ; (4)

        not     b0                            ; ds:b0      ; (5)
        not     w0                            ; ds:w0      ; (6)
        mov     bx, 2                          ; (7)
        not     b[bx]                        ; ds:bx      ; (8)

        mov     si, 0                          ; (9)
        mov     di, 2                          ; (10)
        cld                                         ; (11)
        movsb   3                            ; ds:si -> es:di ; (12)

test2: int      3                              ; (13)
        fill    64                             ; (14)
        int     3                              ; (15)

        push    w0                            ; ds:w0 -> ss:sp ; (16)
        push    2                             ; -> ss:sp      ; (17)
        call    subr                          ; ip -> ss:sp    ; (18)

test3: int      3                              ; (19)
        fill    64                             ; (20)
        int     3                              ; (21)

```

```

not  es:b0          ; es:b0          ; (22)
ss not w0           ; ss:w0          ; (23)
cs not w2           ; cs:w2          ; (24)

cs movsb 4          ; cs:si -> es:di  ; (25)
not    b [bp]       ; ss:bp          ; (26)
not    b cs:[bp]    ; cs:bp          ; (27)

push   cs:[w0]       ; cs:w0 -> ss:sp ; (28)
push   1             ;              ; (29)
es call subr         ; ??           ; (30)

mov     ah, 04c       ; exit          ; (31)
int     021           ;              ; (32)

subr:
enter   2             ; push bp / mov bp, sp / sub sp, 2 ; (34)

mov     w[bp-2], 0     ; ss:bp-2          ; (35)
mov     ax, w[bp+6]    ; ss:bp+6          ; (36)
mov     cx, w[bp+4]    ; ss:bp+4          ; (37)
ror     ax, cl         ;              ; (38)
leave                   ; sp := bp / pop bp ; (39)
ret     4              ; ret / add sp, 4   ; (40)

```

Откройте программу в отладчике и, сойдя с точки останова (2), продолжите выполнение по шагам. Выполнив (3), мы увидим, где находится вершина стека. Команды (5–6) и (8) используют регистр `ds`, так что изменения видны во второй строке отображения данных. Команда (12) копирует из второй строки отладчика в третью, с дополнительным смещением, поскольку `si` и `di` различаются на 2.

Перейдя на (14), выполните `g <Enter>`. В строке (14) задана инициализация 64 байта от начала `PSP`, потому что дальше идет стек, и его лучше не трогать³⁰. Перейдите на (16). Выполните (16–17) и войдите в подпрограмму (33–40). Здесь показан пример использования стека для передачи параметров и резервирования локальных переменных подпрограммы, а также роль регистра `bp` для доступа к этим данным. Команда (34) сохраняет `bp` в стеке, затем приравнивает его к текущему `sp` и дополнительно смещает `sp` на указанное число байтов — это место для локальных переменных подпрограммы. В результате `bp` указывает на собственную копию в стеке, выше (по адресу `[bp+2]`) находится адрес возврата, записанный при выполнении (18), а еще выше — параметры: число 2 (по адресу `[bp+4]`), а над ним копия `w0` (по адресу `[bp+6]`). Ниже (с адреса `[bp-2]`) — место для локальных переменных.

В строке (35) обнуляем слово, зарезервированное под локальные данные, — чтобы увидеть его местоположение в строках 4 и 6 отображения данных. Считываем (36–37) параметры из стека в регистры и что-то с ними делаем (38), формируя результат в `ax`. Команда (39) переустанавливает³¹ `sp` так, что он указывает на копию `bp` в сте-

ке, а затем восстанавливает `bp`. Команда (40) считывает адрес возврата, увеличивает `sp`, тем самым освобождая место, занятое параметрами вызова, и выполняет возврат³².

В третьем опыте (20–30) используем префиксы переназначения сегмента. Обычно принято задавать префикс перед операндом через двоеточие, как в (22) и (27–28). В `a86` и `nasn` разрешено также записывать префикс перед командой, как в (23–25) и (30), что более соответствует машинному представлению. В (30) префикс не имеет смысла, т. к. `ds` в этом варианте инструкции `call` не используется³³. В (24) и (27–28) указан `cs`, причем в (27) он заменяет `ss`.

Кстати, в листинге 9.12а регистр `cs` заменяет `ds` — в команде (10), которая выполняет запись в `PSP`. Это сделано потому, что в листинге 9.12б регистр `ds` смещен на один параграф относительно начала `PSP`, в то время как `cs` как раз указывает на начало.

В рассмотренных примерах номер сегмента нигде не был задан числом — только значением сегментного регистра. Числовое обозначение номера сегмента возможно лишь в командах дальней (межсегментной) передачи управления (листинг 9.13).

Листинг 9.13. Непосредственное задание сегментного адреса в командах (`reboot.8`)

```
inc    w 0:[di]          ; ?!          ; (1)
dec    b 2:1             ; ?! (bug)    ; (2)
jmp     0f000:0ffff0     ; (3)
```

Команду (1) `a86` сочтет ошибкой, но (2) почему-то пропустит, хотя сгенерирует нечто странное. Закомментируйте (1–2), повторите трансляцию и выполните `reboot.com`.

Для изменения значений в сегментных регистрах применяются только команды пересылки, а для изменения `cs` — только команды передачи управления (листинг 9.14)³⁴.

Листинг 9.14. Изменения значений сегментных регистров (`seg_reg.8`)

```
a86_macros:                                     ; (1)
    mov     es, 0b800                          ; (2)
    mov     es, cs                             ; (3)
    mov     cs, ds                             ; ?!          ; (4)
no_macro:                                       ; (5)
    mov     ax, 070                            ; mov es, 070      ; (6)
    mov     es, ax                             ; (7)

    mov     ax, es                             ; mov ds, es      ; (8)
    mov     ds, ax                             ; (9)

    push    cs                                ; mov ds, cs      ; (10)
    pop     ds                                ; (11)
```

```

far_ptr:                                ; (12)
    les    di, d[far_adr]                ; es:di    ; (13)

    lds    si, d[far_adr]                ; ds:si    ; (14)
    lds    si, d[far_adr]                ; ?        ; (15)
    cs lds  si, d[far_adr]                ; !        ; (16)
exit:                                     ; (17)
    mov     ah, 04c                      ; (18)
    int     021                          ; (19)

far_adr dd                                ; (20)
        dw     2, 0b800                  ; (21)

```

Команды (2–3) не имеют машинного выражения — это встроенные макросы `a86`. (Если один из операндов команды пересылки — сегментный регистр, то в машинном коде другим операндом может быть только регистр общего назначения.) Команда (4) недопустима: запись в регистр `cs` равносильна передаче управления, а для этого есть `jmp`.

Закомментируйте (4), повторите трансляцию и войдите в `d86` с `seg_reg.com`. В строках 2–3 отображения данных выводятся байты с адреса `es:0` и `ds:0`. Изначально это первые байты `PSP`. После выполнения (2) во второй строке отображается видеобuffer (верхняя строка самого отладчика). После (3) возвращаемся к исходному изображению.

Все последующие команды не являются встроенными макросами, т. е. для них есть машинное представление. Пара (6–7) записывает в сегментный регистр непосредственное значение. Пары (8–9) и (10–11) демонстрируют два варианта передачи значений между сегментными регистрами: через регистр общего назначения и через стек. Во всех случаях `cs` может быть только источником.

Команды (13–16) загружают сразу пару регистров: сегментный регистр, обозначенный в мнемонике команды³⁵, и регистр общего назначения, указанный первым операндом. Вторым операндом задан адрес двойного слова, где записано смещение (эффективный адрес) в младшем слове и сегментный адрес (номер параграфа) в старшем слове. После команды (13) `es = 0b800`, а `di = 2`. После (14) аналогичные значения примут `ds` и `si`. Но, поскольку `ds` изменился, что сразу видно по данным, отображаемым в строке 1, команда (15) запишет в `ds:si` случайные значения. Все потому, что при обращении к `far_adr` по умолчанию используется регистр `ds`. Поэтому в (16) явно указан префикс `cs`. Регистр `cs` в `com`-программе всегда³⁶ адресует начало `PSP`. Потому именно его применяют для восстановления `ds` (10–11) и `es` (3).

Задачи

1. Взяв за точку отсчета начало `PSP` (листинг 9.15), вычислите (и проверьте в отладчике), к каким адресам обратятся следующие команды? По какому адресу будет выполнен переход?

Листинг 9.15. Определение адреса памяти относительно начала PSP (task1.8)

```
include prep.8                                ; (1)

task1:  int      3                             ; (2)

        push     ax                           ; (3)
        mov      di, 2                         ; (4)
        mov      si, 4                         ; (5)
        not      w [bp+di]                     ; (6)
        cs movsw                                ; (7)
        mov      bx, 6                         ; (8)
        mov      cs:[bx], 01234                ; (9)
        es mov    [2], bx                      ; (10)
        cs jmp    [bx]                         ; (11)
```

2. Найдите ошибки и лишние (или бессмысленные) префиксы в листинге 9.16.

Листинг 9.16. Ошибки при использовании сегментов и префиксов (task2.8)

```
include prep.8                                ; (1)

task2:  int      3                             ; (2)

        cs push  0                             ; (3)
        cs push w[0]                           ; (4)
        mov      ss:[bp], 0abcd                 ; (5)
        es jmp    w[0]                          ; (6)
        ds not    w[bx+2]                       ; (7)
        rep not   w[0]                           ; (8)
        cs call   d[0]                           ; (9)
```

3. По абсолютному адресу 0417 находится байт с информацией о нажатии управляющих клавиш (<Ctrl>, <Ins>, <Alt> и т. п.). Используя из сегментных регистров только `ds`, скопируйте этот байт в видеопамять. (Если хотите проверять в отладчике, то учтите, что `d86` при каждом действии полностью обновляет экран кроме нижней строки — в нее и можно выводить результаты.) Затем решите эту задачу, применив только сегментный регистр `es`.
4. Закодируйте переход по адресу, составные части которого (смещение и номер параграфа) лежат в памяти по абсолютному адресу 0 (т. е. 0:0). Это адрес системной процедуры обработки переполнения при делении, но требуется вызвать ее не косвенно (делением на ноль), а напрямую при помощи команды `jmp`.

Примечания

- Отсюда и название команды `LEA` — Load Effective Address.
- Начальный линейный адрес области данных в дальнейшем называется ее базовым адресом. В `i80x86` этот адрес 32-битовый. Не путать с косвенным 16-битовым адресом в так называемых базовых регистрах `bx/bp`.

3. Если механизм трансляции адресов выключен, то физический адрес (на адресной шине) равен линейному. Если трансляция адресов включена, то в общем случае физический адрес отличается от линейного, или *виртуального*.
4. При коротком переходе дистанция ограничена 128 байтами вверх и 127 байтами вниз.
5. Команда <Ctrl>+<T> после переноса стека говорит отладчику d86: "вот новая вершина стека", в результате чего нулевая строка данных (отображение стека) очистится.
6. В любом случае адресация стека косвенная.
7. В отличие от регистра `bx`, который адресует *данные*.
8. Сегментный адрес у вас может быть другим, подставьте свой.
9. В отладочном режиме `Bochs` воспринимает команду обмена регистра `bx` с самим собой как точку останова — `magic breakpoint`. Эта команда, по-видимому, не встречается в коде операционной системы и инструментальных программ, иначе бы мы остановились раньше.
10. Пока не включена трансляция адресов, разницы нет.
11. В этом отношении `i8080/85` был безнадежен. Другое решение — страничное преобразование, или трансляция адресов, но для его реализации в РС тогда еще не пришло время.
12. Регистр `cs` занят всегда, т. к. на нем держится адресация исполняемого кода; `ds` чаще всего тоже занят, если только команда не обращается к регистрам и непосредственным значениям; `ss` не используется лишь тогда, когда нет действий со стеком (`call`, `push/pop`, `int`, внешние прерывания).
13. Это команды переходов вроде `jmp ax`. или `jmp w[mem16]`.
14. Как раз тот случай, когда новое значение `cs:ip` задано прямо, а не смещением.
15. Как всегда в `i80x86`, по старшему адресу находится наиболее весомая часть данных.
16. В параметрах командной строки приведен полный текст стихотворения "Bad Morning" (Langston Hughes). В переводе, "Надел сапог не на ту ногу. С утра не везет, вот ей богу".
17. В данных программы изначально записано начало стишка из книги Winnie-the-Pooh (Alan Alexander Milne) — "Я тучка, тучка, тучка, я вовсе не медведь..." в переводе Б. Заходера.
18. То есть отбрасываем верхнюю часть блока 64 Кбайт, распределенного при загрузке.
19. О прерываниях разговор в следующей главе. Что касается вызовов BIOS и DOS, к ним мы обращаемся лишь в крайних случаях, как сейчас. Вывод на экран выполняем только прямой записью в видеопамять, минуя BIOS и DOS.

20. Нельзя установить `sp = 0ffff`, как при загрузке `com`-программы, т. к. это уже за пределами резидентной программы. В качестве стека используем 128 слов `PSP`, этого достаточно.
21. В начале блока памяти находятся 16 байтов управляющей информации (`MCR` — `Memory Control Block`), к нему примыкают собственно данные. Система всегда распределяет на 16 байтов больше запрошенного значения. Записывать в `MCR` ничего нельзя, разрушится динамическая память.
22. Секции здесь совсем небольшие, это учебный пример.
23. Еще одна уникальная черта `a86`. В большинстве ассемблеров пришлось бы задать в конце исходного текста директиву `end main`.
24. С ключом `/c` он преобразует генерируемую `exe`-программу в `com`-формат, если это возможно. Это преобразование требуется тогда, когда `com`-программу собирают из `obj`-модулей, что нам вряд ли понадобится.
25. Имена при отладке `exe`-программ в `d86` часто вводят в заблуждение. Поэтому лучше отказаться от создания `sym`-файла, задав ключ `+S` при запуске `a86`.
26. В строке 3 отображается "(segment D2)", в то время как имени `s` секции `D1` соответствует "(segment D1)". Это все из-за совпадения эффективных адресов в разных секциях.
27. Префикс — это байт перед кодом команды, который влияет на ее работу, как, например, знакомый вам префикс `rep` для строковых команд.
28. Поставить префикс перед командой `stos` (запись `al` в память по адресу `es:di`) можно, но процессор его проигнорирует, поскольку `es` не переназначается. Это бессмыслица, хотя и не ошибка — вроде префикса `rep` перед обычными (не-строковыми) командами.
29. Именно "были", потому что сейчас `sp = 020`.
30. При первой инициализации в `prer.8` вершина стека была далеко за пределами `PSP` (`sp = 0ffff`).
31. При этом освобождается место, занятое локальными переменными.
32. Вариант, когда процедура сама освобождает место, занятое параметрами, характерен для языков семейства `Pascal`. В `C`-подобных языках иначе: *вызывающая* программа увеличивает `sp` — следующим оператором после вызова процедуры.
33. Другое дело в случае `jmp/call w[mem16]` — тогда адрес перехода задан в слове по адресу `ds:mem16`.
34. Прибавить к сегментному регистру или вычесть из него напрямую нельзя — только через промежуточный регистр. Что касается передач управления, то они представляют собой, по сути, те же команды пересылки, только записанные иначе.
35. Команда `LDS` означает `Load DS`, а `LES` — `Load ES`.
36. Кроме как в листинге 9.6 разве что, но это была шутка.

глава 10

```
DB6 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA i c 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:

lea si,b_1
```

Исключения

Булавка туда, булавка сюда —
Любой замок для нас срунда.
(Сериял "Детективное агентство "Лунный свет"¹)

В предыдущих примерах мы использовали команду `int 020` — для возврата в DOS, а также `int 021` — для обращения к функциям операционной системы. Команда `int <n>` выполняется как дальний вызов через специальную таблицу адресов, где `n` — индекс элемента таблицы. Таблица находится по нулевому линейному адресу и содержит 256 указателей по 4 байта каждый, т. е. занимает первый килобайт памяти. Каждый указатель, или "вектор"² — это адрес в формате "сегмент:смещение".

Таблица векторов

Пример таблицы векторов показан рис. 10.1.

0	3643038D	700029	F000FF53	700029	36430351	F000FF53	700068	F000FF53
20	700215	70021B	700221	700227	70022D	700233	700239	70023F
40	C0000152	F000F84D	F000F841	F000E3FE	F000E739	F000F859	F000E82E	F000EFD2
60	F0008F75	F000E6F2	F000FE6E	364303EC	F000FF53	0	70001B	C0001393
80	700038	2171048B	24A06AC1	36430416	9E83001E	700048	700050	217104DB
A0	700029	70018F	700029	700029	700029	700029	700029	700030
C0	700029	700029	700029	700029	700029	700029	700029	700029
E0	700029	700029	700029	700029	700029	700029	700029	700029
100	F000EC59	9FC0003D	F000FF53	C0002593	F000FF53	F000FF53	9FC0004D	F000FF53
120	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53
140	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53
160	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53
180	36430388	36430388	0	0	0	0	0	0
1A0	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53	F000FF53
1C0	700245	70024B	700251	700257	70025D	700263	700269	70026F
1E0	0	0	0	0	0	0	0	0
200	0	0	0	0	0	0	0	0
...								
3E0	0	0	0	0	0	0	0	0
400								

Рис. 10.1. Таблица векторов прерываний

Чтобы ее прочитать, выполните следующие действия. Вызовите редактор памяти `e32` командой: `e32 0 1`, где `0` — номер параграфа (сегментный адрес), а `1` — ограничение в килобайтах (см. приложение 2). Выйдите из редактора, нажав `<Esc>`. Двоичный образ автоматически сохраняется при выходе из редактора в файле `$$$bin`. Для преобразования двоичного образа в текстовый файл вызовите утилиту `b2f` (bin-file to text): `b2f $$$bin 0 > ivt.txt`, где `0` — сегментный адрес.

Слева на рис. 10.1 указан линейный адрес. Каждое 32-битовое слово — это вектор прерывания. (В `i80x86` он содержит только дальний адрес перехода.) Вектор номер `n` находится в таблице по смещению `n*4`. Например, вектор `020`, который мы использовали для возврата в DOS, виден по смещению `080`, а рядом с ним — вектор `021`.

Векторные вызовы

Команда `int <n>` — это разновидность дальнего вызова и выполняется следующим образом:

- в стеке сохраняются флаги, что равносильно выполнению команды `pushf`;
- из таблицы векторов считывается 32-битовое слово по смещению `n*4`;
- в стеке сохраняется пара регистров `cs:ip`;
- в пару `cs:ip` записывается вектор: старшее слово — в `cs`, младшее — в `ip`.

Программа, вызванная таким способом, должна заканчиваться командой `iret`, которая восстановит `cs:ip` (что равносильно команде `retf`) и регистр флагов (`popf`).

На рис. 10.1 обращают на себя внимание часто повторяющиеся значения `700029` и `F000FF53`, а также нули во второй половине таблицы. Посмотрим, куда ведут эти адреса. Вызовите `d86` и наберите команду `jmp 070:029 <Enter>`. Вы перейдете на команду `iret`. Следовательно, везде, где вектор равен `700029`, подпрограмма содержит только команду возврата. Проверьте также адрес `0f000:0ff53`.

Что касается нулей, это зарезервированные векторы. Попытка выполнить `int` с этими номерами (например, `int 080`, с вектором по смещению `080 * 4 = 0200`) приведет к катастрофе: управление будет передано в начало таблицы векторов, и процессор начнет выполнять ее как программу.

Раз эти векторы свободны, мы можем воспользоваться ими для экспериментов. Зарезервируем небольшой участок памяти, запишем туда код программы, которая выводит что-либо на экран прямым отображением в видеопамять. Адрес точки входа в программу запишем в вектор по смещению `0200` и выполним в отладчике команду `int 080`.

Чтобы получить в свое распоряжение блок памяти размером, например, 1 Кбайт, выполните трансляцию и вызовите один раз программу из листинга 10.1.

В `ah` задан номер функции DOS, а в `dx` — размер сохраняемой памяти в параграфах.

Листинг 10.1. Резидентная программа, захватившая 1 Кбайт памяти (tsr1k.8)

```
p_sz equ 1k shr 4
mov ah, 031 ; tsr (Terminate and Stay Resident)
mov dx, p_sz
int 021
```

Теперь выполните команду `FreeDOS mem /u` и найдите в списке резидентных модулей имя `TSR1K`. Прочтите число в начале строки и прибавьте к нему единицу — это и есть сегментный адрес модуля (в дальнейшем обозначаем его `mod_seg`). На рис. 10.2 приведен пример вывода команды `mem /u`.

Segment	Total		Name	Type
0fbc	96	(0K)		free
0fc3	72,224	(71K)	COMMAND	program
2170	12,608	(12K)	VC	program
2499	72,064	(70K)		free
3632	1,056	(1K)	TSR1K	program
3675	72,224	(71K)	COMMAND	program
4818	55,008	(54K)	MEM	program
5587	296,592	(290K)		free

Рис. 10.2. Вывод команды `mem /u`

Сегментный адрес для `TSR1K` равен `3632` в шестнадцатеричном формате. Прибавив к нему единицу, получим `3633`. Это номер параграфа, с которого начинается память, выделенная модулю `tsr1k`. Туда мы загрузим программу `isr_1`³, текст которой приведен в листинге 10.2.

Листинг 10.2. Подпрограмма для вызова командой `int (isr_1.8)`

```
include vect.inc ; (1)

org 0 ; (2)
entry: _brk ; (3)
push es, ds, ax, di ; (4)

set_seg ds, cs ; (5)
set_seg es, 0b800 ; (6)

mov ax, w sym ; (7)
mov di, w pos ; (8)
stosw ; (9)
mov pos, di ; (10)

pop di, ax, ds, es ; (11)
iret ; (12)
```

```
sym      dw      4 by '*'          ; 02a      ; (13)
pos      dw      160*24            ; (14)
         dw      3 dup 05555       ; (15)
```

В строках (5–6) — вызов макроса, определенного в файле `vect.inc` (1). Это загрузка сегментного регистра с использованием `ax` для передачи значения. (Например, (5) выполняется по схеме `cs -> ax -> ds`.) По сравнению со встроенной макрокомандой `mov`, макрос `set_seg` компактнее и не задействует стек⁴, хотя он изменяет регистр `ax`.

Директива (2) задает компиляцию с нулевого адреса, в результате чего `a86` генерирует не `com-`, а `bin`-файл, а смещение точки входа в процедуру (3) равно нулю. При входе в процедуру сохраняем (4) регистры, а на выходе — восстанавливаем (11) в обратном порядке. В строке (6) устанавливаем `es` равным сегментному адресу видеопамати. Зачем нужна установка `ds = cs` в (5)? Дело в том, что мы будем вызывать эту процедуру из другой задачи, например прямо из `d86`. А у другой задачи иная настройка `ds`, и она сама собой при вызове процедуры не изменится. Зато при вызове изменится пара `cs:ip`, и в `cs` будет именно то значение сегментного адреса, которое должно быть в `ds` для правильного доступа (7–10) к данным модуля `isr_1`.

В операторе (9) происходит доступ к видеопамати: `ax -> es:di++`. При этом новое значение `di` сохраняется (10) в переменной `pos`, так что при каждом вызове `isr_1` вывод на экран будет смещаться вправо. В операторе (13) задано значение одного *знако-места* экрана: в младшем байте — код ASCII символа `'*'`, а в старшем — атрибут (цвет символа — в младшей тетраде, затем цвет фона — в трех битах старшей тетрады, а в седьмом бите — мигание). В операторе (14) задана исходная позиция для вывода — начало 24-й строки. Это нижняя строка экрана, которую `d86` резервирует для ввода команд в непосредственном режиме, а также для пользовательского вывода.

Чтобы загрузить файл `isr_1.bin` в память, которую мы захватили вызовом `tsr1k.com`, выполните (подставив вместо `seg` свой сегментный адрес): `f2m seg isr.bin <Enter>`. В ответ должно появиться сообщение: `from offset 0 copied 2A bytes -> [seg:0]`.

Теперь направим выбранный нами вектор `080` на адрес `seg:0`. Для этого вновь вызовите команду `e32 0 1` и, перейдя к смещению `0200`, введите четыре шестнадцатеричные цифры сегментного адреса и четыре цифры смещения (например, `36330000`, что соответствует адресу `03633:0`). Выйдите из `e32`, а затем снова зайдите, чтобы убедиться, что записанное по адресу `0200` сохранилось.

Вызовите `d86` и выполните ручную `int 080`. Если внизу появилась звездочка, повторите вызов, нажимая `<F3>`. Вывод должен смещаться вправо.

Еще один способ вызвать подпрограмму, адрес которой записан в какой-либо вектор, показан в листинге 10.3.

Листинг 10.3. Вызов подпрограммы через вектор 080 (`int080.8`)

```
include vect.inc          ; (1)

n_vect equ 080            ; (2)
```

```

int      n_vect      ; (3)

set_seg es, 0        ; (4)
pushf          ; (5)
es call d[n_vect*4]  ; (6)

_exit              ; (7)

```

В строке (3) показан первый способ, только что проверенный вручную. В (4–6) вызов выполняется по частям: настраиваем (4) доступ к памяти с сегментным адресом 0 через регистр `es` (можно было и через `ds`), сохраняем (5) в стеке флаги и выполняем дальний вызов (6) по значению вектора. В строке (7) — выход в DOS, макрокоманда `_exit` определена в `vect.inc`.

Можно было поставить этот опыт проще: сделать программу `isr_1.8` резидентной (листинг 10.4), и тогда бы обошлось без вызовов `tsr1k` и `f2m`. Только эта программа должна быть исполняемой, т. е. не `bin-`, а `com-`формата.

Листинг 10.4. Резидентная процедура обработки исключения (`isr_2.8`)

```

include vect.inc

mov     ah, 031
mov     dx, (end shr 4) + 1
int     021

entry:
push    es, ds, ax, di
...

```

После запуска `isr_2.com` эта программа тут же завершается, оставляя за собой память до параграфа, следующего за неявной меткой `end` (конец исходного текста). Посмотрите в `lst`-файле, что стало со смещением метки `entry`. Теперь его значение уже не ноль, а `0107`. В редакторе `e32` нужно записать это число в младшую часть вектора, например число `36330107` (адрес `03633:0107`) — по линейному адресу `0200` (вектор `080`).

Запрограммируем то, что мы выполняли вручную. В листинге 10.5 приведена программа, которая настраивает вектор `080` на свою процедуру `entry` и сама же выполняет векторный вызов. Эту программу уже необязательно оставлять в памяти, т. к. все действия она выполняет сама.

Листинг 10.5. Программа для проверки механизма вызова прерываний (`isr_3.8`)

```

include vect.inc                                ; (1)

n_vect equ 080                                  ; (2)

```

```

    set_seg ds, 0                ; (3)
    mov     bx, n_vect*4         ; (4)
    mov     w [bx], offset entry ; (5)
    mov     ax, cs               ; (6)
    mov     [bx+2], ax           ; (7)

    int     n_vect               ; (8)

    pushf                        ; (9)
    call    d[n_vect*4]          ; (10)

    _exit                                ; (11)

entry:                                ; (12)
    push    es, ds, ax, di         ; (13)
    ...

```

Команды в строках (8–11) позаимствованы из листинга 10.3, хотя здесь мы воспользовались тем, что `ds` уже равен нулю после (3). В строках (3–7) мы формируем вектор: устанавливаем (3) в `ds` нулевой базовый адрес, задаем (4) в `bx` смещение вектора, а затем через `bx` записываем (5) смещение подпрограммы и ее сегментный адрес, равный `cs`.

После завершения программы цвет символов на консоли станет красным. Вернуть белый цвет можно командой очистки экрана `cls`.

Во всех предыдущих опытах мы пользовались свободным вектором `080`. Если бы вектор был занят, пришлось бы в начале программы сохранить его исходное значение, а на выходе — восстановить. Самый простой вариант, с сохранением вектора в стеке, приведен в листинге 10.6.

Листинг 10.6. Программа с сохранением и восстановлением вектора (`isr_4.8`)

```

...
set_seg ds, 0                ; (3)
mov     bx, n_vect*4         ; (4)

push    [bx+2], [bx]         ; (5)

mov     w [bx], offset entry ; (6)
mov     ax, cs               ; (7)
mov     [bx+2], ax           ; (8)

int     n_vect               ; (9)
pushf                        ; (10)
call    d[n_vect*4]          ; (11)

pop     [bx], [bx+2]         ; (12)
_exit                                ; (13)
...

```

Дальше обычно говорят о *дополнении* к системной процедуре обработки исключения, когда наша процедура вызывает системную процедуру как подпрограмму или же передает ей управление при своем завершении. На практике такие задачи довольно редки (если речь не идет об исследовании или взломе чужих программ). Пример приведен в листинге 10.7 для вектора 021 (вызов функций операционной системы).

Листинг 10.7. Дополнение к системной процедуре обработки исключения (isr_5a.8)

```
include vect.inc ; (1)

extrn vect_0:d, _ptr:w ; (2)

n_vect equ 021 ; (3)

set_seg es, 0 ; (4)
mov bx, n_vect*4 ; (5)

mov ax, offset entry ; (6)
es xchg ax, [bx] ; (7)
mov w[vect_0], ax ; (8)

mov ax, cs ; (9)
es xchg ax, [bx+2] ; (10)
mov w[vect_0+2], ax ; (11)

mov ah, 2 ; (12)
mov dl, 'H' ; (13)
adr1: int 021 ; (14)
mov dl, 'i' ; (15)
adr2: int 021 ; (16)
mov dl, '!' ; (17)
nop ; (18)
adr3: int 021 ; (19)

mov ax, w[vect_0] ; (20)
es mov [bx], ax ; (21)
mov ax, w[vect_0+2] ; (22)
es mov [bx+2], ax ; (23)

mov ah, 04c ; (24)
adr4: int 021 ; (25)

entry: ; (26)
enter ; bp+6 -> (flags) ; (27)
; bp+4 -> (cs) ; (28)
; bp+2 -> (ip) ; (29)
; bp -> (bp) ; (30)
push ds, ax, di ; (31)
```



```

leave                                ; (40)
iret                                ; (41)

```

Мы обращаемся (27–28) к системной процедуре обработки прерывания, как если бы она была подпрограммой — как в листинге 10.3. Системная процедура заканчивается командой `iret` (41).

Команды `int <n>` позволяют обращаться к резидентным процедурам по номеру `n`, не зная и не интересуясь их расположением в памяти, что упрощает вызов функций BIOS и операционной системы. Тот же механизм дальнего косвенного вызова по таблице векторов работает не только для команды `int`, но и при внешних (аппаратных) прерываниях, и при так называемых *fault*-исключениях.

Исключения и прерывания

Уточним термины. В широком смысле исключение (*exception*) — это *синхронный*⁵ вызов по таблице векторов — либо безусловный (командой `int`), либо из-за ошибки при выполнении команды. Для безусловного вызова есть также термин *trap*-исключение, или ловушка⁶. Исключение в узком смысле слова — всегда по ошибке (переполнение при делении, нарушение защиты памяти и т. д.). Термин "прерывание" (*interrupt*) тоже многозначный. В широком смысле это любой векторный вызов. В узком смысле это прерывание от внешнего устройства или процесса, *асинхронное* по отношению к программе⁷. В документации Intel слово *exception* означает синхронное прерывание, а *interrupt* — асинхронное, внешнее.

В следующем примере (листинг 10.9) команда `into` вызывает исключение по вектору 4, если установлен флаг знакового переполнения (`o = 1`).

Листинг 10.9. Команда `into` и исключение по вектору 4 (`into.8`)

```

include vect.inc                                ; (1)

extrn    vect_0:d, inx:w                        ; (2)

n_vect    equ    4                              ; (3)

    set_seg es, 0                                ; (4)
    mov     bx, n_vect*4                          ; (5)

    mov     ax, offset entry                      ; (6)
    es xchg ax, [bx]                              ; (7)
    mov     w[vect_0], ax                        ; (8)

    mov     ax, cs                                ; (9)
    es xchg ax, [bx+2]                            ; (10)
    mov     w[vect_0+2], ax                      ; (11)

```

```

        call    testing                ; (12)

exit:   mov     ax, w[vect_0]          ; (13)
        es mov  [bx], ax              ; (14)
        mov     ax, w[vect_0+2]       ; (15)
        es mov  [bx+2], ax            ; (16)

        _exit                          ; (17)

testing:                                ; (18)
        mov     al, 07e               ; (19)
        add     al, 1 ; o = 0         ; (20)
adr1:   into    ; (21)
        add     al, 1 ; o = 1         ; (22)
adr2:   into    ; (23)
        nop     ; (24)
adr3:   int     4                     ; (25)
        ret     ; (26)

entry:                                ; (27)
        enter   ; (28)
        push    ax, bx                ; (29)

        mov     ax, [bp+2]            ; (30)
        mov     bx, inx               ; (31)
        shl     bl, 1                 ; (32)
        jc      >l1                   ; (33)
        mov     trace[bx], ax         ; (34)
        inc     inx                   ; (35)
        jmp     >l2                   ; (36)
l1:     mov     w [bp+2], exit         ; (37)
l2:     pop     bx, ax                 ; (38)
        leave   ; (39)
        iret     ; (40)

vect_0  dd     ?                     ; (41)
inx      dw     0                     ; (42)
trace    dw     ; (43)

```

Индекс трассы (он же счетчик записей) определен в (42) и используется в (31–35). Команда (32) делает из индекса смещение, а (33) проверяет, остается ли смещение в пределах 0100. Если нет, то в (37) мы подменяем адрес выхода — так, чтобы после (38–40) оказаться на (13), где, восстановив системный вектор, можно завершить программу.

Выполните `into.com` в `d86`. Из полученной трассы видно, что команда (21) не вызвала исключение, а (23) и (25) вызвали. Так и должно быть, поскольку в (21) флаг `o` сброшен, в (23) — установлен, а в (25) вызов безусловный.

Эта программа годится для исследования всех исключений, необходимо только изменить номер вектора (3) и команды для проверки исключения (19–25). Пример такой модификации приведен в листинге 10.10 — это исследование команды для проверки границ массива⁸.

Листинг 10.10. Команда `bound` и исключение по вектору 5 (`bound.8`)

```

...
n_vect equ    5                                ; (3)
...

testing:                                       ; (18)
    mov     ax, -3                            ; (19)
adr1:  bound ax, range                        ; (20)
    mov     ax, 2                             ; (21)
adr2:  bound ax, range                        ; (22)
    mov     ax, -4                            ; (23)
adr3:  bound ax, range                        ; (24)
    mov     ax, 3                             ; (25)
adr4:  bound ax, range                        ; (26)
    ret                                         ; (27)

...
range  dw     -3, 2                           ; (42)
...
```

Команда `bound` вызывает исключение по вектору 5, если значение проверяемого регистра лежит вне диапазона, заданного в двух смежных словах. Первое исключение должно произойти в (24) — при `ax = -4`.

Выполнив `bound.com` в `d86`, получим трассу с повторяющимся значением, равным `adr3`. Команда `bound` относится к той категории исключений, которые сохраняют в стеке адрес не следующей, а *текущей* команды — той самой, которая вызвала исключение. Предполагается, что в процедуре мы скорректируем ошибку и повторим команду, на этот раз уже правильно. Этот подход применяется, например, при подкачке страниц виртуальной памяти, сохраненных на диске: при обращении к отсутствующей странице происходит исключение (с сохранением в стеке адреса текущей команды), затем процедура обработки считывает данные с диска в память и возвращается на ту же команду, которая на этот раз выполняется успешно. В случае с `bound` непонятно, что делать, оставим как есть.

Теперь спровоцируем ошибку при делении `div/ldiv`. При переполнении результата, что проще всего добиться делением на ноль, `i80x86` генерирует исключение номер ноль. Начните с простой программы, которая делит не ноль на ноль — в ответ FreeDOS выдаст сообщение "Divide error" и завершит программу. Потом, взяв за основу листинг 10.10, включите деление в процедуру (18–27) и задайте правильный номер вектора (3). Выясните, адрес какой команды сохраняется в стеке — следующей за делением или текущей?

Полный перечень исключений приведен в [2, Table 8-1]. Какие-то из них возможны лишь в защищенном режиме, хотя вызвать исключение 13 (GP, или General Protection Fault — исключение общей защиты) можно и в реальном режиме. Для этого нужно взять 32-битовый регистр-указатель и задать в нем число больше 0ffff, как в листинге 10.11. Поскольку тут используется 32-битовый регистр, недоступный в а86, программа написана на nasm.

Листинг 10.11. Проверка общей защиты в реальном режиме (gp.asm)

```
xor    ax, ax                ; 0                ; (1)
mov     es, ax                ; (2)
mov     edi, 0xb8000 + (160 * 24) ; (3)
mov     byte [es:edi], '@'      ; (4)

mov     ah, 0x4c              ; (5)
int     0x21                  ; (6)
```

Для трансляции вызовите команду: `nasm gp.asm -f bin -o gp.com`. Команда довольно длинная, и чтобы не набирать ее заново при каждом вызове, рекомендуется добавить в файл `vs.ext` (каталог `tools` в образе диска `C`) следующую строку:

```
asm: nasm !.! -f bin -o !.com
```

После чего эту команду можно вызвать, нажав <Enter> на файле с расширением `asm`.

Запустив `gp.com`, вы увидите в консоли Bochs бесконечно повторяющуюся строку с фразой "write beyond limit". Симулятор Bochs замечателен тем, что сообщает причину сбоя, хотя мы даже не программировали обработку исключения 13. (При работе в FreeDOS без Bochs мы бы просто "повисли", а о причинах пришлось бы догадываться.) Причина сбоя в том, что в реальном режиме сегмент ограничен 64 КБайт (поэтому максимальное смещение в нем — 0ffff), а мы попытались записать что-то по смещению 0xb8000 с хвостиком.

Попробуем запрограммировать обработку GP по аналогии с листингом 10.9 (сначала на а386, а потом на nasm). Если у вас нет а386/d386, воспользуйтесь готовыми результатами трансляции (`gp_2.com` и `gp_2.sym`), а в отладчике d86 вам достаточно выполнить команду `g` и (при наличии файла `gp_2.d8k` с нажатиями клавиш) оценить результат в строках отображения данных⁹. Обработку исключения общей защиты демонстрирует листинг 10.12.

Листинг 10.12. Обработка исключения общей защиты, вариант для а386 (gp_2.8x)

```
...
n_vect equ 13                ; (3)

set_seg es, 0                ; (4)
mov     bx, n_vect*4          ; (5)
```

```

mov     ax, cs                      ; (6)
shl     eax, 16                    ; (7)
mov     ax, offset entry           ; (8)
es xchg eax, [bx]                  ; (9)
mov     vect_0, eax                ; (10)

call    testing                   ; (11)

exit:   mov     eax, [vect_0]       ; (12)
        es mov  [bx], eax          ; (13)
        _exit                                ; (14)

testing:                                ; (15)
        set_seg es, 0              ; (16)
        mov     edi, 0b8000 + (160 * 24) ; (17)
        mov     byte [es:edi], '@'   ; (18)
        ret                                ; (19)
        ...

```

Здесь, по сравнению с листингом 10.10, скорректированы номер вектора (3) и код в процедуре тестирования (15–19). Пользуясь случаем, я также упростил работу с векторами за счет применения регистра `eax`. Этот регистр — 32-битовое расширение регистра `ax`: младшее слово `eax` по-прежнему доступно как `ax`, хотя у старшего слова нет имени, чтобы к нему обратиться. Поэтому в (6–8) мы формируем в `eax` новое значение вектора, пользуясь командой сдвига (7): после (6–7) в старшем слове `eax` находится старшая часть вектора, и остается только записать младшую часть в `ax` (8).

Проверьте работу `gp_2.com` в d386 или в d86. Экран d386 для программы `gp_2.com` показан на рис. 10.3. (В d86 отображение *данных* будет таким же.)

На рис. 10.3 видны 32-битовые расширенные регистры `eax-esp`. Появились два новых сегментных регистра `fs` и `gs`. Это дополнительные регистры для адресации данных, подобно `es`.

После выполнения команды `g` <Enter> в массиве `trace` — многократно повторенный адрес команды, пытающейся обратиться к памяти за пределами сегмента `es`.

Ассемблер *nasm*

Начиная с этого примера, мы все чаще будем обращаться к 32-битовым возможностям i80x86, ориентируясь на ассемблер `nasm`¹⁰. Переход от a86/a386 к `nasm` не совсем прост, поскольку в них заложены противоположные принципы. Имена в a86/a386 типизированы (`abs`, `byte`, `word`, `dword` и т. д.), и машинные команды генерируются исходя из типа операнда. Напротив, в `nasm` от типов отказались вовсе и задают размерность операнда в *каждой* команде при помощи уточнений `byte`, `word`, `dword`. Переменная (операнд в памяти) записывается *всегда* в квадратных скобках,

+ 0100 MOV AX,0	# 0126 INT 021
0103 MOV ES,AX	TESTING:
0105 MOV BX,034	0128 MOV AX,0
0108 MOV AX,CS	012B MOV ES,AX
010A SHL EAX,010	012D MOV EDI,000B_8F00
010E MOV AX,ENTRY	0133 ES MOV B[EDI],040
0111 ES XCHG D[BX],EAX	0138 RET
0115 MOV VECT_0,EAX	ENTRY:
0119 CALL TESTING	0139 ENTER 0,0
EXIT:	013D PUSH AX
011C MOV EAX,VECT_0	013E PUSH BX
0120 ES MOV D[BX],EAX	013F MOV AX,W[BP+2]
0124 MOV AH,04C	0142 MOV BX,INX
EAX 0070_4C33	i e 1: d1,inx,, 128
EBX 0000_0034	IP 0126 2: w8,trace,, 0133 0133 0133 0133 0133 0133 0133 0133
ECX 0009_0000	CS 64C9 3:
EDX 0000_64C9	SS 64C9 4:
ESI 000E_0000	DS 64C9 5:
EDI 000B_8F00	ES 0000 6:
EBP 0000_0000	FS 3B6E 7:
ESP 0000_FFFC	GS 0000 1: 011C

Рис. 10.3. Отладчик d386 после выполнения gp_2.com

иначе ее сочтут числом (непосредственный операнд). Иными словами, если нам нужен адрес байтовой переменной x, то мы так и напишем — x; а если содержимое, то byte [x]. После а86 это непривычно, хотя и несложно, т. к. принцип nasm выра-
жен коротко и ясно.

В листинге 10.13 приведена программа из листинга 10.12, переведенная с а386 на nasm. Знак ! в комментариях поставлен там, где пришлось что-то изменить.

Листинг 10.13. Обработка исключения общей защиты, вариант для nasm (gp_3.asm)

%include "tst_nasm.inc"	; !	; (1)
org 0x100	; !!!	; (2)
n_vect equ 13		; (3)
mov ax, vect_0		; (4)
mov ax, inx		; (5)
mov ax, trace		; (6)
set_seg es, 0		; (7)
mov bx, n_vect*4		; (8)
mov ax, cs		; (9)
shl eax, 16		; (10)

```

        mov     ax, entry           ; !           ; (11)
        xchg    eax, [es:bx]       ; !           ; (12)
        mov     [vect_0], eax      ; (13)

        call    testing            ; (14)

exit:    mov     eax, [vect_0]      ; (15)
        mov     [es:bx], eax       ; !           ; (16)

        _exit                                ; (17)

testing:                                ; (18)
        set_seg es, 0              ; (19)
        mov     edi, 0xb8000 + (160 * 24) ; !           ; (20)
        es mov  byte [edi], '@'    ; !           ; (21)
        ret                                ; (22)

entry:                                ; (23)
        enter   0, 0               ; !           ; (24)
        push    ax                 ; !           ; (25)
        push    bx                 ; !           ; (26)

        mov     ax, [bp+2]         ; (27)
        mov     bx, [inx]          ; !           ; (28)
        shl     bl, 1              ; (29)
        jc      l1                 ; !           ; (30)
        mov     [trace+bx], ax     ; !           ; (31)
        inc     word [inx]         ; !           ; (32)
        jmp     l2                 ; !           ; (33)
l1:      mov     word [bp+2], exit  ; !           ; (34)
l2:      pop     bx                 ; !           ; (35)
        pop     ax                 ; !           ; (36)
        leave                                ; (37)
        iret                                ; (38)

vect_0  dd      0                  ; (39)
inx      dw      0                  ; (40)
trace    dw      0                  ; (41)

```

Чтобы выполнить трансляцию с получением файла-листинга, выполните команду:

```
nasm gp_3.asm -f bin -o gp_3.com -l gp_3.lst
```

Особенности `nasm`, проявившиеся в этой программе, таковы:

- ❑ в строке (1) название директивы записывается с процентом, а имя файла — в кавычках;
- ❑ оператор (2) необходим, т. к. по умолчанию `nasm` начинает трансляцию с нулевого смещения;

- ❑ ключевое слово `offset` в `nasm` не используется¹¹; если требуется задать *адрес* объекта, то запишите его без квадратных скобок, как в (11) и (34);
- ❑ переменная в качестве операнда команды всегда помещается в квадратные скобки, как в (13), (15) и (28);
- ❑ все составляющие косвенного адреса расположены *внутри* квадратных скобок, как в (27), (31) и (34);
- ❑ команда (24) задана со всеми ее операндами, команды (25–26) и (35–36) записываются по отдельности¹²;
- ❑ локальные метки `a86` и опережающую ссылку (`'>'`) на них `nasm` не понимает, метки (34–35) — глобальные¹³;
- ❑ сообщения об ошибках довольно невразумительные, особенно для начинающих;
- ❑ от `lst`-файла мало толку — нет таблицы имен;
- ❑ префикс переназначения сегмента, так же как в `a86`, может записываться вместе с операндом в квадратных скобках (12, 16), а может и перед командой, как в (21).

Зайдите в `d86/d386` с этой программой. У нас нет `sym`-файла, поэтому отсутствует отображение данных. Но обратите внимание на команды (4–6). Они здесь только для того, чтобы нам сразу выяснить значения имен (39–41) и настроить строки отображения данных `d86`, используя *числовые* адреса вместо имен.

Выполнив (4–6), настройте отображение данных, затем пройдите программу целиком: `g <Enter>`. В буфере трассировки (41) должно быть повторяющееся значение адреса команды (21), в счетчике (40) — число 128, а в переменной (39) — 00700233 (вектор номер 13, см. рис. 10.1).

Еще одно отличие `nasm` от `a86` (и даже от традиционных `masm`-совместимых ассемблеров) — свой синтаксис языка макрокоманд. Для примера в листинге 10.14 приведен файл `tst_nasm.inc`, используемый в листинге 10.13.

Листинг 10.14. Макроопределения в `nasm` (`tst_nasm.inc`)

```
%macro set_seg 2                ; (1)
    mov     ax, %2              ; (2)
    mov     %1, ax              ; (3)
%endmacro                       ; (4)

%macro _brk 0                   ; (5)
    xchg    bx, bx              ; (6)
%endmacro                       ; (7)

%macro _exit 0-1 0              ; (8)
    mov     ah, 0x4c            ; (9)
    mov     al, %1              ; (10)
    int     0x21                ; (11)
%endmacro                       ; (12)
```


Макроопределение в `nas`m начинается не с имени, а с ключевого слова `%macro`, затем идет имя и счетчик параметров. В строке (8) число параметров может быть от нуля до одного, и если параметр не задан, он подставляется как 0. Параметры в теле макрокоманды обозначаются по номеру, как в `a86`, только вместо `#` — знак `%`.

И наконец, `nas`m позволяет распределять объекты в области инициализированных данных, за пределами `com`-файла. В листинге 10.15 приведен вариант программы из листинга 10.13 с использованием *секций* со стандартными именами `.text` — для кода, `.data` — для данных, `.bss` — для данных, которые не хранятся в `com`-файле (и потому значения их не определены).

Листинг 10.15. Секции в `nas`m (`gp_4.asm`)

```

...
org      0x100          ; !!!          ; (2)

section .bss              ; (3)
trace:  resw   128        ; dw      128 dup ?      ; (4)
vect_0  resd    1         ; dd      ?          ; (5)

section .data              ; (6)
inx     dw      0         ; (7)

section .text              ; (8)

n_vect  equ     13         ; (9)

mov     ax, vect_0        ; (10)
...
```

В `com`-файле первой секцией всегда будет `.text`, а за ней — `.data`. Секция `.bss` в `com`-файл не попадает, но все ссылки на (4) и (5) будут направлены на данные, начинающиеся сразу за пределами копии `com`-файла в памяти. В секции `.bss` директивы `db/dw/dd` запрещены, а неопределенное значение в виде вопросительного знака `nas`m не понимает. Взамен применяются директивы `resb`, `resw`, `resd` с указанием числа элементов. Обратите внимание: в строке (4) поставлено двоеточие, а в (5) — нет. Для `nas`m это без разницы — ведь здесь нет типов.

Проверьте программу `gp_4.com`, запустив ее в отладчике `d86` или `d386`.

Отладочные исключения

В завершение остановимся на исключениях для отладчика — 1 и 3. Отладчик настраивает эти векторы на себя, записывая в них адреса своих процедур. Точки останова ставятся при помощи команды `int 3`: отладчик заменяет¹⁴ байт по адресу останова кодом команды `int 3`. Почему именно она? В отличие от прочих команд

int, где номер вектора задан во втором байте, эта команда однобайтовая. Поэтому она может заменить любую команду, не затронув следующую. Что было бы в случае двухбайтовой точки останова, иллюстрирует программа в листинге 10.16.

Листинг 10.16. Двухбайтная точка останова на однобайтной команде (brk.8)

```

mov     ax, w brk2           ; (1)
mov     w[adr1], ax         ; (2)

test    ax                   ; (3)
jnz     skip                 ; (4)
adr1:  inc     ax           ; (5)
skip:   mov     dx, ax        ; (6)

brk1:   int     3             ; (7)
brk2:   int     080           ; (8)

```

Командами (1–2) мы записываем по адресу (5) точку останова, но не однобайтовую (7), а двухбайтовую (8). Поскольку команда (5) однобайтовая, то при выполнении (2) мы перезапишем первый байт команды (6). И если программа пойдет по пути (3)→(4)→(6), то в (6) она выполнит совсем не то. Что именно, можно увидеть, пройдя brk.com по шагам в отладчике.

Исключение 1 позволяет, без вмешательства в код программы, выполнить ее трассировку. Это исключение происходит после выполнения любой команды, если флаг `t` = 1. При входе в процедуру обработки исключения флаг `t` автоматически сбрасывается (иначе мы не прошли бы дальше одной команды), а при выполнении команды `iret` восстанавливается. Пример работы с флагом `t` показан в листингах 10.17–18.

Листинг 10.17. Изменение состояния флага `t` (vect.inc)

```

...
t_flag macro                ; (16)
pushf                       ; (17)
pop     ax                   ; (18)
xor     ax, bit 8            ; (19)
push    ax                   ; (20)
popf                                ; (21)
#em                             ; (22)

```

Чтобы изменить значение флага `t` на противоположное, нужно прочесть (17–18) слово состояния в регистр, затем инвертировать (19) бит 8 в регистре и записать результат (20–21) обратно в слово состояния. Макрокоманда (16) дважды вызывается в листинге 10.18.

Листинг 10.18. Трассировка с использованием флага `t` (`trace.8`)

```

...
n_vect equ    1                ; (3)
n_iter equ    5                ; (4)
...
t_flag                ; (13)
call    testing        ; (14)
t_flag                ; (15)

exit:   mov     ax, w[vect_0]    ; (16)
        es mov  [bx], ax        ; (17)
        mov     ax, w[vect_0+2] ; (18)
        es mov  [bx+2], ax      ; (19)

        _exit                ; (20)

testing:                ; (21)
        mov     cx, n_iter      ; (22)
        mov     al, 1           ; (23)

loop1:                ; (24)
        mul     cl              ; ax := al * cl ; (25)
        loop    loop1          ; (26)

        mov     fact, ax        ; (27)
        ret                ; (28)

entry:                ; (29)
...
l1:     mov     w [bp+2], exit    ; (39)
        xor     w [bp+6], bit 8  ; (40)
...
fact    dw     ?                ; (45)

```

В трассируемой процедуре (21) идет подсчет факториала. В строке (4) задано число итераций — небольшое, чтобы программа завершилась до переполнения буфера трассы. Выполните `trace.com` в отладчике. Обратите внимание: в трассу записывается адрес *следующей* команды, и трассировка продолжается при выполнении (15), пока не сбросится флаг `t`.

Если увеличить счетчик итераций (5) хотя бы до 100, то процедура обработки исключения выполнит (39), тем самым подготовив для себя выход в точку (16). А (40) скорректирует копию регистра флагов, сохраненную в стеке. Так что из прерывания мы выйдем в точку (16) и со сброшенным флагом трассировки.

Что если не добавлять (40), а перенести метку `exit` с (16) на (15)? Тогда мы выйдем из прерывания на макрокоманду (15) с установленным флагом трассировки и, отра-

ботав первую из ее команд, снова попадем в прерывание, где повторим команду (39), и так до бесконечности.

Примечания

1. В оригинале — непереводимая игра слов: агентство называется "Blue Moon", а сериал — "Moonlighting", что означает подработку, халтуру.
2. Название, видимо, сложилось в те времена, когда в векторе наряду с обязательным адресом перехода хранились начальные значения других регистров, например, регистра флагов, как в мини-ЭВМ PDP-11.
3. Interrupt Service Routine (ISR) — процедура обработки прерываний.
4. Что существенно при первых опытах с защищенным режимом.
5. Синхронный с выполнением программы, когда можно указать, какая именно команда была причиной.
6. В микро-ЭВМ PDP-11 даже была отдельная команда `TRAP` (с фиксированным номером вектора) для обращения к функциям операционной системы.
7. В этом случае нельзя сказать, какая команда вызвала прерывание, т. к. причина лежит вовне. Прерывание может возникнуть, например, при нажатии клавиши на клавиатуре или срабатывании системного таймера-счетчика.
8. Команда `bound` появилась в i80286 и исчезла в x64 (при включении 64-битового режима она не поддерживается).
9. Поскольку программа местами 32-битовая, то в области кода отладчик может показывать не то, что записано в исходном тексте. Не обращайтесь — даже если отладчику не понятны 32-битовые "новшества", процессор о них знает.
10. Применять `a386` нет смысла, хотя он ближе всех к `a86`. Дело в том, что `a386` не использовался в учебном процессе, и ошибки в нем я обнаружил только при работе над этой книгой. Поздно исправлять — автор больше не поддерживает `a86/a386`. К тому же `a386` ограничен 32-битовым режимом, а впереди у нас еще 64-битовый.
11. Логично: раз нет типов, так и незачем их преобразовывать.
12. Встроенных макрокоманд, как в `a86`, здесь нет.
13. Локальные метки возможны только в макроопределениях, но не в стиле `a86`.
14. Запись точки останова в код возможна лишь в реальном режиме. В защищенном режиме код недоступен для записи, поэтому для точек останова в i80386 были введены специальные отладочные регистры. Перехват управления происходит, когда адрес текущей команды или операнда совпадает со значением в этих регистрах.

ГЛАВА 11

```
DB6_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100 2: =d1,ax,, 1962
CX 0000  CS 6309 3:
DX 6309  SS 6309 4:
SI 011F  DS 6309 5: r5,b_1,, AB ab
DI 0000  ES 6309 6: r5,si,, AB ab
BP 0000  SP FFFE 0:

lea si,b_1
```

Внешние прерывания

Восемнадцать раз за один вечер его выкидывали из пивной "Эксер", и каждый раз он возвращался — дескать, "забыл трубку". Он лез в окна, двери, через кухню, через забор в трактир, через погреб к стойке, где отпускают пиво, и, наверно, спустился бы по дымовой трубе, если б его не сняли с крыши пожарные. Такой был настойчивый, что мог бы стать министром или депутатом!

(Ярослав Гашек. Похождения бравого солдата Швейка)

Внешние прерывания (interrupts), подобно исключениям (exceptions), используют тот же механизм подпрограммных вызовов через таблицу векторов. Разница в том, что прерывания инициируются внешними устройствами, т. е. возникают асинхронно по отношению к выполняемой программе.

Обычно у внешнего устройства есть регистр состояния, в котором отражаются различные события: завершение приема или передачи по каналу связи, обнуление таймера-счетчика и т. д. Изначально прерывания от устройства запрещены, но их можно открыть, установив биты разрешения прерываний в соответствующем регистре. Последующие события в виде заявки на прерывание передаются от устройства на какой-то из входов блока радиальных прерываний (или, проще, контроллера прерываний), который организует приоритетное обслуживание заявок, пропуская их по одной на вход INTR процессора. (Входы контроллера прерываний называются $irq <n>$, где $n = 0-7$, а IRQ означает то же, что и INTR — Interrupt Request.)

Итак, работа с прерываниями подразумевает, помимо установки вектора, еще и настройку устройства и связанного с ним входа контроллера прерываний. Программирование внешних устройств — это отдельная обширная тема, выходящая за рамки книги. Мы возьмем самые простые устройства, уже настроенные при загрузке BIOS и FreeDOS: системный таймер и клавиатуру.

Системный таймер

Нулевой канал системного таймера содержит 16-разрядный счетчик, работающий на вычитание от генератора меандра с тактовой частотой 1,193 МГц. Начальное

значение счетчика — ноль. Он считает такты, пока не дойдет до нуля: 0, 0ffff, 0fffe, 0fffd ... 3, 2, 1, 0 — и генерирует заявку на прерывание, которая поступает на вход irq0 контроллера прерываний, а сам продолжает счет от начального значения вниз. Заявки генерируются с частотой $1\ 193\ 000/65\ 536 \sim 18,2$ Гц.

Входу irq0 назначен вектор прерывания 8. Для начала, чтобы сразу не брать на себя обслуживание контроллера прерываний, мы воспользуемся вектором 01c. Дело в том, что системная процедура обработки прерывания по вектору 8 вызывает *пользовательскую* процедуру по вектору номер 01c. На рис. 10.1, где показана таблица векторов прерываний, найдите вектор 01c (он находится по смещению $01c * 4 = 070$). Значение вектора F000FF53 вам знакомо, и как это уже выяснилось, по адресу 0f000:0ff53 находится команда iret. По аналогии с кодом из листинга 10.4 создадим резидентную процедуру и свяжем ее с вектором 01c (листинг 11.1).

Листинг 11.1. Резидентная процедура обработки прерываний от таймера (i1c_1.8)

```

mov     ah, 031                ; (1)
mov     dx, (end shr 4) + 1    ; (2)
int     021                    ; (3)
entry:                                     ; (4)
cs inc  w sym                  ; (5)
iret                                     ; (6)

sym     dw      ?              ; (7)
```

Команды (1–3) завершают программу, оставляя в памяти ее код и данные. Действие в процедуре обработки прерывания (4–6) — увеличение (5) счетчика (7). В команде (5) префикс задан потому, что вызов процедуры (4–6) будет происходить из другой программы: вектор 8, судя по рис. 10.1, направлен в область BIOS. В результате векторного вызова мы окажемся в строке (4), и в cs будет сегментный адрес нашей программы. Но в других сегментных регистрах — наверняка содержится не этот адрес, а базовые адреса BIOS (070, скорее всего). Тем не менее значения cs достаточно для доступа к своим данным (7), т. к. com-программа транслируется в предположении, что все сегментные регистры настроены одинаково. И взамен ds мы используем для доступа к данным cs в строке (5)¹.

Выполните трансляцию, запустите полученную программу i1c_1.com и при помощи команды mem /u прочтите адрес блока памяти, выделенного модулю i1c_1. Прибавьте к нему 1 — это адрес программного сегмента i_1c, обозначим его четыре цифры как S₁S₂S₃S₄. Из lst-файла читаем смещения объектов (4) и (7) — 0107 и 010d. Выполнив команду e32 0 1 <Enter>, переходим к вектору по смещению 070 и не дрогнувшей рукой² записываем в него подряд восемь цифр — S₁S₂S₃S₄0107 <Enter>. Выходим из e32 и снова входим с сегментным адресом S₁S₂S₃S₄, чтобы посмотреть, как увеличивается счетчик (7). Чтобы e32 обновлял экран, нажимайте пробел. Байт,

где что-то меняется, и есть младшая часть счетчика по смещению 010d от начала сегмента S₁S₂S₃S₄. Оцените темп изменений³. Выйдите из e32, нажав <Esc>.

А сейчас запретим прерывания от системного таймера, закрыв вход irq0 контроллера прерываний (листинг 11.2).

Листинг 11.2. Переключение входного вентиля для irq0 (sw_irq0.8)

```
in      al, 021          ; (1)
mov     dl, al           ; (2)
xor     al, bit 0        ; (3)
out     021, al          ; (4)
...
```

Команды в строках (1) и (3) предназначены для чтения-записи *портов* ввода-вывода. Через это отдельное адресное пространство доступны регистры внешних устройств⁴. При загрузке системы контроллеру прерываний был назначен базовый адрес 020 в пространстве ввода-вывода. Байтовый регистр запрета/разрешения irq-входов находится по смещению +1, т. е. по адресу 021. Мы читаем этот регистр в строке (1), инвертируем (3) в al бит с номером, равным номеру irq, и записываем (4) это число обратно в порт 021. Команда (2) — сохранение прочитанного значения для отображения его в d86.

Зайдите в d86 с этой программой, но не выполняйте ее по шагам, а пройдитесь по коду клавишами-стрелками, чтобы увидеть в строке отображения данных изменения счетчика из листинга 11.1. Затем, вернувшись в начало клавишей <Home>, выполните команду g <Enter>. Оцените значение младшего бита регистра dl в строке отображения данных. Ноль означает, что вход *открыт*. Убедитесь, что счетчик прерываний больше не изменяется — ведь мы заблокировали прерывания от системного таймера на уровне контроллера прерываний. Вновь вернитесь в начало и выполните g <Enter>, после чего счет прерываний должен возобновиться.

На этом закончим опыты с прерыванием 01c и обратимся к их источнику⁵ — прерыванию по вектору 8. Мы заменяем системную процедуру своей. В нашу процедуру надо добавить сброс контроллера прерываний: запись кода 020 в порт по смещению +0, т. е. в порт 020. Пока сброс не выполнен, контроллер блокирует irq-входы с номерами, большими или равными номеру входа, обслуживаемого в данный момент. При обслуживании irq0 заблокированы все входы, в частности irq1 — прерывание от клавиатуры.

Взяв за основу пример из листинга 10.9, добавим в него сброс контроллера прерываний (листинг 11.3).

Листинг 11.3. Программа обработки прерываний по вектору 8 с записью трассы (i8.8)

```
...
n_vect equ 8          ; (3)
```

```

    set_seg es, 0                ; (4)
    mov     bx, n_vect*4        ; (5)

    cli                    ; (6)
    mov     ax, offset entry    ; (7)
    ...
    mov     w[vect_0+2], ax    ; (12)
    sti                    ; (13)

    call    testing            ; (14)

exit:  cli                    ; (15)
    mov     ax, w[vect_0]      ; (16)
    ...
    es mov  [bx+2], ax        ; (19)
    sti                    ; (20)

    _exit                    ; (21)

testing:                    ; (22)
    waste                    ; (23)
    jmp     testing           ; (24)

entry:                    ; (25)
    enter                    ; (26)
    push    ax, bx           ; (27)
    ...

l2:                    ; (36)
    eoi_1                    ; ! ; (37)
    pop     bx, ax           ; (38)
    leave                    ; (39)
    iret                    ; (40)
    ...

```

Сброс контроллера прерываний выполняет макрокоманда (37) из листинга 11.4, приведенного далее. В остальном же процедура обработки прерывания (25–26) не изменилась. Перед фрагментами (7–12) и (16–19), где вектор перезаписывается, добавлен запрет внешних прерываний (6, 15). Как только вектор записан, прерывания вновь разрешаются (13, 20). Код, при выполнении которого могут произойти прерывания, содержит макрорасширение (23) из листинга 11.4, которое выполняется в бесконечном цикле (22–24).

Листинг 11.4. Дополнительные макроопределения для программы i8.8 (vect.inc)

```

    ...
waste macro                ; (23)
    even                    ; (24)

```



```

#rxl(10)                                ; (25)
        jmp      >m1                     ; (26)
m1:                                           ; (27)
#em                                           ; (28)

eoi_1   macro                             ; (29)
        mov     al, 020                   ; (30)
        out     020, al                   ; (31)
#em                                           ; (32)

```

Команда перехода на 0 байтов вперед (25–27) не то же самое, что пустая команда: она очищает очередь предвыборки, заставляя процессор заново считывать команды из памяти. В результате все команды перехода, составляющие тестируемый фрагмент в листинге 11.4, не только замедляются, но и выполняются одинаковое время, благодаря чему трасса покажет равномерно распределенные значения.

Выполните программу i8.com в отладчике. Видно, что адреса, сохраненные в массиве трассировки, распределены более-менее равномерно. А если исправить в листинге 11.4 команду перехода (26) на команду `nop`, то в трассе будет одно и то же значение, т. к. выполнение десяти пустых операций — ничто по сравнению с командой перехода в конце этой последовательности.

Клавиатура

Следующий пример — прерывание от клавиатуры. Так же как и в случае с системным таймером, нам не нужно настраивать устройство. Это уже сделано в BIOS. Но если при работе с системным таймером мы ограничились сбросом контроллера прерываний, а само устройство не требовало от нас никаких действий, то с клавиатурой чуть сложнее: в процедуре обработки мы должны послать контроллеру клавиатуры подтверждение, т. е. разблокировать его (листинг 11.5).

Листинг 11.5. Программа обработки прерывания от клавиатуры (i9.8)

```

...
n_vect equ 9                                ; (3)
...
entry:                                       ; (25)
        enter                               ; (26)
        push    ax, bx                     ; (27)

        in      al, 060                     ; (28)
        mov     ah, al      ; scan code    ; (29)

        in      al, 061                     ; (30)
        or      al, bit 0                    ; (31)
        out     061, al      ; ack          ; (32)

```


кания равен коду нажатия + 080. Непонятно только, что такое 9с в начале массива. Судя по величине, это код отпускания, и мы его получили, когда выполняли команду `g <Enter>`. Чтобы в этом убедиться, заново зайдите в отладчик с этой программой и, запустив ее, нажмите несколько раз `<Enter>`, а потом `<Esc>`. Действительно, за 9с следуют парами коды 1с и 9с. Значит, наша программа успела поймать отпускание клавиши `<Enter>`, нажатие которой ее запустило.

Проверьте работу автоповтора: нажав клавишу, удерживайте ее до переполнения буфера.

Интересно, что в опытах с ведущим контроллером прерываний нам даже не пришлось открывать входы `irq0` и `irq1` — они уже открыты.

Часы реального времени

Можно было бы продолжить опыты с устройствами, подключенными ко второму (ведомому) контроллеру прерываний, выход которого заведен на вход `irq2` ведущего контроллера. Наиболее подходящее для исследований устройство, подключенное к ведомому контроллеру, — это часы реального времени⁸.

К сожалению, в Bochs это устройство имитируется неточно: регистр состояния не отражает временных событий (всегда читается как ноль), хотя прерывания, связанные с этими событиями, генерируются. Этот факт подтверждает программа `rtc.8`, которая по прерываниям выводит счетчики событий в левом верхнем углу экрана (завершается по нажатии `<Esc>`).

Проделанных опытов достаточно, чтобы в дальнейшем программировать внешние прерывания в защищенном режиме.

Примечания

1. У резидентной программы нет своего стека, поэтому мы пользуемся стеком прерванной программы, который на этот случай создается с запасом.
2. Если ошибетесь, придется сеанс Bochs-FreeDOS перезапустить.
3. Такое впечатление, что в Bochs счет идет быстрее, чем 18 раз в секунду, как на PC.
4. Большинство внешних устройств PC изначально отображалось в адресное пространство ввода-вывода, т. е. в порты. Видеоадаптер же всегда находился в адресном пространстве основной памяти, где сейчас также расположены сетевые карты и контроллер xHCI (USB 3.0).
5. Дело в том, что детали вызова прерывания `01с` нам не известны. Системное прерывание может по-своему настроить сегментные регистры и даже переключиться на свой стек. Все это осложняет опыты, заставляя нас считаться с причудами операционной системы.

6. Клавиатура генерирует не ASCII-коды, а номера клавиш. Нумерация — слева направо сверху вниз, и под номером один обычно находится клавиша <Esc>.
7. Входы контроллера прерываний реагируют не на уровень, а на перепад из 0 в 1, поэтому заявка на прерывание, обычно связанная с состоянием внешнего устройства, должна быть в устройстве сброшена. Исключение составляет системный таймер, который устанавливает заявку, досчитав до нуля, и сбрасывает ее сам после автоматической перезагрузки начального значения.
8. Принято нумеровать входы *ведомого* контроллера, начиная от 8, т. е. irq8, irq9 и т. д. до irq15. Поэтому можно сказать, что часы реального времени, связанные с нулевым входом ведомого контроллера, подключены к линии irq8.

ГЛАВА 12

```
DB6 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA i c 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:

lea si,b_1
```

32-битовые данные и адреса

А на завтрак я съел яичко —
Стало кругленьким моё личико,
За обедом я съел ватрушки —
Стали пухлыми мои ушки,

А на полдник я съел биточки —
Стали толстыми мои щёчки,
А на ужин я съел пирог —
Стали жирными пальцы ног...
(Дина Бурачевская. Диета)

Жил человек полнеющий,
А так вообще вполне еще.
(Рената Муха)

С появлением процессора i80386 стали доступны, даже в реальном режиме, 32-битовые расширения регистров общего назначения¹. Хотя некоторые регистры остаются 16-битовыми: во всех режимах это сегментные регистры, а в реальном режиме — ip, sp и flags.

Префиксы размерности операнда и адреса

Выясним, чем отличаются коды команд с 16- и 32-битовыми операндами. В листинге 12.1 показан фрагмент lst-файла, а386 создал его при трансляции программы cmd_1.8x.

Листинг 12.1. Кодирование 16- и 32-битовых операндов и адресов (cmd_1.lst)

```
1                                cmd1:
2 0100 40                        inc     ax
3 0101 66 40                     inc     eax
4 0103 66 40                     o4 inc  ax
5                                cmd2:
6 0105 FE 07                     inc     b[bx]
7 0107 67 FE 03                  inc     b[ebx]
8 010A FE 03                     inc     b[bp+di]
```

9	010C	67	FE 03	a4	inc	b[bp+di]
10	010F	67	FE 44 3D 00		inc	b[ebp+edi]
11				cmd3:		
12	0114	FF	07		inc	w[bx]
13	0116	67	FF 03		inc	w[ebx]
14	0119	FF	03		inc	w[bp+di]
15				cmd4:		
16	011B	66	FF 07		inc	d[bx]
17	011E	67 66	FF 03		inc	d[ebx]
18				cmd5:		
19	0122	FE	06 00 00		inc	b[0]
20	0126	FF	06 00 00		inc	w[0]
21	012A	66	FF 06 00 00		inc	d[0]
22				cmd6:		
23	012F	67 66	FF 05 78 56		inc	d[012345678]
	0135	34	12			
24				cmd7:		
25	0137	66	B8 78 56 34 12		mov	eax, 012345678

Коды команд (2–3) различаются только начальным байтом 066. Это префикс размерности *операнда*, в реальном режиме он превращает 16-битовые операнды в 32-битовые. (А на 8-битовые никак не влияет.) Возникает вопрос: можно ли расширить 16-битовую команду до аналогичной 32-битовой, просто поставив префикс 04 (или 032 в *nas*m).

Команда в строке (6) обращается к байту по 16-битовому адресу, заданному регистром *bx*. Команда (7) тоже обращается к байту, но через 32-битовый указатель, и в ее коде появляется префикс размерности *адреса* 067. Но следующие два байта (7) не соответствуют коду (6)! Они совпадают с кодом (8), где указаны совсем другие адресные регистры. Из этого следует, что для преобразования 16-битового адреса в 32-битовый мало поставить префикс *a4*, иногда должен измениться код команды.

В строке (9) мы поставили префикс изменения размерности адреса (*a4* в *a386* означает, что адрес 4-байтовый). Думая расширить *bp/di* до *ebp/edi*, получили код команды (7). Дело в том, что, увидев префикс *a4*, *a386* генерирует байт 067 и транслирует команду так, как если бы префикса вообще не было. (Эту ситуацию *nas*m считает ошибкой; пример приведен в листинге 12.2.) А то, что мы хотели получить, запрограммировано в строке (10).

В строках (12–14) то же самое, только операнды 16-битовые. В (17) — 32-битовые и операнд, и адрес, поэтому добавлены сразу два префикса: 066 и 067. В командах (19–21) адрес 16-битовый и кодируется двумя байтами. В строке (21) сгенерирован префикс размерности операнда, т. к. команда обращается к двойному слову. В строке (23) — та же команда, но адрес задан 32-битовым числом, в результате добавляются оба префикса, а после кода операции идет 4-байтовый адрес.

Префикс размерности операнда в *a386* обозначается 04, а в *nas*m — 032. Префикс размерности адреса в *a386* — *a4*, в *nas*m — *a32*. Если размерности операнда и адре-

са очевидны, как в листинге 12.1, ассемблер сам выберет префиксы и сгенерирует корректный код.

В листинге 12.2 — примеры 32-битовых префиксов в `nasm`. Исходный текст для `nasm`, повторяющий программу на `a386`, содержится в файле `cmd_2.asm`.

Листинг 12.2. 32-битовые префиксы в `nasm` (`cmd_2.lst`)

```

1                                cmd1:
2 00000000 40                    inc     ax
3 00000001 6640                  inc     eax
4 00000003 6640                  o32 inc ax
5      *****                  warning: invalid operand size prefix
6                                cmd2:
7 00000005 FE07                  inc     byte [bx]
8 00000007 67FE03               inc     byte [ebx]
9 0000000A FE03                  inc     byte [bp+di]
10      ;                        a32 inc byte [bp+di]
11 0000000C 67FE443D00           inc     byte [ebp+edi]
12                                cmd3:
13 00000011 FF07                  inc     word [bx]
14 00000013 67FF03               inc     word [ebx]
15 00000016 FF03                  inc     word [bp+di]
16                                cmd4:
17 00000018 66FF07               inc     dword [bx]
18 0000001B 6667FF03            inc     dword [ebx]
19                                cmd5:
20 0000001F FE060000             inc     byte [0]
21 00000023 FF060000             inc     word [0]
22 00000027 66FF060000          inc     dword [0]
23                                cmd6:
24 0000002C 66FF067856           inc     dword [0x12345678]
25      *****                  warning: word data exceeds bounds
26                                cmd7:
27 00000031 66B878563412         mov     eax, 0x12345678

```

На префикс в строке (4) следует предупреждение, а команду (10) `nasm` считает ошибкой. Судя по листингу 12.1, это справедливо — хотя это не ошибка, а, скорее, самообман. Остальной код сходится с листингом 12.1 — кроме усечения адреса в (24) до 16 бит. Эта вивисекция, которую `nasm` произвел над длинным адресом в (24), основана на предположении, что в реальном режиме адрес *всегда* ограничен 16 битами². Но в режиме `unreal` (разновидность реального режима) это ограничение снято, и вспомогательный сегмент данных (`es`, `fs` или `gs`) допускает смещение до `0xffffffff`. И в этом случае программа из листинга 10.11 не вызывает срабатывания общей защиты.

Можно сделать вывод, что префиксы `a4` и `o4` проставлять самим не следует. Но есть исключения: строковые команды и команды `call/ret`. В строковых командах раз-

мерность операнда обозначена суффиксом b/w/d. Например, команда `movsd` копирует из памяти в память двойное слово: `d[ds:si] -> d[es:di]`. Префикс `a4` воздействует на адрес, так что вместо `si/di` используются их 32-битовые расширения `esi/edi`. А если перед строковой командой стоит префикс повторения `rep[z|nz]`, то `a4` увеличивает размерность не только эффективного адреса, но и счетчика повторов: вместо `cx` используется `ecx`.

Как это проверить, оставаясь в реальном режиме? Сначала следует задать в старшей части `esi` ненулевое значение и выполнить команду `a4 lodsb` — должна сработать общая защита (исключение 13). Потом задать `esi` в пределах 16 бит, `ecx = 010000` и выполнить один шаг команды `rep lodsb`. Если `ecx` станет равным `0ffff`, то используется именно он. А иначе, если задействован `cx`, `ecx` и `edi` останутся без изменений, поскольку `cx = 0` — повторы закончились (листинг 12.3)³.

Листинг 12.3. Действие префикса `a4` на строковую команду (`a4_str.8x`)

```

mov     esi, 010000      ; (1)
lodsb                   ; (2)

mov     ecx, 010003      ; (3)
cld                                           ; (4)
rep lodsb                ; (5)

int     3                ; (6)
a4 lodsb                 ; (7)

and     esi, 0ffff       ; (8)
mov     ecx, 010001      ; (9)
rep a4 lodsb             ; (10)
...
```

Вариант для `nasm` приведен в файле `a32_str.asm`.

И еще несколько команд, в которых нет явной информации о размерности операнда или адреса. Помимо строковых команд, это `loop[z|nz]` и `xlat`.

Какой регистр используется в `loop[z|nz]` — `cx` или `ecx`? Это зависит от текущей размерности и может быть изменено для 16-битового режима одним из префиксов⁴ `o4` или `a4`. Как проверить команду цикла в реальном режиме? Видимо, нужно записать в `ecx` число `010000`, и если команда использует `cx`, то в `ecx` после ее первого выполнения будет `01ffff`. А если задействован `ecx`, то в нем окажется `0ffff`. Программа для проверки приведена в листинге 12.4.

Листинг 12.4. Префиксы `a4` и `o4` для команды `loop` (`cx_ecx.8x`)

```

mov     ecx, 010000      ; (1)
loop    >m1              ; (2)
nop                                           ; (3)
```



```

m1:                                     ; (4)
      mov     ecx, 010000               ; (5)
      jcxz    >m1                       ; (6)
      nop                                     ; (7)
m1:                                     ; (8)
      jecxz    >m1                       ; (9)
      nop                                     ; (10)
m1:                                     ; (11)
      o4 loop >m1                       ; ? ; (12)
      nop                                     ; (13)
m1:                                     ; (14)
      mov     ecx, 010000               ; (15)
      a4 loop >m1                       ; ! ; (16)
      nop                                     ; (17)
m1:                                     ; (18)
      ...

```

Команды в строках (6) и (9) проверяют соответственно 16- и 32-битовый регистр-счетчик. Код у этих команд, судя по содержимому файла `sx_esc.lst`, одинаковый, только перед (9) поставлен префикс `a4` (067).

Для команды `xlat` имеет смысл префикс `a4` (или `a2` в 32-битовом режиме): он изменяет размерность указателя таблицы `bx/ebx`. Для проверки запишем в `ebx` адрес 010000, а в `al` — ноль. (Для команды `xlat` регистр `al` — это индекс в таблице байтов, и он же — результат чтения из таблицы.) Команда без префикса должна обратиться к началу `PSP`, а команда с префиксом — вызвать срабатывание общей защиты. Тестовая программа приведена в листинге 12.5.

Листинг 12.5. Префикс `a4` в команде `xlat` (`xlat.8x`)

```

mov     b[0], 055
mov     ebx, 010000
mov     al, 0
xlat
mov     al, 0
a4 xlat
...

```

Результат выполнения первой команды `xlat` показан на рис. 12.1. К сожалению, в отладчике обе команды `xlat` выглядят одинаково, как будто у `d386` нет слов, чтобы выразить наличие префикса перед второй командой.

В командах `div/ldiv` и `mul/imul` один операнд явный, а другой подразумевается. Префикс `o4` не требуется, т. к. размерность операции следует из первого операнда. Так, команда `mul ebx` выполняет умножение `eax*ebx`, а результат помещает в пару `edx:eax`; команда `div ebx` делит `edx:eax` на `ebx` и записывает частное в `eax`, а остаток в `edx`. Программа для проверки приведена в листинге 12.6.

```
0100 MOV B[0],055
0105 MOV EBX,0001_0000
010B MOV AL,0
010D XLATB
# 010E MOV AL,0
0110 XLATB
0112 INT 020

EAX 0000_0055 i 1:
EBX 0001_0000 IP 010E 2:
```

Рис. 12.1. Фрагмент экрана d386 после выполнения первой команды `xlat`

Листинг 12.6. Команды `div` и `mul` с 32-битовым операндом (`mul_div.8x`)

```
mov    eax, 123456
mov    ebx, 543210
mul    ebx           ; 67,062,533,760 = 0xF_9D3D_0E80
mov    ebx, 1000000
div    ebx           ; 67,062 = 0x1_05F6 (533,760 = 0x8_2500)
...
```

Результаты выполнения иллюстрирует рис. 12.2. Флаги переполнения после умножения показывают, что результат не поместился целиком в `eax`, т. е. в `edx` есть значащие цифры.

0100 MOV EAX,0001_E240	0100 MOV EAX,0001_E240
0106 MOV EBX,0008_49EA	0106 MOV EBX,0008_49EA
010C MUL EBX	010C MUL EBX
# 010F MOV EBX,000F_4240	010F MOV EBX,000F_4240
0115 DIV EBX	0115 DIV EBX
0118 INT 020	# 0118 INT 020
EAX 9D3D_0E80 o is c	EAX 0001_05F6 o is c
EBX 0008_49EA IP 010F	EBX 000F_4240 IP 0118
ECX 0000_0000 CS 64C9	ECX 0009_0000 CS 64C9
EDX 0000_000F SS 64C9	EDX 0008_2500 SS 64C9

Рис. 12.2. Фрагменты экрана d386 после умножения (слева) и деления (справа)

В завершение разговора о префиксах размерности `a4/o4` — еще один пример: команда возврата `ret/retf`. В 32-битовом режиме короткий вызов подпрограммы сохраняет в стеке копию 32-разрядного указателя инструкций `eip`. (В 16-битовом режиме — копию 16-разрядного `ip`.) Дальний вызов в 32-битовом режиме сначала сохраняет регистр `cs`, но затем продвигает `sp` на четыре (хотя размерность `cs` — два байта). В листинге 12.7 приведен один пример для `ret` и два примера для `retf`.

Листинг 12.7. 32-битовый возврат из подпрограммы (o4_ret.8x)

```

        push    0, m1                ; (1)
        o4 ret                          ; (2)
        nop                                ; (3)
m1:      nop                                ; (4)
        push    0, cs, 0, m2          ; (5)
        o4 retf                       ; (6)
        nop                                ; (7)
m2:      nop                                ; (8)
        mov     ax, cs                ; (9)
        dec     ax                    ; (10)
        push    0, ax, 0, m3+010      ; (11)
        o4 retf                       ; (12)
m3:      nop                                ; (13)
        int     3                     ; (14)
        int     020                   ; (15)

```

При каждом выполнении `ret/retf` мы оказываемся все ниже — на ближайшей метке `m <n>`.

Косвенная адресация через 32-битовые регистры

Наиболее существенное отличие 32-разрядных процессоров i80x86 от 16-разрядных — это возможность косвенной адресации с использованием *любого* 32-битового регистра общего назначения. Косвенная адресация по сумме двух регистров теперь допустима для любой пары 32-битовых регистров, включая сочетание регистра с самим собой⁵. Наконец, для одного из регистров⁶ можно задать масштабный множитель 2, 4 или 8. Примеры косвенной адресации с использованием 32-битовых регистров приведены в листинге 12.8.

Листинг 12.8. Косвенная адресация с использованием 32-битовых регистров (ind32.8x)

```

        mov     edx, 1                ; (1)
        mov     ebx, 8                ; (2)
        not     b [edx]               ; (3)
        not     b [edx*3]             ; [edx*2+ edx] ; (4)
        not     b [edx*5]             ; [edx*4+ edx] ; (5)
        not     b [edx*9]             ; [edx*8+ edx] ; (6)
        not     b [edx+ebx]           ; (7)
        not     b [edx*2+ebx]         ; (8)
        not     b [edx*4+ebx]         ; (9)
        not     b [edx*8+ebx]         ; (10)
        ...

```

Фрагмент экрана d386 перед началом серии команд с косвенной адресацией показан на рис. 12.3. Выделенные байты будут инвертированы в порядке возрастания

адреса. Байту в конце первой строки отображения данных достанется два раза под-
ряд: командами (6–7).

В листинге 12.9 приведена программа поиска в массиве 32-битовых слов, состав-
ленная по мотивам программы `find_2.8` для 16-битовых слов (см. листинг 6.2).

```
0100 MOV EDX,0000_0001
0106 MOV EBX,0000_0008
# 010C NOT B[EDX]
010F NOT B[EDX+2*EDX]
0113 NOT B[EDX+4*EDX]
0117 NOT B[EDX+8*EDX]
011B NOT B[EDX+EBX]
011F NOT B[EBX+2*EDX]
0123 NOT B[EBX+4*EDX]
0127 NOT B[EBX+8*EDX]

EAX 0000_0000    i          1: b10,0,, CD 20 64 99 00 EA 70 00 70 00
EBX 0000_0008    IP 010C    2: "000A  D0 3E 33 36 1C 00 F6 99 1E 00
EDX 0000_0001
```

Рис. 12.3. Фрагменты экрана d386 до выполнения команд с косвенной адресацией

Листинг 12.9. Поиск в массиве с использованием масштабируемого индекса (find_3.8x)

```
x          equ      012345678                                ; (1)
max32      equ      07fffffff          ; signed              ; (2)
min32      equ      080000000                                ; (3)

          jmp      start                                        ; (4)

array      dd      0, max32, min32, x-1, x+1, x              ; (5)
item_sz    equ      type array                                ; (6)
item_no    equ      ($ - array) / item_sz                    ; (7)

start:     mov      cx, item_no                                ; (8)
          mov      esi, -1                                    ; (9)

l1:                                                ; (10)
          inc      esi                                        ; (11)
          mov      eax, [esi*item_sz+array]                    ; (12)
          cmp      eax, x                                      ; (13)
          loopne   l1                                         ; (14)
          jne      miss                                        ; (15)

found:     int      3                                          ; (16)
miss:      int      020                                        ; (17)
```

Для адресации элементов массива используется 32-битовый регистр с масштабным коэффициентом (12). Вместо *esi* можно взять *edx*, *eax* и т. п. — любой 32-битовый регистр общего назначения. Пару команд (12–13) можно было бы записать в одну, но тогда в отладчике не видно, к какому элементу массива мы обращаемся.

Новые команды

Для дальнейшего знакомства с 32-битовыми командами вернитесь к задачам из главы 7. Выберите такую, где речь идет об обработке 16-битовых слов, и запрограммируйте вариант для 32-битовых данных. Интерес также представляют задачи на обработку битов, т. к. в 32-разрядных вариантах *i80x86* для этого появились новые возможности.

Новые команды, начиная с *i80386*, перечислены в [9, 22.13], а назначение раскрыто в разделе "The 86 Instruction Set" в файле *a86manu.txt* из архива *a86.zip*. Перечислим некоторые наиболее интересные команды, используя сокращения: *r* — регистр, *m* — память, *imm* (immediate) — непосредственное значение, *dst* — приемник, *src* — источник.

- ❑ *cwde* — копирование *ax* → *eax* с расширением знака.
- ❑ *pushad/popad* — сохранение/восстановление 32-битовых регистров общего назначения (по аналогии с *pusha/popa* для 16-битовых регистров).
- ❑ *movzx dst, src* — копирование в приемник большей размерности с обнулением его старшей части.
- ❑ *movsx dst, src* — копирование в приемник большей размерности с расширением знака.
- ❑ *bt {m/r}, {r/imm}* — чтение бита (Bit Test) из массива бит (размерностью 8, 16 или 32) во флаг переноса *c*; положение массива задано первым операндом, а номер бита — вторым. Имеются варианты с изменением проверенного бита, независимо от его текущего значения: *btc* — complement (инвертирование), *btr* — reset (сброс), *bts* — set (установка).
- ❑ *bsf dst, src* — поиск бита в битовом массиве *src* (не более 32 битов), от младшего бита к старшим (Bit Scan Forward). Если поиск успешный, то флаг *z* = 0 и номер записан в *dst*. Если ничего не нашли (при *src* = 0), то флаг *z* = 1.
- ❑ *bsr* — аналогично, только поиск от старшего бита к младшему (Bit Scan Reversed).
- ❑ *set<c> dst* — запись в *dst* нуля или единицы в зависимости от условия *<c>*, например *setnz al*.
- ❑ *sh{l|r}d dst, src, cl/imm* — команда двойного сдвига: биты, вытолкнутые из копии *src* (сам *src* не изменяется), попадают в *dst*. Счетчик сдвигов задан в конце команды.
- ❑ *xadd {m/r}, r* — обмен местами и сложение.
- ❑ *cmov<c> src, dst* — условное копирование.

В листинге 12.10 приведены примеры перечисленных команд.

Листинг 12.10. Команды, появившиеся в 32-разрядных процессорах (new_cmd.8x)

```

mov     al, -2                                ; (1)
cbw                                ; al -> ax      ; (2)
cwd                                ; ax -> dx:ax    ; (3)
cwde                                ; ax -> eax    ; (4)

mov     eax, 080                            ; (5)
movsx   bx, al                             ; (6)

mov     eax, 0aa550000                      ; (7)
movzx   eax, bx                             ; (8)

pushd   1, 2, 3, 4, 5, 6, 7, 8              ; (9)
popad                                ; (10)

mov     esi, bit 30                         ; (11)
mov     ecx, 32                             ; (12)
11:    bt     esi, ecx                       ; (13)
jc      >l2                                ; (14)
loop    l1                                 ; (15)
12:                                ; (16)
stc                                ; (17)
setc    ch                                ; (18)
cmovc   bx, bp                            ; (19)
clc                                ; (20)
setc    ch                                ; (21)
cmovc   bx, dx                            ; (22)

xadd     eax, esi                          ; (23)

mov     eax, 0                             ; (24)
bsf     dx, eax                            ; (25)
mov     eax, bit 16 + bit 15               ; (26)
bsf     dx, eax                            ; (27)
bsr     dx, eax                            ; (28)

mov     ax, 02076                           ; (29)
mov     dx, 3                              ; (30)
shrd    ax, dx, 3                          ; (31)

mov     ax, 02076                           ; (32)
mov     dx, 0c000                           ; (33)
shld    ax, dx, 2                          ; (34)

int     020                                ; (35)

```

Результат выполнения команды в строке (2): `ax = 0fffe`. После (3) `dx = 0ffff`, а после (4) `eax = 0ffff fffe`.

После строки (6) `bx = 0ff80`, а после (8) `eax = 0ffff ff80`.

Восемь 32-битовых значений, записанных (9) в стек, при выполнении (10) снимаются из стека в регистры в следующем порядке: edi, esi, ebp, ebx, edx, ecx, eax; последнее значение никуда не записывается.

Цикл (16–15) продолжается, пока `ecx` не уменьшится до 30, адресуя единичный бит в `esi`.

Команды (18–19) и (21–22) связаны с флагом переноса `c`. Вообще, условие может быть любым — из тех, что допускаются в командах ветвлений, кроме `cxz` (`cx = 0`).

Команда (25) демонстрирует неудачный поиск единичного бита, а (27–29) — удачный и в разных направлениях.

Выполнение операторов (31) и (34) иллюстрирует рис. 12.4. Сначала записываем шестнадцатеричное число 02076 по битам, разделяя тетрады пробелом. Потом убираем пробелы и добавляем слева три младших бита числа 3 (сдвиг вправо), группируем полученную последовательность по четыре бита слева направо. Три бита справа лишние, они были вытолкнуты при сдвиге. В правой части рис. 12.4 — аналогично для сдвига влево на два разряда.

```

      2    0    7    6
      0010 0000 0111 0110
      0010000001110110
0011 -> 0010000001110110          0010000001110110 <- 1100
      0110010000001110110          001000000111011011
      0110 0100 0000 1110 -> 110    00 <- 1000 0001 1101 1011
      6    4    0    e                8    1    d    b

```

Рис. 12.4. Выполнение команд (31) и (34) из листинга 12.10

Примечания

1. Расширенные (32-битовые) регистры пишутся с буквы `e`.
2. Горе от ума! Если команда имеет машинное выражение, ассемблер обязан ее генерировать, а не изображать из себя "заботливую няньку".
3. Без `d386` этот код можно проверить в `d86` или в отладчике Bochs — для этого нужно поставить в начале программы команду `xchg bx, bx` и запустить сеанс командой `__debug.bat`.
4. Кажется очевидным, что если речь идет о размерности операнда, то нужен префикс `o4`. Но возможны исключения.
5. В `i8086/286` допускалось только сочетание одного базового `bp/bx` и одного индексного `si/di` регистра.
6. Обычно это регистр, который в программе используется в качестве индекса или смещения.


```

DB6_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA    i    c    1: =e1,ax,, 10101010
BX 0000    IP 0100  2: =d1,ax,, 1962
CX 0000    CS 6309  3:
DX 6309    SS 6309  4:
SI 011F    DS 6309  5: r5,b_1,, AB ab
DI 0000    ES 6309  6: r5,si,, AB ab
BP 0000    SP FFFE  0:

lea si,b_1

```

ЧАСТЬ II

Защищенный режим

Глава 13. Код в защищенном режиме

Глава 14. Данные и стек

Глава 15. Исключения и прерывания

Глава 16. LDT и TSS

Глава 17. Преобразование адресов

Глава 18. Привилегии

ГЛАВА 13

```
DB6 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100  2: =d1,ax,, 1962
CX 0000  CS 6309  3:
DX 6309  SS 6309  4:
SI 011F  DS 6309  5: r5,b_1,, AB ab
DI 0000  ES 6309  6: r5,si,, AB ab
BP 0000  SP FFFE  0:

lea si,b_1
```

Код в защищенном режиме

Дабы узнать, что же это за слова, он поднес лист бумаги к огню на тот случай, если письмо написано раствором нашатыря. <...> Затем он натер его молоком женщины, кормившей свою перворожденную дочь, — на тот случай, если письмо написано кровью жабы. <...> Затем он натер другой уголок серой из уха на тот случай, если письмо написано желчью ворона. <...> Затем он вымочил письмо в уксусе на тот случай, если оно написано касторовым маслом. <...> Затем он осторожно окунул его в таз с холодной водой и сейчас же вынул — на тот случай, если оно написано квасцами.

(Франсуа Рабле. Гаргантюа и Пантагрюэль)

Гребешков вынул из жилетного кармана кусочек грязного мыла и сунул его в полуштоф. Когда водка вспенилась и замутилась, он принялся всыпать в нее всякую дрянь. В полуштоф посыпались селитра, нашатырь, квасцы, глауберова соль, сера, канифоль и другие "специи"... Комик пялил глаза на Гребешкова и страстно следил за движениями полуштофа.

(А. П. Чехов. Средство от запоя)

Защищенный режим появился в 16-битовом варианте еще в процессоре i80286. Мы начнем с 32-битового защищенного режима, впервые реализованного в i80386 и поддерживаемого как режим совместимости в 64-разрядных процессорах AMD-Intel.

В реальном режиме сегмент, или участок памяти для кода, данных или стека занимает 64 Кбайт, а его базовый адрес задан 16-битовым номером параграфа. При доступе к памяти 16-битовый сегментный адрес сдвигается на четыре бита влево [2, Figure 1-5]. Адресное пространство при 20-разрядных линейных адресах ограничено величиной 1 Мбайт, зато прикладным программам предоставлена полная свобода: можно записывать в любой участок памяти, включая области операционной системы.

В защищенном режиме базовый адрес сегмента — 32-битовый, а размер можно задать любым в пределах 32 бит (от одного байта до 4 Гбайт). Запись в сегмент кода запрещена, но можно разрешить чтение. И наконец, введена система привилегий, чтобы окончательно загнать пользовательскую программу в рамки, не дающие ей разрушить другие задачи и операционную систему.

Для адресации памяти сохранены те же сегментные регистры, только теперь они называются селекторами¹. Почему? Теперь их значения — это индексы в специаль-

ной таблице, где содержатся описания (дескрипторы) сегментов. Общесистемная таблица GDT (Global Descriptor Table — глобальная таблица дескрипторов) может находиться где угодно в физической памяти [2, Figure 1-5], но ее адрес и размер хранятся в регистре GDTR (GDT Register). При доступе² к памяти процессор использует соответствующий селектор для чтения дескриптора из GDT.

В первых примерах мы используем только сегмент кода, не обращаясь к данным в памяти и не задействуя стек. Задача — дойти до конца программы. Воспользуемся отладочным вариантом Bochs: `__debug.bat`. Первый пример работы в защищенном режиме приведен в листинге 13.1.

Листинг 13.1. Переключение в защищенный режим, вариант для a386 (pm0.8x)

```
include a386.inc ; (1)

        jmp     start ; (2)

        even    8 ; (3)

gdt: ; (4)
dsc0: ; Null ; (5)
    dd    0, 0 ; (6)
dsc1: ; Code 32 bit ; (7)
    dw    1k ; limit low ; (8)
    dw    ? ; base low ; (9)
    db    ? ; base middle ; (10)
    db    1001_1000xb ; P=1, DPL=00, S=1 | 1, C, xr, A ; (11)
    db    0100_0000xb ; G=0, 32 bit, 00 | limit high ; (12)
    db    0 ; base high ; (13)

gdtr: ; (14)
    dw    $ - gdt - 1 ; size(gdt) ; (15)
    dd    ? ; laddr(gdt) ; (16)

;----- ; (17)

start: _brk ; (18)

    xor     eax, eax ; (19)
    mov     ax, cs ; (20)
    shl     eax, 4 ; eax := laddr(ps) ; (21)

    mov     edx, eax ; (22)

    lea     bx, dsc1+2 ; dsc1.laddr ; (23)
    mov     w[bx], ax ; low ; (24)
    shr     eax, 8 ; (25)
    mov     b[bx+2], ah ; middle ; (26)
```

```

    lea    bx, gdtr+2      ; gdtr.laddr      ; (27)
    add    edx, gdt        ; (28)
    mov    [bx], edx       ; (29)

    lgdt   [gdtr]          ; (30)

    cli                                ; (31)

    mov    eax, cr0        ; (32)
    xor    al, 1           ; (33)
    mov    cr0, eax        ; PM             ; (34)

    jmp    (1*8):0200      ; sel = 1, rpl = 0 ; (35)

;-----                                ; (36)

    org    0200            ; (37)
    use32                                ; (38)

pm:    _brk                                ; (39)
    clr    ebx, ecx        ; (40)
12:    inc    w[ebx]        ; ?!           ; (41)
    dec    ecx             ; (42)
    jmp    12              ; (43)

```

В подключаемом файле (1) определен макрос для задания точки останова, его вызовы находятся в начале (18) и в конце (39) программы.

В строке (3) задано выравнивание таблицы³ GDT, находящейся по адресу (4). Каждый дескриптор занимает 8 байтов. Начальный (нулевой) дескриптор (5) зарезервирован: попытка записать ноль в регистр-селектор вызовет срабатывание общей защиты⁴. Начиная с (7) задаем поля следующего (первого) дескриптора — для адресации кода в защищенном режиме.

Поля дескриптора кода показаны в [2, Figure 4-14]⁵. Первым словом (8) заданы младшие 16 бит предела. Число 1k (1024) взято с запасом, оно гораздо больше объема, занимаемого в памяти программой pm.com (размер com-файла + 256 байтов PSP)⁶. Старшие четыре бита предела заданы младшей тетрадой предпоследнего байта (12) — нули. (Значок '|' в комментариях в (11–12) поставлен между тетрадами байта.)

В строках (9–10) и (13) заданы составляющие базового линейного адреса кода. Старшая часть (13) во всех опытах равна нулю, т. к. в реальном режиме программа может быть запущена лишь в пределах первого мегабайта. Поля (9–10) заранее не известны, они зависят от расположения программы в памяти. В строках (19–21) мы вычисляем базовый линейный адрес кода, сдвинув на четыре бита влево значение сегментного регистра кода. Результат по частям записываем (23–26) в поля (9–10).

В управляющей структуре данных (14–16), которая будет записана в регистр `GDTR`, адрес `GDT` в поле (16) тоже должен быть линейным. Воспользовавшись значением базового линейного адреса, сохраненным в (22), прибавляем к нему (28) смещение `GDT` и записываем (29) результат в поле (16) структуры (14–16). Наконец, командой (30) копируем структуру (14–16) в регистр `GDTR`⁷.

В итоге регистр `GDTR` содержит линейный адрес и размер глобальной таблицы дескрипторов. В дескрипторе (7–13) сформированы адресные поля, также представляющие базовый линейный адрес программы. (Назначение битовых полей в (11–12) выяснится чуть позже.)

Мы все подготовили в реальном режиме и можем переключить процессор в защищенный режим. Внешние прерывания⁸ запрещаем (31). Для перехода в защищенный режим устанавливаем (32–34) младший бит в управляющем регистре `cr0`.

И тут же окажемся в другой реальности, где адресация кода идет по цепочке от регистра `GDTR` через таблицу `GDT`, а смещение⁹ дескриптора в таблице задано регистром-селектором `cs`.

Правда, непонятно, как будет выполнена команда (35) — она ведь скомпилирована для 16-битового режима. Да и зачем она? Затем, что это единственный способ изменить значение `cs`, не считая команды `retf`. И другая причина: необходимо очистить очередь предвыборки. Параллельно¹⁰ с выполнением команд процессор считывает из памяти последующие коды¹⁰ — по возрастанию адресов. Он не только заполняет очередь предвыборки, но и декодирует находящиеся в ней команды. К моменту выполнения (34) команда (35) уже расшифрована, причем согласно 16-битовой кодировке. При выполнении (35) в `cs` записывается 8 (смещение `dsc1` от начала `GDT`), а в `ip` — 0200 (эффективный адрес следующей команды). При выполнении (35) также сбрасывается очередь предвыборки¹¹. Теперь процессор заново заполняет очередь предвыборки, дешифруя команды (39–43) по 32-битовой схеме. (Если бы не промежуток между (35) и (39), образовавшийся в результате директивы (37), можно было бы сказать так: в очереди предвыборки к моменту выполнения (35) находятся команды из последовательности (39–42), декодированные согласно прежней 16-битовой схеме, т. е. неправильно.)

Выполните трансляцию программы `pm0.8x`. В отсутствие `a386` можно воспользоваться либо готовым файлом `pm0.com`, либо ассемблером `nasm` (см. файл `pm-0.asm`, а комментарии к нему — чуть позже).

Запустите сеанс `Bochs` в отладочном варианте: `__debug.bat`. Первая остановка произойдет при запуске "процессора". Для продолжения нажмите клавишу `<F5>` или кнопку **Continue**, и в пользовательском окне `Bochs` должен отобразиться процесс загрузки. Затем вызовите программу `pm0.com` — напрямую¹², без `d86`. Остановка должна произойти на инструкции, следующей за (18). В окне отладчика `Bochs` выполните команду **View/Linear Dump** (`<F7>`) и задайте адрес, равный `cs*010 + 0100`. (Например, если `cs = 0362a`, то начальный адрес программы — `0x363a0`.) В окне справа появится шестнадцатеричное отображение кода и данных программы со смещения `0100`. Выполните команду **Options/Dump 'Wordsize'/8 bytes** (`<Alt>+<8>`), чтобы видеть 8-байтовые слова¹³. В начале отображения — код (2) и добавочные

байты, заданные в строке (3). Следующий 8-байтовый элемент — `dcs0`, заполненный нулями, за ним `dsc1 = 0040_9800_0000_0400`, а следом `gdtr = db87_0000_0000_000f`. Выполните по шагам (нажимая клавишу `<F11>` или кнопку **Step**) команды (19–30).

Тут обнаруживается недостаток отладчика Bochs: он не отображает изменений в данных, а команда **Refresh** (обновление окон кода и данных) и нажатия клавиши `<Esc>` (обновление окна данных) бесполезны. Остается только заново нажимать `<F7>` или `<Ctrl>+<F7>`, а затем, когда появится диалоговое окно адреса, сразу нажать `<Enter>`, благо введенный адрес там остался.

Пройдя команды (19–30), проверьте изменения в данных. Теперь в `dsc1` записан линейный адрес программы (`0040_9803_62a0_0400`), а в `gdtr` — линейный адрес GDT (`db87_0003_63a8_000f`). После команды (30) выполните **View/GDT** (`<Ctrl>+<F2>`). Первый дескриптор в GDT сформирован, как предполагалось: базовый адрес, предел сегмента, его тип и разрядность — все правильно.

Продолжите программу до точки (40). Мы перешли в защищенный режим. (Перед тем как сдвинуться с точки (18), можно было выполнить команду **Break on CPU mode change** (`<Shift>+<F6>`), и тогда точка останова (39) оказалась бы лишней.) И еще одна хорошая новость — при переходе в защищенный режим Bochs переключает схему дешифрации команд и в результате правильно дизассемблирует (39–43).

Интересно, что в (41) задано обращение к памяти, и оно работает (можно посмотреть, как меняется слово в начале `PSP`). Но ведь мы его не настраивали, т. е. ничего не писали в `ds`. В этом-то все и дело: мы не трогали `ds`, поэтому в нем сохранился базовый адрес нашей программы, сформированный загрузчиком, а поле предела изначально (для реального режима) равно `0ffff`. Ведь дескрипторы действуют и в реальном режиме, только у программиста нет доступа к полям предела и атрибутов, а базовый адрес ограничен 20 битами. И до тех пор, пока в защищенном режиме мы ничего не пишем в `ds`, в его кеш-памяти остается то, что было сформировано в реальном режиме.

По-хорошему нужно добавить в GDT дескриптор сегмента данных и записать его смещение в `ds` — сразу после (38). Но пока что, сохраняя программу в первозданной простоте, мы проведем ряд опытов для проверки атрибутов сегмента (11–12) и защиты памяти.

В листинге 13.2 приведен вариант рассмотренной программы, переписанный на ассемблере `nasm`, с некоторыми усовершенствованиями. С ним мы и будем работать дальше.

Листинг 13.2. Переключение в защищенный режим, вариант для `nasm` (`pm-0.asm`)

```
%include "nasm.inc" ; (1)

org 0100h ; !+ ; (2)

jmp start ; (3)
```

```

        align    4                ; !                ; (4)
gdt:                                ; (5)
dsc0:                                ; Null          ; (6)
        dd      0, 0              ; (7)
dsc1:                                ; Code 32 bit    ; (8)
        dw      end - 1           ; limit low     ! ; (9)
        dw      0                 ; base low      ! ; (10)
        db      0                 ; base middle   ! ; (11)
        db      1001_1000b        ; P, dpl, S | 1, c, xr, a ! ; (12)
        db      0100_0000b        ; g, 32 bit, 00 | limit high ! ; (13)
        db      0                 ; base high     ; (14)
gdtr:                                ; (15)
        dw      $ - gdt - 1       ; (16)
        dd      gdt               ; +            ; (17)

;-----                                ; (18)
start: _brk                          ; (19)

        laddr_cs                   ; +            ; (20)
        mov     edx, eax           ; (21)

        wr_base dsc1              ; +            ; (22)
        add     [gdtr+2], edx      ; !            ; (23)
        lgdt    [gdtr]            ; (24)

        cli                        ; (25)
        switch_mode                ; +            ; (26)
        jmp     8:pm               ; +            ; (27)

;-----                                ; (28)
;      org      0200h              ; +            ; (29)
;      use32                                ; (30)

pm:      _brk                      ; (31)
        clr     ebx, ecx           ; (32)
12:      inc     word [ebx]         ; (33)
;      cs mov    al, [dsc1+5]       ; ?            ; (34)
;      waste    4*1024              ; ?            ; (35)
        dec     ecx               ; (36)
        jmp     12                 ; (37)
;-----                                ; (38)
end:                                ; !            ; (39)

```

Отличия от листинга 13.1 помечены знаками '!', '+' и '?'. Восклицанием отмечены изменения, вынужденные различиями между a386 и nasm, плюсом — усовершенствования, а вопросом — команды, которые мы пустим в ход при последующих опытах.

Директива в строке (2) обязательна¹⁴, после `a86` к ней приходится привыкать. Директива выравнивания (4) в `nasm`, как видите, пишется иначе, также мы заменяем избыточное 64-битовое выравнивание 32-битовым. В строке (9) задаем точное значение предела для сегмента кода. Это адрес метки (39), которую в `nasm` приходится определять самим, минус 1. (Поле предела в дескрипторе — всегда на единицу меньше.) В (10–11) задаем 0, т. к. `nasm` не понимает знака вопроса. В строках (12–13) двоичные числа заданы суффиксом `'b'`, а в комментариях *величина* битового поля обозначена строчной или прописной буквой — для экономии места. То есть `'P'` в (12) означает `P=1`, а `'G'` в (13) — `G=0`.

Три команды для вычисления линейного базового адреса кода собраны в макрокоманду (20), см. файл `nasm.inc`. В этом файле также определены макросы: (22) для записи базового адреса из `eax` в дескриптор, заданный параметром, и (26) для переключения младшего бита в регистре `cr0`. Формирование в поле (17) линейного адреса GDT упростилось: смещение GDT задано при трансляции, остается только прибавить (23) базовый линейный адрес программы. Директива (29) закомментирована¹⁵. В строке (27) указана метка (31) начала кода защищенного режима.

По сравнению с листингом 13.1 вариант, приведенный в листинге 13.2, дает более компактный код, к тому же его работоспособность не зависит ни от размера фрагмента, выполняемого в реальном режиме, ни от объема программы в целом¹⁶.

Опыты с дескрипторами и защитой памяти

В начале каждого опыта копируйте `pm-0.asm` в другой файл, например `x.asm`, и делайте исправления в нем. Причина сбоя будет видна в консоли, например `jump_protected: cs == 0`.

- ☐ Исправьте в строке (9) единицу на двойку. При выполнении (37) будет нарушена граница сегмента кода.
- ☐ Тот же опыт, но ближе к практике: закомментируйте (37).
- ☐ Измените в (16) знак доллара на `dsc1`. Размер GDT, загружаемый в регистр `GDTR`, теперь меньше на один дескриптор, и при выполнении (27) произойдет сбой.
- ☐ Исправьте в строке (27) число 8 на 0.
- ☐ Закомментируйте строку (22), в результате чего в (10–11) останутся нули вместо линейного адреса программного сегмента¹⁷. В итоге базовый адрес кода равен нулю, и (27) передаст управление в начало физической памяти. Результат непредсказуемый: например, возможно сообщение: `write_virtual_checks(): write beyond limit, r/w`.

Вернемся к исходному варианту и разберем атрибуты сегмента, заданные в (12–13).

- ☐ Бит `P = 1` означает присутствие сегмента в памяти. При обращении к сегменту с `P = 0` возникнет исключение, и операционная система в ответ должна загрузить в этот сегмент код из внешней памяти и установить `P = 1`. Задайте в (12) код `1_1000b`. Последует сообщение: `check_cs(0x0008): code segment not present !`

- ❑ Бит номер 1 в (12) задает дополнительные возможности доступа. Единица для сегмента кода позволяет не только исполнять код (x — `eXecute`), но и читать его (r — `Read`). Добавьте единицу в первом бите (12): `1001_1010b`. И уберите комментарий в начале строки (34), эта команда будет читать байт (12) из первого дескриптора. Не отключая команду (34), восстановите исходное значение в (12); последует сообщение: `read_virtual_checks(): execute only`.
- ❑ Младший бит в (12) — A (`Accessed`), изначально равный нулю, должен уже стать единицей, поскольку к сегменту обращались. Если добавить команду, которая в цикле обнуляет¹⁸ бит A , то можно убедиться, что A снова устанавливается в единицу.
- ❑ Бит G (`Granularity`¹⁹) задает масштаб предела. При $G = 0$ предел задан в байтах, а при $G = 1$ — в страницах по 4 Кбайт. Только при $G = 1$ можно расширить предел сегмента до максимального значения 4 Гбайт. Задайте в (13) `1100_0000b`, а в (9) — ноль. Предел сегмента на единицу больше значения, заданного в (9) и (13). Сейчас, учитывая $G = 1$, предел равен $1 * 4$ Кбайт. Программа будет работать без сбоев. Но если убрать комментарий в (35), программа должна прерваться на смещении `0x1000` с сообщением: `prefetch: EIP [00001000] > CS.limit [00000fff]`. Верните первоначальное значение в (13), тем самым сократив предел до одного байта, и сбой произойдет уже при выполнении (27): `branch_far: RIP > limit`.
- ❑ Для сегмента кода бит 6 в (13) определяет разрядность кода. Единица — код 32-битовый, ноль — 16-битовый. Задайте ноль в (13) и закомментируйте (30). После переключения в защищенный режим проверьте результат: **View/GDT**.
- ❑ Для сегмента кода бит $S = 1$, т. к. это дескриптор общего назначения²⁰. Типы дескрипторов, в зависимости от S , сгруппированы в [2, Table 4-2], а в [2, Table 4-5] они показаны для $S = 0$ более подробно.

Первые опыты с привилегиями

Поле `DPL` (`Descriptor Privilege Level`) задает уровень привилегий дескриптора. Высшая привилегия имеет код ноль, а низшая — три²¹.

- ❑ `DPL` (`descriptor privilege level`) — статус, заданный в дескрипторе сегмента; в операторе (12) листинга 13.2 поле `DPL = 0`.
- ❑ `CPL` (`current privilege level`) — статус текущего исполняемого кода; хранится в битах 0–1 селектора `CS` и может быть прочитан с любого уровня привилегий.
- ❑ `RPL` (`requestor privilege level`) — статус, заданный в битах 0–1 регистра-селектора [2, Figure 4-3].

Напрямую можно обратиться к сегменту кода, только если его статус в дескрипторе (`DPL`) совпадает с текущим (`CPL`). Дополнительное условие: $RPL \leq CPL$. Предположим, $DPL = CPL = 2$. Если при этом $RPL = 3$, то в доступе будет отказано. Если $RPL = 0-2$, доступ разрешен.

Задайте в (12) число 1111_1000b (DPL = 3). Сейчас CPL = 0²², и в доступе будет отказано: check_cs(0x0008): non-conforming code seg descriptor dpl != cpl, dpl=3, cpl=0.

При DPL = 3 задайте RPL = 3, для этого исправьте в (27) число 8 на 8 + 3. Последует сообщение: check_cs(0x000b): non-conforming code seg selector rpl>cpl, rpl=3, cpl=0.

Можно попытаться добавить в (12) признак C = 1, но это тоже бесполезно. Если бы мы находились на уровне 3, то могли бы обратиться к сегменту кода с DPL = 0 и C = 1. Такой сегмент называется конформным (Conforming), при его выполнении CPL не меняется²³. Но у нас ситуация обратная: мы находимся на верхнем уровне иерархии (CPL = 0). Спуститься вниз — задача не из простых, мы к ней вернемся позже.

Переключение сегментов кода и вентили вызова

Добавим дополнительный сегмент кода с процедурой, которую мы будем вызывать из первого сегмента. В листинге 13.3 приведен пример прямого вызова.

Листинг 13.3. Прямое обращение к процедуре из другого сегмента кода (pm-1.asm)

```

...
jmp      start                      ; (3)

use32                                ; (4)
subr:                                       ; (5)
inc      ecx                          ; (6)
retf                                         ; (7)
end2:                                       ; (8)

use16                                ; (9)
...
dsc2:                                       ; Code 32 bit                ; (21)
dw      end2 - 1                      ; limit low                    ; (22)
dw      0                             ; base low                     ; (23)
db      0                             ; base middle                   ; (24)
db      1001_1000b                    ; P, dpl, S | 1, C, XR, A     ; (25)
db      0100_0000b                    ; G, 32 bit, 00 | limit high  ; (26)
db      0                             ; base high                     ; (27)
...

start: _brk                            ; (32)
mov     sp, 0100h                      ; (33)

ladr_cs                                ; eax := ladr(cs)              ; (34)
mov     edx, eax                       ; (35)

```

```

wr_base dsc1                                ; (36)

mov     eax, edx                             ; (37)
wr_base dsc2                                ; (38)

add     [gdtr+2], edx                       ; (39)
lgdt    [gdtr]                             ; (40)

cli                                           ; (41)
switch_mode                                ; (42)
jmp     8:pm                                ; (43)

;-----
use32                                        ; (44)
                                        ; (45)

pm:    _brk                                ; (46)
        clr     ebx, esi                    ; (47)
l2:    inc     word [ebx]                   ; (48)
        inc     esi                        ; (49)
        call    2*8:subr                    ; via dsc2 ; (50)
        jmp     l2                          ; (51)
end:                                         ; (52)

```

```

; cs,0 (4) +8 ; (7)
; eip (4) +4 ; (8)
; ebp (4) +0 ; (9)
mov ax, [ebp+14] ; (10)
mov cx, [ebp+12] ; (11)
rol ax, cl ; (12)
leave ; (13)
retf ; (14)
end2: ; (15)
...
pm: _brk ; (53)
clr ebx, esi ; (54)
mov ebp, 0x5555aaaa ; (55)
12: ; (56)
inc word [ebx] ; (57)
inc esi ; (58)

push word [ebx] ; (59)
push si ; (60)
call 2*8:subr ; via dsc2 ; (61)

jmp 12 ; (62)
end1: ; (63)

```

Процедуре передаются два 16-битовых параметра. Они записываются в стек (59–60) как 16-битовые. В результате вызова в стек добавляются восемь байтов адреса возврата (селектор, 16-битовый ноль и `eip`), а после (5) дополнительно сохраняется `ebp` и выполняется присвоение `ebp := esp`, так что `ebp` указывает на свое сохраненное значение. В строках (5–9) показаны данные в стеке, а также их размерность и смещение относительно указателя `ebp`. Параметры начинаются со смещения +12. Команды (10–11) считывают параметры в рабочие регистры, а (13) ликвидирует последствия выполнения (5), т. е. восстанавливает `ebp` и настраивает `esp` так, чтобы тот указывал на адрес возврата.

Если запустить цикл (56–62) бесконечно, без точек останова, то вскоре произойдет сбой. Судя по диагностике в консоли `Bochs`, он связан со стеком. Поставьте точку останова на строке (62) и посмотрите, как уменьшается `esp`. Чтобы понять причину, обратитесь к листингу 9.126.

В листинге 13.5 приведен еще один вариант межсегментного вызова процедуры — через так называемый вентиль вызова (`Call Gate`) [2, Figure 4-17].

Листинг 13.5. Вентиль вызова (`cg1.asm`)

```

...
dsc1: ; Code 32 bit ; (14)
dw end - 1 ; limit low ; (15)
dw 0 ; base low ; (16)

```

```

        db      0                ; base middle                ; (17)
        db      1001_1000b       ; P, dpl, S | 1, c, xr, a    ; (18)
        db      0100_0000b       ; g, 32 bit, 00 | limit high ; (19)
        db      0                ; base high                ; (20)

dsc2:
        ; Code 32 bit                ; (21)
        dw      end2 - 1         ; limit low                ; (22)
        dw      0                ; base low                 ; (23)
        db      0                ; base middle              ; (24)
        db      1001_1000b       ; P, dpl, S | 1, c, xr, a    ; (25)
        db      0100_0000b       ; g, 32 bit, 00 | limit high ; (26)
        db      0                ; base high                ; (27)

cgl:
        ; Call gate                ; (28)
        dw      subr             ; low adr                 ; (29)
        dw      16               ; selector (dsc2)         ; (30)
        db      0                ; (rsrv)                  ; (31)
        db      10001100b        ; P, dpl, s | Call Gate    ; (32)
        dw      0                ; high adr                ; (33)
        ...

pm:    _brk                      ; (52)
        clr      ebx, esi        ; (53)
l2:    inc      word [ebx]        ; (54)
        inc      esi             ; (55)
        call    3*8:0            ; via cgl                 ; (56)
        call    3*8:0ffffh       ; (57)
        jmp     l2               ; (58)
end:
; (59)

```

Вентиль вызова — это еще один дескриптор (28–33) в GDT. Он нестандартный ($s = 0$): в нем нет информации о базовом адресе и пределе, зато есть адрес межсегментного перехода (в полях Target Code-Segment Selector и Target Code-Segment Offset [2, Figure 4-17]). Поскольку адрес целиком задан в вентиле, то в командах (56–57) достаточно задать селектор вентиля, а смещение может быть каким угодно, процессор его игнорирует.

Вентили позволяют определить точки входа для обращения к привилегированным процедурам. Они и были задуманы для вызова функций операционной системы ($DPL = 0$) с пользовательского уровня²⁵ ($CPL = 3$), но в дальнейшем для этого появились другие средства²⁶. В листинге 13.5 приведен относительно простой вариант, когда уровень привилегий у всех дескрипторов одинаков и особой необходимости в вентилях вызова нет.

Обратное переключение режима

Эта возможность позволяет включить так называемый *unreal mode*: в защищенном режиме загрузить в *gs* или *fs* дескриптор с нулевым базовым адресом и пределом 4 Гбайт, потом вернуться в реальный режим и до завершения программы²⁷ пользоваться префиксом *gs* или *fs* с 32-битовым эффективным адресом. Этот режим мы рассмотрим в следующей главе, а пока — просто возврат в реальный режим.

Проще всего выполнить этот опыт, если сегмент, заданный *dsc1*, 16-битовый. Идея обратного переключения та же, что и прямого: изменить нулевой бит регистра *cr0* и выполнить дальний переход, указав в качестве селектора значение *cs*, соответствующее режиму. Если мы запустили программу при *cs = 0x362a*, то нужно это же число задать в команде межсегментного перехода, следующей за сбросом нулевого бита в *cr0*. Пример приведен в листинге 13.6.

Листинг 13.6. Возврат в реальный режим (*pm_rm.asm*)

```

...
dsc1:                ; Code 16 bit                ; (8)
    dw      0ffffh    ; limit low                ; (9)
    dw      0          ; base low                ; (10)
    db      0          ; base middle             ; (11)
    db      1001_1000b ; P, dpl, S | 1, c, xr, a        ; (12)
    db      0000_0000b ; g, 16 bit, 00 | limit high       ; (13)
    db      0          ; base high                ; (14)
...
start: _brk          ; (19)
    mov     ax, cs    ; (20)
    mov     [cmd+3], ax ; (21)
...
    switch_mode        ; (28)
    jmp     8:pm        ; (29)
...
pm:     _brk          ; (31)
    switch_mode        ; (32)
cmd:    jmp     0:rm    ; (33)
...
rm:     _brk          ; (35)
    _exit             ; (36)

```

Мы сразу записали (20–21) исходное значение *cs* в код команды перехода (33). Для включения и выключения нулевого бита в *cr0* используются вызовы (28) и (32) одной и той же макрокоманды, которая переключает этот бит. Обратите внимание, что предел сегмента в строке (9) задан точно таким же, как в реальном режиме. Иначе говоря, данные, записанные в теневой регистр²⁸ *cs* из *dsc1* при выполнении (29), совпадают с его данными в реальном режиме. Но смысл числа, записанного в видимую часть *cs*, разный: в защищенном режиме это селектор (смещение

дескриптора в GDT), а в реальном режиме — сегментный адрес (номер параграфа, с которого начинается сегмент).

Примечания

1. Большая удача для Intel, что слова "segment" и "selector" начинаются с одной буквы.
2. А здесь уже точно: дескриптор считывается только при *записи* в регистр-селектор один раз. Прочитанное сохраняется в кеш-памяти, где 16-битовый регистр селектора — не более чем точка доступа к структуре данных [2, Figure 4-4]. Кеш-память, где хранится копия дескриптора, называют также *теневым* (shadowed) регистром.
3. Для эффективного доступа к таблице достаточно выравнивания по 32-битовой границе, но с (3) таблица имеет лучший вид при просмотре в отладчике Bochs.
4. Исключение 13 — GPF (General Protection Fault). Обращение к селектору 0 происходит в результате довольно частой программистской ошибки: использования нулевого указателя данных.
5. Поля дескриптора в общем случае см. в [2, Figure 4-13].
6. В следующих примерах это число будет подсчитываться автоматически при трансляции.
7. Непривычно то, что в этом регистре хранится не одно, а два значения, причем разной размерности. А при записи в регистры-селекторы структура данных еще обширнее — целый дескриптор.
8. Они обрабатываются иначе, чем в реальном режиме.
9. Это действительно *смещение* в байтах от начала GDT. Иногда говорят об *индексе* в массиве 8-байтовых дескрипторов, что справедливо только для 32-битового режима. А в 64-битовом режиме в одной таблице могут соседствовать 8- и 16-байтовые дескрипторы.
10. Это началось еще с процессора i8088, где очередь предвыборки составляла 4 байта. При дальнейших модификациях i80x86 емкость очереди увеличивалась.
11. Побочный эффект передачи управления.
12. Отладчик d86/d386 с этого момента больше не понадобится.
13. Эта настройка сохраняется, но отображение данных придется задавать заново при каждом входе в Bochs.
14. По умолчанию `pasn` транслирует с нулевого смещения, а не со смещения 256, как `a86/a386`.
15. В ней больше нет необходимости. Кроме того, `pasn` считает ее ошибкой, т. к. он может справиться только с одной такой директивой — в операторе (2).
16. Кроме того, `a386` отказывается транслировать команду (15), и ее приходится задавать в кодах (*приложение 4*).

17. Такая простота, когда в (10–11) и (17) не требуется прибавлять линейный базовый адрес, возможна разве что при программировании начального загрузчика, который работает при `cs = 0`. Но у нас `com`-программа, и значение `cs` заранее не известно.
18. Это означает запись в программный сегмент. При использовании `cs` запись невозможна, но `ds`, ранее заданный в реальном режиме, разрешает нам и чтение, и запись. По этой причине в (34) можно было обойтись без префикса `cs`.
19. В переводе "зернистость", а по существу это масштаб: либо 1 байт, либо 4 Кбайт.
20. Согласно документации `S` означает `System`. Но `S = 1` как раз обозначает обычный дескриптор, неспецифический. На мой взгляд, проще полагать, что `S` — это `Standard`. Стандартные дескрипторы служат для описания сегментов кода, данных и стека. А нестандартные дескрипторы (`S = 0`) — это вентили вызова и прерываний, а также `LDT` и `TSS`, о которых речь впереди.
21. Нулевой (высший) уровень предназначен для операционной системы, а третий (низший) — для пользовательских приложений. Механизм привилегий в `i80x86` напоминает классы пассажиров на корабле: обитатели трюма могут подняться на общую смотровую площадку, не становясь при этом пассажирами 1-го класса, но аристократия обязана оставаться у себя на верхней палубе.
22. Нулевое значение `CPL` унаследовано от реального режима.
23. Аналогия с пассажирами третьего класса, поднявшимися на общую смотровую площадку на верхней палубе.
24. В 16-битовом режиме сохраняются `cs` и `ip`, а при выполнении команды `enter`, также `bp`. В 32-битовом режиме сохраняются `cs` (как 32-битовое значение с нулями в старшем слове), `eip`, а при отработке `enter`, еще и `ebp`.
25. В обратную сторону вентили не работают. Вообще обратиться с уровня `CPL = 0` к процедуре с `DPL > 0` нельзя ни напрямую, ни через вентиль.
26. Пары команд `SYSENTER/SYSEXIT` и `SYSCALL/SYSRET`. Первая пара предназначена для 32-битового режима, а вторая — для 64-битового, их мы рассмотрим впоследствии.
27. Похоже, что `FreeDOS` восстанавливает настройки предела сегментов `0xffff`. Поэтому действие `unreal mode` после *возврата* в `FreeDOS` не гарантируется.
28. То есть в кеш-память регистра-селектора. Теневой регистр — это невидимая, недоступная программе часть селектора. Туда (при записи числа `x` в селектор) копируется весь дескриптор, расположенный в `GDT` по смещению `x`.

ГЛАВА 14

```
DB6 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA i c 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:

lea si,b_1
```

Данные и стек

Продукты нужны самые простые — яйца, сахар, мука, — но важна последовательность и осторожность. И вдохновение, да! И есть один секрет...
(Татьяна Москвина. Не делайте бисквиты в плохом настроении)

До сих пор мы пользовались сегментом данных и стека, унаследованных от com-программы в реальном режиме: в теневых регистрах ds и ss остается линейный базовый адрес программного сегмента, а в поле предела — 0xffff.

Дескрипторы данных

Начнем с данных. В листинге 14.1 добавлены дескрипторы dsc4 (для доступа к данным внутри программы) и dsc5 (для доступа ко всему 32-битовому адресному пространству, от базового адреса ноль до предела 0xffff_ffff).

Листинг 14.1. Дескрипторы данных (ds_es.asm)

```
...
smth dw -1 ; (5)
endl: ; (6)

subr: ; (9)
inc ecx ; (10)
retf ; (11)
...

dsc4: ; Data (local), r/w ; (39)
dw endl - 1 ; limit low ; (40)
dw 0 ; base low ; (41)
db 0 ; base middle ; (42)
db 1001_0010b ; P, dpl, S | data, e, RW, a ; (43)
db 0000_0000b ; g, b, 00 | limit high (0) ; (44)
db 0 ; (45)
```

```

dsc5:                                ; Data (flat), r/w                                ; (46)
    dw    -1                        ; limit low (max)                            ; (47)
    dw    0                         ; base low                                  ; (48)
    db    0                         ; base middle                             ; (49)
    db    1001_0010b                 ; P, dpl, S | data, e, RW, a          ; (50)
    db    1100_1111b                 ; G, B, 00 | limit high (max)         ; (51)
    db    0                         ; (52)
    ...

    laddr_cs    eax := laddr(cs)                                ; (59)
    mov     edx, eax                                              ; (60)

    wr_base dsc1                                              ; (61)

    mov     eax, edx                                              ; (62)
    wr_base dsc2                                              ; (63)

    mov     eax, edx                                              ; (64)
    wr_base dsc4                                              ; (65)

    ...

    jmp     (dsc1 - gdt):pm                                      ; (70)

    use32                                              ; (72)
pm:    _brk                                              ; (73)

    set_seg ds, dsc4 - gdt                                      ; (74)
    set_seg es, dsc5 - gdt                                      ; (75)

    mov     esi, 080h                ; cmd string                                ; (76)
    mov     edi, 0b8000h              ; (77)
    clr     ecx                      ; (78)
    cld                                     ; (79)
    lodsb                                     ; (80)
    mov     cl, al                    ; (81)
    jcxz    l2                      ; (82)

l1:    movsb                                     ; (83)
    inc     edi                        ; (84)
    loop    l1                        ; (85)

l2:    not     word [smth]              ; (86)
    call    (cg1 - gdt):subr           ; call gate                        ; (87)
    jmp     l2                        ; (88)

end:                                     ; (89)

```

Данные, принадлежащие программе, ограничены меткой (6), т. е. включают в себя область PSP, слово (5) и то, что в промежутке. Имя в строке (6) использовано в определении предела (40) дескриптора (39–45). Предел для следующего дескриптора (46–52) задан максимальным, причем $G = 1$ (масштаб предела равен 4 Кбайт). Дескрипторы данных отличаются третьим битом в (43) и (50): этот бит в дескрипторах кода установлен, а в дескрипторах данных сброшен. Бит номер 1 в стандартных дескрипторах открывает дополнительные возможности доступа, но если для сегмента кода это был доступ на чтение (а не только на выполнение), то для сегмента данных — доступ на запись (а не только на чтение). Что означает для сегмента данных второй бит (E), мы выясним чуть позже. Бит 7 в (44) и (51) — это по-прежнему выбор между 16- и 32-битовым режимом, но применительно к данным это означает, что адрес для доступа к сегменту — 16-битовый или 32-битовый.

Запустите отладочный сеанс Bochs, а программу ds_es.com вызовите с параметрами в командной строке. В результате выполнения (74–85) параметры будут скопированы из области PSP в начало видеобuffers и станут видны в левом верхнем углу экрана. А программа уйдет в бесконечный цикл (86–88), выполняя чтение-запись (86) своих данных (5).

Настройка регистров для доступа к данным производится (74–75) прямой записью числа (смещение дескриптора от начала GDT) в регистр-селектор. Результат можно проверить, выполнив команду **View/GDT** и сравнив значения селекторов со смещениями дескрипторов в GDT. Затем настраиваем регистры-указатели: *esi* — на начало копии параметров командой строки в PSP, где записана длина ее параметров; а *edi* — на линейный адрес видеобuffers, с учетом того, что базовый адрес в ds равен нулю. Не *si/di*, а *esi/edi*, потому что текущем 32-битовом сегменте кода строковые команды (80) и (83) используют пары регистров *ds:esi* и *es:edi*. Загрузив длину и проверив ее на ноль (80–82), копируем (83) байт за байтом из области PSP в видеобuffer, пропуская (84) нечетные байты видеобuffers, где находятся цветные атрибуты.

Заметим, что в отладчике Bochs команда (82) отображается как *jecxz*. Это действительно так, в 32-битовом режиме команда (82) проверяет не *cx*, а *ecx*. Поэтому в (78) мы обнуляем *ecx*. Пожалуй, в области действия директивы (72) имеет смысл отказаться от 16-битовых регистров в пользу байтовых и 32-битовых.

Выполним пару опытов для получения ошибок.

- ❑ Исправьте в строке (86) ключевое слово *word* на *dword*, что должно привести к нарушению границы сегмента данных и сообщению: `write_virtual_checks(): write beyond limit, r/w`.
- ❑ Не исправляя эту ошибку, установите в строке (44) бит $G = 1$. Посмотрите, какое значение предела показывает команда **View/GDT**. Результатом будет модификация команды (10), примыкающей к (5), и разрушение программы.

Дескриптор стека

Определяя поле предела (limit) в дескрипторе, мы во всех рассмотренных примерах задавали верхнюю границу сегмента, так что допустимые смещения в сегменте ограничены диапазоном $[0 \dots \text{limit} + 1]$. Оказывается, что при $E = 1$ (Expandable) предел, заданный в дескрипторе, определяет *нижнюю* границу, а верхняя при этом всегда равна 0xffff (при $B = 0$) или 0xffff_ffff (при $B = 1$). Вариант с $E = 1$ позволяет защититься от растущего стека¹, поскольку стек расширяется вниз. Мы можем проверить эту идею на сегменте данных: в листинге 14.2 он допускает смещения от 0xffff вниз до метки (89), т. е. содержит все неинициализированные данные компилятора².

Листинг 14.2. Защищенный доступ к неинициализированным данным (bss.asm)

```

...
dsc4:                                ; Data (local), r/w                ; (36)
    dw      end - 1                 ; limit low                    ; (37)
    dw      0                      ; base low                  ; (38)
    db      0                      ; base middle               ; (39)
    db      1001_0110b             ; P, dpl, S | data, E, RW, a ; (40)
    db      0000_0000b             ; g, b, 00 | limit high (0)  ; (41)
    db      0                      ;                             ; (42)
    ...

    use32                           ; (69)

pm:   _brk                          ; (70)
    set_seg es, dsc4 - gdt          ; (71)
    mov     edi, 0ffffh             ; (72)
    std     ;                        ; (73)

l1:                                       ; (74)
    mov     ax, di                  ; (75)
    stosb                                ; (76)
    cmp     edi, end                ; (77)
    jae     l1                      ; (78)

l2:   _brk                          ; (79)
    jmp     l2                      ; (80)

    align   16                      ; (81)
    db      'Finita '               ; (82)
end:   db      'la Comedia'         ; (83)

```

Настроив пару `es:edi` (71–72) и задав отрицательное приращение (73) индексных регистров в строковых командах, мы пишем байт за байтом (76), двигаясь вниз в сторону метки (83). Строка в (83) будет перезаписана, но (82) останется в целости. Проверьте работу программы в отладчике Bochs, задав отображение данных кла-

вишей <F7>. Найти (82–83) нетрудно, тем более что она выровнена (81) для удобства наблюдения³.

Чтобы проверить срабатывание защиты, исправьте в (78) условный переход на безусловный, и тогда бесконечный цикл закончится сообщением: `write_virtual_checks(): write beyond limit, r/w expand down`.

Бит `В` в дескрипторе данных имеет смысл лишь при `Е = 1`. Посмотрим, в чем этот смысл. Сначала при `В = 0` пройдем по памяти в прямом направлении: от (83) до смещения `0xffff` (листинг 14.3).

Листинг 14.3. Исследование верхнего предела сегмента с `Е = 1` и `В = 0` (`bss2.asm`)

```

...
dsc4:                                ; Data (local), r/w                ; (36)
    dw    end - 1                    ; limit low                    ; (37)
    dw    0                          ; base low                  ; (38)
    db    0                          ; base middle               ; (39)
    db    1001_0110b                 ; P, dpl, S | data, E, RW, a ; (40)
    db    0000_0000b                 ; g, b, 00 | limit high (0)   ; (41)
    db    0                          ;                             ; (42)
...

    set_seg es, dsc4 - gdt            ; (71)
    mov     edi, end                  ; (72)
    cld                                ; (73)
11:                                ; (74)
    mov     ax, di                    ; (75)
    stosb                                ; (76)
    cmp     edi, 0ffffh               ; (77)
    jbe     11                        ; (78)
...

```

Цикл (74–78) завершился при `edi = 0x10000`, но записи (76) по этому смещению не было. Если в (77) увеличить предел (хотя бы на 1), то сбой произойдет на команде (76) при `edi = 0x10000`. Сообщение такое же: `write_virtual_checks(): write beyond limit, r/w expand down`.

При увеличенном пределе в (77) установите `В = 1`, задав `0100_0000b` в (41). В этом случае сбоя не случилось. Итак, верхняя граница сегмента при `Е = 1` и `В = 0` — это смещение `0xffff`, а при `Е = В = 1` верхней границы вообще нет, кроме естественно-го предела для 32-битового адреса⁴.

В листинге 14.4 приведен пример определения сегмента стека. Здесь `dsc4` — снова дескриптор данных программы, а для стека определен отдельный дескриптор `dsc6`. Сегмент стека занимает всю область неинициализированных данных из предыдущего опыта: от `0xffffe` вниз до метки `end`.

Листинг 14.4. Определение и проверка сегмента стека (err1.asm)

```

...
dsc4:                                ; Data (local), r/w                ; (37)
    dw      end - 1                  ; limit low                    ; (38)
    dw      0                        ; base low                  ; (39)
    db      0                        ; base middle               ; (40)
    db      1001_0010b               ; P, dpl, S | data, e, RW, a ; (41)
    db      0000_0000b               ; g, b, 00 | limit high (0)  ; (42)
    db      0                        ;                             ; (43)
...

dsc6:                                ; (51)
stk:                                  ; Stack, r/w                ; (52)
    dw      end - 1                  ; limit low                    ; (53)
    dw      0                        ; base low                    ; (54)
    db      0                        ; base middle                 ; (55)
    db      1001_0110b               ; P, dpl, S | data, E, RW, a ; (56)
    db      0000_0000b               ; g, b, 00 | limit high (0)  ; (57)
    db      0                        ;                             ; (58)
...

    set_seg ds, dsc4 - gdt           ; (79)
    mov     byte [end-1], '-'        ; (80)

    set_seg ss, stk - gdt            ; (81)
    mov     sp, end + 8              ; (82)

    push    eax                      ; (83)
    push    edx                      ; (84)
    push    ax                       ; (85)

12:    _brk                          ; (86)
    jmp     12                       ; (87)

    align   16                       ; (88)
    db      'Finita '                ; (89)
end:    db      'la Comedia'         ; (90)

```

При выполнении команды (85) произойдет запланированный сбой: `stackPrefetch(2): access [0x000001c5] > SS.limit [0x000001c6]`. Но команды (83–84) должны были затереть строку (90), а этого не произошло. Причина в том, что базовый адрес стека равен нулю. Это видно по значению селектора `ss` в консоли и обнаруживается командой **View/GDT** после выполнения (81).

Исправьте ошибку. В строке (90) после выполнения (83–84) должны уцелеть лишь две последние литеры.

Обнулите в (56) признак `RW` (бит 1). Добавьте между (82) и (83) команду `pop`. При выполнении какой команды произойдет сбой? Какое сообщение последует?

Верните `RW = 1`, прокомментируйте (82) и уберите команду `pop` — пусть стек будет таким же, как в `com`-программе. Убедитесь, что при выполнении (85) указатель стека увеличивается на 2. Вопрос: если вместо регистра в (85) задать число 1, то с какой разрядностью оно запишется в стек: 16 бит или 32 бита? Можно ли изменить разрядность команды (85) префиксами `a16/a32` и `o16/o32`?

Режим *unreal*

В листинге 14.5 показан пример переключения в режим `Unreal`⁵ на основе программы из листинга 13.5.

Листинг 14.5. Включение и проверка режима `unreal` (`unreal.asm`)

```

...
dsc2:                                     ; Data (flat), r/w                ; (15)
dw      -1                               ; limit low (max)                ; (16)
dw      0                                 ; base low                       ; (17)
db      0                                 ; base middle                    ; (18)
db      1001_0010b                       ; P, dpl, S | data, e, RW, a    ; (19)
db      1100_1111b                       ; G, B, 00 | limit high (max)    ; (20)
db      0                                 ;                                ; (21)
...

mov     ax, cs                           ; (27)
mov     [cmd+3], ax                       ; (28)
...

switch_mode                             ; (35)
jmp     8:pm                             ; (36)

pm:     _brk                             ; (38)
set_seg fs, dsc2 - gdt                   ; (39)
switch_mode                             ; (40)
cmd:    jmp     0:rm                      ; (41)

rm:     _brk                             ; (43)
mov     ebx, 0xb8000                     ; (44)
fs inc  byte [ebx]                       ; (45)

mov     ebx, 0x100000                     ; (46)
fs mov  eax, [ebx]                       ; (47)
fs mov  [ebx], edx                       ; (48)

mov     ebx, 0                           ; (49)
fs mov  eax, [ebx]                       ; (50)

_exit                                     ; (51)

```

Войдя (35–36) в защищенный режим (сегмент кода 16-битовый, чтобы проще было возвращаться), записываем (39) в селектор `fs` смещение дескриптора (15–21), где задан нулевой базовый адрес и максимальный предел. Возвращаемся (40–41) в реальный режим и проверяем: сначала доступ к видеопамяти (44–45) с указателем больше `0xffff`, а затем, с еще большим значением (46) указателя, доступ к памяти за пределами 1 Мбайт⁶. При чтении (47) по адресу `0x100000` получаем 0. (Действительно, если посмотреть память с адреса `0xfffff`, увидим, что после `0x100000` идут сплошные нули.) Записываем (48) что-нибудь по этому адресу и смотрим память с нулевого адреса. Если там не то же самое, что мы записали по адресу `0x100000`, значит, адресная линия A20 разблокирована⁷.

Адресная линия A20

В Bochs, как мы только что убедились, линия A20 открыта изначально. Но в PC скорее всего нет. Самый простой способ управления линией A20 показан в листинге 14.6, хотя ни один из способов не дает гарантии.

Листинг 14.6. Макросы для управления адресной линией A20 (`nasm.inc`)

```
...
%macro a20_on 0
    in      al, 092h
    test    al, 2
    jnz     %%m1
    or      al, 2
    out     092h, al

%%m1:
%endmacro

%macro a20_off 0
    in      al, 092h
    test    al, 2
    jz      %%m1
    and     al, 1111_1101b
    out     092h, al

%%m1:
%endmacro
...
```

Блокировка и разблокировка выполняются через порт `0x92`. Единица в первом бите означает, что линия A20 открыта. Макросы сделаны так, чтобы лишний раз в порт `0x92` ничего не писать. (Имена с двойным процентом — локальные, т. е. можно вызвать макрокоманду много раз, не опасаясь дублирования имени `m1`.) В листинге 14.7 мы открываем линию и выполняем тест памяти, затем закрываем ее и повторяем тест.

Листинг 14.7. Проверка макрокоманд управления линией A20 (a20.asm)

```

...
dsc2:                                ; Data (flat), r/w                ; (15)
    dw    -1                        ; limit low (max)              ; (16)
    dw    0                         ; base low                    ; (17)
    db    0                         ; base middle                ; (18)
    db    1001_0010b                ; P, dpl, S | data, e, RW, a  ; (19)
    db    1100_1111b                ; G, B, 00 | limit high (max)  ; (20)
    db    0                         ;                               ; (21)
    ...

    switch_mode                      ; (33)
    jmp     8:pm                     ; (34)

    use32                           ; (36)

write_compare:                       ; (37)
    mov     ebx, 0x100000            ; (38)
    mov     edx, 0x10                ; (39)
    cld                                  ; (40)
    mov     edi, ebx                 ; (41)
    mov     ecx, edx                 ; (42)
11:                                           ; (43)
    mov     eax, edi                 ; (44)
    stosb                                  ; (45)
    loop    11                      ; (46)

    clr     esi                      ; (47)
    mov     edi, ebx                 ; (48)
    mov     ecx, edx                 ; (49)
    repe    cmpsb                    ; (50)
    sete    al                      ; al = 1 if identical            ; (51)
    ret                                  ; (52)

pm:    _brk                          ; (53)
    set_seg ds, dsc2 - gdt          ; (54)
    set_seg es, dsc2 - gdt          ; (55)

    a20_on                          ; (56)
    call    write_compare            ; al = 0 ?                    ; (57)
    _brk                                  ; (58)

    a20_off                          ; (59)
    call    write_compare            ; al = 1 ?                    ; (60)
12:    _brk                          ; (61)
    jmp     12                      ; (62)

```

Переключившись (33–34) в защищенный режим, записываем (54–55) в селекторы `es` и `ds` дескриптор (15–21) сегмента данных, который охватывает все адресное пространство `i80x86`. Включаем (56) линию `A20` и вызываем (57) процедуру (37–52), которая записывает (38–46) с адреса `0x100000` несколько байтов с возрастающими значениями. Если линия `A20` заблокирована, то запись пойдет с нулевого физического адреса. В строках (47–50) мы сравниваем между собой пары байтов по адресам `0` и `0x100000`, `1` и `0x100001` и т. д. При совпадении всех пар команда (51) устанавливает `al = 1` (линия `A20` закрыта), а если нет, то `al = 0` (линия `A20` открыта).

Вообще-то код (47–51) лишний. Можно было бы проверить совпадение данных, просматривая их в отладчике `Bochs` после выполнения (38–46). Остановившись в точке (61), можно оценить разницу между режимами отображения памяти в `Bochs`. Нажав клавишу `<F7>`, мы видим то, что видит процессор: по адресам `0x100000` и `0` — одни и те же данные. А нажав комбинацию клавиш `<Ctrl>+<F7>`, мы видим уже физическую память, и по адресам `0x100000` и `0` — разные данные.

Привилегии сегментов данных

Теперь выполним опыт с привилегиями сегментов данных. Напомним, что в сегментах кода передачи управления ограничены текущим уровнем привилегий, т. е. уровни не могут напрямую вызывать друг друга⁸. С данными иначе: верхние уровни могут обращаться к данным своего и низшего уровня⁹. Мы с нашего уровня `CPL = 0` можем обратиться к сегменту данных с `DPL = 0–3`. Можно взять, например, программу предыдущего опыта и задать в дескрипторе данных `DPL = 2` (листинг 14.8).

Листинг 14.8. Обращение к данным низшего уровня привилегий (`dpl.asm`)

```

...
dsc2:                                ; Data (flat), r/w                ; (15)
    dw    -1                        ; limit low (max)                ; (16)
    dw    0                          ; base low                    ; (17)
    db    0                          ; base middle                 ; (18)
    db    1101_0010b                ; P, DPL=10, S | data, e, RW, a ; (19)
    db    1100_1111b                ; G, B, 00 | limit high (max)    ; (20)
    db    0                          ;                               ; (21)
...

pm:    _brk                          ; (53)
    set_seg fs, (dsc2 - gdt) + 0      ; (54)
    set_seg ds, (dsc2 - gdt) + 1      ; (55)
    set_seg es, (dsc2 - gdt) + 2      ; (56)
    set_seg es, (dsc2 - gdt) + 3      ; (57)
...

```

При записи в регистр-селектор (54–57) мы проверяем всевозможные значения `RPL` — уровня запрашиваемых привилегий. (Задавая `RPL > 0` при `CPL = 0`, мы, можно

сказать, скромничаем.) Команды (54–56) проходят, но на (57) последует сбой с сообщением: `load_seg_reg(ES, 0x0013): RPL & CPL must be <= DPL`. Действительно, $RPL = 3$, $CPL = 0$, $\max(RPL, CPL) = 3$, и это меньше, чем $DPL = 2$. Мы не имеем права обращаться к данным второго уровня от имени третьего уровня. По сравнению с кодом, где запрещено вызывать третий уровень со второго уровня, у сегментов данных иерархия обратная.

В файле `_dsc.asm` собраны все дескрипторы, с которыми мы познакомились: дескрипторы кода, данных, стека и вентиль вызова. Также добавлены вызовы макрокоманды `a20_on` и подпрограмм для вывода шестнадцатеричных чисел прямым отображением в видеопамять, чтобы можно было проверить программу не только в отладчике Bochs, но и на PC. Дальнейшие примеры — это развитие `_dsc.asm` с добавлением в GDT изучаемых дескрипторов.

В листинге 14.9 приведен фрагмент `_dsc.asm` с примером вызова процедур из файла `_output.asm`¹⁰, а сами процедуры показаны в листинге 14.10.

Листинг 14.9. Вывод дескрипторов (`_dsc.asm`)

```
%include "nasm.inc" ; (1)
%define _SHOW_ ; (2)

        org      0100h ; (3)
        jmp      start ; (4)

;----- subroutines ; (5)
        use32 ; (6)

#ifdef _SHOW_ ; (7)
#include "_output.asm" ; (8)
#endif ; (9)

subr:    mov      eax, 0x212850e ; (10)
        retf ; (11)

end2: ; (12)

;----- system data ; (13)
...

dsc5: ; (50)
flat: ; Data (flat), r/w ; (51)
        dw      -1 ; limit low (max) ; (52)
        dw      0 ; base low ; (53)
        db      0 ; base middle ; (54)
        db      1001_0010b ; P, dpl, S | data, e, RW, a ; (55)
        db      1100_1111b ; G, B, 00 | limit high (max) ; (56)
        db      0 ; (57)
...
```

```

start:                                     ; (70)
#ifdef _SHOW_                             ; (71)
    cls                                  ; (72)
#endif                                    ; (73)
    _brk                                ; (74)
    ...

pm:    _brk                              ; (91)
        set_seg ds, dsc4 - gdt           ; (92)
        set_seg es, flat - gdt          ; (93)
        set_seg ss, stk - gdt           ; (94)
        a20_on                           ; (95)

#ifdef _SHOW_                             ; (96)
    clr     edx                          ; (97)
    mov     esi, gdt                     ; (98)
    mov     ecx, (gdtr - gdt) / 4        ; (99)
    cld                                         ; (100)
.l1:                                           ; (101)
    lodsd                                ; eax - value ; (102)
    call    wr_hex                        ; edx - position 0-1, 8-9 ; (103)
    inc     edx                          ; (104)
    test    dl, 2                         ; (105)
    jz      .l2                          ; (106)
    add     dl, 6                         ; (107)
.l2:                                           ; (108)
    loop    .l1                          ; (109)
#endif                                    ; (110)

    _brk                                ; (111)
    call    (cg1 - gdt):0                 ; (112)
#ifdef _SHOW_                             ; (113)
    mov     edx, 199                      ; (114)
    call    wr_hex                        ; (115)
    mov     esi, msg                      ; (116)
    mov     edx, 108                      ; (117)
    call    wr_str                        ; (118)
#endif                                    ; (119)

p9:    _brk                              ; (120)
        jmp     p9                        ; (121)

msg     db      '*** GDT ***', 0          ; (122)
end:                                         ; (123)

```

Определение в строке (2) имени `_SHOW_` в дальнейшем используется как условие трансляции операторов (8), (72), (97–109) и (114–118):

- ❑ В область процедур (5–12) добавляется (8) код и данные подпрограмм вывода.
- ❑ При запуске очищается экран (72), заметим, что эту макрокоманду можно вызывать только в реальном режиме, т. к. она обращается к BIOS.
- ❑ После завершения инициализации (92–95) на экран выводятся текущие дескрипторы. Вызов (103) выводит на экран число, заданное в регистре `eax`, в позиции, заданной регистром `edx`. (Селектор `es` при этом должен быть настроен на дескриптор (50–57) с нулевым базовым адресом.) Экран поделен на 200 позиций: от 0 (левый верхний угол) до 199 (правый нижний угол). В каждой строке экрана — 8 позиций по 10 знако-мест. Команды (105–107) добавлены для того, чтобы `edx` принимал значения из ряда 0, 1, 8, 9, 16, 17 и т. д., чтобы 8-байтовые дескрипторы отображались по одному в строке.
- ❑ Команды (114–115) выводят в правый нижний угол экрана число — неважно какое, только для проверки максимальной позиции 199. Команды (116–118) демонстрируют вызов процедуры для вывода строки, адресуемой указателем `esi`, в позиции, заданной регистром `edx`.

Вывод на экран предусмотрен в большинстве последующих примеров, что позволяет запускать программы на PC. Если предстоит отладка в Bochs, можно закомментировать (2), но это мало что дает при достаточном количестве заранее поставленных точек останова (74), (91), (111) и (120).

Подключаемый файл, указанный в строке (1), вторично подключается директивой (8). В результате имена из (1) транслируются дважды, что вызывает ошибку повторного определения. Поэтому в начале файла `nasm.inc` добавлено условие трансляции всего исходного текста, истинное только при первой попытке¹¹. (В листинге 14.10 оператор в строке (1) — директива подключения `nasm.inc`.)

Листинг 14.10. Процедуры для вывода 32-битовых слов (`_output.asm`)

```
%include "nasm.inc"                                ; (1)

;                                                    ; (2)
;      Entry: eax - value                            ; (3)
;                                                    ; (4)
hex2str:                                             ; (5)
    pusha                                           ; (6)

    mov     ebx, hexs                               ; (7)
    mov     edi, _str                               ; (8)
    mov     edx, eax                                ; (9)
    call    four_digits                             ; (10)
    mov     byte [edi], '_'                          ; (11)
    inc     edi                                      ; (12)
    call    four_digits                             ; (13)
    mov     byte [edi], 0                            ; (14)
```

```

        popa                                ; (15)
        ret                                ; (16)

_str    resb    10                          ; (17)
hexs    db      '0123456789abcdef'          ; (18)

four_digits:                                ; (19)
        mov     ecx, 4                      ; (20)
.ll:    ; (21)
        rol     edx, 4                      ; (22)
        mov     al, dl                      ; (23)
        and     al, 0fh                     ; (24)
        xlat                                         ; (25)
        mov     [edi], al                   ; (26)
        inc     edi                         ; (27)
        loop    .ll                        ; (28)
        ret                                ; (29)

;                                           ; (30)
;      Entry:  edx - position (dl)          ; (31)
;              esi - string ptr            ; (32)
;              es  - flat segment           ; (33)
wr_str: ; (34)
cyan    equ    3                            ; (35)
white   equ    7                            ; (36)

        pusha                                ; (37)

        mov     al, 20                      ; (38)
        mul     dl                          ; (39)
        movzx   edi, ax                     ; (40)
        add     edi, laddr_video    ; !!!   ; (41)
        cld                                         ; (42)

        mov     ah, cyan                    ; (43)
        test    dl, 1                      ; (44)
        jz      .ll                        ; (45)
        mov     ah, white                   ; (46)
.ll:    ; (47)
        lodsb                                         ; (48)
        test    al, al                      ; (49)
        jz      .l2                        ; (50)
        stosw                                         ; (51)
        jmp     .ll                        ; (52)
.l2:    ; (53)
        popa                                ; (54)
        ret                                ; (55)

```



```

; ; (56)
; hex2str + wr_str ; (57)
; ; (58)
wr_hex: ; (59)
    call hex2str ; (60)
    push esi ; (61)
    mov esi, _str ; (62)
    call wr_str ; (63)
    pop esi ; (64)
    ret ; (65)

```

Процедура (5) преобразует число, заданное в `eax`, в строку (17) в шестнадцатеричном формате. Команды (6) и (37) сохраняют регистры общего назначения, а (15) и (54) — восстанавливают. Команда (7) устанавливает указатель `ebx` на строку (18), что используется в (25) для преобразования числа 0–15 в шестнадцатеричную цифру 0–f. (Входное значение и результат (25) — регистр `al`.) При вызовах (10) и (13) к строке добавляются четыре такие цифры. Хотя можно было поступить проще: поставить (20–28) на место (10–13) — но захотелось добавить подчеркивание (11–12) в середине. В (14) к строке добавляется завершающий ноль.

Процедура (34) выводит строку, адресуемую парой `ds:esi`, в позиции, заданной `edx` (точнее, регистром `di`). Смещение от начала видеобuffers вычисляется¹² в (38–39). Результат копируется (40) в `edi` и к нему прибавляется (41) линейный адрес видеобuffers, определенный в `asm.inc`. В (43–46) выбирается цвет для вывода¹³ — в зависимости от четности `edx`.

Процедура (59) объединяет вызовы предшествующих двух: преобразует число из `eax` в строку (17) и, задав (62) ее адрес в `esi`, выводит ее на экран в позиции `edx`. Процедуры (34) и (59) предназначены для пользователя, а (5) и (19) — служебные.

В листингах 14.9 и 14.10 появились метки с точкой. Эти метки в `asm` локальны: они существуют в промежутке между двумя обычными метками. Так, например, первый экземпляр метки (21) уместен лишь в пределах (19–34), а второй (47) — от (34) до конца исходного текста¹⁴.

Выполним еще один опыт над привилегиями. Мы проверили, что с `CPL = 0` можно обратиться к сегменту данных, имеющему низший статус, т. е. с `DPL > 0`. Проверим, допустимо ли такое обращение со стеком (листинг 14.11).

Листинг 14.11. Дескриптор стека и настройка селектора `ss` (`_dsc.asm`)

```

...
stk: ; Stack, r/w ; (59)
    dw end - 1 ; limit low ; (60)
    dw 0 ; base low ; (61)
    db 0 ; base middle ; (62)
    db 1001_0110b ; P, dpl, S | data, E, RW, a ; (63)
    db 0000_0000b ; g, b, 00 | limit high (0) ; (64)
    db 0 ; (65)
...

```

```
set_seg ss, stk - gdt                ; (94)
...
```

Задайте в поле `dpl` (63) двоичный код 11. При записи (94) дескриптора (59–65) в `ss` выяснится, допустима ли ситуация $CPL > DPL$, как при обращении к данным. После чего повторите опыт, вернув в (63) $DPL = 0$, но добавив в (94) $RPL > 0$.

Итак, если ситуация $CPL > DPL$ при обращении к данным допустима, то при работе со стеком правило $CPL = DPL = RPL$ соблюдается неукоснительно. Если в системе есть программы с разным уровнем привилегий, то у каждого уровня свой дескриптор стека и отдельная память под стек¹⁵.

Примечания

1. Именно так: мы защищаем не стек, а данные и код, расположенные под ним.
2. Изначально `sp = 0xffffe`, но мы его переносим вниз — на вершину `PSP`. Поэтому память от конца программы до смещения `0xffff` свободна.
3. Не забывайте: обновить отображение данных можно, только заново нажав `<F7>` и `<Enter>`.
4. Кроме того, при `B = 0` эффективный адрес берется из регистра `sp`, а при `B = 1` — из `esp`.
5. В [16] также упоминается режим `Big Real`. В этом режиме сегмент дополнительных данных начинается за пределами первого мегабайта, хотя его размер по-прежнему ограничен величиной 64 Кбайт.
6. Мы вынужденно используем косвенную адресацию, а не прямую — потому, что в 16-битовом режиме `nasm` считает недопустимым прямой адрес больше `0xffff`. Он выдает предупреждение и отсекает старшую часть адреса.
7. Сведения о линии `A20` в IBM PC и способах ее разблокировки можно найти, например, на сайте <https://en.wikipedia.org/>, задав поиск "A20 line", или в статье "A20 — a pain from the past".
8. Снизу вверх еще куда ни шло, для этого есть вентили вызова и конформные сегменты на верхних уровнях. А в обратную сторону — только с ухищрениями, но об этом позже.
9. Элита так и живет: пользуется трудом низших сословий, никак с ними не общаясь.
10. При трансляции `nasm` выведет предупреждение: `uninitialized space declared in .text section: zeroing [-wzeroing]`. Оно относится к директиве (17) в подключаемом файле `_output.asm` и сообщает, что зарезервированная этой директивой память будет заполнена нулями. Что нас устраивает, и можно отключить эти предупреждения, добавив в конец командной строки опцию `-w-zeroing`.
11. Точь-в-точь как в `h`-файлах языка C.

12. Множитель 20 — это ширина поля вывода (10 знако-мест), помноженная на число байтов в знако-месте (каждое требует двух байтов, где в младшем — код ASCII, а в старшем — код цвета).
13. Одноцветный вывод массива *чисел* плохо читается, но загромождать интерфейс процедуры, добавляя цветовой параметр, мне не хотелось. Пусть цвет выбирается *автоматически* по номеру позиции вывода.
14. Если метка с точкой поставлена до первой "глобальной" метки, то все ее экземпляры `asm` считает ошибкой. Объяснение простое: `asm` добавляет имя локальной метки к последней глобальной, например `four_digits.ll`; но если еще не к чему прибавлять (как в начале программы), идет в отказ.
15. При передаче управления между уровнями происходит переключение стека, но об этом позже.

ГЛАВА 15

```
DB6 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100  2: =d1,ax,, 1962
CX 0000  CS 6309  3:
DX 6309  SS 6309  4:
SI 011F  DS 6309  5: r5,b_1,, AB ab
DI 0000  ES 6309  6: r5,si,, AB ab
BP 0000  SP FFFE  0:

lea si,b_1
```

Исключения и прерывания

Совершенно откровенно

Тронул я ее колено.

Тут же получил по роже,

Честно и открыто тоже.

(Олег Григорьев)

...Неприятно было то, что среди всех этих розовых надежд Иван Ильич вдруг открыл в себе еще одну неожиданную способность: именно плевать. По крайней мере слюна вдруг начала выскакивать из его рта совершенно помимо его воли. Заметил он это на Акиме Петровиче, которому забрызгал щеку и который сидел, не смея сейчас же утереться из почтительности.

(Ф. М. Достоевский. Скверный анекдот)

В реальном режиме путь вызова процедур обработки исключений и прерываний проходит через глобальную таблицу векторов, которая начинается от линейного адреса 0 и содержит 256 элементов — составных адресов, содержащих 16-битовый номер параграфа в старшем слове и 16-битовое смещение в младшем слове.

Дескрипторы прерываний и исключений

В защищенном режиме путь вызова проходит через глобальную таблицу IDT (Interrupt Descriptor Table), в которой записаны вентили прерываний. В вентиле хранится дескриптор кода и эффективный адрес процедуры. Сама таблица IDT может находиться где угодно, но ее адрес и размер должны быть записаны в регистр IDTR, здесь совершенная аналогия с GDT.

Номера прерываний те же, что в реальном режиме, хотя в защищенном режиме число исключений при ошибках¹ больше [2, Table 8-1]. В реальном режиме номер прерывания — это порядковый номер вектора. В защищенном режиме номер прерывания — это порядковый номер вентилей в IDT. Как следствие, размер IDT определяется наибольшим номером обслуживаемого прерывания.

Типы дескрипторов при $s = 0$ приведены в [2, Table 4-5], а формат вентилей — в [2, Figure 4-18]. Тут есть знакомый вентиль вызова (32-bit Call Gate), а теперь нам понадобятся еще два типа: 32-bit Interrupt Gate (код 1110) и 32-bit Trap Gate

(код 1111). В чем разница? Только в том, что при вызове процедуры через Interrupt Gate автоматически сбрасывается флаг разрешения прерываний (i), а через Trap Gate — не сбрасывается².

В листинге 15.1 приведен пример обработки исключений с номерами ноль и один. В позициях экрана с этими же номерами отображаются счетчики исключений.

Листинг 15.1. Обработка исключений 0 и 1 (isr_2.asm)

```

..
; ----- isr ; (13)

i_cnt: resd 2 ; (14)

isr0: push edx ; (15)
      clr edx ; (16)
      jmp isr ; (17)

isr1: push edx ; (18)
      mov edx, 1 ; (19)
      jmp isr ; (20)

; to be continued... ; (21)

isr: _brk ; (22)
     push eax ; (23)
     lea eax, [i_cnt + (edx * 4)] ; (24)
     inc dword [eax] ; (25)

#ifdef _SHOW_ ; (26)
     mov eax, [eax] ; (27)
     call wr_hex ; (28)
#endif ; (29)
     pop eax ; (30)
     pop edx ; (31)
     iret ; (32)
     ...

;----- IDT ; (90)
idt: ; (91)
i0: ; (92)
     dw isr0 ; low adr ; (93)
     dw code - gdt ; selector ; (94)
     db 0 ; (rsrv) ; (95)
     db 1000_1111b ; P, dpl, 0 | Trap ; (96)
     dw 0 ; high adr ; (97)

```

```

il:                                     ; (98)
    dw    isr1                         ; low adr      ; (99)
    dw    code - gdt                   ; selector    ; (100)
    db    0                           ; (rsrv)       ; (101)
    db    1000_1111b                   ; P, dpl, 0 | Trap ; (102)
    dw    0                           ; high adr     ; (103)

idtr  dw    $ - idt - 1                ; (104)
      dd    idt                        ; (105)
      ...

      add    [gdtr+2], edx              ; (121)
      add    [idtr+2], edx              ; (122)

      lgdt   [gdtr]                    ; (123)
      lidt   [idtr]                    ; (124)
      ...

pm:    _brk                            ; (130)
      ...

      int    1                         ; (135)
      int    0                         ; (136)
      int    0                         ; (137)
      mov     ax, 0101h                 ; (138)
      div     al                        ; 0x101 / 1 -> overflow ; (139)

#ifdef _SHOW_                           ; (140)
      mov     esi, msg                  ; (141)
      mov     edx, 199                  ; (142)
      call    wr_str                    ; (143)
#endif                                  ; (144)

p9:    _brk                            ; (145)
      jmp     p9                        ; (146)

msg     db    '*** Fin ***', 0         ; (147)
end:                                          ; (148)

```

Счетчики исключений определены в (14). В строках (15–17) и (18–20) — код, выполняемый при входе в прерывание: сохранение `edx` в стеке, запись в `edx` номера прерывания и переход в точку сборки (22). Здесь, сохранив (23) `eax` в стеке, мы записываем (24) в `eax` адрес счетчика прерывания, номером которого задан в `edx`. Увеличиваем (25) счетчик, и если требуется (26) вывод на экран, то перед вызовом процедуры отображения (28) переписываем счетчик в `eax`. Позиция вывода `edx` равна номеру прерывания.

Вентили прерываний определены в IDT, начиная с (91). (Метки (92) и (98) можно было не ставить, они нигде не используются.) Каждый вентиль занимает восемь байтов, где заданы: эффективный адрес (93, 99) процедуры обработки прерывания, ссылка (94, 100) на дескриптор ее кода, а также признаки (96, 102) того, что сам вентиль присутствует в памяти и предназначен для обработки исключений (тип 1111). Структура данных (104–105) после коррекции (122) линейного адреса IDT записывается (124) в регистр IDTR.

Если в этот момент выполнить команду **View/IDT**, то вентили будут показаны неправильно, т. к. мы еще не перешли в 32-битовый режим. Выполните **View/IDT** в точке (130).

В строках (135–137) вызываем исключения напрямую. На экране в позициях 0 и 1 отобразятся счетчики исключений. Команда (139) тоже вызывает исключение 0, но при этом в стеке сохраняется адрес этой же команды. Поэтому мы вновь и вновь возвращаемся на (139), а счетчик в позиции 0 растет бесконечно. Закомментируйте (139) — и программа, пройдя (135–138), выведет строку (147) в последней позиции экрана. Так как ширина последней позиции ограничена десятью литерами, а в строке (147) их одиннадцать, то последняя литера '*' на экране не видна.

В этой программе хотелось бы расширить код (15–20) и данные (92–103) так, чтобы перехватывать все прерывания в диапазоне номеров от нуля до, предположим, 39. Первые 32 — это исключения, а последние 8 мы оставим на будущее для внешних прерываний. Это расширение несложно запрограммировать с помощью макрокоманд, как показано в листинге 15.2.

Листинг 15.2. Обработка 40 исключений (isr_n.asm)

```
%include "nasm.inc" ; (1)
%define _SHOW_ ; (2)
%define max_int 39 ; (3)
...

; ----- isr ; (14)

%macro set_isr 1 ; (15)
isr%1: _brk ; (16)
      push    edx ; (17)
      mov     edx, %1 ; (18)
      jmp     isr ; (19)
%endmacro ; (20)

%macro set_isr 1 ; (21)
      align 4 ; (22)
i_cnt: resd %1 ; (23)
%assign i 0 ; (24)
%rep %1 ; (25)
      set_isr i ; (26)
      %assign i i+1 ; (27)
```



```

%endrep                                     ; (28)
%endmacro                                   ; (29)

        set_isrs max_int + 1                ; (30)

isr:    push    eax                         ; (31)
        lea     eax, [i_cnt + (edx * 4)]    ; (32)
        inc     dword [eax]                 ; (33)
        ...

;----- IDT                                ; (98)

%macro  igate  1                            ; (99)
i%1:                                         ; (100)
        dw      isr%1                      ; low adr      ; (101)
        dw      code - gdt                 ; selector      ; (102)
        db      0                          ; (rsrv)        ; (103)
        db      1000_1111b                 ; P, dpl, 0 | Trap ; (104)
        dw      0                          ; high adr      ; (105)
%endmacro                                   ; (106)

%macro  igates 1                            ; (107)
%assign i 0                                ; (108)
%rep %1                                     ; (109)
        igate   i                          ; (110)
        %assign i i+1                      ; (111)
%endrep                                     ; (112)
%endmacro                                   ; (113)

idt:                                         ; (114)
        igates  max_int + 1                ; (115)

idtr    dw      $ - idt - 1                 ; (116)
        dd      idt                        ; (117)
        ...

pm:     _brk                                ; (142)
        ...
        int     0                          ; (147)
        int     0                          ; (148)
        int     max_int                    ; (149)
;        int     max_int + 1                ; error ! ; (150)
        ...

```

Максимальный номер прерывания задан в (3). Количество вентилей (115) и процедур (30) — на 1 больше. Макроопределение (99–106) определяет данные одного вентиля, номер прерывания задан в параметре вызова. Макроопределение (107) со-

держит вызовы макроса (99) с возрастающим параметром; число вызовов задано в (115). Макроопределения (15–29) для генерирования входного кода процедур обработки прерываний составлены аналогично; вызов — в (30).

В результате прямых вызовов (147–149) в позициях 0 и 39 выводятся счетчики — числа 2 и 1.

А теперь уберите комментарий в начале (150). Эта команда вызовет срабатывание общей защиты, т. к. номер вектора в (150) превышает число вентилях в IDT. (Это число равно значению (117), поделенному на восемь.) Сообщение в консоли Bochs следующее: `interrupt(): vector must be within IDT table limits, IDT.limit = 0x13f`. Начинается серия сообщений о срабатывании общей защиты³, но вскоре она заканчивается сообщением: `stackPrefetch(4): access [0x00000682] > SS.limit [0x00000684]`. Это значит, что стек закончился.

Коды ошибок для исключений

Причина в том, что исключение 13 дополнительно записывает в стек 32-битовое слово с информацией об ошибке. Не все исключения так поступают, иначе такая же ситуация возникла бы на команде (39) в листинге 15.1. Да и само исключение 13 в реальном режиме не должно так себя вести⁴, хотя это не проверено, поскольку в программе `gr_3.asm` (см. листинг 10.13) число повторных прерываний ограничено значением 256 и до исчерпания стека дело не доходит. Удалите в `gr_3.asm` команды (24–37), оставив только возврат из прерывания, и проверьте результат. Вызовы исключения 13 должны стать бесконечными.

Судя по [2, Table 8-6], в разных ситуациях исключение общей защиты записывает либо код ошибки в формате селектора [2, Figure 8-2], либо ноль. В любом случае это 32-битовое значение.

Проверим. Вернемся к программе в листинге 15.2. Поставьте комментарий в начале строки (2), а в (150) уберите, затем запустите программу в отладочном сеансе Bochs. Остановившись на (142), задайте `esp = 0xf010`. При выполнении (150) отладчик встанет сразу после (16) и `esp = 0xff00`. Изменение `esp` составляет $0x10 = 16$, т. е. было сделано четыре записи. Посмотрим, что было записано. Линейный базовый адрес сегмента сохранился в `edx`, и нужно прибавить к нему $0xf000$, что можно сделать в уме: $362a0 + 10000 - 1000 = 462a0 - 1000 = 452a0$. Открыв клавишей <F7> окно памяти с адреса `0x452a0`, видим (по возрастанию адресов):

- 00000142 — код ошибки согласно [2, Figure 8-9];
- 000005D4 — эффективный адрес возврата;
- 00000008 — значение селектора `cs`;
- 00010002 — флаги процессора.

Код ошибки записан последним, и чтобы удалить его из стека, достаточно увеличить `esp` на 4.

Выясним, что означает код ошибки `0x142` согласно [2, Figure 8-2]. В битах 3–15 записано смещение (относительно начала GDT, LDT или IDT) того дескриптора,

который привел к ошибке. В нашем случае смещение равно 0×140 . Бит 0 (EXT) сброшен — значит, источник исключения внутренний. Бит 1 (IDT) установлен⁵, поэтому речь идет об IDT. Разделив смещение 0×140 на 8 (размер вентилей в байтах), получим номер прерывания — $0 \times 28 = 40$. Действительно, это прерывание и было вызвано в (150).

Придумайте опыт, где источником исключения 13 была бы таблица GDT, и проверьте код ошибки. Затем спровоцируйте какое-нибудь другое fault-исключение, тоже с кодом ошибки. Сведения о кодах ошибок для всевозможных исключений приведены в [2, 8.2.1–8.2.24].

Теперь выполним опыт с привилегиями. Задайте в (104) код `1110_1111b` (`DPL = 3`). Ничего не случится, программа по-прежнему работает. Установив в вентилях `DPL = 3`, мы сделали их доступными для программ любого уровня привилегий. Однако в дескрипторе `dsc1`, на который ссылаются вентили, поле `DPL = 0`. А что если сослаться на такой же дескриптор, только с `DPL = 3`? Тогда получим сообщение: `interrupt(): not accessible or not code segment cs=0x0038`. Вывод: передать управление непривилегированной процедуре обработки прерывания нельзя.

Внешние прерывания

Перейдем к внешним прерываниям. Убедимся сначала, что счет в системном таймере идет (листинг 15.3). Если да, то должна быть и заявка на прерывание при каждом обнулении счетчика.

Листинг 15.3. Проверка системного таймера (tm1.asm)

```

...
pm:    _brk                                ; (130)
...

      mov     ecx, 0x2_0000                ; (135)
.11:                                     ; (136)
      in      al, 0x40                     ; timer low      ; (137)
      mov     ah, al                       ; (138)
      in      al, 0x40                     ; timer high      ; (139)
      xchg    al, ah                       ; (140)
#ifdef _SHOW_                             ; (141)
      movzx   eax, ax                      ; timer          ; (142)
      clr     edx                           ; (143)
      call    wr_hex                       ; (144)

      mov     eax, ecx                     ; loop counter    ; (145)
      inc     edx                           ; (146)
      call    wr_hex                       ; (147)
#endif                                     ; (148)
      loop    .11                          ; (149)
      ...

```

Команды (137–139) опрашивают по частям 16-битовый счетчик таймера, а команды (140, 142) формируют его значение в `eax`, которое выводится (144) в нулевой (143) позиции экрана. В соседней позиции выводится (145–147) счетчик цикла `ecx`. Видно, что значение в нулевой позиции меняется, поэтому можно приступить к обработке прерывания по входу `irq0`.

Сейчас входу `irq0` назначен номер прерывания 8, программа для реального режима (см. листинг 11.4) использует этот вектор. Но в защищенном режиме номера 0–31 заняты исключениями. (В частности, номер 8 отдан исключению Double Fault.) Поэтому контроллер прерываний придется перепрограммировать — так, чтобы его номера шли, например, от 32. Но мы сделаем это чуть позже, а для начала воспользуемся все-таки прерыванием 8, надеясь не вызвать *исключения* с этим номером.

Обработка прерываний от таймера включена в программу из листинга 15.2, результат показан в листинге 15.4.

Листинг 15.4. Прерывание от системного таймера (tm2.asm)

```

...

isr:  push    eax                      ; (31)
      lea     eax, [i_cnt + (edx * 4)] ; (32)
      inc     dword [eax]              ; (33)
      mov     eax, [eax]               ; (34)

#ifdef _SHOW_                          ; (35)
      call    wr_hex                   ; (36)
#endif                                ; (37)
      and     dl, 1111_1000b           ; (38)
      cmp     dl, 1000b                ; [8..15] ; (39)
      jnz     .11                     ; (40)
      eoi                      ; !      ; (41)
.11:                                     ; (42)
      pop     eax                     ; (43)
      pop     edx                     ; (44)
      iret                      ; (45)
...

;----- IDT                          ; (103)
%macro  igate  1                      ; (104)
i%1:                                     ; (105)
      dw      isr%1                   ; low adr ; (106)
      dw      code - gdt              ; selector ; (107)
      db      0                       ; (rsrv) ; (108)
      %if %1 >= 8 && %1 <= 15         ; (109)
      db      1000_1110b              ; P, dpl, 0 | Intr ; (110)

```

```

%else ; (111)
        db      1000_1111b      ; P, dpl, 0 | Trap ; (112)
%endif ; (113)
        dw      0                ; high adr      ; (114)
%endmacro ; (115)
...

pm:      _brk ; (151)
        ...
        mov     ebx, i_cnt + 8*4 ; (156)
        sti     ; (157)
.11:     mov     eax, [ebx] ; (158)
        cmp     al, 91      ; 5 sec ; (159)
        jb      .11 ; (160)

#ifdef _SHOW_ ; (161)
        mov     esi, msg ; (162)
        mov     edx, 42 ; (163)
        call    wr_str ; (164)
#endif ; (165)
...

```

Для прерываний с номерами 8–15, соответствующим входам irq0–irq7, сделаны два дополнения:

- ❑ Изменение типа вентиля на Interrupt Gate, для чего в макроопределении (104–115) добавлено ветвление (109–113). Теперь при входе в процедуру обработки будет сбрасываться флаг разрешения внешних прерываний (i).
- ❑ Вызов в строке (41) макрокоманды для сброса контроллера прерываний, но только для внешних прерываний, а не для исключений — для чего добавлена проверка (38–40).

Поскольку таймер продолжает счет, а вход irq0 в контроллере прерываний открыт⁶, достаточно разрешить прерывания (157) на входе процессора. Мы намерены задержаться в цикле (158–160), пока счетчик прерываний от таймера не дойдет до 91, что при частоте 18,2 Гц соответствует пятисекундной задержке⁷.

При выходе из цикла выводится сообщение о завершении, но прерывания продолжают и счетчик увеличивается. Более того, рядом с ним может появиться еще один — с единичным значением. Это прерывание от клавиатуры, возникшее при *отпуске* клавиши <Enter> при запуске программы. (Если счетчик не появился, нажмите любую клавишу.) Дальнейшие прерывания от клавиатуры заблокированы, т. к. мы не послали сигнал подтверждения в ее контроллер (см. листинг 11.6).

Прихорашивать программу в листинге 15.4 уже нет смысла — пора уже перестроить контроллер прерываний, чтобы заявки irq0...irq7 соответствовали номерам прерываний 32...39, за пределами диапазона номеров fault-исключений (листинг 15.5).

Листинг 15.5. Обработка прерываний с перенастройкой контроллера (tm3.asm)

```

...
%define trap_no pic_vect                ; (3)
%define intr_no 8                      ; (4)
...

; ----- isr                          ; (15)
        align    4                    ; (16)
i_cnt:  resd     trap_no + intr_no    ; (17)

%macro  set_trap 1                     ; (18)
trap%1: _brk                          ; (19)
        push     edx                  ; (20)
        mov      edx, %1              ; (21)
        jmp      exit                 ; (22)
%endmacro                             ; (23)

%macro  set_traps 0                    ; (24)
%assign i 0                           ; (25)
%rep trap_no                          ; (26)
        set_trap i                    ; (27)
        %assign i i+1                 ; (28)
%endrep                               ; (29)
%endmacro                             ; (30)

%macro  set_intr 1                     ; (31)
intr%1: _brk                          ; (32)
        push     edx                  ; (33)
        mov      edx, %1 + trap_no    ; (34)
        jmp      irq%1                ; (35)
%endmacro                             ; (36)

%macro  set_intrs 0                    ; (37)
%assign i 0                           ; (38)
%rep intr_no                          ; (39)
        set_intr i                    ; (40)
        %assign i i+1                 ; (41)
%endrep                               ; (42)
%endmacro                             ; (43)

        set_traps                     ; (44)
        set_intrs                      ; (45)

irq0:                                     ; (46)
irq2:                                     ; (47)
irq3:                                     ; (48)

```

```

irq4:                                     ; (49)
irq5:                                     ; (50)
irq6:                                     ; (51)
irq7:                                     ; (52)
        jmp      irq                     ; (53)

irq1:                                     ; (54)
        push     eax                     ; (55)
        in       al, 060h                 ; (56)
        mov      ah, al                   ; scan code ; (57)
        in       al, 061h                 ; state   ; (58)
        or       al, 1                     ; (59)
        out      061h, al                 ; ack      ; (60)
#ifdef _SHOW_                             ; (61)
        movzx    eax, ah                   ; (62)
        add      edx, 40                   ; (63)
        call     wr_hex                     ; (64)
        sub      edx, 40                   ; (65)
#endif                                     ; (66)
        pop      eax                     ; (67)
        jmp      irq                     ; (68)

irq:    eoi                               ; (69)
exit:   push     eax                     ; (70)
        lea      eax, [i_cnt + (edx * 4)] ; (71)
        inc      dword [eax]              ; (72)
        mov      eax, [eax]               ; (73)

#ifdef _SHOW_                             ; (74)
        call     wr_hex                     ; (75)
#endif                                     ; (76)
        pop      eax                     ; (77)
        pop      edx                     ; (78)
        iret                               ; (79)
        ...

;----- IDT                             ; (137)

%macro  trap_gate 1                       ; (138)
        dw      trap%1                    ; low adr ; (139)
        dw      code - gdt                ; selector ; (140)
        db      0                          ; (rsrv)   ; (141)
        db      1000_1111b                 ; P, dpl, 0 | Trap ; (142)
        dw      0                          ; high adr  ; (143)
%endmacro                                  ; (144)

%macro  trap_gates 0                      ; (145)
%assign i 0                               ; (146)

```

```

%rep trap_no                                     ; (147)
    trap_gate i                                 ; (148)
    %assign i i+1                               ; (149)
%endrep                                         ; (150)
%endmacro                                       ; (151)

%macro intr_gate 1                               ; (152)
    dw      intr%1                             ; low adr      ; (153)
    dw      code - gdt                         ; selector        ; (154)
    db      0                                 ; (rsrv)         ; (155)
    db      1000_1110b                        ; P, dpl, 0 | Intr ; (156)
    dw      0                                 ; high adr       ; (157)
%endmacro                                       ; (158)

%macro intr_gates 0                             ; (159)
%assign i 0                                     ; (160)
%rep intr_no                                    ; (161)
    intr_gate i                                 ; (162)
    %assign i i+1                               ; (163)
%endrep                                         ; (164)
%endmacro                                       ; (165)

idt:                                           ; (166)
    trap_gates                                 ; (167)
    intr_gates                                 ; (168)

idtr    dw      $ - idt - 1                    ; (169)
        dd      idt                            ; (170)
        ...

pm:      _brk                                  ; (195)
        ...

        int      0                             ; (200)
        int      0                             ; (201)
        int      trap_no - 1                   ; (202)

        set_pic1                               ; (203)
        mask                                           ; (204)
        unmask   0                                 ; irq0          ; (205)
        unmask   1                                 ; irq1          ; (206)

        mov      ebx, i_cnt + pic_vect*4        ; (207)
        sti                                           ; (208)
.l1:     mov      eax, [ebx]                       ; (209)
        cmp      al, 182                          ; 10 sec       ; (210)
        jb       .l1                              ; (211)
        ...

```


В IDT (166–168) теперь 32 вентиля типа Trap Gate (142) и восемь — типа Interrupt Gate (156). Они различаются не только типом, но и точками входа (139) и (153) в процедуру обработки. Точки входа для исключений определены в (19) и создаются при вызовах (27), а для внешних прерываний — в (153) и (162) соответственно. Счетчики исключений и внешних прерываний определены в (17).

Обработка исключений остается такой же, как в листинге 15.2, только имена изменились. Все исключения начинаются с команд (19–22) и сходятся к (70). Путь обработки прерываний чуть сложнее: все они начинаются с команд (32–34), но тут же расходятся по меткам `irq0–irq7` (46–52) и (54). По этим адресам находится код для обслуживания конкретного *устройства*. Таймер (`irq0`) не требует обслуживания, но с клавиатурой иначе: мы читаем (56–57) скан-код нажатой или отпущенной клавиши, выдаем подтверждение (58–60) в контроллер клавиатуры, а при условии (61) еще и выводим (62–65) скан-код в позиции 41 экрана. Внешние прерывания сходятся в точке (69), где выполняется сброс контроллера прерываний, а дальше — выход, общий для прерываний и исключений (70).

Теперь проверим утверждение, что при входе в процедуру обработки прерывания через Interrupt Gate флаг разрешения внешних прерываний (`i`) сбрасывается, а через Trap Gate — нет. Для этого будем выводить не счетчики прерываний, а регистр состояния, как показано в листинге 15.6.

Листинг 15.6. Проверка разницы между Interrupt и Trap Gate (`eflags.asm`)

```

...
align    4                                ; (16)
eflags dd    0                            ; (17)
...

irq:  eoi                                ; (69)
exit: push    eax                          ; (70)

      pushf                                ; (71)
      pop     eax                          ; (72)
      mov     [eflags], eax                ; (73)

#ifdef _SHOW_                                ; (74)
      call    wr_hex                       ; (75)
#endif                                       ; (76)
      pop     eax                          ; (77)
      pop     edx                          ; (78)
      iret                                   ; (79)
...

pm:   _brk                                ; (195)
      ...
      set_pic1                             ; (200)
      mask                                  ; (201)
```

```

unmask 0                ; irq0                ; (202)
unmask 1                ; irq1                ; (203)

sti                                ; (204)
int     0                    ; (205)
int     trap_no - 1          ; (206)

#ifdef _SHOW_                    ; (207)
mov     esi, msg              ; (208)
mov     edx, 182              ; (209)
call    wr_str                ; (210)
#endif                          ; (211)

...

```

Состав регистра `rflags` показан в [2, Figure 3-7]; `eflags` — его младшая 32-битовая половина. Флаг `i` — это бит 9, и ему соответствует число `0x200`. (Заодно гляньте, где находится поле `IOPL`, оно нам понадобится в следующем опыте.)

Вместо массива счетчиков у нас теперь одна переменная (17) для копии регистра флагов, чтение которого выполняют (71–72) в общей части процедур обработки исключений и прерываний. (Если включен вывод на экран, то переменная (17) лишняя.) В главной программе убран цикл опроса счетчика прерываний таймера. Флаг `i` устанавливается (204) до вызова исключений (205–206). В позициях экрана, соответствующих исключениям, выводится число большее или равное `0x200`. Это означает, что флаг `i` при входе в исключение не сбрасывается. А в позициях внешних прерываний число меньше `0x200` — флаг `i` обнулен.

Поле *IOPL* в регистре флагов

И в завершение выясним, что означает поле `IOPL` (I/O Privilege Level). Оно касается портов ввода-вывода, через которые мы обращаемся (командами `in/out`) к регистрам контроллера прерываний и контроллера клавиатуры. Согласно документации `IOPL` задает уровень привилегий, достаточный для обращения к портам. Судя по выводу программы из листинга 15.6, сейчас `IOPL` = 0. Так как наш уровень привилегий `CPL` = 0, то, как ни настраивай `IOPL`, его изменений мы не почувствуем.

В листинге 15.7 показана макрокоманда для установки поля `IOPL`. Добавьте в `eflags.asm` вызов этой макрокоманды с параметром 3 — между операторами (203) и (204), и убедитесь, что программа работает как прежде.

Листинг 15.7. Макрокоманда для установки поля `IOPL` в регистре флагов (`nasm.inc`)

```

%macro set_iopl 1
    pushf
    pop     eax
    and     eax, ~(3 << 12)
    or      eax, (%1 & 3) << 12

```

```
        push    eax
        popf
%endmacro
```

Примечания

1. Так называемые fault exceptions.
2. Отсюда следует, что мы можем применять Interrupt Gate при работе с исключениями, но не наоборот: при использовании Trap Gate для обработки внешних прерываний есть опасность рекурсивного вызова процедуры обработки, особенно если сброс контроллера прерываний запрограммирован в начале процедуры. Вход в прерывание от типа вентиля никак не зависит, т. е. исключения 0 и 13 в любом случае запишут в стек адрес инструкции, вызвавшей сбой, а не адрес следующей инструкции.
3. Вывод счетчика исключений можно заметить в отладочном сеансе между точками останова.
4. Взгляните на файл exception.cc в папке CPU дерева проекта Bochs. Четвертый оператор снизу говорит о том, что в *реальном* режиме ошибка в стек не записывается.
5. Когда бит 1 сброшен, речь или о GDT (если бит 2 тоже сброшен), или об LDT (если бит 2 равен единице).
6. Все это наследие FreeDOS, откуда мы запускаемся.
7. Может оказаться, что в Bochs таймер работает быстрее и пауза будет короче.

ГЛАВА 16

LDT и TSS

```
DB6 BUFFER:
0108 MOV SI,011F
010E INT 3

AX 07A0 IP C 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:
```

```
lea si,b_1
```

Швейк побывал в нескольких лавочках, но стоило ему произнести: "Будьте любезны, бутылочку елей, освященного епископом", — всюду или фыркали ему в лицо, или в ужасе прятались под прилавком... Он решил попытать счастья в аптеках. Из первой велели его вывести. В другой хотели вызвать по телефону карету "скорой помощи", а в третьей провизор ему сказал, что на Длугой улице — торговля маслами и лаками — там на складе наверняка найдется нужный елей.

(Ярослав Гашек. Похождения бравого солдата Швейка)

"Лавра" привели к другому берегу, и старпом снова проревел в трубу:

— Кто вы?

— Мы — посланные на... — дружно ответствовали островитяне.

— Приходится констатировать, — пожал плечами Суер, — что это действительно люди, посланные на...

— А за что вас послали? — крикнул старпом.

— А по разным причинам, — дружелюбно поясняли островитяне, — а больше без причин.

(Юрий Коваль. Суер)

Дескрипторы LDT (Local Descriptor Table) и TSS (Task State Segment) — существенные усложнения программной модели i80x86, которые не нашли практического применения. Одной из причин отказа от аппаратной многозадачности, предоставляемой TSS, называют низкую скорость, почему-то не приводя данных измерений... А мы померяем. Но начнем не с TSS, а с LDT.

Дескрипторы LDT

Дескриптор LDT находится в GDT и определяет базовый адрес, размер и атрибуты сегмента, который хранит еще одну таблицу дескрипторов — так называемую локальную. Системная программа, готовя к запуску новую задачу, выделяет ей память под ее *собственную* таблицу дескрипторов и формирует в GDT один дескриптор для ссылки на эту таблицу. В результате образуются два уровня дескрипторных таблиц, а GDT не загромождается дескрипторами прикладных задач — ведь у каждой задачи как минимум три дескриптора.

Кстати, в TSS есть поле, ссылающееся на LDT, и при переключении задач автоматически переключается и LDT. Что не обязательно — LDT и TSS могут обойтись друг без друга¹.

В листинге 16.1 приведен простейший пример, где в LDT задан только один дескриптор².

Листинг 16.1. Простой пример LDT с одним дескриптором (ldt.asm)

```

...
gdt:                                     ; (84)
dsc0:                                     ; Null                               ; (85)
      dd      0, 0                       ; (86)
...
dsc7:                                     ; !                               ; (134)
ltl1:                                     ; Data, r/w                         ; (135)
      dw      ldt_sz                     ; limit low                         ; (136)
      dw      0                         ; base low                          ; (137)
      db      0                         ; base middle                       ; (138)
      db      1000_0010b                 ; P, dpl=00, s | ldt               ; (139)
      db      0                         ; flags + limit high                ; (140)
      db      0                         ; (141)

gdtr:                                     ; (142)
      dw      $ - gdt - 1                ; size(gdt)                        ; (143)
      dd      gdt                       ; + laddr(cs)                      ; (144)

;----- LDT                             ; (145)
ldt:                                     ; !                               ; (146)
l_dsc0:                                 ; Data (video), r/w                ; (147)
      dw      160                       ; limit low                         ; (148)
      dw      0x8000                    ; base low (0xb8000)               ; (149)
      db      0xb                      ; base middle                       ; (150)
      db      1001_0010b                 ; P, dpl=00, S | Data, e, RW, a    ; (151)
      db      0100_0000b                 ; g, 32 bit, 00 | limit high (0)   ; (152)
      db      0                         ; (153)

ldt_sz equ      $ - ldt                 ; (154)
...
;-----                                ; (189)
start:                                  ; (190)
...
      mov     eax, edx                  ; (204)
      add     eax, ldt                  ; (205)
      wr_base ltl1                      ; !                               ; (206)

      add     [gdtr+2], edx             ; (207)
      add     [ldtr+2], edx             ; (208)

      lgdt    [gdtr]                  ; (209)
      lidt    [ldtr]                  ; (210)
...

```

```

pm:      _brk                                     ; (216)
          set_seg ds, dsc4 - gdt                  ; (217)
          set_seg ss, stk - gdt                   ; (218)

          mov     ax, lt1 - gdt                    ; !      ; (219)
          lldt    ax                               ; (220)
          mov     ax, (l_dsc0 - ldt) + use_ldt     ; (221)
          mov     es, ax                           ; (222)

          clr     edi                               ; (223)
          mov     ecx, 80                          ; (224)
          mov     al, '.'                          ; (225)
          mov     ah, 7                            ; (226)
          cld                                         ; (227)
          rep     stosw                             ; (228)

          set_seg es, flat - gdt                   ; (229)
          a20_on                                     ; (230)
          ...

p9:      _brk                                     ; (248)
          call    (cg1 - gdt):0                    ; (249)
          jmp     p9                                ; (250)
          ...

```

В GDT добавлен (134–141) нестандартный дескриптор ($s = 0$), описывающий расположение, размер и атрибуты LDT. Поля в строках (137–138) и (141) формируются командами (204–206) как сумма линейного базового адреса и смещения LDT. Сама таблица LDT определена в (146–153) и содержит всего лишь один дескриптор данных, который адресует вторую строку видеопамати³. В отличие от регистров GDTR и IDTR, представляющих собой структуру (142–144), содержащую размер таблицы и ее линейный адрес, в регистр LDTR пишется (219–222) всего лишь 16-битовое *смещение* от начала GDT. Чтобы воспользоваться дескриптором (147), в селектор записываем (221–222) смещение дескриптора от начала LDT вместе с установленным битом 2, означающим⁴ выбор дескриптора из LDT.

Остановившись на строке (216), выполните команду **View/GDT** — там должен появиться дескриптор типа LDT, заданный (134–141) и ссылающийся на (147). В строках (217–218) устанавливаем селекторы данных и стека, а затем инициализируем LDT и записываем в селектор `es` нулевой дескриптор из LDT — в случае с LDT это не ошибка. В строках (223–228), пользуясь этим селектором, выводим точку во второй строке экрана и перенастраиваем (229) селектор на дескриптор из GDT.

Обратите внимание, что команды (209–210) выполняются в реальном режиме, а (220) — в защищенном, поскольку команда (220) в реальном режиме не поддерживается. Зато в регистр LDTR можно записать ноль, как это делают при запуске Linux⁵ и Fysos [12]. Этот ноль означает, что ссылка на LDT недопустима, поскольку путь к ней лежит через нулевой дескриптор GDT (листинг 16.2).

Листинг 16.2. Запрет использования LDT (nasm.inc)

```
%macro   ldt_off 0
        clr     ax
        lldt    ax
%endmacro
```

Выполните следующие два опыта, выполняя макрокоманду из листинга 16.2 в пошаговом режиме.

- ❑ Закомментируйте команды в строках (219–220) и поставьте под ними вызов макрокоманды из листинга 16.2. Последует сообщение `fetch_raw_descriptor: LDTR.valid=0` и исключение 13.
- ❑ Вернувшись к исходному тексту `ldt.asm`, вставьте вызов макроса между строками (209) и (210), т. е. в код реального режима. Последует сообщение: `LLDT: not recognized in real or virtual-8086 mode.`

Доступ к LDT — дважды косвенный: используя регистр `LDTR` как смещение в таблице GDT (адресуемой регистром `GDTR`), мы обращаемся к сегменту LDT, селектором выбирая из GDT искомый дескриптор. Но по этому пути процессор проходит не при каждом обращении, а только один раз при записи в регистр-селектор числа с установленным вторым битом. В дальнейшем используется копия дескриптора, сохраненная в кеш-памяти — в так называемом теновом регистре.

Попробуйте самостоятельно исследовать пример из листинга 16.1: удалить строки (148–153), перенести из GDT в LDT дескрипторы `dsc4–dsc6`, удалить (223–228), а (217–222) и (229) скорректировать так, чтобы селекторы были настроены на дескрипторы из LDT.

Поскольку LDT связана с конкретной задачей, она не может содержать глобальные объекты, каковыми являются вентили прерываний и исключений. Но вентиль вызова она может содержать⁶. Для проверки перенесите дескриптор `cg1` из GDT в LDT и скорректируйте вызов (249).

На этом все с LDT. Прежде чем перейти к сегментам задач TSS, на которых базируется аппаратная многозадачность, рассмотрим пример программной реализации многозадачности.

Программное переключение контекста

Многозадачность в однопроцессорной системе означает передачу управления между задачами, или сопрограммами, так что задача продолжается из того состояния, в котором она находилась в момент приостановки. Передача управления происходит либо по инициативе программы, либо от внешнего прерывания, например от таймера.

Состояние, или контекст задачи, применительно к `i80x86` включает в себя все регистры CPU, а также состояние и данные FPU и других математических расширений, если они применяются.

Разберем самый простой случай, когда две сопрограммы находятся в пределах сопрограммы, не используют арифметических расширений и выполняются в реальном режиме. Контекст определяют 32-битовые регистры общего назначения, 16-битовые регистры `flags`, `sp`, `ip` и основные сегментные регистры.

В листинге 16.3 приведен простейший пример переключения — для двух сопрограмм.

Листинг 16.3. Программное переключение задач (sw.asm)

```

...
%define _SHOW_                                ; (2)
...
n_switch equ 1000000                          ; (4)

struc ts                                       ; (5)
_edi resd 1 ; pushad / popad                  ; (6)
...
_eax resd 1 ; (13)
_es resw 1 ; push es / pop es                  ; (14)
_ip resw 1 ; int / ired                        ; (15)
_cs resw 1 ; (16)
_flags resw 1 ; (17)
endstruc ; (18)

n_vect equ 4 ; (19)
sp2 equ 0100h - ts_size ; <sp> for task2      ; (20)
;----- ; (21)
        jmp start ; (22)

next_sp dw sp2 ; <sp> of suspended task      ; (23)

trap:  push es ; (24)
       pushad ; (25)
       _brk ; (26)

       mov ax, sp ; swap stacks ; (27)
       cs xchg [next_sp], ax ; (28)
       mov sp, ax ; (29)

       popad ; (30)
       pop es ; (31)
       ired ; (32)

start: ; (33)
       clr_psp ; (34)
       mov bp, sp2 ; (35)
       mov dword [_esp+bp], -1 ; (36)

```

```

mov     ax, cs                                ; (37)
mov     word [_cs+bp], ax                     ; (38)
mov     word [_es+bp], 0b800h                 ; (39)
mov     word [_ip+bp], thread                 ; (40)

pushf                                         ; (41)
pop     word [_flags+bp]                     ; (42)

mov     dword [_eax+bp], n_switch             ; (43)
mov     dword [_ebx+bp], 0x2222_2222         ; (44)
...
mov     dword [_ebp+bp], 0x7777_7777         ; (49)

set_seg ds, 0                                ; set vect ; (50)
mov     ebx, n_vect * 4                      ; (51)
mov     word [bx], trap                      ; (52)
mov     ax, cs                               ; (53)
mov     word [bx+2], ax                      ; (54)

set_seg ds, cs                               ; ! ; (55)
mov     sp, 0x80                             ; ! ; (56)
set_seg es, 0xb820                           ; (57)

clr     eax                                  ; ! ; (58)
mov     ebx, 0x0202_0202                     ; (59)
...
mov     ebp, 0x0707_0707                     ; (64)

thread:                                       ; (65)
.ll1:                                         ; (66)
        int     n_vect                       ; (67)
#ifdef _SHOW_                                ; (68)
        and     edi, 0x3f                    ; (69)
        stosw                                     ; (70)
#endif                                       ; (71)
        dec     eax                           ; (72)
        jnz     .ll1                         ; (73)

        flood   0                             ; (74)
.ll2:                                         ; (75)
        _brk                                     ; (76)
        jmp     .ll2                         ; (77)

```

Объявление структуры (5–18) не резервирует память, а лишь создает имена (6–17) со значениями от нуля по возрастающей⁷. Также `asm` порождает имя `size_ts`, равное размеру структуры, оно затем используется в (20).

Вначале обнуляем (34) область PSP. В ней мы разместим два стека для сопрограмм: один по со смещению 0x100, второй по смещению 0x80, по 128 байт каждый. (Команды (35–49) обсудим позже.) В строках (50–54) устанавливаем вектор прерывания 4 на процедуру (24), а указатель стека главной⁸ задачи переносим (56) на верхушку первого стека — по смещению 0x80. Командами (58–64) записываем в регистры общего назначения всякую всячину — только для удобства наблюдения в отладчике. В цикле (66–73) выполняем переключение (67) на сопрограмму — до обнуления вычитаемого регистра (72–73).

Что произойдет при вызове (67)? В стек запишется содержимое 16-битовых регистров flags, cs и ip, а в результате выполнения (24–25) — также es и содержимое 32-битовых регистров общего назначения. Стек главной задачи при первой остановке в точке (27) показан на рис. 16.1. Проверьте, запустив sw.com в отладочном сеансе Bochs.

....				0606	0606	0505	0505
				<---edi--->		<---esi--->	
0707	0707	0078	0000	0202	0202	0404	0404
<---ebp--->		<---esp--->		<---ebx--->		<---edx--->	
0303	0303	0000	0000	B820	01BF	3633	0246
<---ecx--->		<---eax--->		<es>	<ip>	<cs>	<flags>

Рис. 16.1. Содержимое стека главной задачи

Чтобы переключиться на сопрограмму, нужно сохранить текущее значение sp и записать (27–29) в него указатель вершины стека следующей задачи (23). Содержимое стека побочной задачи после инициализации (35–49) показано на рис. 16.2.

....				6666	6666	5555	5555
				<---edi--->		<---esi--->	
7777	7777	FFFF	FFFF	2222	2222	4444	4444
<---ebp--->		<---esp--->		<---ebx--->		<---edx--->	
3333	3333	4240	000F	B800	01BD	3633	0246
<---ecx--->		<---eax--->		<es>	<ip>	<cs>	<flags>

Рис. 16.2. Содержимое стека вторичной задачи

Фокус в том, что вторичная задача еще не начала выполняться, а мы уже сформировали в ее стеке данные и подготовили в (23) такое значение sp, как если бы эта задача уже была выполнена (67) и (24–25). После изменения (27–29) указателя стека команды (30–32) обращаются к стеку вторичной задачи. В поле _ip там задана (40) точка (65). И вторичная сопрограмма

попадет в тот же цикл (66–73), где выполняется главная программа! Разница только в значениях регистра `eax`: для главной программы это 0, заданный в (58), а для сопрограммы — константа (4), записанная в `eax` командой (43).

Значения регистров в отладчике — это и есть текущий контекст. Сейчас это контекст главной задачи. Нажав кнопку **Continue**, вы переключитесь на вторичную задачу и вновь остановитесь на (27). Значения регистров должны измениться, и это уже контекст вторичной задачи.

На рис. 16.2 бросается в глаза немыслимое значение, приготовленное для указателя стека, — `0xffff_ffff`. Оказывается, что при выполнении (30) это значение вообще не используется — так устроена эта команда. Хотя парная ей команда (25) сохраняет все восемь регистров общего назначения, включая указатель стека⁹. (Вы убедитесь в этом через пару остановок: во вторичном стеке на месте `esp` будет `0x100 - 40 = 0xd8`.) Команда (30) сохраненное значение `esp` игнорирует, а стек переключается только благодаря изменению переменной (23) в точке (28).

При каждой остановке на (27) видно, что число в `eax` уменьшилось на единицу.

Запустите программу в рабочем режиме. Сопрограммы будут переключаться, пока одна из них не досчитает до нуля — и тогда экран заполнится (74) звездочками¹⁰. Первой к финишу придет вторичная сопрограмма, т. к. главная вынуждена считать от 0 до 0.

Обратите внимание, что значения `es` у сопрограмм немного разные. Поэтому вывод на экран, выполняемый командами (69–70), будет направлен в разные его области¹¹.

Закомментируйте строку (2) и запустите полученную программу `sw.com` на PC. Измерьте время выполнения. Если оно слишком мало, увеличьте счетчик (4). Это же значение надо будет подставить в программу `hw3.asm` в листинге 16.7, которая тоже выполняет переключение между задачами, но аппаратно — с использованием TSS.

Дескрипторы TSS

В рассмотренном примере из листинга 16.3 состояние задачи, или ее контекст, описывается структурой (5–18). В него входят значения регистров, которые первый раз считываются при запуске задачи, а в дальнейшем — при каждом ее продолжении после приостановки. В общем случае это разделяемые вычислительные ресурсы: регистры `i80x86`, регистры `i80x87/MMX` и т. д. В рассмотренном примере мы не включили в контекст регистры `ss`, `ds`, `fs` и `gs`, дав себе слово не изменять их значений при работе сопрограмм¹².

В следующих примерах используется структура данных TSS [2, Figure 12-6]. Она содержит поля для значений известных нам сегментных регистров и 32-битовых регистров общего назначения. Помимо этих полей TSS включает в себя:

- LDT Selector — смещение в GDT дескриптора LDT; если там не ноль, то при переключении задач будет переключаться и LDT¹³.

- ❑ CR3 — адрес таблицы трансляции адресов¹⁴; записав ноль, мы отказываемся от переключения таблиц¹⁵.
- ❑ Link (Prior TSS Selector) — селектор предыдущей задачи.
- ❑ I/O-Permission Bitmap Base Address — смещение массива I/O-Permission Bitmap, где заданы номера портов ввода-вывода, запрещенных для доступа при CPL > 0.
- ❑ Три пары значений SS<i>/ESP<i> — это адреса стеков для уровней привилегий 0–2. При переключении задач с разным уровнем привилегий автоматически переключается стек. Мы не определяем эти поля, т. к. все происходит (пока что) на нулевом уровне.

Все перечисленные дополнительные поля мы обнуляем, не вдаваясь в детали. Последние два пункта списка имеют отношение к TSS в 64-битовом режиме, и мы к ним вернемся, когда будем разбираться с привилегиями.

Чтобы запустить задачу (пока что одну), требуются два экземпляра TSS. Один TSS мы заполняем начальными значениями регистров запускаемой задачи. Другой TSS — одноразовый: при переключении на новую задачу в него копируется текущий контекст. В дальнейшем переключений не будет¹⁶.

В листинге 16.4 показан запуск одной задачи, которая выводит в нулевой позиции экрана счетчик.

Листинг 16.4. Аппаратное переключение на единственную задачу (hw1.asm)

```

...
struc    tss                                ; !                                ; (16)
_link    resw    2                          ; Back-link to calling TSS        ; (17)
_esp0     resd    1                          ; Stack of level 0 pointer          ; (18)
_ss0      resw    2                          ; (19)
...
_cr3      resd    1                          ; Page Directory Base Register     ; (24)
_eip      resd    1                          ; (25)
_eflags   resd    1                          ; (26)
_eax      resd    1                          ; (27)
...
_edi      resd    1                          ; (34)
_es       resw    2                          ; (35)
...
_gs       resw    2                          ; (40)
_ldt      resw    2                          ; (41)
_flags    resw    1                          ; TSSFlagsTrap (bit 0)             ; (42)
_iomapb   resw    1                          ; (43)
endstruc                                     ; (44)

;----- GDT                                ; (45)
...
```

```

dsc8:                                ; !                                ; (104)
ts_work:                             ; tss0 segment, r/w                ; (105)
    dw    tss_size                    ; limit low                      ; (106)
    dw    0                          ; base low                      ; (107)
    db    0                          ; base middle                   ; (108)
    db    1000_1001b                 ; P, dpl=00, s | free 32-bit TSS ; (109)
    db    0100_0000b                 ; g, 32 bit, 00 | limit high (0) ; (110)
    db    0                          ;                               ; (111)

dsc9:                                ; !                                ; (112)
ts_dump:                             ; tss0 segment, r/w                ; (113)
    dw    tss_size                    ; limit low                      ; (114)
    dw    0                          ; base low                      ; (115)
    db    0                          ; base middle                   ; (116)
    db    1000_1001b                 ; P, dpl=00, s | free 32-bit TSS ; (117)
    db    0100_0000b                 ; g, 32 bit, 00 | limit high (0) ; (118)
    db    0                          ;                               ; (119)
    ...
;----- TSS    !                                ; (123)

tss0:  resb    tss_size                    ; (124)
tss1:  resb    tss_size                    ; 1-st regular task            ; (125)

;-----                                ; (126)

start:                                ; (127)
    ...
    mov     eax, edx                    ; (140)
    add     eax, tss1                   ; (141)
    wr_base ts_work                     ; !                             ; (142)

    mov     eax, edx                    ; (143)
    add     eax, tss0                   ; (144)
    wr_base ts_dump                     ; !                             ; (145)
    ...

%macro  cp_r16  1-*                      ; copy reg16 to TSS            ; (153)
%rep  %0                                      ; (154)
    push    %1                          ; (155)
    pop     word [_%1 + ebx]             ; (156)
%rotate 1                                  ; (157)
%endrep                                      ; (158)
%endmacro                                    ; (159)

%macro  prep_ts 2                          ; (160)
    mov     ebx, %1                      ; (161)
    cp_r16  es, cs, ss, ds               ; (162)
    mov     dword [_eip+ebx], %2         ; (163)
%endmacro                                    ; (164)

```

```

pm:                                     ; (165)

    ...

    mov     ax, ts_dump - gdt          ; !          ; (172)
    ltr     ax                          ; !          ; (173)

    prep_ts tss1, entry1               ; (174)
    jmp     (ts_work - gdt):0          ; !          ; (175)

#ifdef _SHOW_                          ; (176)
    mov     esi, msg                   ; (177)
    mov     edx, 182                   ; (178)
    call    wr_str                     ; (179)
#endif                                ; (180)

dead:  _brk                           ; (181)
       call (cgl - gdt):0              ; (182)
       jmp  dead                       ; (183)

entry1:                                ; (184)
.ll:    _brk                           ; (185)
       inc     eax                     ; (186)
#ifdef _SHOW_                          ; (187)
       call    wr_hex                  ; (188)
#endif                                ; (189)
       jmp     .ll                     ; (190)

msg     db     '*** Done ***', 0       ; (191)
end:                                          ; (192)

```

Структура TSS определена в строках (16–44), а сами TSS, заполненные нулями, заданы в (124–125). В GDT добавлены два TSS-дескриптора (104–119), где размер TSS задан статически (106, 114), а линейный адрес в полях (107–108, 111) и (115–116, 119) формируется при выполнении команд (140–145).

Регистр `TR` (Task Register) — это селектор текущего TSS. Команды (172–173) настраивают `TR` на дескриптор (113–119). Макровызов (174) подготавливает в `tss1` поля со значениями основных селекторов и точку входа в задачу. (Остальные поля обнулены изначально.) На этот TSS мы переключимся командой (175) — и попадем в результате в точку (184). Задача входит в бесконечный цикл (185–190), выводя в нулевой позиции¹⁷ экрана счетчик `eax`.

Если закомментировать строку (175), то программа выведет сообщение (191) и войдет в бесконечный цикл (181–183).

При аппаратном переключении между двумя сопрограммами одна из них является продолжением главной задачи — точно так же, как при программном переключении. Данные TSS вторичной задачи мы подготавливаем сами, а TSS главной задачи заполняется автоматически при первом переключении на вторичную задачу (листинг 16.5).

Листинг 16.5. Аппаратное переключение между двумя задачами (hw2.asm)

```

...
#define _SHOW_ ; (2)
n_switch equ 1000000 ; (3)
...

dsc8: ; (105)
ts_wrk1: ; tss0 segment, r/w ; (106)
    dw tss_size ; limit low ; (107)
    dw 0 ; base low ; (108)
    db 0 ; base middle ; (109)
    db 1000_1001b ; P, dpl=00, s | free 32-bit TSS ; (110)
    db 0100_0000b ; g, 32 bit, 00 | limit high (0) ; (111)
    db 0 ; (112)
dsc9: ; (113)
ts_wrk0: ; tss0 segment, r/w ; (114)
    dw tss_size ; limit low ; (115)
    dw 0 ; base low ; (116)
    db 0 ; base middle ; (117)
    db 1000_1001b ; P, dpl=00, s | free 32-bit TSS ; (118)
    db 0100_0000b ; g, 32 bit, 00 | limit high (0) ; (119)
    db 0 ; (120)
...
;----- TSS ! ; (124)

tss0: resb tss_size ; main program ; (125)
tss1: resb tss_size ; 1-st switch to ; (126)

;----- ; (127)
start: ; (128)
...
    mov eax, edx ; (141)
    add eax, tss1 ; (142)
    wr_base ts_wrk1 ; ! ; (143)

    mov eax, edx ; (144)
    add eax, tss0 ; (145)
    wr_base ts_wrk0 ; ! ; (146)
...
%macro prep_ts 3 ; (161)
    mov ebx, %1 ; (162)
    cp_r16 es, cs, ss, ds ; (163)
    mov dword [_eip+ebx], %2 ; (164)
    mov dword [_esp+ebx], %3 ; (165)
%endmacro ; (166)

```



```

pm:                                     ; (167)
    ...
    set_tr  ts_wrk0 - gdt                ; (174)
    prep_ts tss1, entry1, 0xf000        ; (175)
    jmp     (ts_wrk1 - gdt):0            ; !   ; (176)

    mov     eax, n_switch                 ; (177)
#ifdef _SHOW_                             ; (178)
    inc     edx                           ; (179)
#endif                                    ; (180)
.ll1:                                     ; (181)
    _brk                                    ; (182)
#ifdef _SHOW_                             ; (183)
    call    wr_hex                        ; (184)
#endif                                    ; (185)
    jmp     (ts_wrk1 - gdt):0             ; (186)
    dec     eax                           ; (187)
    jnz     .ll1                          ; (188)

    flood   laddr_video                  ; (189)
.ll2:                                     ; (190)
    _brk                                    ; (191)
    jmp     .ll2                          ; (192)

entry1:                                  ; (193)
.ll1:                                     ; (194)
    _brk                                    ; (195)
#ifdef _SHOW_                             ; (196)
    call    wr_hex                        ; (197)
#endif                                    ; (198)
    jmp     (ts_wrk0 - gdt):0             ; (199)
    inc     eax                           ; (200)
    jmp     .ll1                          ; (201)

end:                                     ; (202)

```

Названия дескрипторов (106, 114) изменились, что отразилось на командах (143, 146). Поскольку у каждой задачи должен быть свой стек, в макроопределение (161) добавлен еще один параметр: значение указателя стека. При подготовке `tss1` (175) мы записываем в поле `_esp` число `0xf000`, а в главной задаче `esp = 0xffff` изначально.

Главная программа в точке (176) переключается на сопрограмму, которая в цикле (193–201) переключается (199) обратно. Главная программа возобновляется в точке (177), входит в цикл (181–188) и на каждой итерации передает управление (186) на сопрограмму, вычитает счетчик (187) и при его обнулении (188) выводит на экран сообщение о завершении (189).

Хотя программа hw2.com в Vochs работает нормально, при выполнении на PC она сразу приводит к перезагрузке системы. Чтобы выяснить причину сбоя, в hw2.asm пришлось добавить обработку fault-исключений (примерно так, как это сделано в программе isr_n.asm из листинга 15.2) с выводом счетчиков и дополнительно кода ошибки, а также отображением содержимого TSS (листинг 16.6).

Листинг 16.6. Программа для диагностики сбоя при использовании TSS (err2.asm)

```

...
#define trap_no      32                ; (3)
#define trap_limit 100                ; (4)
...
#ifdef _SHOW_                        ; (11)
view_tss:                          ; esi = ptr(ts), edx - pos ; (12)
    cld                             ; (13)
    mov     ecx, tss_size / 4        ; (14)
.l1:                                  ; (15)
    lodsd                             ; (16)
    call    wr_hex                   ; (17)
    inc     edx                       ; (18)
    loop    .l1                     ; (19)
    ret                                     ; (20)
#endif                              ; (21)
...
; ----- trap                      ; (25)
%macro set_trap 1                    ; (26)
trap%1: _brk                         ; (27)
    enter  0, 0                      ; (28)
    push   edx                       ; (29)
    mov     edx, %1 * 2              ; (30)
    jmp     trap                     ; (31)
%endmacro                          ; (32)
...
trap:                                ; (44)
    push    eax                      ; (45)
    lea     eax, [i_cnt + (edx * 4)] ; (46)
    inc     dword [eax]              ; (47)
    mov     eax, [eax]               ; (48)
.l2:        _brk                     ; (49)
    cmp     al, trap_limit           ; (50)
    ja      .l2                     ; (51)

#ifdef _SHOW_                        ; (52)
    call    wr_hex                   ; (53)
    mov     eax, [ebp+4]             ; err_code ; (54)
    inc     edx                       ; (55)
    call    wr_hex                   ; (56)

```

```

%endif ; (57)
    pop    eax ; (58)
    pop    edx ; (59)
    leave  ; (60)
    iret   ; (61)
    ...

    mov    eax, n_switch ; (250)
.11: ; (251)
    _brk   ; (252)
    jmp    (ts_wrk1 - gdt):0 ; (253)
    dec    eax ; (254)
    jnz    .11 ; (255)

    mov    edx, 198 ; (256)
    mov    esi, msg ; (257)
    call   wr_str ; (258)

.12: _brk ; (259)
    jmp    .12 ; (260)

entry1: ; (261)
.11: _brk ; (262)
#ifdef _SHOW_ ; (263)
    mov    edx, 72 ; (264)
    mov    esi, tss0 ; (265)
    call   view_tss ; (266)

    mov    edx, 112 ; (267)
    mov    esi, tss1 ; (268)
    call   view_tss ; (269)
#endif ; (270)
    jmp    (ts_wrk0 - gdt):0 ; (271)
    inc    edi ; (272)
    jmp    .11 ; (273)

msg    db    '*** Done ***', 0 ; (274)
end: ; (275)

```

Процедура (12–20), вызываемая командами (264–269) из вторичной задачи, выводит содержимое обоих массивов TSS. При обработке исключений (27–61) добавлена команда (28), которая сохраняет в стеке регистр `ebp` и устанавливает его равным `esp`, в результате чего код ошибки доступен по смещению +4 от адреса в `ebp`. Сведения об исключении теперь выводятся не в одной, а в двух позициях. Команда (30) задаст начальную позицию, и в ней, как прежде, выводится (53) счетчик. За ним в следующей позиции выводится то, что прочла¹⁸ команда (54).

Если запустить программу на PC, поля сегментных регистров в `tss1` выглядят как на рис. 16.3.

На рис. 16.3 обращают на себя внимание слишком большие значения в регистрах `fs/gs`. Это явно не селекторы, это сегментные адреса реального режима, унаследованные от FreeDOS¹⁹. При обратном переключении на главную задачу процессор попытается записать (восстановить) эти значения в `fs/gs`, но в защищенном режиме эти числа превышают размер GDT, что вызовет срабатывание общей защиты. Во избежание сбоя нужно обнулить регистры `fs/gs` в начале программы. Ноль в селекторе не ошибка, пока тот не используется для доступа к памяти, а `fs/gs` в нашей программе не задействованы.

esi	edi	es	cs	ss	ds	fs	gs
0000_02c4	0011_fffe	0000_0028	0000_0008	0000_0030	0000_0020	0000_3a23	0000_3654

Рис. 16.3. Поля сегментных регистров в `tss1`

Исправленный вариант (`hw3.asm`) отличается от (`err2.asm`) парой операторов (листинг 16.7).

Листинг 16.7. Инициализация всех сегментных регистров (`hw3.asm`)

```
pm:      ...                                     ; (239)
        _brk                                     ; (240)
        set_seg ds, dsc4 - gdt                   ; (241)
        set_seg ss, stk - gdt                   ; (242)
        set_seg es, flat - gdt                  ; (243)
        set_seg fs, 0                            ; !   ; (244)
        set_seg gs, 0                            ; !   ; (245)
        ...
```

Теперь можно закомментировать определение `_SHOW_` в начале программы и задать `n_switch = 10.000.000`. Результаты выполнения на PC следующие²⁰: `sw.com` — 72 сек, `hw3.com` — 39 сек. Таким образом, время программного переключения составляет 7,2 мкс, а аппаратного — 3,9 мкс.

Аппаратное переключение задач, причем с сохранением всех регистров `i80x86`, оказалось быстрее программного переключения в самом легком его варианте. Если же добавить в контекст `sw.com` регистры `ds`, `fs`, `gs`, а также `ss`, с участием которого будем переключать стек, то можно предположить и большую разницу. Хотя при частоте переключений, скажем, 1000 Гц потери от программной реализации — всего 0,5%.

Еще одно улучшение в программе `hw3.asm` показано в листинге 16.8: начальное значение `esp` у сопрограмм одинаково. Такое возможно при условии, что стек в моменты переключения пуст, а иначе записи, оставшиеся в стеке, будут изменены другой задачей. (Как следствие, нельзя вызывать переключение из подпрограммы.)

Листинг 16.8. Одинаковая установка указателя стека для сопрограмм (hw3.asm)

```

...
%macro prep_ts 2                                ; (233)
    mov     ebx, %1                             ; (234)
    cp_r16  es, cs, ss, ds                      ; (235)
    mov     dword [_eip+ebx], %2                ; (236)
    mov     dword [_esp+ebx], esp               ; (237)
%endmacro                                       ; (238)

...
    prep_ts tss1, entry1                       ; (248)
...

```

При программном переключении стеки разделены, поскольку в них хранится контекст. В результате при переключении задач переключается и стек. Каждый стек должен иметь в запасе M байт, и тогда произведение $M N_i$, где N_i равно количеству задач, есть перерасход памяти при программном переключении²¹.

Теперь посмотрим, как меняется значение регистра TR и состояния дескрипторов задач при переключениях. Изначально дескрипторы задач содержат в байте по смещению 5 код 1001 в младшей тетраде. Если выполнить команду **View/GDT** до загрузки TR , она покажет, что оба дескриптора задач свободны (Available). В результате загрузки TR указанный в ней дескриптор станет занятым (Busy) — код 1001 в нем изменится на 1011. При переключениях автоматически корректируется значение TR , а занятый и свободный дескрипторы меняются местами.

Попытка переключиться на занятый дескриптор — это ошибка. Проверьте, скорректировав одну из команд переключений. Можно поступить проще: установить в одном из TSS код 1011, т. е. сразу пометить TSS как занятый — и тогда сбой произойдет либо при загрузке TR , либо при первом переключении.

Проверить действие поля `iomap`, которое ограничивает доступ к портам ввода-вывода, мы пока не можем, т. к. его действие распространяется лишь на уровни привилегий 1–3. Для нулевого уровня, на котором мы пока что находимся, доступ к портам разрешен всегда²².

Примечания

1. Если в задаче LDT не используется, то в соответствующее поле TSS следует записать ноль.
2. Программа `ldt.asm` сделана на основе `tm3.asm` из предыдущей главы.
3. Ссылка на нулевой дескриптор LDT не приводит к сбою, в отличие от GDT.
4. До сих пор при записи в селекторы бит 2 был сброшен, что означало выбор дескриптора из GDT.
5. По крайней мере с версии 2.x.
6. Также LDT может содержать вентиль задачи, но этот тип вентилей мы не изучаем, поскольку в 64-битовом режиме они упразднены. Достаточно того, что мы разберемся с TSS для 64-битового режима.

7. Чтобы убедиться в этом, выполните трансляцию `ts.8` и посмотрите `lst`-файл. Обратите внимание, что поля `_ds` здесь нет, поэтому любое изменение `ds` при выполнении сопрограммы отразится на другой сопрограмме. Чтобы избежать взаимного влияния сопрограмм, вводим следующее ограничение: `ds` к моменту запуска сопрограммы равен `cs` и не изменяется в дальнейшем.
8. Главная она постольку, поскольку выполняет инициализацию. Но как только начнется взаимное переключение, сопрограммы станут равноценными.
9. Сохраняется исходное значение `esp`, каким оно было в начале команды.
10. Макрокоманда для вывода на экран определена в `pasm.inc`. При ее вызове в реальном режиме `es` должен содержать сегментный адрес видеобуфера, а параметр вызова — 0. В защищенном режиме `es` — селектор дескриптора `flat`, а параметр вызова — `0xb8000`.
11. Поскольку значения `eax` тоже различны, то и вывод будет различаться. При этом код и пути его выполнения у двух сопрограмм одни и те же.
12. Регистр `ds` убран для того, чтобы на рис. 16.1 и 16.2 не было разрыва 32-битовых значений на границах строк. С той же целью можно было *включить* в контекст четыре сегментных регистра, но хотелось сделать самый быстрый вариант программного переключения — чтобы преимущество TSS было еще убедительнее.
13. Записывая туда ноль, мы отказываемся от LDT.
14. Преобразование линейных адресов в физические — тема следующей главы.
15. Если мы не собираемся использовать страничное преобразование, это поле необходимо обнулить.
16. Можно поступить проще: создать один TSS и записать его адрес в регистр `TR` (Task Register). В этом случае никаких переключений задач нет, а TSS используется как хранилище адресов стека для программ разного уровня привилегий, а также массива, ограничивающего доступ к портам для `CPL > 0`. Так мы поступим при исследовании TSS в 64-битовом режиме.
17. Поскольку `tss1._edx = 0`.
18. Если исключение не записывает код ошибки, то это поле содержит случайное значение (`eip` на момент прерывания).
19. В коде FreeDOS, в отличие от предыдущих реализаций DOS, встречаются 32-битовые команды реального режима — так что не исключена запись в `fs` и `gs`.
20. Intel Core 2 Duo, E8400 (3.00 GHz).
21. В предположении, что размер контекста примерно одинаков при любом способе переключения.
22. Следующие вопросы уже не имеют отношения к `x64`, и мы их не рассматриваем: переключение задач командой вызова вместо команды перехода, а также вызов задачи по исключению или прерыванию за счет добавления вентиль задачи в IDT.

ГЛАВА 17

```
DB6 BUFFER:
0100 MOV SI,011F
010E INT 3

RX 07AA i c 1: =e1,ax,, 10191019
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:
```

```
lea si,b_1
```

Преобразование адресов

Матушка всегда знала, где находится, просто заблудилось все остальное.
(Терри Пратчетт. *Вещие сестрички*)

Вот плачет бедная стиральная машина
 Всем своим женским скрытым существом,
 А я надмирным неким существом
 Стою над ней, чтоб подвиг совершила,
 Поскольку мне его не совершить.
 Она же плачет, но и совершает...

(Дмитрий А. Пригов)

В защищенном режиме можно настроить процессор i80x86 так, что линейный адрес будет подвергаться преобразованию при доступе к физической памяти. С какой целью? Затем, чтобы создать у каждой программы иллюзию, будто она выполняется с нулевого линейного адреса, хотя находится в произвольной области физической памяти¹. А поскольку преобразование действует в ограниченном диапазоне линейных адресов, программа не сможет выйти за указанные ей пределы.

Преобразование адресов упрощает подготовку и обеспечивает взаимную изоляцию программ, выполняемых в многозадачной среде. Программа может быть размещена в любом свободном блоке физической памяти, подходящем по размеру, такие программы принято называть перемещаемыми.

В реальном режиме перемещаемость достигается за счет настройки базового линейного адреса (20-битового). Для размещения com-программы нужно найти свободный блок памяти размером 64 Кбайт и записать его начальный адрес (номер параграфа) в сегментные регистры кода, данных и стека — и все.

В 32-битовом защищенном режиме применяются, по сути, те же селекторы и сегменты, только с дополнительным ограничением длины. Так или иначе, пока есть сегменты, перемещаемость достижима и без трансляции адресов². Если отказаться от сегментов, как в архитектуре x64, то вопрос о перемещаемости программ возникает вновь. Преобразование адресов — приемлемый ответ³.

Аппаратное преобразование линейного адреса в физический — это альтернатива сегментной модели памяти и основа работы x64 в 64-битовом режиме. Тут возникает вопрос: преобразование выполняется для *каждого* байта? (Например: байт с ли-

нейным адресом 0 находится в физической памяти по адресу 0x10000, байт с линейным адресом 1 — по физическому адресу 0x427 и т. д.) Для управления столь подробным отображением потребовался бы вектор размером с программу⁴, где были бы перечислены все физические адреса: 0x10000, 0x427 и т. д.

Чтобы сократить размер данных, управляющих преобразованием, решили⁵ отображать память блоками (страницами) по 4 Кбайт⁶. Например, байты с линейными адресами 0–0xffff отображаются в физическую память по адресам 0x10000–0x10fff, а байты с линейными адресами 0x1000–0x1041 (предположим, тут программа заканчивается) — в страницу, расположенную по физическим адресам 0x4000–0x4fff. Страницы выровнены по естественной границе. Так, для страницы размером 4 Кбайт три младшие шестнадцатеричные цифры начального адреса должны быть нулями⁷. Для отображения программы размером 0x1041 байт понадобится 0x2000 байт физической памяти и вектор из двух элементов: 0x10000 и 0x4000. Неиспользованный остаток второй страницы равен $0x2000 - 0x1041 = 4031$ байт.

Первый вариант трансляции адресов

Начнем с первого варианта, реализованного еще в i80386. Также дополнительно упростим задачу: настроим преобразование так, чтобы физические адреса совпадали с линейными. (Такое преобразование далее называется тождественным.) Если после включения механизма трансляции система не перезагрузится, то настройка правильная.

В листинге 17.2 приведена часть программы pg0.asm, которая в защищенном режиме подготавливает таблицы для отображения адресов⁸ и включает механизм отображения. А в листинге 17.1 приведены макросы для программы pg0.asm.

Листинг 17.1. Макроопределения для программы pg0.asm (nasm.inc)

```
%macro set_cr3 1
    mov     eax, %1
    mov     cr3, eax
%endmacro

%macro paging_on 0
    mov     eax, cr0
    or      eax, 1 << 31
    mov     cr0, eax
%endmacro

%macro store 2
    mov     edi, %1
    mov     al, %2
    stosb
%endmacro
```


Первая макрокоманда записывает в управляющий регистр `cr3` заданное значение. Вторая включает трансляцию адресов. К этому моменту регистр `cr3` должен быть инициализирован. Третья не имеет отношения к трансляции адресов, а просто записывает по заданному смещению заданный байт, используя селектор `es`.

Листинг 17.2. Тожественное отображение адресов (`pg0.asm`)

```

...
%define dir 0x100000                                ; (3)
%define pt0 (dir + 0x1000)                          ; (4)
%define pg_flags 7                                  ; (5)
...
    _brk                                            ; (169)
    mov     edi, pt0                                ; (170)
    mov     eax, 0 + pg_flags    ; phadr := 0        ; (171)
    mov     ecx, 1024                                ; (172)
.11:                                            ; (173)
    stosd                                ; pt0[i] := phadr ; (174)
    add     eax, 4096    ; phadr + 4k                ; (175)
    loop    .11                                    ; (176)

    mov     eax, pt0 + pg_flags                    ; (177)
    mov     edi, dir                                ; (178)
    stosd                                ; dir[0] := phadr(pt0) ; (179)
    mov     ecx, 1023    ; dir[1..1023] := 0        ; (180)
    clr     eax                                    ; (181)
    rep     stosd                                ; (182)

    set_cr3 dir                                    ; !          ; (183)
    store   0xb8000, '*'                          ; (184)
    paging_on                                ; !          ; (185)
    store   0xb8000 + 4, '#'                      ; (186)

.12:    _brk                                            ; (187)
        jmp     .12                                    ; (188)
end:                                          ; (189)

```

Отображением адресов управляют одномерные массивы⁹, которые хранятся в выровненных страницах по 4 Кбайт. Элемент массива — 32-битовый *физический* адрес, число элементов — 1024, а индексом является какая-то из частей *линейного* адреса [2, Figure 5-4]. Старшие 10 бит линейного адреса — это индекс в векторе Page Directory, следующие 10 бит — индекс в векторе Page Table, а младшие 12 бит — смещение в байтах от начала страницы.

Вектор Page Directory¹⁰, или каталог таблиц страниц содержит 32-битовые физические адреса таблиц страниц (Page Tables). Место каталога задано оператором (3). По значению в старших 10 битах линейного адреса из каталога считывается адрес

вектора Page Table. В примере предполагается, что биты 22–31 линейного адреса всегда равны нулю¹¹, поэтому мы инициализируем (177–179) только нулевой элемент каталога, а остальные обнуляем¹² (180–182).

Таблица страниц, на которую ссылается нулевой элемент каталога, находится в памяти (4) сразу под каталогом. В зависимости от числа в битах 12–21 линейного адреса выбирается элемент таблицы — 32-битовый физический адрес страницы. В этом примере мы транслируем адреса один в один, поэтому в нулевой элемент таблицы страниц пишем ноль¹³, в первый — 0x1000, во второй — 0x2000 и т. д. до элемента под номером 1023 (0x3ff), куда мы записываем число 0x3ff000. Эти действия выполняются¹⁴ командами (170–176).

Подготовив массивы, записываем (183) адрес каталога в регистр cr3. Пока что трансляция адресов не включена, и мы "напрямую" выводим (184) звездочку в углу экрана. Затем записью старшего бита в cr0 включаем (185) трансляцию адресов и выводим "решетку" во второй позиции экрана. Остановившись в отладчике на (187), имеет смысл взглянуть на линейную (<F7>) и физическую (<Ctrl>+<F7>) память от адреса 0x100000 и 0x101000, а также выполнить команду **View/Page Table**.

Так что же задано в (5)? Это слагаемое присутствует во всех элементах массивов, кроме обнуленных элементов каталога. Обратившись к [2, Figure 5-5] и [2, Figure 5-6], выясняем, что 7 = 111b означает¹⁵:

- ❑ Present (бит 0) — страница присутствует в памяти;
- ❑ R/W (бит 1) — для третьего уровня привилегий (user) разрешен доступ на чтение и запись; а остальным уровням и так все разрешено;
- ❑ User level (бит 2) — страница доступна не только с уровней привилегий 0–2 (supervisor), но и с уровня 3 (user).

Поскольку мы находимся на уровне supervisor, биты 1–2 можно обнулить. Существенное значение имеет нулевой бит: если он сброшен, элемент исключается из преобразования¹⁶.

В следующем примере, в листинге 17.3, добавлено перенаправление страницы 0x12 линейных адресов в видеобuffer (физическая страница 0xb8).

Листинг 17.3. Трансляция адресов с перенаправлением (pg1.asm)

```

...
set_cr3 dir                                ; (183)
store  0xb8000, '*'                        ; (184)
paging_on                                  ; (185)

pg_rdr  equ 0x12                            ; (186)
adr_rdr equ pg_rdr * 0x1000                 ; (187)

store  adr_rdr + 2, '-'                    ; (188)

mov     eax, 0xb8000 + pg_flags            ; 0xb80000 <=      ; (189)
es mov  [pt0 + (pg_rdr * 4)], eax          ; <= 0x12000      ; (190)

```

```

%if 0 ; (191)
    es invlpg [adr_rdr] ; (192)
%endif ; (193)
    store adr_rdr + 2, '@' ; (194)
    store 0xb8000 + 4, '#' ; (195)
    ...

```

В строке (186) задан номер страницы в пространстве линейных адресов, а в (187) — ее линейный адрес. После включения (185) страничного преобразования запись (188) байта по линейному адресу $0x12002$ направляется в память по тому же физическому адресу — $0x12002$ (куда-то за пределы нашей программы). В строках (189–190) меняем значение элемента таблицы, который отображает страницу $0x12$: задаем там физический адрес видеобuffers.

Конечно, запись (195) попадет в видеобuffer, ведь элемент номер $0xb8$ мы не трога-ли, и для него по-прежнему преобразование тождественное. Интересно, куда на-правится запись (194)? Ведь мы исправили элемент таблицы уже после включения преобразования.

Проверьте — никакого символа '@' появиться не должно. Будет только '#' в третьей позиции экрана. Закомментируйте (188), и символ '@' появится. То есть корректи-ровка таблиц после (185) — еще не ошибка. Но если мы уже обратились (188) к па-мяти, то сведения о выполненной трансляции адресов попадут в кеш-память¹⁷. Там останется запись о том, что обращение к линейному адресу $0x12002$ направлено по физическому адресу $0x12002$, и команда (194) будет выполнена без повторного чте-ния таблицы страниц. Обновление таблицы страниц командами (189–190) останет-ся незамеченным! Когда же мы закомментировали (188), то команда (194) первой обратилась к адресу $0x12002$, что и заставило процессор прочитать таблицу не из кеш-памяти, а из основной памяти.

Вернитесь к исходному варианту `pg1.asm` и исправьте в строке (191) 0 на 1. Коман-да (192), где указан линейный адрес $0x12000$, сотрет из кеш-памяти данные о преоб-разовании адресов, попавшие туда при выполнении (188).

Убедитесь, что вариант с командой (192) работает. Тут возникает вопрос: сведения в кеш-памяти хранятся с точностью до байта или с точностью до страницы? Верни-тесь к исходному варианту `pg1.asm` и исправьте в строке (194) число 2 на 20, на-пример. В результате (188) и (194) пишут в одну и ту же страницу, но по разным линейным адресам. Проверим, стирает ли команда (192) кеш-память для всех адре-сов страницы или только для одного.

Запись по адресу $0x12020$ на экран не попала. Как и следовало ожидать, буфер трансляции помнит о преобразовании адреса с точностью до *страницы*. Так что в команде (192) можно указать любой адрес в пределах страницы. Проверьте это, добавив в квадратные скобки слагаемое от 1 до $0xffff$.

Остановившись в цикле после (195), выполните команду **View/Page Table**.

Прежде чем двигаться дальше, договоримся о сокращениях. В документации аб-бревиатуры PDE и PTE означают Page Directory Entry (элемент каталога таблиц

страниц) и Page Table Entry (элемент таблицы страниц) [2, Figure 5-4], а PDE.P соответствует признаку P (Present) в PDE [2, Figure 5-5].

Вариант PSE

Начиная с Pentium, можно выбрать для каждой страницы ее размер. В современных процессорах для 32-битового режима возможны варианты 4 Кбайт, 2 и 4 Мбайт (см. строку Legacy Mode в [2, Table 5-1]). В листинге 17.4 задано тождественное отображение, используется одна страница размером 4 Мбайт.

Листинг 17.4. Отображение адресов одной страницей размером 4 Мбайт (pg2.asm)

```

...
#define dir 0x100000                                ; (3)
#define ps (1 << 7)                                ; (4)
#define pg_flags 111b                               ; (5)
...
    _brk                                            ; (169)
    cld                                           ; dir[0] := (0, large) ; (170)
    mov     edi, dir                               ; (171)
    mov     eax, 0 + pg_flags + ps                 ; (172)
    stosd                                         ; (173)

    clr     eax                                   ; disable dir[1-1023] ; (174)
    mov     ecx, 1023                             ; (175)
    rep stosd                                     ; (176)

    set_cr4_pse                                   ; !                ; (177)
    set_cr3 dir                                   ; (178)
    store    0xb8000, '*'                         ; (179)
    paging_on                                     ; (180)
    store    0xb8000 + 2, '!'                     ; (181)

    mov     edi, 0x40_0000 - 4                     ; (182)
    mov     eax, 0x12345678                         ; (183)
    stosd                                         ; (184)
.12:                                             ; (185)
    _brk                                            ; (186)
%if 0                                           ; (187)
    stosd                                         ; (188)
    es mov  eax, [edi]                             ; (189)
%endif                                           ; (190)
    jmp     .12                                     ; (191)
end:                                             ; (192)

```

Страница расширяется, если ее PDE.PS = 1 (Page Size, бит 7), а в регистре CR4 установлен бит PSE (Page-Size Extensions). Если CR4.PSE = 0, то признак PDE.PS на раз-

мер страницы не влияет, тот остается равным 4 Кбайт. Если же $CR4.PSE = 1$, то для тех PDE, у которых $PS = 1$, картина отображения изменяется [2, Figure 5-7]. Как видите, PDE (при $PS = 1$) напрямую выбирает страницу размером 4 Мбайт, а таблицы страниц при этом исчезли.

Прокомментируем код в листинге 17.4. В строке (4) определен бит `PS`, и с его добавлением мы формируем (171–173) `PDE` в начале каталога. Остаток каталога дописываем нулями (174–176). Устанавливаем `CR4.PSE = 1` макрокомандой (177), определенной в `pasm.inc`. Включение трансляции адресов и ее проверка (178–181) — как в предыдущих примерах. В (182–184) выполняем запись в последнее 32-битовое слово страницы — убеждаемся, что размер страницы не меньше 4 Мбайт. Если исправить в (187) ноль на единицу, то (188) должна привести к сбою. Действительно, на экране появятся числа, соответствующие исключениям 13 и 14. Если выполнять программу в отладчике, видно, что первым срабатывает исключение 14 — `#PF` (Page-Fault Exception). При этом, согласно [2, 8.2.15], в регистр `CR2` записывается линейный адрес, вызвавший сбой, а в стек — код ошибки из [2, Figure 8-3]. Действительно, в отладчике `CR2 = 400000`, а показанный на экране код ошибки 2 соответствует единице в бите `R/W`, т. е. сбой произошел при записи. Если закомментировать (188), то сбой возникнет при чтении, бит `R/W` в коде ошибки будет сброшен, а сам код ошибки равен нулю, проверьте.

В том, что команда (184) выполнена успешно, можно убедиться нажав клавиши <Ctrl>+<F7>. Результат чтения физической памяти с адреса 0x3ffff0 показан на рис. 17.1.

0x003FFFF0	00000000	00000000	00000000	12345678
0x00400000	00000000	00000000	00000000	00000000

Рис. 17.1. Результат выполнения команды (184)

В следующем примере в листинге 17.5 используются: страница размером 4 Мбайт и, вплотную к ней, с физического адреса 0x40_0000, еще одна страница размером 4 Кбайт, которая требует для своего отображения таблицу страниц. Картина отображения, можно сказать, двойится: в зависимости от значения PDE.PS она мгновенно переключается¹⁸ между схемой, показанной на [2, Figure 5-7] для нулевого PDE, и схемой на [2, Figure 5-4] — для следующего PDE.

Листинг 17.5. Использование страниц двух размеров (pg3.asm)

```

...
#define dir 0x100000 ; (3)
#define ps (1 << 7) ; (4)
#define pg_flags 11b ; (5)
#define pt0 (dir + 0x1000) ; (6)
...

_brk ; (170)
cld ; dir[0] := (0, large) ; (171)

```

```
mov     edi, dir                                ; (172)
mov     eax, 0 + pg_flags + ps                  ; (173)
stosd                                       ; (174)

mov     eax, pt0 + pg_flags                      ; (175)
stosd                                ; dir[1] := (pt0[0], small) ; (176)

clr     eax                                ; disable dir[1-1023] ; (177)
mov     ecx, 1022                            ; (178)
rep     stosd                                ; (179)

mov     edi, pt0                                ; pt0[0] ; (180)
mov     eax, 0x40_0000 + pg_flags              ; (181)
stosd                                       ; (182)

mov     ecx, 1023                                ; disable pt0[1-1023] ; (183)
clr     eax                                ; (184)
rep     stosd                                ; (185)

set_cr4_pse                                ; ! ; (186)
set_cr3 dir                                ; (187)
store   0xb8000, '*'                        ; (188)
paging_on                                ; (189)
store   0xb8000 + 2, '!'                    ; (190)

mov     edi, 0x40_0000 - 4                    ; (191)
mov     eax, 0x12345678                      ; (192)
stosd                                       ; (193)
stosd                                       ; (194)
.12:                                       ; (195)
    _brk                                    ; (196)
    jmp     .12                             ; (197)
end:                                       ; (198)
```

Местоположение таблицы страниц определено в (6) — сразу за каталогом. В строках (171–174) заполняется нулевой элемент PDE, в (175–176) — первый, в (177–179) — остальные. Первый элемент таблицы формируется в (180–182), остаток — в (183–185). На рис. 17.2 показана картина отображения после включения преобразования адресов (189).

Команда (193) записывает 32-битовое слово в конец страницы 4 Мбайт, а (194) — в начало прилегающей к ней страницы 4 Кбайт. В точке между (193) и (194) можно

L.Address	is mapped to P.Address
0x00000000 - 0x00400FFF	0x0000000000000000 - 0x0000000000400FFF

Рис. 17.2. Результат выполнения команды View/Page Table после (189)

проверить, в каких PDE и PTE установлен бит 5 (Access) — признак того, что к странице обращались. По логике программы, между (193) и (194) бит A должен быть установлен для нулевой страницы, а для первой — сброшен. После (194) бит A будет установлен также и для первой страницы¹⁹.

Можно ли сразу за страницей 4 Кбайт отобразить страницу 4 Мбайт? Нет, поскольку все страницы должны быть выровнены по их естественным границам. Если нужна еще одна страница в 4 Мбайт, придется заполнить промежуток между адресами 0x40_0000 и 0x80_0000 — довести число страниц размером 4 Кбайт до 1024.

Увеличьте число страниц размером 4 Кбайт и проверьте отображение при помощи команд, аналогичных (191–193).

Какой смысл в наличии страниц разного размера? В страницу большего размера можно поместить ядро операционной системы, а в страницах поменьше — пользовательские приложения.

Вариант отображения, показанный в [2, Figure 5-7], самый простой. Если во всех действующих²⁰ PDE признак PS = 1, то уровень отображения всего один. Этот вариант, хотя он и использует 32-битовые PDE, позволяет с их помощью задать адресные биты 32–39 [2, Figure 5-8], расширяя адресное пространство до 40 бит.

Вариант PAE

На смену PSE-отображению (Physical Size Extension) приходит PAE-отображение (Physical Address Extension). Это максимум, что можно было выжать из 32-битового PDE. В этом варианте элементы таблиц отображения уже не 32-битовые, а 64-битовые. (Хотя мы по-прежнему остаемся в рамках i80x86, т. е. линейный адрес 32-битовый²¹.) Судя по [2, Figure 5-9] и [2, Figure 5-13], PAE-отображение вводит дополнительный уровень преобразования: появляется таблица Page Directory Pointer Table²², предшествующая знакомому нам каталогу Page Directory Table. Таблицы, управляющие отображением, остались того же размера (4 Кбайт), но содержат теперь 512 элементов по 64 бита (а было 1024 по 32)²³.

Режим PAE включается установкой бита 5 в регистре CR4 (CR4.PAE = 1). При CR4.PAE = 1 признак CR4.PSE игнорируется. Если PDE.PS = 0, то размер страницы — 4 Кбайт. Если PDE.PS = 1, то размер — 2 Мбайт, независимо от значения CR4.PSE.

Простейший пример PAE-отображения, когда размер страницы равен 2 Мбайт [2, Figure 5-13], приведен в листинге 17.6.

Листинг 17.6. Пример PAE-отображения страницы размером 2 Мбайт (pg4.asm)

```
...
%define dir 0                                ; (3)
%define pdp (dir + 0x1000)                  ; (4)
...
    _brk                                     ; (170)
    cld                                     ; (171)
```

```
mov     edi, pdp                                ; (172)
mov     eax, dir + 1                            ; (173)
stosd                                ; pdp[0].low := (dir, P=1) ; (174)
clr     eax                                    ; (175)
mov     ecx, 7                                ; (176)
rep     stosd                                ; pdp[1..3] := 0 ; (177)

mov     eax, 0 + pg_flags + ps                 ; (178)
mov     edi, dir                               ; (179)
stosd                                ; dir[0].low := (0, ps) ; (180)
clr     eax                                    ; (181)
mov     ecx, 1023                             ; dir[1..511] := 0 ; (182)
rep     stosd                                ; (183)

set_cr3 pdp                                ; ! ; (184)
set_cr4_pae                                ; ! ; (185)

store   0xb8000, '*'                        ; (186)
paging_on                                ; (187)
store   0xb8000 + 4, '#'                    ; (188)

mov     edi, 0x20_0000 - 4                   ; (189)
mov     eax, 0x12345678                     ; (190)
stosd                                ; (191)

.l2:   _brk                                ; (192)
stosd                                ; (193)
jmp     .l2                                ; (194)
end:                                       ; (195)
```

Каталог таблиц теперь находится в начале памяти (3), его размер по-прежнему равен 4 Кбайт, но элементы уже 64-битовые [2, Figure 5-15]. В каталоге мы заполняем нулевой элемент (178–180), обнуляем его старшую часть, а затем и остальные 511 элементов. Таблица PDPT (Page Directory Pointer Table) расположена в памяти сразу после каталога, она содержит всего 4 элемента по 64 бита. В нулевой элемент пишем (172–174) адрес каталога + бит P. Обратите внимание: в отличие от PDE [2, Figure 5-15], биты 1–2 в PDPE [2, Figure 5-14] зарезервированы, там должны быть нули.

В регистр CR3 записываем (184) адрес PDPT, устанавливаем (185) признак CR4.PAE и включаем (187) отображение адресов. На рис. 17.3 видно, что запись в последнее 32-битовое слово прошла. Команда (193) пытается выполнить запись уже за пределами отображения адресов, что приводит к сбою.

0x001FFFF0	00000000	00000000	00000000	12345670
0x00200000	00000000	00000000	00000000	00000000

Рис. 17.3. Содержимое памяти на границе страницы после (191)

Результаты опыта соответствует ожиданиям, но команда **View/Page Table** показывает нечто странное (рис. 17.4).

Такое отображение должно привести к катастрофе. Согласно ему мы замкнуты в области физических адресов 0...0xffff, которая не имеет отношения к нашей программе. Или отображение настроено неправильно, или Bochs неверно его показывает.

LAddress	is mapped to P.Address
0x00000000-0x00000FFF	0x0000000000000000-0x0000000000000FFF
0x00001000-0x00001FFF	0x0000000000000000-0x0000000000000FFF
...	
0x001FF000-0x001FFFFF	0x0000000000000000-0x0000000000000FFF

Рис. 17.4. Результат выполнения команды **View/Page Table** в точке (192)

Чтобы выяснить причину, добавим в программу тест памяти: запись в заданные линейные адреса (при включенном преобразовании адресов) и чтение из физической памяти (уже при выключенном преобразовании) с выводом результатов на экран.

Пробный вариант, сделанный для одного адреса, приведен в листинге 17.7.

Листинг 17.7. Тест памяти для одного адреса (pg4_b.asm)

```

...
paging_on                                     ; (187)
store    0xb8000 + 4, '#'                     ; (188)

mov      edi, 0x20_0000 - 4                    ; (189)
mov      eax, 0x12345678                      ; (190)
stosd                                         ; (191)

paging_off                                    ; (192)

#ifdef _SHOW_                                  ; (193)
    es mov  eax, [0x20_0000 - 4]                ; (194)
    mov     edx, 80                             ; (195)
    call    wr_hex                               ; (196)
#endif                                         ; (197)
...

```

Число, записанное (189–191) в последнее двойное слово отображаемой памяти, после выключения трансляции адресов (192) считывается (194) из памяти по тому же адресу и выводится на экран (195–196) в позиции 80, т. е. в начале строки 10.

В листинге 17.8 проверяемые адреса заданы в массиве, ограниченном –1.

Листинг 17.8. Тест памяти для массива адресов (pg5_a.asm)

```

...
    _brk                                     ; (186)
    mov     esi, ladr                       ; (187)
    mov     edx, 80                        ; (188)
    paging_off                             ; (189)
check:                                     ; (190)
    lodsd                                     ; (191)
    test    eax, eax                       ; (192)
    js      .l2                             ; (193)

    mov     ebx, eax                       ; (194)
    paging_on                               ; corrupt eax ; (195)
    es mov  [ebx], ebx                     ; (196)
    paging_off                             ; corrupt eax ; (197)
    es mov  eax, [ebx]                     ; (198)
    _brk                                     ; (199)
#ifdef _SHOW_                               ; (200)
    call    wr_hex                         ; (201)
    inc     edx                             ; (202)
#endif                                     ; (203)
    jmp     check                          ; (204)

.l2:   _brk                               ; (205)
    jmp     .l2                            ; (206)

ladr:  dd    0x3ffc, 0x4000, 0x4ffc, 0x5000, 0x1f_fff ; (207)
       dd    0x20_0000, 0x20_3ffc, 0x3f_fff ; (208)
       dd    0x40_0000                     ; (209)
       dd    -1                             ; (210)
end:                                       ; (211)

```

Очередной элемент массива (207–210) считывается командой (191). Если элемент отрицательный (192–193), выходим из цикла. Если неотрицательный, то включаем отображение (195) и записываем (196) по этому линейному адресу значение адреса. Затем, выключив отображение (197), считываем данные (198) из того же адреса (на сей раз физического) и выводим (201) на экран, после чего, увеличив (202) номер позиции для следующего вывода, переходим (204) к повтору цикла.

Пока все сходится: по адресу 0x3ffc читаем 0x3ffc и т. д. Сбой происходит на строке (209), но исключение успешно диагностируется и блокируется после ста повторов. Эта программа проверена на PC без Bochs, результат тот же. Однако по-прежнему команда **View/Page Table** в Bochs показывает ерунду, да и чтение памяти по <F7> (виртуальная память) и <Ctrl>+<F7> (физическая память) дает несовпадающие результаты, хотя преобразование тождественное.

Прежде чем делать выводы, необходим еще один опыт — с нетождественным преобразованием. В листинге 17.9 в отображении участвуют две страницы по 2 Мбайт.

Если для первой страницы преобразование адресов тождественное, то вторая (с линейными адресами 0x20_0000...0x3f_ffff) переводится в физические адреса 0x40_0000...0x5f_ffff.

Листинг 17.9. Тест памяти для нетождественного отображения (pg5_b.asm)

```

...
_brk                                     ; (170)
cld                                     ; (171)
mov     edi, dir                         ; (172)
clr     eax                             ; (173)
mov     ecx, 2048                       ; (174)
rep     stosd                           ; (175)

es mov  dword [pdp], dir + 1             ; (176)

mov     eax, 0 + pg_flags + ps          ; (177)
es mov  [dir], eax                       ; (178)
add     eax, 0x40_0000                   ; (179)
es mov  [dir + 8], eax                   ; (180)

set_cr3 pdp                             ; ! ; (181)
set_cr4_pae                             ; ! ; (182)

store   0xb8000, '*'                     ; (183)
paging_on                                     ; (184)
store   0xb8000 + 4, '#'                  ; (185)

mov     esi, l_adr                       ; (186)
wr_loop:                                     ; (187)
lods    ; (188)
test    eax, eax                         ; (189)
js      .l1                              ; (190)
es mov  [eax], eax                       ; (191)
jmp     wr_loop                           ; (192)
.l1:                                         ; (193)

paging_off                                 ; (194)
mov     esi, ph_adr                      ; (195)
#ifdef _SHOW_                               ; (196)
mov     edx, 80                           ; (197)
#endif                                     ; (198)
rd_loop:                                     ; (199)
lods    ; (200)
test    eax, eax                         ; (201)
js      .l2                              ; (202)
es mov  eax, [eax]                       ; (203)
_brk                                         ; (204)

```

```

#ifdef _SHOW_ ; (205)
    call    wr_hex ; (206)
    inc     edx    ; (207)
#endif      ; (208)
    jmp     rd_loop ; (209)

.l2:    _brk ; (210)
    jmp     .l2 ; (211)

l_addr: dd    0x3ffc, 0x4000, 0x4ffc, 0x5000, 0x1f_fff ; (212)
        dd    0x20_0000, 0x20_3ffc, 0x3f_fff ; (213)
        dd    -1 ; (214)
ph_addr: dd    0x3ffc, 0x4000, 0x4ffc, 0x5000, 0x1f_fff ; (215)
        dd    0x40_0000, 0x40_3ffc, 0x5f_fff ; (216)
        dd    -1 ; (217)
end:    ; (218)

```

Чтобы упростить формирование таблиц, в начале мы их полностью обнуляем (171–175). Затем формируем (176) начальный PDPE, а прочие PDPE остаются нулями, т. е. не используются. Записываем (177–178) нулевой адрес в нулевой элемент каталога, а в следующий элемент — адрес²⁴ 0x40_0000. Включив в строке (184) преобразование, пишем (186–193) в линейные адреса из массива (212–214), так что в линейный адрес 0x3ffc попадает число 0x3ffc, а в линейный адрес 0x3f_fff — число 0x3f_fff. Затем, *выключив* преобразование адресов (194), считываем и выводим (195–209) данные из физических адресов, перечисленных в массиве (215–217).

Выведенные данные совпадают — как в Bochs, так и на PC. Преобразование адресов работает, как предполагалось, и теперь уже можно сделать вывод, что Bochs при включенном режиме PAE (для больших страниц) неправильно выполняет две команды из меню **View: Page Table** и **Linear Dump**.

В листинге 17.10 приведен пример варианта PAE со страницами стандартного размера 4 Кбайт. В этом варианте Bochs обрабатывает команды из меню **View** правильно.

Листинг 17.10. Вариант PAE для страниц размером 4 Кбайт (pg6.asm)

```

...
bzero    dir, 0x3000 ; (172)

es mov    dword [pdp], dir + 1 ; pdp ; (173)

mov      eax, pt0 + pg_flags ; dir ; (174)
es mov    [dir], eax ; (175)

mov      edi, pt0 ; pt0 ; (176)
mov      eax, 0 + pg_flags ; (177)
mov      ecx, 512 ; ! ; (178)

```

```

.11:                                     ; (179)
      stosd                           ; (180)
      add     eax, 0x1000               ; (181)
      add     edi, 4                   ; ! ; (182)
      loop    .11                     ; (183)
      ...
ph_adr:                               ; (217)
l_adr:  dd     0x3ffc, 0x4d2c, 0x1a_bcde, 0x1f_fffc ; (218)
;      dd     0x20_0000                ; (219)
      dd     -1                       ; (220)
end:                                     ; (221)

```

В этом варианте число таблиц, управляющих отображением, достигает трех [2, Figure 5-9]. Помимо таблицы PDPT, начальный элемент которой ссылается (173) на каталог, появляется таблица страниц, на которую ссылается (174–175) нулевой элемент каталога²⁵. Таблица страниц заполняется значениями физических адресов от нуля (177) с шагом 0x1000 (181). Только число элементов теперь вдвое меньше (178), а сами они вдвое длиннее, что учитывается командой (182). Затем выполняется тестирование, как в листинге 17.9. Поскольку преобразование адресов тождественное, массивы линейных и физических адресов объединены (217–218).

Мы проверили все варианты преобразования линейного адреса в физический, доступные для 32-битового защищенного режима i80x86.

Плоская модель памяти

Следующий пример — попытка смоделировать режим flat-памяти, когда базовые линейные адреса кода, данных и стека равны нулю. В результате, программа порождает линейные адреса, близкие к нулю, но благодаря механизму трансляции адресов может располагаться в произвольном участке физической памяти.

Воспользуемся чуть измененным методом²⁶ из главы 10. Требуется выделить память по границе страниц 4 Кбайт, но DOS этого не умеет. Поэтому распределяем 8 Кбайт, а внутри этой области найдем выровненные 4 Кбайт. Программа для распределения 8 Кбайт приведена в листинге 17.11.

Листинг 17.11. Резидентная программа, сохраняющая за собой 8 Кбайт памяти (tsr8k.8)

```

p_sz    equ     8k shr 4
        mov     ah, 031
        mov     dx, p_sz
        int     021

```

Выполните трансляцию и запустите tsr8k.com. Затем выполните **mem /u** и найдите имя TSR8K в списке резидентных модулей. Пример списка приведен на рис. 17.5.

К числу 0x3632 нужно прибавить единицу — это и есть сегментный адрес искомого блока памяти. Добавив ноль справа, получим линейный адрес — 0x36330.

Segment	Total		Name	Type
0fbc	96	(0K)		free
0fc3	72,224	(71K)	COMMAND	program
2170	12,608	(12K)	VC	program
2499	72,064	(70K)		free
3632	8,224	(8K)	TSR8K	program
3835	72,224	(71K)	COMMAND	program
49d8	55,008	(54K)	MEM	program
5747	289,424	(283K)		free

Рис. 17.5. Список резидентных модулей, полученных командой mem /u

Блок памяти размером 0x2000 = 8192 байта занимает адреса [0x36330...0x38330[. Внутри него по адресам [0x37000...0x38000[находится выровненная страница 4 Кбайт. В нее мы загрузим тестовую программу, приведенную в листинге 17.12.

Листинг 17.12. Программа для работы в плоской памяти (test.asm)

```
%include "nasm.inc"                                ; (1)
    org      0x100                                  ; (2)
    use32                                           ; (3)

    cld                                             ; (4)
    _brk                                           ; (5)
    mov      ax, [sym]                             ; (6)
    mov      edi, [pos]                            ; (7)
    add      edi, 0xb8000                          ; (8)
    mov      ecx, 80*3                             ; (9)
    rep      stosw                                 ; (10)
exit:                                           ; (11)
    _brk                                           ; (12)
    jmp      exit                                  ; (13)

sym      db      '*', 7                            ; 02a + color      ; (14)
pos      dd      80*2*12                          ; line 12             ; (15)
          dd      0                                ; (16)
          dd      0x55555555                       ; signature           ; (17)
```

Оператор (3) задает 32-битовый режим процессора i80x86. Команда (6) считывает значение выводимого символа (14) с атрибутом, команда (7) — его смещение (15) от начала видеопамати (8), затем этим символом заполняются (9–10) три строки экрана. Учитывая начальное смещение (2), характерное для com-программ, полученный com-файл следует загружать с линейного адреса 0x37100, т. е. с сегментного адреса 0x3710 — командой f2m 3710 test.com. После загрузки вызовите редактор памяти: e32 3710 1, где 1 — ограничение редактируемой области в килобайтах. На

рис. 17.6 показаны две верхние строки редактора, с адреса 0x37100 начинается код программы, что можно проверить по lst-файлу. Число (17), выделенное на рис. 17.6 полужирным шрифтом, отмечает конец com-файла.

Чтобы запустить этот код, нужно сделать программу, которая настроит преобразование адресов 0 -> 37000, обнулит базовые линейные адреса данных и стека и передаст управление по линейному адресу 0x100, при этом также обнулив базовый линейный адрес кода. Программа приведена в листинге 17.13.

37100	DB8766FC	123A166	3D8B0000	125	8000C781	F0B9000B	F3000000	8766AB66
37120	2AFBEBDB	78007	0	55555500	7290755	7310720	7390739	72D0731

Рис. 17.6. Программа test.com в памяти с адреса 0x37100

Листинг 17.13. Программа для вызова модуля test.com (pg7.asm)

```

...
%define tsr_seg 0x3632 ; (3)
%define dir 0 ; (4)
%define pt0 (dir + 0x1000) ; (5)
...
dsc8: ; (116)
flat_code: ; Code (flat) ; (117)
    dw -1 ; limit low (max) ; (118)
    dw 0 ; base low ; (119)
    db 0 ; base middle ; (120)
    db 1001_1000b ; P, dpl=00, S | Code, c, xr, a ; (121)
    db 1100_1111b ; G, 32 bit, 00 | limit high (max); (122)
    db 0 ; (123)
    ...
    bzero dir, 0x2000 ; (179)

    mov edi, pt0 ; (180)
    mov eax, 0 + pg_flags ; phadr := 0 ; (181)
    mov ecx, 1024 ; (182)
.11: ; (183)
    stosd ; pt0[i] := phadr ; (184)
    add eax, 0x1000 ; phadr + 4k ; (185)
    loop .11 ; (186)

    mov eax, pt0 + pg_flags ; (187)
    mov edi, dir ; (188)
    stosd ; dir[0] := phadr(pt0) ; (189)

    tsr_pg equ 1 + ((tsr_seg + 1) >> 8) ; (190)
    tsr_adr equ tsr_pg * 0x1000 ; (191)

```

```

    _brk                                     ; (192)
    mov     eax, tsr_adr + pg_flags         ; tsr <==      ; (193)
    es mov  [pt0], eax                      ; <== 0        ; (194)

    set_seg ds, flat - gdt                 ; (195)
    set_seg ss, flat - gdt                 ; (196)
    mov     esp, 0x100                     ; (197)

    set_cr3 dir                             ; (198)
    paging_on                              ; (199)

    jmp     (flat_code - gdt):0x100        ; (200)
end:                                       ; (201)

```

Адрес резидентного модуля TSR8K (см. рис. 17.5) задан в (3) без добавления единицы. В строках (190–191) по его значению вычисляются номер и адрес страницы 4 Кбайт, куда мы предварительно поместили модуль test.com. Эту страницу надо отобразить с нулевого линейного адреса. Для трансляции адресов выбрана классическая схема, как в листинге 17.2, только таблицы находятся в начале памяти²⁷. Код (180–189) такой же, только чуть короче за счет предварительного обнуления таблиц (179). В (179–189) мы задаем тождественное преобразование адресов, но команды (193–194) корректируем начальный PTE, тем самым направив линейные адреса 0...0xffff в ту страницу физической памяти, где находится test.com.

Команды (195–196) обнуляют базовый линейный адрес сегментов данных и стека, расширяя размер этих сегментов до максимума²⁸. Указатель стека мы изменяем (197) так, чтобы он адресовал физическую память ниже модуля test.com, как мы уже не раз поступали, перенося стек в область PSP. После окончательной настройки и включения отображения (198–199) передаем управление (200) на сегмент кода, чей дескриптор (116–123) соответствует flat-модели памяти. Линейный адрес после выполнения (200) равен 0x100, а физический адрес — 0x37100. Если запустить программу pg7.com в отладочном сеансе Vochs, после строки (200) мы остановимся на команде (5) из листинга 17.12.

В многозадачной среде каждой прикладной программе требуется свое отображение, т. е. свои управляющие таблицы. Операционная система размещает программы и их таблицы в памяти и, переключая задачи, перенастраивает регистр cr3, записывая в него адрес таблицы верхнего уровня для выбранной задачи²⁹.

Мы рассмотрели преобразование адресов в 32-битовом режиме настолько подробно потому, что без него нельзя перейти в 64-битовый режим. Число уровней управляющих таблиц в 64-битовом режиме возрастает, поскольку размерность адреса увеличилась.

Примечания

1. Операционная система "знает", где находится программа, а программа о своем местоположении даже не "подозревает".
2. Такое впечатление, что Intel не был уверен в сегментной модели памяти и добавил к ней альтернативу — механизм трансляции адресов. И не напрасно — вскоре появились работоспособные операционные системы с плоской (flat) моделью памяти, где базовые адреса сегментов равны нулю и пределы расширены до максимума, а разделение физической памяти идет через механизм трансляции адресов. Эта же схема принята в архитектуре x64.
3. И опять, как будто преобразования адресов мало, в AMD64 ввели еще и *относительную* адресацию данных (когда в машинном коде задано расстояние от текущей команды, как в командах переходов) — для разработки кода, свободно перемещаемого без сегментации.
4. Точнее, в четыре раза больше, т. к. для каждого байта в пространстве линейных адресов должен быть задан 32-битовый физический адрес.
5. Страничное преобразование было разработано еще до Intel, но в микроЭВМ не применялось.
6. Это как минимум, а в i80x86/64 возможны схемы преобразования со страницами порядка мегабайт.
7. Например, 0x5100 не может быть адресом страницы размером 4 Кбайт, а 0x5000 — может.
8. Это можно сделать и до перехода в защищенный режим, если таблицы находятся в пределах 1 Мбайт.
9. Обычно их называют "таблицами", но это, по сути, вектор — одномерный массив, где индексом является какая-то часть линейного адреса, а элементом — 32-битовый физический адрес.
10. Это массив, где хранятся физические адреса таблиц страниц. Вместо "каталога страниц" точнее "каталог таблиц страниц".
11. Работая в FreeDOS, мы находимся в пределах 20 разрядов адреса (биты 0–19).
12. Если там окажутся случайные значения, это может вызвать сбой при включении механизма трансляции адресов; эту ситуацию мы проверим чуть позже. А ноль означает, что элемент таблицы — нерабочий.
13. Не считая признаков в младших битах, о которых чуть позже.
14. Заполнив всего лишь одну таблицу страниц, мы уже подготовили отображение адресного пространства большего, чем было доступно в реальном режиме.
15. Аббревиатуры PDE и PTE означают Page Directory Entry и Page Table Entry, т. е. элементы каталога и таблиц страниц.
16. Что и происходит при обнулении элемента целиком.
17. Translation Lookaside Buffer — TLB.

18. При условии $CR4.PSE = 1$.
19. По идее можно проследить частоту обращения к странице, если программно сбрасывать бит A , а потом проверять его на единицу.
20. У действующих PDE и PTE признак P (Present) равен единице.
21. В 64-битовом режиме AMD64 вариант PAE расширен до 64-битового линейного адреса.
22. Элемент этой таблицы называется PDPE (Page Directory Pointer Entry).
23. Судя по [2, Figure 5-9] и [2, Figure 5-13], из 64 бит используются только 52. Это временное аппаратное ограничение, поскольку на сегодня 52 бит хватает.
24. Если бы мы хотели тождественного отображения, то записали бы адрес на $0x20_0000$ больше, а не на $0x40_0000$.
25. У нас на высших уровнях — всего по одной таблице, т. к. DOS-программы работают в пределах первого мегабайта.
26. Там мы загружали процедуру в память как резидентную, выясняли ее адрес командой `mem /u` и, записав его в вектор прерывания $0x80$, выполняли векторный вызов резидентной процедуры.
27. Так даже интереснее: программа `test.com` работает вблизи нулевого *линейного* адреса, притом что с нулевого *физического* адреса хранятся данные, управляющие трансляцией адресов.
28. Для дополнительного сегмента данных (селектор `es`) это уже было сделано.
29. При использовании TSS перезапись $CR3$ происходит автоматически — в TSS есть соответствующее поле. Но этот вариант мы не рассматриваем, т. к. в 64-битовом режиме аппаратная многозадачность упразднена.

ГЛАВА 18

```
006_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i  c  1: =e1,ax,, 10101010
BX 0000  IP 0100  2: =d1,ax,, 1962
CX 0000  CS 6309  3:
DX 6309  SS 6309  4:
SI 011F  DS 6309  5: r5,b_1,, AB ab
DI 0000  ES 6309  6: r5,si,, AB ab
BP 0000  SP FFFE  0:

lea si,b_1
```

Привилегии

Ведь вот, скажем, яйца и мука. Они разной природы. Они в общем друг друга не хотят, скажем прямо. Их надо приручить, уговорить, уболтать... заморочить, выбить из них это упрямство, эту их косность, нежелание мешаться друг с другом.

(Татьяна Москвина. Не делайте бисквиты в плохом настроении)

Опыты с привилегиями в предыдущих главах были ограничены высшим (нулевым) уровнем. Чтобы довести эти опыты до конца, надо понизить привилегию текущего исполняемого кода, т. е. увеличить значение CPL — младшие два бита в селекторе кода.

Уровни привилегий для сегментной модели памяти определяются следующими битовыми полями¹:

- ❑ DPL (descriptor privilege level) — уровень сегмента, задан в его дескрипторе;
- ❑ CPL (current privilege level) — уровень текущего исполняемого кода, хранится в битах 0–1 селектора CS и может быть прочитан с любого уровня привилегий;
- ❑ RPL (requestor privilege level) — запрашиваемый уровень, в битах 0–1 селектора [2, Figure 4-3].

Изменение уровня привилегий

Как изменить уровень текущего исполняемого кода CPL? При вызове подпрограмм через вентиль вызова можно повысить уровень², но не наоборот. Но если такой вызов *поднял* уровень привилегий кода (уменьшил число в CPL), то возврат из подпрограммы должен *низвести* нас на прежний уровень (увеличить CPL). Пользуясь этим свойством команды `retf`, можно подготовить в стеке значения CS и EIP для "возврата" с нулевого уровня на уровень $n = 1-3$. Число n должно быть указано в поле RPL (запрашиваемый уровень привилегий) селектора CS, а также в дескрипторе кода (поле DPL), на который ссылается CS. Выполнив команду `retf`, мы совершим переход, установив при этом $CPL = n$.

Но есть одно затруднение: изменение CPL *всегда* сопровождается аппаратным переключением стека. Зачем вообще понадобились отдельные стеки для разных уров-

ней привилегий? Если бы стек был общим, то можно было бы исчерпать его до дна (подойдя к самому его пределу) и обратиться к операционной системе, спровоцировав ее гибель из-за нехватки места в стеке³.

Переключение стека для i80x86 означает следующее:

- ❑ пара `ss:esp` примет новые значения, указывая на вершину стека *n*-го уровня привилегий;
- ❑ в этом стеке будут сохранены⁴ также прежние значения `ss:esp` [2, Figure 4-34].

Пример переключения на третий уровень привилегий приведен в листинге 18.1.

Листинг 18.1. Переход на низший уровень привилегий (`cpl_3.asm`)

```

...
dsc1:                                     ; (8)
code0:                                   ; Code, ring 0           ; (9)
    dw    end - 1                        ; limit low          ; (10)
    dw    0                             ; base low             ; (11)
    db    0                             ; base middle          ; (12)
    db    1001_1010b                    ; P, dpl, S | 1, C, xr, A ; (13)
    db    0100_0000b                    ; g, 32 bit, 00 | limit high ; (14)
    db    0                             ; base high            ; (15)
dsc2:                                     ; (16)
code3:                                   ; Code, ring 3         ; (17)
    dw    end - 1                        ; limit low          ; (18)
    dw    0                             ; base low             ; (19)
    db    0                             ; base middle          ; (20)
    db    1111_1010b                    ; P, DPL, S | 1, C, xr, A ! ; (21)
    db    0100_0000b                    ; g, 32 bit, 00 | limit high ; (22)
    db    0                             ; base high            ; (23)
dsc3:                                     ; (24)
stk0:                                   ; Stack, ring 0        ; (25)
    dw    end - 1                        ; limit low          ; (26)
    dw    0                             ; base low             ; (27)
    db    0                             ; base middle          ; (28)
    db    1001_0110b                    ; P, dpl, S | Data, E, RW, a ; (29)
    db    0000_0000b                    ; g, b, 00 | limit high (0) ; (30)
    db    0                             ; (31)
dsc4:                                     ; (32)
stk3:                                   ; Stack, ring 3        ; (33)
    dw    0                             ; limit low          ; (34)
    dw    0                             ; base low             ; (35)
    db    0                             ; base middle          ; (36)
    db    1111_0110b                    ; P, DPL, S | Data, E, RW, a ! ; (37)
    db    0000_0000b                    ; g, b, 00 | limit high (0) ; (38)
    db    0                             ; (39)

```

```

gdt:
    dw      $ - gdt - 1      ; size(gdt)      ; (40)
    dd      gdt               ; + laddr(cs)      ; (42)
    ...

proc_3:
.ll:
    _brk
    inc     ecx               ; (64)
    push    ecx               ; (65)
    pop     ecx               ; (66)
    jmp     .ll               ; (67)

pm:
    set_seg ss, stk0 - gdt    ; (69)
    push    stk3 - gdt + 3    ; old SS          ; (70)
    push    0x100             ; old ESP        ; (71)
    push    code3 - gdt + 3   ; old CS         ; (72)
    push    proc_3            ; old EIP         ; (73)

    _brk
    clr     ecx               ; (75)
    retf                          ; (76)

end:                          ; (77)

```

Здесь определены по две пары дескрипторов кода и стека: строки (9–15) и (25–31) — для нулевого уровня привилегий, а (17–23) и (33–39) — для третьего уровня. Эти пары различаются значением `DPL` — либо `00b`, либо `11b`. Что касается стека, то для нулевого уровня его нижний предел (26) граничит с вершиной программы, а начальное значение `esp = 0xffffe` — такое же, как всегда при запуске `com`-программы. Стек третьего уровня мы разместим в `PSP`, поэтому его нижний предел (34) равен нулю.

После перехода в защищенный режим, находясь на нулевом уровне привилегий, мы настраиваем (69) стек нулевого уровня и записываем в него (70–73) четыре значения согласно [2, Figure 4-34]: значение селектора сегмента стека, начальное значение `esp`, значение селектора сегмента кода и начальное значение `eip`. Слагаемое 3 в (70) и (72) — это поле `RPL = 3`.

Остановившись на строке (74), выполните команду **View/GDT** и **View/Stack**. Четыре элемента в стеке: `0x19d`, `0x13`, `0x100` и `0x23` — это результат (70–73). Выполнив (76), мы попадаем в процедуру (61–67). Команды (65–66) включены в процедуру, чтобы проверить работоспособность стека третьего уровня привилегий.

В табл. 18.1 показано, как изменились указатель стека и селекторы при переключении стека. Заметим, что в селекторах кода и стека биты 0–1 теперь установлены в единицу. Для селектора кода это означает, что `CPL = 3`, чего мы и добивались.

Таблица 18.1. Изменения регистров в результате выполнения команды возврата

Регистр	Исходное значение	Новое значение
esp	0000fffe	00000100
cs	0008	0013
ds	3633	0000
es	3633	0000
ss	0018	0023

Обратите внимание на селекторы данных. Их значения, оставшиеся от реального режима, теперь обнулены. Так действует команда возврата при переключении стека: она обнуляет селекторы данных, *неприменимые* на новом уровне привилегий.

В листинге 18.2 приведена та же программа, дополненная выводом на экран (по минимуму) с уровней 0 и 3.

Листинг 18.2. Вывод на экран с уровней 0 и 3 (cpl_3b.asm)

```
...
proc_3:                                     ; (7)
.ll:                                       ; (8)
    _brk                                   ; (9)
%ifdef _SHOW_                             ; (10)
    es mov [0xb8000 + 3*2], al            ; (11)
%endif                                    ; (12)
    inc al                                ; (13)
    push eax                              ; (14)
    pop  eax                              ; (15)
    jmp  .ll                              ; (16)
...

dsc5:                                     ; (56)
flat:                                     ; Flat data, ring 0-3 ; (57)
    dw -1                                 ; limit low (max) ; (58)
    dw 0                                  ; base low       ; (59)
    db 0                                  ; base middle    ; (60)
    db 1111_0010b                         ; P, DPL, S | Data, e, RW, a ; (61)
    db 1100_1111b                         ; G, B, 00 | limit high (max) ; (62)
    db 0                                   ; (63)
...

pm:                                       ; (88)
    _brk                                   ; (89)
    set_seg ss, stk0 - gdt                ; (90)
%ifdef _SHOW_                             ; (91)
    set_seg es, flat - gdt                ; (92)
    es mov byte [0xb8000], '0'            ; (93)
%endif                                    ; (94)
...

```

Добавлен один дескриптор (57–63) сегмента данных с $DPL = 3$. Этот сегмент — общий для всех уровней привилегий. Однако селектор для доступа к сегменту инициализируется один раз (92) с $RPL = 0$ и в таком виде используется как в (93) при $CPL = 0$, так и в (11) при $CPL = 3$. Причина в том, что при доступе к данным вычисляется максимум из RPL и CPL , и результат должен быть не больше DPL . В нашем случае $DPL = 3$. При выполнении (93) $CPL = 0$ и $RPL = 0$ их максимум равен нулю и не превышает DPL . При выполнении (11) $CPL = 3$, а $RPL = 0$, их максимум равен трем и опять не превышает DPL . Понятно, что при любом сочетании CPL и RPL их максимум не превысит $DPL = 3$, вот почему дескриптор (57–63) годится для кода любого уровня⁵.

Вызов привилегированной процедуры

Следующая задача — добавить в программу процедуры вывода из модуля `_output.asm`. В идеале код и данные этого модуля должны быть изолированы от пользовательского уровня, т. е. находиться на нулевом уровне.

С третьего уровня мы можем обратиться к процедуре нулевого уровня двумя способами.

1. Передать управление на *конформный* сегмент кода (в его дескрипторе бит $C = 1$) с $DPL = 0$. При входе в процедуру значение CPL по-прежнему равно трем, а стек не переключается.
2. Обратиться к *обычному* сегменту кода через вентиль вызова⁶. При передаче управления обнулится CPL , а стек переключится; новые значения `SS:ESP` берутся из дескриптора текущей задачи⁷.

Пока что о подключении модуля `_output.asm` не думаем, только проверяем перечисленные способы.

Простейший пример первого варианта показан в листинге 18.3. Данные размещены в сегменте с $DPL = 3$ (доступны с любого уровня), а код — в конформном сегменте кода с $DPL = 0$.

Листинг 18.3. Обращение к конформному сегменту кода (`conf_1.asm`)

```

...
x      dd      0x0102_0304                ; (7)
subr0:                                     ; (8)
      inc      dword [x]                  ; (9)
      retf                                     ; (10)

proc_3:                                     ; (11)
      cld                                     ; (12)
.ll:                                       ; (13)
      _brk                                     ; (14)
#ifdef _SHOW_                             ; (15)
      mov      edi, 0xb8000                ; (16)
```

```

        mov     esi, x                ; (17)
        mov     ecx, 4                ; (18)
.l12:                                     ; (19)
        movsb                               ; (20)
        inc     edi                    ; (21)
        loop    .l12                  ; (22)
%endif                                     ; (23)
        call    (sbr - gdt):subr0      ; conforming ; (24)
        jmp     .l11                  ; (25)
end2:                                     ; (26)
        ...
dsc6:                                     ; (74)
sbr:                                     ; Subroutines, conforming ; (75)
        dw     end2 - 1               ; limit low      ; (76)
        dw     0                     ; base low      ; (77)
        db     0                     ; base middle   ; (78)
        db     1001_1110b            ; P, dpl, S | 1, C, XR, a ; (79)
        db     0100_0000b            ; g, 32 bit, 00 | limit high ; (80)
        db     0                     ; base high     ; (81)
dsc7:                                     ; (82)
lcl:                                     ; Data (local), ring 0-3 ; (83)
        dw     end - 1               ; limit low      ; (84)
        dw     0                     ; base low      ; (85)
        db     0                     ; base middle   ; (86)
        db     1111_0010b            ; P, DPL, S | Data, e, RW, a ; (87)
        db     0100_0000b            ; g, B, 00 | limit high (0) ; (88)
        db     0                     ; (89)
        ...
pm:                                     ; (118)
        _brk                          ; (119)
        set_seg ss, stk0 - gdt        ; (120)
        set_seg ds, lcl - gdt        ; (121)
#ifdef _SHOW_                          ; (122)
        set_seg es, flat - gdt       ; (123)
#endif                                ; (124)
        ...

```

Конформный сегмент для процедур определен в (75–81), признаки $C = 1$ и $DPL = 00$ заданы в (79). В строках (83–89) определен сегмент внутренних данных программы с $DPL = 11$, т. е. доступный с любого уровня. Как и в предыдущих примерах, после переключения на третий уровень, попадаем в точку (11), где в бесконечном цикле выводим (16–22) на экран байты, составляющие 32-битовое слово (7), и вызываем (24) процедуру (8) из конформного сегмента с $DPL = 00$. Эта процедура увеличивает слово (7), в результате чего символы на экране изменяются. Если в отладчике остановиться на точке (9), видно, что младшие биты селекторов не обнуляются, а величина CPL по-прежнему равна трем.

Недостаток конформных сегментов в том, что пользователь может передать управление в любую точку сегмента. Можно в (24) указать произвольное смещение (в пределах сегмента), например не (8), а (7), — и разрушить систему. Второй недостаток: поскольку CPL при выполнении конформного кода остается прежним, то доступ к данным нулевого уровня невозможен. Можно сделать вывод, что конформный сегмент — это временное пристанище для процедур без статической памяти.

Второй вариант, с использованием вентиля вызова, приведен в листинге 18.4. Он хоть и сложнее, но куда более защищенный, т. к. вентиль фиксирует точку входа в процедуру, а данные определены в сегменте с DPL = 0, т. е. недоступны с третьего уровня.

Листинг 18.4. Использование вентиля вызова (cg_1.asm)

```

...
x      dd      0x0102_0304                      ; (7)
subr0:                                     ; (8)
      set_seg ds, lcl0 - gdt                    ; (9)
      inc      dword [x]                        ; (10)
      retf                                       ; (11)

proc_3:                                     ; (12)
      cld                                       ; (13)
.l1:                                       ; (14)
      _brk                                     ; (15)
      set_seg ds, lcl3 - gdt                    ; (16)
      not      byte [x + 3]                     ; (17)
      ...
      call     (cg - gdt + 3):0                  ; non-conforming ; (27)
      jmp      .l1                             ; (28)
      ...

dsc6:                                     ; (77)
sbr:                                       ; Subroutines, non-conforming ; (78)
      dw      end2 - 1                          ; limit low      ; (79)
      dw      0                                ; base low          ; (80)
      db      0                                ; base middle       ; (81)
      db      1001_1010b                       ; P, dpl, S | 1, c, XR, a ; (82)
      db      0100_0000b                       ; g, 32 bit, 00 | limit high ; (83)
      db      0                                ; base high         ; (84)
dsc7:                                     ; (85)
lcl3:                                     ; Data (local), ring 0-3    ; (86)
      dw      end - 1                          ; limit low      ; (87)
      dw      0                                ; base low          ; (88)
      db      0                                ; base middle       ; (89)
      db      1111_0010b                       ; P, DPL, S | Data, e, RW, a ; (90)
      db      0100_0000b                       ; g, B, 00 | limit high (0) ; (91)
      db      0                                ; (92)

```

```

dsc8:                                     ; (93)
lcl0:                                     ; Data (local), ring 0 ! ; (94)
    dw    end - 1                         ; limit low         ; (95)
    dw    0                               ; base low           ; (96)
    db    0                               ; base middle        ; (97)
    db    1001_0010b                      ; P, dpl, S | Data, e, RW, a ; (98)
    db    0100_0000b                      ; g, B, 00 | limit high (0) ; (99)
    db    0                               ; (100)
dsc9:                                     ; (101)
cg:                                       ; Call gate          ; (102)
    dw    subr0                           ; low adr            ; (103)
    dw    sbr - gdt                       ; selector           ; (104)
    db    0                               ; prm_cnt (0-31)     ; (105)
    db    1110_1100b                      ; P, DPL, s | Call Gate ! ; (106)
    dw    0                               ; high adr           ; (107)
dsc10:                                    ; (108)
ts_work:                                 ; tss0 segment, r/w  ; (109)
    dw    tss_size                        ; limit low         ; (110)
    dw    0                               ; base low           ; (111)
    db    0                               ; base middle        ; (112)
    db    1000_1001b                      ; P, dpl=00, s | free 32-bit TSS ; (113)
    db    0100_0000b                      ; g, 32 bit, 00 | limit high (0) ; (114)
    db    0                               ; (115)
    ...
tss0:  resb    tss_size                    ; (120)
    ...
    mov     eax, edx                       ; (141)
    add     eax, tss0                      ; (142)
    wr_base ts_work                        ; ! ; (143)
    ...
pm:                                       ; (151)
    set_seg ss, stk0 - gdt                ; (153)
    set_seg ds, lcl0 - gdt                ; (154)
    ...
    mov     ebx, tss0                     ; (158)
    mov     dword [_esp0 + ebx], 0xfffe    ; (159)
    mov     dword [_ss0 + ebx], stk0 - gdt ; (160)

    mov     ax, ts_work - gdt              ; ! ; (161)
    ltr     ax                             ; (162)
    ...

```

Сегмент процедур с DPL = 0 определен в (79–84), и он обычный (82), не конформный. Определены два сегмента локальных данных: один (86–92) — с DPL = 3, второй (94–100) — с DPL = 0. Первый используется в коде (12–28) с CPL = 3, второй — в процедуре (8–11) с CPL = 0, вызываемой с третьего уровня через вентиль. Вентиль вызова определен в (102–107) с DPL = 3, т. е. он доступен для любого уровня. Вызов

задан в (27). В вызове указан RPL равный трем, что необязательно: при вызове через вентиль эффективный уровень привилегий равен $\max(RPL, CPL)$. (Проверьте, убрав слагаемое 3 из (27).) А вот $DPL = 3$ в *определении* вентиля (106) — обязательно.

Но прежде чем выполнить (27), нужно сформировать в TSS пару `SS0:ESP0` для стека нулевого уровня привилегий и связать этот TSS с регистром `TR`. Определение структуры TSS находится в `iasm.inc`, место под TSS зарезервировано в (120) и, судя по предупреждениям ассемблера, заполнено нулями. Соответствующий дескриптор задачи определен в (109–115), линейный адрес TSS в дескрипторе формируется командами (141–143). После перехода в защищенный режим заполняются (158–160) поля TSS — только те, которые необходимы для задания адреса стека нулевого уровня. Наконец, в регистр `TR` записывается (161–162) селектор TSS.

В табл. 18.2 показаны изменения регистров при вызове процедуры (8) через вентиль с уменьшением CPL. Значение регистра `ss` не изменяется, потому что равно значению поля `SS0` в TSS.

Таблица 18.2. Изменения регистров при вызове подпрограммы с уменьшением CPL

Регистр	Значение в точке (9)	Значение в точке (28)
esp	0000ffee	00000100
cs	0030	0013
ds	0038	0000
es	0028	0028
ss	0018	0023

В строке (9) в селектор данных записали `0x40`, но при возврате из процедуры он автоматически обнуляется, т. к. сегмент данных с $DPL = 0$ на третьем уровне недоступен. Стек в точке (9) содержит числа `0x13e` (`eip`), `0x13` (`cs`), `0x100` (`esp`) и `0x23` (`ss`). Взяв за основу пример с вентилем вызова, добавим в программу модуль `_output.asm` с некоторыми изменениями, показанными в листинге 18.5.

Листинг 18.5. Изменения в модуле `_output.asm`

```
...
wr_hex:                                     ; (59)
    push    ds                             ; (60)

    push    eax                             ; (61)
    set_seg ds, lcl0 - gdt ; !              ; (62)
    pop     eax                             ; (63)

    call    hex2str                         ; (64)
    push    esi                             ; (65)
    mov     esi, _str                       ; (66)
```

```

call    wr_str                      ; (67)
pop     esi                        ; (68)

pop     ds                          ; !      ; (69)
retf                      ; !      ; (70)

```

Для доступа к локальным данным нулевого уровня добавлена настройка в строке (62), а в (70) поставлен дальний возврат, поскольку эта процедура вызывается через вентиль. В служебных процедурах (64) и (67) оставлен ближний возврат. Изменения в главной программе отражены в листинге 18.6.

Листинг 18.6. Вентильный вызов подпрограммы из модуля _output.asm (cg_2.asm)

```

...
%include "_output.asm"                ; (7)

pos      dd      82                    ; (8)

proc_3:                                     ; (9)
    mov     edx, [pos]                 ; (10)
    clr     eax                       ; (11)
.ll1:                                     ; (12)
    _brk                                         ; (13)
    call    (cg - gdt):0                ; (14)
    dec     eax                       ; (15)
    jmp     .ll1                       ; (16)
end2:                                     ; (17)

...

dsc9:                                     ; (89)
cg:                                     ; Call gate ; (90)
    dw      wr_hex                      ; low adr      ; !      ; (91)
    dw      sbr - gdt                  ; selector    ; (92)
    db      0                          ; prm_cnt (0-31) ; (93)
    db      1110_1100b                 ; P, DPL, s | Call Gate ; (94)
    dw      0                          ; high adr    ; (95)

```

В дескрипторе вентиль (90–95) изменилась точка входа (91), а сам вызов (14) остался прежним⁸.

В итоге мы вызываем с третьего уровня процедуру нулевого уровня, причем статические данные процедуры находятся также на нулевом уровне. Правда, в этом примере локальные данные с DPL = 0 и DPL = 3 совмещены в памяти, и при желании с третьего уровня можно добраться до статических данных процедуры, выяснив их имена из _output.asm. Чтобы защитить локальные данные модуля _output, следует разместить их отдельно от данных (8), используемых на третьем уровне. Тогда можно точнее задать границы сегментов в дескрипторах.

Следующие опыты выполняются при $CPL = 3$ — то, что мы не смогли проверить в предыдущих главах. За основу берем только что рассмотренную программу `cg_2.asm` с выводом на экран.

Обращение к портам ввода-вывода

В листинге 18.7 — попытка обратиться к портам ввода-вывода с уровня 3. Доступность портов зависит в первую очередь от поля `IOPL` (I/O Privilege Level) в регистре флагов. Если $CPL \leq IOPL$, доступ разрешен. Если $CPL > IOPL$, то к проверке подключается массив бит `IOPB` (I/O Permission Bitmap) в сегменте `TSS` текущей задачи. Если в массиве `IOPB` бит n установлен в единицу, доступ к порту номер n запрещен.

Листинг 18.7. Доступ к портам без настройки `IOPB` (`io_1.asm`)

```

...
proc_3:                                     ; (8)
.12:                                       ; (9)
        clr     edx                         ; (10)
        mov     ecx, 200                   ; (11)
.11:                                       ; (12)
        _brk                                         ; (13)
        in      al, dx                      ; (14)
        mov     eax, edx                   ; (15)
        call    (cg - gdt):0               ; (16)
        inc     edx                         ; (17)
        loop    .11                       ; (18)
        jmp     .12                       ; (19)
...
        _brk                                         ; (154)
        set_iopl 2                             ; (155)
...

```

Поле `IOPL` находится в битах 12–13 регистра флагов. Макрос (155) для установки `IOPL` определен в `nasm.inc`. Выполняя его по шагам, можно заметить, что изначально $IOPL = 0$. Если в (155) задано число 0–2, то будут прочитаны все порты от 0 до 199. Если в (155) задать число 3, то произойдет сбой, но не сразу, а на порте `0x21`.

Почему? Посмотрите на определение структуры `TSS` в `nasm.inc`. Последнее поле — это смещение массива `IOPB`. Поскольку оно равно нулю (мы ведь его не задавали), система считает, что `IOPB` находится в *начале* `TSS`. То есть `IOPB` оказался совмещен в памяти с другими полями `TSS`, и его значения случайные. Бит 33 — это первый попавшийся единичный бит (если быть точным, то он находится в поле `esp0`, которое расположено по смещению 4 и содержит `0xffffe`).

Нужно разместить `IOPB` правильно: на вершине `TSS`, т. е. по смещению `0x66`. И, поскольку процессор читает `IOPB` 16-битовыми словами, число байтов в `IOPB` должно быть четным⁹. Проверим действие `IOPB` для портов 0–7 в отладчике (листинг 18.8).

Листинг 18.8. Добавление двухбайтового массива IOPB к TSS (io_2.asm)

```

...
ts_work:                ; tss0 segment, r/w                ; (100)
    dw    tss_size + 2  ; limit low                        !    ; (101)
    dw    0              ; base low                        ; (102)
    db    0              ; base middle                     ; (103)
    db    1000_1001b     ; P, dpl=00, s | free 32-bit TSS   ; (104)
    db    0100_0000b     ; g, 32 bit, 00 | limit high (0)  ; (105)
    db    0              ;                                ; (106)
...
tss0:    resb    tss_size                ; (111)
iopb:    db      1010_0000b              ; !    ; (112)
         db      -1                      ; !    ; (113)
...
    _brk                ; (154)
    mov    word [_iomapb + ebx], iopb - tss0 ; !    ; (155)

```

Не исправляя определение TSS в `nasm.inc`, мы добавили (112–113) к его данным два байта IOPB. А в строке (155) записали в TSS смещение IOPB от начала TSS. Обратите внимание, что в (155) пишется 16-битовое слово, а не 32-битовое.

Сбой произойдет при обращении к порту 5, что соответствует установке бита 5 в (112).

Что случится при чтении порта за пределами массива IOPB? Задайте в (112–113) нули и заново запустите программу. Порты 0–15 читаются, а при обращении к порту 16 — сбой, т. к. за пределом IOPB доступ к портам запрещен.

Выполните еще два опыта:

- ☐ Чтобы совсем закрыть доступ к портам, надо расположить на вершине TSS пустой IOPB. Чтобы длина IOPB была равна нулю, необходимо убрать слагаемое в строке (101). Проверьте.
- ☐ Измените IOPB так, чтобы стали доступны порты системного таймера 0x40...0x43, причем только они.

Привилегии при страничном отображении

Осталось выяснить возможности доступа к страницам (при включенном преобразовании адресов) для уровня `CPL = 3`. Заметим, что для механизма преобразования адресов есть лишь два уровня привилегий: `user (CPL = 3)` и `supervisor (CPL = 0...2)`.

Доступом к страницам управляют следующие биты PDE:

- ☐ Present (бит 0) — страница присутствует в памяти.
- ☐ R/W (бит 1) — уровню `user` разрешен доступ и на чтение, и на запись, а на уровне `supervisor` этот бит не влияет, там и так все разрешено.
- ☐ User level (бит 2) — страница доступна с любого уровня.

В листинге 18.9 приведен пример, объединяющий программы из листингов 17.3 и 18.2.

Листинг 18.9. Ограничения доступа к страницам (pg_1.asm)

```

...
#define pg_present      1                ; (5)
#define pg_user_rw      2                ; (6)
#define pg_user_level    4                ; (7)
#define pg_work (pg_present + pg_user_rw + pg_user_level) ; (8)
#define pg_test (pg_present + pg_user_level) ; (9)
...
proc_3:                                ; (14)
.ll:                                    ; (15)
    _brk                                ; (16)
#ifdef _SHOW_                            ; (17)
    es mov [0xb8000 + 3*2], al           ; (18)
#endif                                  ; (19)
    inc al                               ; (20)
    es mov ah, [0]                        ; read page 0 ; (21)
    es mov [0], al                        ; write page 0 ; (22)
    jmp .ll                               ; (23)
    ...
    mov edi, pt0                          ; (104)
    mov eax, 0 + pg_test ; !              ; (105)
    stosd                                  ; (106)

    mov eax, 0 + pg_work                   ; (107)
    mov ecx, 1023                          ; (108)
.ll:                                       ; (109)
    add eax, 4096                          ; phadr + 4k ; (110)
    stosd                                  ; pt0[i] := phadr ; (111)
    loop .ll                               ; (112)

    mov eax, pt0 + pg_work                 ; (113)
    mov edi, dir                           ; (114)
    stosd                                  ; dir[0] := phadr(pt0) ; (115)
    mov ecx, 1023                          ; dir[1..1023] := 0 ; (116)
    clr eax                                 ; (117)
    rep stosd                              ; (118)
    ...

```

В строках (5–7) определены биты PDE, в (8) — их объединение, которое открывает уровню user доступ на чтение-запись. Оно используется в (113) и (107) при инициализации каталога и таблицы страниц с номерами 1–1023. В строке (9) задано частичное объединение, которое разрешает доступ только на чтение, оно используется в (105) при инициализации нулевой страницы памяти.

В строке (21) задано чтение нулевой страницы, которое должно пройти успешно, а в (22) — попытка записи, приводящая к сбою. Проверив исходный вариант `pg_1.asm`, скорректируйте (9) так, чтобы сбой произошел раньше — на строке (21).

Примечания

1. Все поля двухбитовые и содержат код от 00 (высший уровень) до 11 (низший, пользовательский уровень).
2. То есть уменьшить число в `CPL`.
3. Можно нарочно — и я бы обязательно так поступил, будь я тестировщиком. А можно нечаянно, отлаживая рекурсивную процедуру, в которой есть вызовы операционной системы.
4. Более сложный вариант [2, Figure 4-33] возникает, если в вентиле вызова указан ненулевой счетчик параметров, передаваемых через стек. Чтобы понизить `CPL`, мы используем, конечно, самый простой вариант — [2, Figure 4-34]. И пока что мы оставляем в стороне вопрос, откуда берутся новые значения `ss:esp` при вызовах.
5. Кстати, в отладчике видно, что при переключении уровня селектор `es` не обнулится. Это означает, что по результатам проверки он признан годным для третьего уровня, пусть даже его `RPL = 0`.
6. Если сегмент конформный, то как к нему ни обращайся (напрямую или через вентиль), `CPL` не изменится, а стек переключаться не будет.
7. Из полей `SS0` и `ESP0` дескриптора `TSS`, на который указывает регистр `TR`.
8. Разве что убрали ненужное слагаемое `RPL = 3`.
9. Если оно нечетное, нужно добавить байт со значением `0xff`.


```

DBG_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA    i    c    1: =e1,ax,, 10101010
BX 0000    IP 0100    2: =d1,ax,, 1962
CX 0000    CS 6309    3:
DX 6309    SS 6309    4:
SI 011F    DS 6309    5: r5,b_1,, AB ab
DI 0000    ES 6309    6: r5,si,, AB ab
BP 0000    SP FFFE    0:

lea si,b_1

```

ЧАСТЬ III

64-битовые режимы

Глава 19. Переход в режим совместимости

Глава 20. Переход в 64-битовый режим

Глава 21. Особенности 64-битового режима

ГЛАВА 19

```
DB6_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA i c 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:

lea si,b_1
```

Переход в режим совместимости

— Боцман! Ну вы закончили там? Отклоните прислоненное!
— А куда после деть?
— Это меня не касается. Сказано "отклонить" — отклоните немедленно и давайте куда хотите.
(Юрий Коваль. Сур)

В процессорах AMD64 предусмотрены два "длинных" режима: режим совместимости¹ (compatibility mode) и собственно 64-битовый режим (64-bit mode). Эти режимы доступны после переключения в так называемый длинный режим (long mode²).

Переход в режимы long

Переход в длинные режимы выполняется так: переключаемся в 32-битовый защищенный режим, устанавливаем в регистре³ `EFER` [2, Figure 3-8] бит разрешения длинного режима `LME` (Long Mode Enable), затем включаем преобразование адресов в варианте [2, Figure 5-17] или [2, Figure 5-22]. В ответ процессор установит в регистре `EFER` бит подтверждения `LMA` (Long Mode Active). И теперь при выборе сегмента кода, в дескрипторе которого бит `L` (Long) сброшен, действует режим совместимости, а при `L = 1` включается 64-битовый режим.

Но прежде чем переходить в режимы `long`, следует убедиться, что процессор их поддерживает. Сведения о возможностях процессора, полученные посредством команд `CPUID`, выводятся при запуске `Bochs` в его консоли, как показано на рис. 19.1.

```
...
000000000000i[CPU0 ] CPUID[0x80000000]: 80000018 68747541 444d4163 69746e65
000000000000i[CPU0 ] CPUID[0x80000001]: 00060f82 0000059f 0000011b ebd3fbff
...
000000000000i[CPU0 ] CPUID[0x80000018]: 00000000 00000000 00000000 00000000
000000000000i[CPU0 ] CPU Features supported:
000000000000i[CPU0 ] x87
...
000000000000i[CPU0 ] longmode
...
```

Рис. 19.1. Сведения о 64-разрядном процессоре Turion64 Tyler

Сначала выводятся результаты выполнения команды `cpuid`: в квадратных скобках указан номер функции в регистре `eax`, а справа от двоеточия — результат в четырех регистрах: `eax`, `ebx`, `ecx` и `edx`. Описание функций `cpuid` приведено в [10]. Чтобы выяснить, поддерживается ли режим `long`, нужно выполнить функцию `0x80000001` и проверить бит 29 в `edx`. Но сначала — вызвать функцию `0x80000000`, которая вернет в `eax` максимально допустимое значение номера функции⁴. Эта проверка необходима при программировании для PC, а в Bochs достаточно найти слово `longmode` в списке `CPU Features supported`.

Для сравнения на рис. 19.2 показаны результаты `cpuid` для 32-разрядного процессора⁵. В списке `CPU Features supported` слово `longmode` отсутствует.

```
...
000000000000i[CPU0 ] CPUID[0x80000000]: 80000008 00000000 00000000 00000000
000000000000i[CPU0 ] CPUID[0x80000001]: 00000000 00000000 00000001 00000000
...
```

Рис. 19.2. Сведения о 32-разрядном процессоре (model=atom_n270)

В листинге 19.1 приведены макросы из файла `nasm.inc`:

- 1. `chk_longmode` — для проверки, поддерживает ли процессор режимы `long`.
- 2. `set_efer_lme` — для установки бита `EFER.LME`.
- 3. `chk_efer_lma` — для проверки бита подтверждения `EFER.LMA`.

Для макросов 1 и 3 значение проверяемого бита отражено во флаге переноса⁶.

Листинг 19.1. Макроопределения для переключения в режим `long` (`nasm.inc`)

```
%macro    chk_longmode 0                                ; (234)
...
    mov    eax, 0x8000_0000                            ; (243)
    cpuid                                ; (244)
    cmp    eax, 0x8000_0001                        ; (245)
    cmc                                ; (246)
    jnc    %%exit                                ; (247)
    mov    eax, 0x8000_0001                        ; (248)
    cpuid                                ; (249)
    bt     edx, 29                                ; (250)
%%exit:                                ; (251)
%endmacro                                ; (252)

_efer    equ    0xc000_0080                        ; (253)
_lme     equ    8                                  ; (254)
_lma     equ    10                                 ; (255)

%macro    set_efer_lme 0                            ; (256)
    mov    ecx, _efer                                ; (257)
```

```

        rdmsr                                ; (258)
        bts      eax, _lme                   ; (259)
        wrmsr                                ; (260)
%endmacro                                    ; (261)

%macro  chk_efer_lma 0                       ; (262)
        mov      ecx, _efer                 ; (263)
        rdmsr                                ; (264)
        bt       eax, _lma                  ; (265)
%endmacro                                    ; (266)

```

Для записи единицы в `EFER.LME` используется команда (259) проверки и установки бита, а для оценки ответа в `EFER.LMA` — команда (265) проверки бита, значение бита копируется во флаг переноса. Переход из (247) в (251) происходит, если после выполнения `CPUID`-функции с номером `0x8000_0000` число в `eax` не стало больше `0x8000_0000`.

Программа для проверки макросов приведена в листинге 19.2.

Листинг 19.2. Проверка макрокоманд из листинга 19.1 (`chk_1.asm`)

```

...
start:                                     ; (40)
        _brk                                ; (41)
        chk_longmode                        ; (42)
        jc      .l1                         ; (43)
        _exit                                ; (44)
.l1:                                         ; (45)
...
pm:                                         ; (61)
        set_seg ss, stk0 - gdt              ; (62)
        set_seg ds, flat - gdt              ; (63)
#ifdef _SHOW_                              ; (64)
        set_seg es, flat - gdt              ; (65)
#endif                                     ; (66)

        _brk                                ; (67)
        clr      eax                        ; (68)
        cpuid                                ; vendor string ; (69)
        mov      [0], ebx                    ; (70)
        mov      [4], edx                    ; (71)
        mov      [8], ecx                    ; (72)
#ifdef _SHOW_                              ; (73)
        clr      esi                        ; (74)
        mov      edi, 0xb8008                ; (75)
        cld                                         ; (76)
        mov      ecx, 12                      ; (77)

```

```

.10:    movsb                                ; (78)
        inc      edi                        ; (79)
        loop     .10                       ; (80)
%endif                                ; (81)

        set_efer_lme                        ; (82)

waiting:                                ; (83)
.11:                                ; (84)
%ifdef _SHOW_                           ; (85)
        store    0xb8000, byte [0]         ; (86)
        inc      byte [0]                  ; (87)
%endif                                ; (88)
        chk_efer_lma                        ; (89)
        jnc      .11                       ; (90)

long_mode:                              ; (91)
.12:    _brk                                ; (92)
%ifdef _SHOW_                           ; (93)
        store    0xb8004, byte [0]         ; (94)
        inc      byte [0]                  ; (95)
%endif                                ; (96)
        jmp      .12                       ; (97)

end:                                ; (98)

```

Вызов (42) выполняется в реальном режиме, но он возможен и защищенном режиме — команда `CPUID` непротивительна. Если проверка (42) не прошла, программа сразу возвращается (44) в DOS. Если же с (42) все в порядке, то программа очищает экран и, перейдя в защищенный режим, выполняет (68–69) функцию 0 команды `CPUID`. Результат — строка с названием фирмы копируется (70–72) в начало физической памяти⁷, а затем выводится (74–80) на экран, начиная с третьего знака-места. После вызова (82) попадаем в цикл (84–90) ожидания `EFER.LMA = 1`. В цикле выводим (86) в углу экрана байт с возрастающим (87) значением. Вывод произойдет хотя бы раз, т. к. он предшествует проверке (89–90).

Когда символ в углу экрана остановится и начнет меняться (94–95) соседний символ, это будет означать, что режим `long` включен. Пока что этого не произойдет, потому что между (82) и (83) необходимо еще включить преобразование адресов согласно [2, Figure 5-17] или [2, Figure 5-22]. Код, который следует добавить, приведен в листинге 19.3, здесь выбран вариант [2, Figure 5-17] с размером страниц 4 Кбайт.

Листинг 19.3. Четырехступенчатое страничное преобразование (`long_1.asm`)

```

...
#define pml4 0x100000                      ; (3)
#define pdp (pml4 + 0x1000)                ; (4)

```

```

#define dir (pdp + 0x1000) ; (5)
#define pt0 (dir + 0x1000) ; (6)
#define pg_flags 111b ; (7)
...
set_efer_lme ; (89)
prep: ; (90)
    bzero pml4, 0x4000 ; (91)

    mov dword [pml4], pdp + pg_flags ; (92)
    mov dword [pdp], dir + pg_flags ; (93)
    mov dword [dir], pt0 + pg_flags ; (94)

    mov edi, pt0 ; (95)
    mov eax, 0 + pg_flags ; (96)
    mov ecx, 512 ; (97)
.ll: ; (98)
    stosd ; (99)
    add eax, 0x1000 ; (100)
    add edi, 4 ; (101)
    loop .ll ; (102)

start_long: ; (103)
    _brk ; (104)
    set_cr4_pae ; (105)
    set_cr3 pml4 ; (106)
    paging_on ; (107)
    ...

```

В строках (3–6) заданы адреса таблиц, управляющих страничным преобразованием. Они расположены вплотную друг к другу, поэтому обнуляем их одной макрокомандой (91). Берем элемент в начале таблицы PML4, соответствующий нулям в битах 39–47 линейного адреса, и записываем (92) в него адрес таблицы PDP. Аналогично — для таблицы PDP (93) и каталога (94). Таблицу страниц заполняем целиком (95–102), с шагом 0x1000, все 512 элементов. Напомним, что элементы всех таблиц здесь 64-битовые, поэтому указатель записи при заполнении (98–102) таблицы страниц дополнительно увеличивается на четыре (101), пропуская старшую часть 64-битового слова.

Затем, уже при включенном (89) бите `EFER.LME`, устанавливаем (105) бит `CR4.PAE`, а в регистр `CR3` пишем (106) адрес таблицы верхнего уровня и, наконец, в регистре `CR0` включаем (107) преобразование адресов. На этот раз `EFER.LMA = 1` — переход в режим `long` подтвержден. И мы продолжаем работать в режиме совместимости, без каких-либо дополнительных настроек.

Чтобы войти в 64-битовый режим, нужно передать управление на сегмент кода, в дескрипторе которого установлен бит `L`. Согласно [2, Figure 4-20] это бит 21 в старшем 32-битовом слове дескриптора. В 32-битовом режиме бит 21 зарезервирован [2, Figure 4-14], и во всех прежде рассмотренных примерах там ноль.

Особенности режима совместимости

Рассмотрим некоторые особенности режима совместимости, в котором мы оказались. По идее, код, созданный для 32-битового защищенного режима, должен работать в режиме совместимости⁸. Но без модификаций тут не обойтись, т. к. в "длинных" режимах вентили вызова [2, Figure 4-23] и вентили прерываний и исключений [2, Figure 4-24] должны быть расширены так, чтобы в них помещался 64-битовый адрес⁹. В результате дескрипторы вентилей удваиваются в размере.

В листинге 19.4 — попытка сделать универсальный вентиль вызова, работающий и в 32-битовом режиме, и в длинных режимах.

Листинг 19.4. Вентиль вызова в режиме совместимости (long_2.asm)

```

...
subr:                                     ; (12)
    clr     eax                           ; (13)
    cpuid   ; vendor string               ; (14)
    ...
    retf    ; (26)
    ...

dsc3:                                     ; (50)
cgl:                                     ; Call gate (32/64 bit) ; (51)
    dw      subr                          ; low adr       ; (52)
    dw      code0 - gdt                   ; code selector  ; (53)
    db      0                            ; (rsrv)         ; (54)
    db      10001100b                     ; P, dpl, s | Call Gate ; (55)
    dw      0                            ; high adr       ; (56)
    dq      0                            ; 64-bit extension ! ; (57)

dsc4:                                     ; (58)
flat:                                     ; Flat data, ring 0-3 ; (59)
    ...

pm:                                     ; (91)
    set_seg ss, stk0 - gdt                ; (92)
    set_seg ds, flat - gdt                ; (93)
    set_seg es, flat - gdt                ; (94)
    ldt_off                                ; (95)
    a20_on                                ; (96)

    _brk                                  ; (97)
    mov     edi, 8                        ; (98)
    call    (cgl - gdt):0                  ; ! ; (99)

    set_efer_lme                          ; (100)
    ...

long_mode:                               ; (126)
.12:     _brk                              ; (127)

```



```

#ifdef _SHOW_ ; (128)
    store 0xb8004, byte [0] ; (129)
    inc byte [0] ; (130)
#endif ; (131)
    mov edi, 40 ; (132)
    call (cgl - gdt):0 ; ! ; (133)
    jmp .l2 ; (134)
end: ; (135)

```

Дескриптор вентиля вызова (50–57) помещен в середине GDT, добавочные восемь байтов заданы в (57). В GDT соседствуют 8- и 16-байтовые дескрипторы, что не возбраняется. Для ссылки на дескриптор по-прежнему используется его смещение от начала GDT — в строках (92–94), (99) и (133).

Первый вызов (99) через вентиль выполняется в защищенном режиме; подпрограмма (12–26) обрабатывает нулевую функцию `CPUID` и выводит полученную строку на экран с позиции 8. Вторая попытка (133) вентильного вызова, но уже в длинном режиме, заканчивается сбоем. На рис. 19.3 показано содержимое консоли Bochs.

```

00204125142e[CPU0 ] call_gate64: not 64-bit code segment in call gate 64
...
00204125142i[CPU0 ] CPU is in compatibility mode (active)
...
00204125142i[CPU0 ] EFER    = 0x00000500
00204125142i[CPU0 ] | RAX=00000000000000542  RBX=0000000068747541
00204125142i[CPU0 ] | RCX=00000000c0000080  RDX=0000000000000000
00204125142i[CPU0 ] | RSP=000000000000ffffe  RBP=0000000000000000
00204125142i[CPU0 ] | RSI=0000000000000000c  RDI=0000000000000028
00204125142i[CPU0 ] | R8=0000000000000000    R9=0000000000000000
00204125142i[CPU0 ] | R10=0000000000000000   R11=0000000000000000
00204125142i[CPU0 ] | R12=0000000000000000   R13=0000000000000000
00204125142i[CPU0 ] | R14=0000000000000000   R15=0000000000000000

```

Рис. 19.3. Сбой при вентильном вызове кода с $L = 0$

Первая строка на рис. 19.3 говорит о том, что вентиль вызова в режиме совместимости не работает. Если он и функционирует, то только для вызова 64-битового кода¹⁰ ($L = 1$).

В отладчике мы впервые видим 64-битовые расширения регистров общего назначения, а с ними еще восемь ранее недоступных регистров: `r8–r15`. Но пользоваться ими в режиме совместимости мы не можем, они доступны только в 64-битовом режиме.

Примечания

1. 32-битовый, но с 64-битовым преобразованием адресов.
2. Это название принято в документации AMD, а Intel предпочитает "IA-32e". В дальнейшем режим совместимости и 64-битовый режим будем называть "длинным" режимом или режимами, что точнее.
3. Регистр `EFER` (Extended Feature Enable Register) относится к группе регистров, специфических для конкретной модели процессора (MSR — Model Specific Registers). Для доступа к ним предназначены команды `wrmsr` и `rdmsr`.
4. Вдруг это значение равно `0x80000000`? Тогда функция `0x80000001` недоступна. Да и сама команда `CPUID` под вопросом, ведь до Pentium ее не было. Можно проверить: попытаться установить и сбросить бит 21 в регистре флагов. Если удалось, значит, процессор поддерживает команду `CPUID`.
5. В файле `bochssrc` в строке `cpu` (поле `model`) задали имя `atom_n270`.
6. Значение `c = 1` в обоих случаях означает удачный исход.
7. В этой программе базовые линейные адреса сегментов данных настроены на ноль.
8. Хотя в документации от AMD не сказано, что этот код можно запустить сразу, без доделок.
9. Понятно почему — ведь вентиль должен позволять вызовы 64-битового кода (`L = 1`), который может находиться за пределами 4 Гбайт.
10. Об этом — в следующей главе.

ГЛАВА 20

```
DB6 BUFFER:
0108 MOV SI,011F
010E INT 3

AX 07A0 IP C 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:
```

Переход в 64-битовый режим

Когда меня аудитор спросил, зачем мы ворвались в квартиру господина Какона, я ему на это ответил просто: "Я полагал, что мы ближе всего познакомимся с господином Каконом, если будем ходить к нему в гости".
(Ярослав Гашек. Похождения бравого солдата Швейка.)

В 64-битовом режиме сегментная модель памяти упраздняется. Взамен воцаряется плоская модель, в которой базовые линейные адреса всех сегментов обнулены, независимо от значений селекторов¹. Пределы сегментов не контролируются². Дескрипторы остались, но в них учитываются главным образом только битовые поля **P** (Present), **DPL** (Descriptor Privilege Level), **L** (Long), **C** (Conforming) и **D** (Data size)³. Адресная информация используется только в вентилях, причем она расширена до 64 бит. Размер системных регистров также увеличен до 64 бит. Для вентиля размер дескриптора — 16 байт, но для данных и кода остался прежним (8 байт), т. к. адресная информация этих дескрипторов игнорируется.

Загрузчик *ld0*

Эксперимент с плоской моделью памяти в *главе 17* был довольно громоздким, потребовался целый набор программ для выполнения промежуточных действий. Проще написать программу-загрузчик⁴, которая считывает com-файл, копирует его в физическую память по абсолютному адресу 0x100 и, обнулив сегментные регистры, передает управление по этому адресу. Базовые линейные адреса загруженной программы равны нулю, поэтому переключение в 64-битовый режим на адресацию не повлияет.

Программа загрузки на ассемблере а86 приведена в листинге 20.1.

Листинг 20.1. Запись файла по адресу 0:0100 (ld0.8)

```

buf_sz    equ      0f800 - buf                                ; (1)

dos        macro                                           ; (2)
            mov      ah, #1                                    ; (3)
            int      021                                       ; (4)
#em                                                ; (5)

```

```

mov     bx, 0                ; stdin                ; (6)
lea     dx, buf              ; (7)
mov     cx, buf_sz           ; (8)
dos     03f                  ; read              ; (9)
jc      >19                  ; (10)

mov     cx, ax               ; b_cnt            ; (11)
jcxz    >19                  ; (12)
cmp     ax, buf_sz           ; (13)
je      >19                  ; (14)

mov     ax, 3                ; cls            ; (15)
int     010                  ; (16)

lea     si, buf              ; (17)
mov     ax, 0                ; (18)
mov     es, ax               ; buf ->        ; (19)
mov     di, 0100             ; -> 0:100      ; (20)
cld                                ; (21)
cli                                ; !             ; (22)
rep     movsb                ; (23)

mov     ds, ax               ; (24)
mov     ss, ax               ; (25)
jmp     0:0100               ; (26)
19:                                ; (27)
dos     04c                  ; exit          ; (28)

buf     db                   ; (29)

```

Начало буфера для чтения файла определено в конце программы (29), а размер в строке (1) задан с учетом того, что стек растет от смещения `0ffffe` вниз, в сторону (29). Файл читается (6–9) из стандартного входного потока, поэтому при вызове `ld0.com` перед именем загружаемого файла надо ставить значок `<`. После чтения проверяем ошибки (10–14), возврат (28) в DOS произойдет при ошибке чтения (10) или если размер файла нулевой (11–12) или максимальный⁵ (13–14). Если проверки пройдены, очищаем экран (15–16) и копируем (17–23) содержимое буфера в начало физической памяти со смещения 0100. Поскольку в реальном режиме в этой области находятся векторы прерываний, то перед началом копирования запрещаем⁶ внешние прерывания (22). Обнуляем сегментные регистры данных и стека (24–25) и передаем управление (26) загруженной `com`-программе.

Выполните трансляцию программы `ld0.8` и переместите файл `ld0.com` в каталог `C:\tools`.

Проверим работу `ld0.com` на простом примере⁷, который приведен в листинге 20.2. Эта программа написана на `nasm`, а расширение `"a"` вместо `"asm"` придумано для

того, чтобы получать не com-, а bin-файл⁸. Такой файл не запустишь случайно, как com-файл, а по расширению *.bin мы подключим⁹ загрузчик ld0.com.

Листинг 20.2. Программа для проверки загрузчика ld0.com (test.a)

```

org      0x100                                ; (1)

mov      ax, 0xb800                            ; (2)
mov      es, ax                                ; (3)
mov      di, 0                                 ; (4)
mov      si, msg                                ; (5)
cld                                             ; (6)
mov      cx, msg_sz                             ; (7)
11:                                             ; (8)
movsb                                          ; (9)
inc      di                                     ; (10)
loop     11                                    ; (11)

19:                                             ; (12)
es mov    [di], al                             ; (13)
inc      al                                    ; (14)
jmp      19                                    ; (15)

msg      db      "Start from 0:100... "        ; (16)
msg_sz   equ     $ - msg                       ; (17)

```

Выполните трансляцию, а затем загрузку:

```

nasm test.a -f bin -o test.bin
ld0 <test.bin.

```

Программа отобразит (2–11) на экране сообщение (16) и уйдет в бесконечный цикл (12–15), выводя (13) справа от сообщения изменяющийся (14) символ.

В качестве теста можно взять любую из рассмотренных ранее программ защищенного режима — кроме тех, которые пишут в начало памяти от адреса 0100. Например, подойдет программа long_1.asm из предыдущей главы. Хотя она копирует результат команды CPUID (функция 0) в начало памяти, но все-таки до адреса 0100. Проверьте: переименуйте long_1.asm в long_1.a, выполните трансляцию в bin-файл и загрузку посредством ld0.com.

С этого момента мы работаем с а-файлами и пользуемся загрузчиком ld0. Поскольку nasm для DOS не поддерживает 64-битовый режим, используем последнюю версию nasm для Windows. Это кросс-компилятор, и его следует запускать не из сеанса Bochs-FreeDOS, а непосредственно в Windows¹⁰. Теперь в образ диска нужно переносить только результаты трансляции, т. е. bin-файлы.

Программа, показанная в листинге 20.3, после включения режимов long переходит к 64-битовому кодовому сегменту.

Листинг 20.3. Переход в 64-битовый режим (jmp64_1.a)

[illegible]

```

        use32                                ; (84)
pm:                                           ; (85)
    ...
long_mode:                                  ; (118)
    _brk                                    ; (119)
#ifdef _SHOW_                                ; (120)
    store    0xb8002, byte [20]              ; (121)
    inc      byte [20]                      ; (122)
#endif                                       ; (123)
    mov      edi, 30                        ; (124)
    jmp      (code64 - gdt):entry64         ; (125)
end:                                         ; (126)

```

В строках (50–56) задан дескриптор 64-битового кода, в котором бит $L = 1$, а $D = 0$ (55).

В промежутке (119–125) действует режим совместимости. В строке (125) задан переход в 64-битовый сегмент кода в программу (12–33), которая выясняет имя производителя (14–18) и выводит его на экран (20–25), а затем в бесконечном цикле (27–33) увеличивает символ в третьем знако-месте экрана. Единственное отличие от 32-битового режима — это применение регистра `r15`, доступного только в 64-битовом режиме. В строках (16–17) используется базовая регистровая адресация со смещением, а в (18) — прямая¹¹.

В отладчике Bochs видно, как в момент перехода в длинный режим изменяется содержимое панели регистров: размерность большинства 32-битовых регистров удвоилась, и появились новые 64-битовые регистры. Команда **View/GDT** показывает наличие 64-битового сегмента кода с нулевым базовым адресом и максимальным значением предела, как предполагалось. Вывод команды **View/Page Table** также обнадеживает.

Можем ли мы вернуться из 64-битового режима в режим совместимости? Для этого, согласно [2, Figure 1-6], следует перейти в кодовый сегмент, в дескрипторе которого признак $L = 0$. Достаточно команды дальнего перехода, где указано значение селектора, с которым мы вошли в режим совместимости. Трудность в том, что 64-битовый режим не поддерживает межсегментные переходы, где адрес задан константой. Поэтому придется воспользоваться косвенным переходом, сначала записав полный адрес (64-битовое смещение и 16-битовый селектор) в память, как показано в листинге 20.4.

Листинг 20.4. Возврат в режим совместимости (jmp64_2.a)

```

    ...
    use64                                ; (11)

addr80 dq    long_mode                  ; (12)
        dw    code32 - gdt              ; (13)

```

```

entry64:                                     ; (14)
    _brk                                     ; (15)
    clr     eax, r15                         ; (16)
    cpuid                                ; vendor      ; (17)
    mov     [0+r15], ebx                    ; (18)
    mov     [4+r15], edx                    ; (19)
    mov     [8], ecx                        ; (20)
#ifdef _SHOW_                               ; (21)
    clr     esi                             ; (22)
    add     edi, 0xb8000                    ; (23)
    mov     ecx, 12                         ; (24)
.ll:      movsb                             ; (25)
    inc     edi                             ; (26)
    loop    .ll                            ; (27)
    store   0xb8004, byte [21]              ; (28)
    inc     byte [21]                       ; (29)
#endif                                     ; (30)
    jmp     far qword [addr80]              ; !      ; (31)
    ...
    use32                                   ; (82)
pm:                                             ; (83)
    ...
long_mode:                                   ; (116)
    _brk                                     ; (117)
#ifdef _SHOW_                               ; (118)
    store   0xb8002, byte [20]              ; (119)
    inc     byte [20]                       ; (120)
#endif                                     ; (121)
    mov     edi, 30                         ; (122)
    jmp     (code64 - gdt):entry64          ; (123)
end:                                           ; (124)

```

Команда (123) включает 64-битовый режим, т. к. ссылается на дескриптор кода с $L = 1$. Напротив, команда (31) возвращает в режим совместимости, поскольку в ней есть ссылка (12–13) на исходный дескриптор кода с $L = 0$. Интересно, что команда (31) передает управление в точку (118), даже если в (13) задать ноль¹². Селектор, указанный в (13), несет информацию о состоянии бита L и ничего более.

Те же действия в листинге 20.5 выполнены с помощью вентиля вызова¹³.

Листинг 20.5. Переключение режимов при помощи вентиля вызова (sg64_1.a)

```

...
;----- 64-bit subroutines                  ; (10)

    use64                                    ; (11)
    align 8                                 ; (12)

```



```

subr_1:                                                    ; (13)
    clr     eax, r15                                       ; (14)
    cpuid                                ; vendor          ; (15)
    ...
    retfq                                ; !              ; (29)

;----- system data                                     ; (30)

    use16                                           ; (31)
    ...

dsc2:                                                    ; (45)
code64:                                                    ; (46)
    dw      0                                           ; (limit low)      ; (47)
    dw      0                                           ; (base low)       ; (48)
    db      0                                           ; (base middle)    ; (49)
    db      1001_1010b  ; P, dpl, S | 1, c, XR, (a)      ; (50)
    db      0010_0000b  ; 0, d, L, 0 | (limit high) !    ; (51)
    db      0                                           ; (base high)      ; (52)

dsc3:                                                    ; (53)
cg1:                                                    ; (54)
    dw      subr_1                                       ; Call gate 64 bit ; (55)
    dw      code64 - gdt  ; L-code selector              ; (56)
    db      0                                           ; (rsrv)           ; (57)
    db      1000_1100b  ; P, dpl, s | Call Gate          ; (58)
    dw      0                                           ; high adr         ; (59)
    dq      0                                           ; 64-bit extension ! ; (60)
    ...

long_mode:                                                ; (121)
.l2:  _brk                                              ; (122)
#ifdef _SHOW_                                             ; (123)
    store   0xb8002, byte [20]                          ; (124)
    inc     byte [20]                                    ; (125)
#endif                                                  ; (126)
    mov     edi, 30                                       ; (127)
    call    (cg1 - gdt):0                                ; !              ; (128)
    jmp     .l2                                           ; (129)
end:                                                    ; (130)

```

В дескрипторе 64-битового вентиля (54–60) задана, в операторах (55) и (59–60), точка входа (13) в подпрограмму, а в строке (56) — селектор 64-битового дескриптора кода (46–52). Обратите внимание на команду возврата (29): именно в таком ее варианте из стека считывается и адрес возврата, и селектор¹⁴.

Убедившись в отладчике, что (29) возвращает нас в режим совместимости, проверим заодно вложенный вызов. Можно обойтись одним вентиляем, если вызывать процедуру (13) рекурсивно, как показано в листинге 20.6.

Листинг 20.6. Вложенный вызов 64-битовой подпрограммы (cg64_2.a)

```

...
#define delay    1000000                                ; (8)
...
    use64                                                ; (12)
    align    8                                          ; (13)

addr80    dq    0                                          ; (14)
          dw    cg1 - gdt                                ; (15)

iter      db    8                                          ; (16)

subr_1:                                          ; (17)
.l1:      _brk                                          ; (18)
#ifdef _SHOW_                                          ; (19)
    mov     al, [iter]                                  ; (20)
    add     al, '0'                                      ; (21)
    mov     [edi + 0xb8000], al                        ; (22)

    mov     rcx, delay                                  ; (23)
.l13:     jmp     .l14                                  ; (24)
.l14:     loop    .l13                                  ; (25)
#endif                                              ; (26)
    cmp     byte [iter], 0                              ; (27)
    ja      .l2                                          ; (28)
    retfq                                         ; (29)
.l12:                                          ; (30)
    dec     byte [iter]                                  ; (31)
    add     edi, 2                                       ; (32)
    call    far qword [addr80]                        ; (33)
    sub     edi, 2                                       ; (34)
    jmp     .l1                                          ; (35)
...

long_mode:                                          ; (127)
.l12:     _brk                                          ; (128)
#ifdef _SHOW_                                          ; (129)
    store   0xb8002, byte [1]                          ; (130)
    inc     byte [1]                                    ; (131)
#endif                                              ; (132)
    mov     edi, 10                                      ; (133)
    call    (cg1 - gdt):0                                ; ! ; (134)
    jmp     .l2                                          ; (135)
end:                                          ; (136)

```

Мы попадаем в точку (17) из режима совместимости (134), как в предыдущей программе. Выводим (20–22) число, заданное в (16), выполняем задержку¹⁵ (23–25),

заданную в (8), а затем, проверив переменную (16) на ноль (27–28), выходим (29) из подпрограммы. Но прежде чем выйти, мы выполняем рекурсивный вызов (33), уменьшая в (31) значение переменной (16) и увеличивая в (32) смещение выводимого символа от начала экрана. Заметим, что вентильный вызов (33) в 64-битовом режиме пришлось задать как косвенный, а прямой вызов (136) доступен только в режиме совместимости.

Применимость 64-битовых данных

Итак, мы вплотную приблизились к 64-битовым данным. Возникает вопрос: в каких приложениях действительно требуется такая разрядность¹⁶, неужели не достаточно 32 бит? Есть ли еще какие-нибудь очевидные применения, кроме криптографии, где данные шифруются 64-битовыми блоками?

Возможное применение — действительные числа с *фиксированной* точкой. Вещественные числа в физических задачах, в том числе в задачах управления, обычно ограничены. Скажем, скорость вращения авиационной турбины не превышает 32 000 об/мин. Если нас устроит точность 1 об/мин, то для представления этой величины достаточно 16-битового числа. Если требуемая точность 0,001 об/мин, то придется либо масштабировать величину (например, 29 567 000 теперь означает 29 567 об/мин, и нам хватит 32-битового целого), либо сразу переходить к формату вещественных чисел с *фиксированной* точкой. Берем 64-битовое слово, целая часть вещественного числа находится в старших 32 битах, а дробная — в младших. Теперь 3.625 выглядит как 000...011 (целая часть в старшей половине) и 1010...00 (дробная часть в младшей половине). Точность — не ниже чем у 64-битового формата с плавающей точкой, хотя диапазон не настолько велик: от 07fff ffff (~2.1e+9) до 1/(07fff ffff) (~4.6e–10). Отрицательные числа задаем в дополнительном коде, как обычно. Арифметика целочисленная¹⁷, операции выполняет центральный процессор, что наверняка быстрее i80x87 и прочих арифметических расширений. При разработке, например, цифровых фильтров, где достаточно операций умножения, сложения, арифметической инверсии или вычитания, выигрыш в производительности несомненный.

Примечания

1. Для `fs` и `gs` сделано исключение: соответствующие сегменты могут иметь ненулевой базовый адрес.
2. Теперь изоляция программ достигается лишь за счет преобразования адресов, без которого длинные режимы не работают.
3. При `L = 1` адрес становится 64-битовым, но размер данных по умолчанию — по-прежнему 32 бита (`D = 0`). Для доступа к 64-битовым данным используется префикс расширения размерности `REX`. Сочетание `L = D = 1` зарезервировано и пока не определено.
4. Можно даже оформить программу как *начальный* загрузчик. Он записывается в нулевой сектор загрузочного диска (USB flash), а BIOS копирует его по адресу

0000:7c00 и передает туда управление. Некоторые самоучители по защищенному режиму основаны на этом варианте, но для начинающих он плох тем, что при использовании нулевого сегмента не чувствуешь разницы между линейным и эффективным адресом. Правда, эта разница, характерная для сегментной модели памяти, в 64-битовом режиме как раз исчезает.

5. Когда размер прочитанного в точности равен размеру, указанному при вызове, может оказаться, что файл вычитан не до конца, и загружать его в таком случае нельзя.
6. Иначе первое же прерывание от системного таймера разрушит нашу конструкцию.
7. Это обычная com-программа, и она может быть выполнена без участия ld0.com, в любом участке памяти. Но те программы, которые мы изготовим в дальнейшем для проверки 64-битового режима, работают только в начале физической памяти и соответственно должны загружаться посредством ld0.com.
8. Реакция файлового менеджера на расширение "a" должна быть:

```
nasm !.! -f bin -o !.bin.
```
9. Реакция файлового менеджера на расширение "bin" должна быть: `ld0 <!.!`
10. В качестве файлового менеджера рекомендуется Far Manager — он подобен тому, которым мы пользовались в FreeDOS, и тоже поддерживает автоматическую подстановку командной строки по расширению файла (настройка по команде <F9>/**Commands/File associations**).
11. Согласно сетевой публикации одного известного в прошлом хакера, прямая адресация в 64-битовом режиме не поддерживается. Я было поверил, потому и начал проверку с косвенной адресации.
12. Только при этом не произойдет обратного переключения в режим совместимости, и последующий 32-битовый код будет интерпретирован как 64-битовый, т. е. неправильно.
13. То, что не получилось в *главе 19*.
14. Оба записаны в стек как 64-разрядные числа, поскольку в 64-битовом режиме стек принимает только значения максимальной размерности.
15. По умолчанию команда (25) использует в качестве счетчика не 32-, а 64-разрядный регистр.
16. Кстати, кто знаком с ассемблером для PLC [6], знает, что при логическом управлении понятие разрядности вовсе упраздняется — процессор однокбитовый. Только когда к обработке подключаются аналоговые величины, в ход идут дополнительные 32-битовые арифметические регистры, и этой разрядности для промышленных динамических систем чаще всего хватает.
17. Сложение, инвертирование и вычитание выполняются за одну команду. Но после умножения (и перед делением) требуются некоторые дополнительные действия. (Попробуйте применить команду двойного сдвига для коррекции результата 64-битового умножения.)

ГЛАВА 21

```
DB6 BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA  i   c   1: =e1,ax,, 10101010
BX 0000  IP 0100 2: =d1,ax,, 1962
CX 0000  CS 6309 3:
DX 6309  SS 6309 4:
SI 011F  DS 6309 5: r5,b_1,, AB ab
DI 0000  ES 6309 6: r5,si,, AB ab
BP 0000  SP FFFE 0:

lea si,b_1
```

Особенности 64-битового режима

...Как арбуз я теперь, и даже
В дверь на улицу не пролажу!
(Дина Бурачевская. Диета)

Отличий 64-битового режима от 32-битового защищенного режима довольно много:

- ❑ Существующие регистры расширены до 64 бит и появились новые 64-битовые регистры r8–r15.
- ❑ Стек принимает только 64-битовые данные.
- ❑ Сегментов больше нет, за исключением fs и gs, у которых базовый адрес может отличаться от нуля. Для всех остальных селекторов память начинается с нулевого линейного адреса, а предел расширен до максимума. Из полноценных дескрипторов остались: вентили¹, TSS и LDT²; их размер увеличен вдвое и составляет 16 байт, т. к. в них должно быть место для 64-битового адреса. В дескрипторе кода остались только атрибуты P, DPL, L, D и C, в дескрипторе стека — P и DPL.
- ❑ Отсутствует команда прямого межсегментного перехода³.
- ❑ Появилась пара команд SYSCALL и SYSRET для обращения к привилегированным процедурам.
- ❑ В вентилях исключений и прерываний появился индекс IST. При IST = 0 переключение стека происходит так же, как в защищенном режиме, т. е. при изменении уровня привилегий. При IST = 1...7 выбирается одно из семи значений указателя стека — из текущего TSS.
- ❑ Переключение задач упразднено, но один экземпляр TSS используется для хранения указателей стека исключений и прерываний, а также для настройки ограничений доступа к портам ввода-вывода при CPL > 0.
- ❑ При обработке исключений и прерываний в стеке *всегда* сохраняются копии ss и rsp, даже если текущий уровень привилегий остается прежним.
- ❑ Ограничения доступа к данным сняты: любые данные доступны с любого уровня привилегий⁴.


```

        mov     rax, rdx                                ; (38)
        or      rax, 0x89ab_cdef                        ; 0xffff_ffff_89ab_cdef ? ; (39)

        mov     rax, rdx                                ; (40)
        mov     edx, 0x89ab_cdef                        ; 0x0000_0000_89ab_cdef ; (41)
        or      rax, rdx                                ; (42)

%if 0                                           ; (43)
illegal:                                       ; (44)
        mov     ah, sil                                ; (45)
        mov     r8b, ah                                ; (46)
        mov     r0, rdx                                ; r0 ? ; (47)
        mov     r8, rip                                ; rip ? ; (48)

        jcxz    .l1                                    ; (49)
.l1:      into                                       ; (50)
        bound   ax, [mem]                            ; (51)

%endif                                           ; (52)
mem64:                                         ; (53)
        mov     edx, [rel mem]                        ; relative ; (54)
        default rel                                    ; (55)
        mov     eax, [mem]                            ; (56)
        mov     rax, [mem]                            ; (57)

        mov     ecx, [abs mem]                        ; (58)
        default abs                                    ; (59)
        mov     eax, [mem]                            ; absolute ; (60)
        mov     rax, [mem]                            ; (61)

_rcx:                                           ; (62)
        mov     rcx, 0x1_0000_0000                    ; (63)
        jecxz   .l1                                    ; ecx ; (64)
        nop                                           ; (65)
.l1:      jrcxz   .l2                                    ; (66)
        nop                                           ; (67)
.l2:      inc     rcx                                    ; 0x1_0000_0001 ; (68)
        loop    .l3                                    ; rcx ; (69)
        nop                                           ; (70)
.l3:      inc     rcx                                    ; (71)
        a32 loop .l4                                    ; ecx ; (72)
        nop                                           ; (73)
.l4:                                           ; (74)
        ...

```

Составляющие регистров `r8–r15` обозначены суффиксом `d` (dword, младшие 32 бита), `w` (word, младшие 16 бит) или `b` (byte, младшие 8 бит). Примеры приведены в (24–26). В строках (28) и (45) — ссылка на младший байт регистра `si`, возможная лишь в 64-битовом режиме.

В строках (45–51) — примеры недопустимых обозначений (47–48) и сочетаний операндов (45–46), а также упраздненных команд (49–51). (Чтобы проверить, нужно в директиве (43) исправить ноль на единицу и четыре раза повторить трансляцию, по одной убирая команды под меткой (44), т. к. `nas` останавливается на первой ошибке.) В `nas` не поддерживаются⁵ обозначения `r0...r7` и `rip`. Привычные сочетания младшего и старшего байтов 16-битового регистра допустимы (27), хотя новшества (45–46) более ограничены в этом отношении.

Обратите внимание, что при записи (19), (39) и (41) в младшие 32 бита 64-битового регистра его старшая часть обнуляется. Это касается не только команд копирования, но и других — например, (25). Напротив, запись младших 8/16 бит в 16/32-битовый регистр не влияет на старшую часть регистра. Вопрос: как поместить в 64-битовый регистр 32-битовые составляющие?

В строках (30–32) показано, как записать в 32-битовый регистр его 16-битовые части. Этот способ работает, потому что при выполнении (32) старшая часть 32-битового регистра сохраняется. Аналогичные операции (35) и (37) над 64-битовым регистром не пройдут, т. к. (37) обнулит старшие 32 бита. А если применить дизъюнкцию (39), непосредственный операнд искажается при трансляции⁶. В конце концов, при помощи (40–42) цель достигнута.

В строках (63–72) исследуются команды, неявно использующие регистр `cx/ecx/rcx`. Размерность может быть задана в мнемонике команды, как в (64) и (66), или при помощи префикса (72).

Команды (53–61) демонстрируют два варианта прямой адресации данных. По умолчанию для 64-битового режима `nas` генерирует эффективный адрес, т. е. смещение от начала сегмента. Это абсолютная адресация. При относительной адресации в коде тоже задано смещение ячейки памяти, но оно отсчитывается от текущей команды⁷. Вариант адресации можно выбрать для одной команды, как в (54) и (58), или задать по умолчанию, как в (55) и (59).

Эксперименты с относительной адресацией и перемещаемыми программами мы выполним чуть позже, а пока обратимся к машинному коду рассмотренной программы. Отображение машинного кода, приведенное в листинге 21.2, скопировано с экрана отладчика `Bochs`.

Листинг 21.2. Машинный код 64-битовых команд из листинга 21.1

```

...
(7) 48C7C2FEFFFFFF      mov rdx, 0xfffffffffffffffe ; (18)
(5) BAFEFFFFFFF        mov edx, 0xfffffffffe       ; (19)
(4) 66BA0200           mov dx, 0x0002             ; (20)
(7) 48C7C0FEFFFFFF      mov rax, 0xfffffffffffffffe ; (21)
(18) 48BAEFCDA8967452381  mov rdx, 0x0123456789abcdef ; (22)
(18) 48B9BBB4444AAAA5555  mov rex, 0x5555aaaa4444bbbb ; (23)
...
(6) 480DEFCDA89        or rax, 0xfffffffff89abcdef ? ; (39)
...
```



```

(6) 8B157FFFFFFF      mov edx, dword ptr ds:[rip-129]; (54)
...
(6) 8B0579FFFFFF      mov eax, dword ptr ds:[rip-135]; (56)
(7) 488B0572FFFFFF      mov rax, qword ptr ds:[rip-142]; (57)
...
(7) 8B0C250B010000     mov ecx, dword ptr ds:0x10b    ; (58)
...
(7) 8B04250B010000     mov eax, dword ptr ds:0x10b    ; (60)
(8) 488B04250B010000     mov rax, qword ptr ds:0x10b    ; (61)
...

```

Слева в скобках — длина команды в байтах, затем — их содержимое в шестнадцатеричном коде, результат дизассемблирования, а также номера операторов⁸ из листинга 21.1. Как видно из (18), (21) и (39), если непосредственный операнд 64-битовой команды содержит единицы в старшей части, то в машинном коде он сокращается вдвое. Смещение в командах с относительной адресацией (54–57) — это знаковое 32-битовое значение⁹, а эффективный адрес в командах с абсолютной адресацией (58–61) — тоже 32-битовое, пока данные находятся в пределах 4 Гбайт. И даже в этих пределах команды с относительной адресацией на байт короче.

Относительная адресация и перемещаемость

При относительной адресации программа становится перемещаемой. Если все адреса переходов и адреса данных заданы как расстояния от команд, то какая разница, по каким абсолютным адресам эти объекты расположены? Пока расстояния внутри их совокупности неизменны, ее можно целиком перемещать по памяти. Для проверки этой возможности нужно выделить перемещаемую часть 64-битового кода и после запуска программы скопировать его в другую область памяти, а затем передать туда управление.

Исходный, непереключаемый вариант приведен в листинге 21.3, включение режима относительной адресации (106) пока что закомментировано.

Листинг 21.3. Абсолютная прямая адресация (abs64.a)

```

...
use64                                     ; (105)
; default rel                          ; default abs! ; (106)

align 0x100                             ; (107)
start64:                               ; (108)

vendor resd 3                           ; (109)

entry64:                               ; (110)
    _brk                               ; (111)

```

```

        clr     eax                                ; (112)
        cpuid                                     ; (113)
        mov     [vendor], ebx                     ; (114)
        mov     [vendor + 4], edx                 ; (115)
        mov     [vendor + 8], ecx                 ; (116)
#ifdef _SHOW_                                     ; (117)
        lea     esi, vendor                       ; (118)
        add     edi, 0xb8000                      ; (119)
        mov     ecx, 12                          ; (120)
.11:     movsb                                     ; (121)
        inc     edi                               ; (122)
        loop    .11                              ; (123)
#endif                                           ; (124)

        add     edi, 20                          ; (125)
.12:                                           ; (126)
        _brk                                     ; (127)
        inc     qword [str8]                     ; (128)
#ifdef _SHOW_                                     ; (129)
        lea     esi, str8                         ; (130)
        mov     ecx, 8                           ; (131)
        push    rdi                               ; (132)
.13:                                           ; (133)
        mov     al, [esi]                         ; (134)
        mov     [edi], al                        ; (135)
        inc     esi                               ; (136)
        sub     edi, 2                           ; (137)
        loop    .13                              ; (138)

        pop     rdi                               ; (139)
#endif                                           ; (140)
        jmp     .12                              ; (141)

str8     db     '00000000'                       ; (142)

end64:                                         ; (143)
end:                                           ; (144)

```

Код 64-битового режима перенесен в конец программы и выровнен¹⁰ (107) так, что смещение метки (108) равно 0x200. После загрузки посредством `ld0.com` этот 64-битовый код запишется в память с линейного адреса 0x300. Результат (113) сохраняется теперь (114–116) в локальных данных (109). Также в локальных данных определен 64-битовый счетчик (142), в цикле (126–141) его возрастающее значение выводится на экран.

Кстати, попробуйте в строках (132) и (139) задать 32-битовый регистр¹¹.

Режим абсолютной адресации действует в `asm` по умолчанию. Проверив работу программы `abs64`, удалите комментарий в начале строки (106), выполните трансля-

цию и проверьте еще раз. В отладчике видно, что (109) в командах (114–116) адресуется отрицательным смещением, а (142) в команде (128) — положительным. В строках (118) и (130) адрес операнда тоже задан смещением относительно указателя инструкции. Все эти команды перемещаемые.

Теперь добавим код, который копирует операторы (109–142) в область памяти, находящуюся на 0x100 байтов впереди (листинг 21.4). Когда мы будем переходить на 64-битовый код, то зададим точку входа, смещенную на 0x100.

Листинг 21.4. Перемещаемая 64-битовая программа (rel64.a)

```

...
#define reloc_ofs 0x100                                ; (8)
...
use32                                                    ; (61)
pm:                                                      ; (62)
...
reloc_sz equ     end64 - start64                        ; (71)

reloc_64_bit_code:                                     ; (72)
    std                                                  ; (73)
    mov     esi, start64 + reloc_sz - 1                 ; (74)
    lea     edi, [esi + reloc_ofs]                      ; (75)
    mov     ecx, reloc_sz                               ; (76)
    rep     movsb                                       ; (77)
    mov     al, 0x55                                    ; (78)
    mov     ecx, reloc_ofs                              ; (79)
    rep     stosb                                       ; (80)
    cld                                               ; (81)
    ...
    use64                                             ; (117)
    default rel                                       ; (118)

    align   0x100                                     ; (119)
start64:                                             ; (120)

vendor     resd     3                                  ; (121)
...
str8       db       '00000000'                        ; (154)
end64:                                           ; (155)

```

Копирование выполняется в 32-битовом режиме, сразу после настройки селекторов. В строке (71) вычисляется размер массива-источника, а (8) задает расстояние между массивами. В общем случае не исключено перекрытие массивов (источника и приемника), поэтому копируем от конца массива к началу. Для этого в (73) задаем обратное направление для строковых команд (77) и (80), а в (74–75) устанавливаем указатели на последние байты источника и приемника. После копирова-

ния (77) заполняем (78–80) кодом 0x55 ('U') промежутком между массивами и сам массив-источник.

На рис. 21.1 показана память после перемещения программы (120–154) от адреса 0x300 на 0x100 байтов вперед, программа уже вывела название производителя (121) и три раза увеличила счетчик (154) — на рис. 21.1 они выделены.

0x300	55555555	55555555	55555555	55555555	UUUUUUUUUUUUUUUU
0x310	55555555	55555555	55555555	55555555	UUUUUUUUUUUUUUUU
...					
0x3F0	55555555	55555555	55555555	55555555	UUUUUUUUUUUUUUUU
0x400	68747541	69746E65	444D4163	31DB8766	AuthenticAMD f..1
...					
0x460	DAEB5FF3	30303033	30303030		. ..30000000

Рис. 21.1. Перемещение программы при относительной адресации

Можете проверить другое значение в строке (8): например¹², 1 или 300. Копирование с перекрытием пройдет корректно, результат выполнения не должен отличаться от варианта в листинге 21.4.

Если вернуться к абсолютной адресации, закомментировав (118), то окажется, что перемещенная программа, обращаясь к данным, промахивается и попадает в первоначальные адреса. Как показывает рис. 21.2, по адресу 0x400, куда должна быть записана строка, остались нули. Строка записалась по адресу 0x300. Счетчик (154) тоже оказался на прежнем месте, поэтому вывод на экран начинается не с нулей, а с буквы 'U' (0x55).

0x300	60747541	69746E65	444D4163	55555555	AuthenticAMD UUUUU
0x310	55555555	55555555	55555555	55555555	UUUUUUUUUUUUUUUU
...					
0x360	55555555	55555555	55 60 5555	55555555	UUUUUUUUUU'UUUUU
...					
0x3F0	55555555	55555555	55555555	55555555	UUUUUUUUUUUUUUUU
0x400	00000000	00000000	00000000	31DB8766	E.1
...					
0x460	EF83C6FF	5FF3E202	3030D8EB	30303030	_..000000
0x470	00003030				00

Рис. 21.2. Попытка перемещения при абсолютной адресации

Мы убедились, что программа с относительной адресацией может быть размещена в любой области памяти, а с абсолютной — только по фиксированному адресу, который задается заранее при трансляции.

Селекторы *fs* и *gs*

Теперь проверим, что для сегмента, доступного через селектор *fs/gs*, базовый линейный адрес может быть ненулевым — даже в 64-битовом режиме. Установим базовый адрес, не равный нулю, как обычно, т. е. в защищенном режиме через дескриптор в GDT (листинг 21.5). Выясним, что станет с адресом после входа в 64-битовый режим.

Листинг 21.5. Первый вариант установки базового адреса сегментов *fs/gs* (fsgs_1.a)

```

...
#ifdef _SHOW_                                ; (3)
#define sel fs                                ; (4)
#endif                                        ; (5)

...
use64                                         ; (14)

vendor resd 3                                ; (15)

entry64:                                     ; (16)
    _brk                                     ; (17)
    clr eax                                  ; (18)
    cpuid                                    ; (19)
    mov [vendor], ebx                        ; (20)
    mov [4+vendor], edx                      ; (21)
    mov [8+vendor], ecx                     ; (22)
#ifdef _SHOW_                                ; (23)
    mov esi, vendor                          ; (24)
    mov ecx, 12                              ; (25)
    mov edi, ecx                             ; (26)
.ll1:                                        ; (27)
    mov al, [esi]                            ; (28)
    sel mov [edi], al                         ; (29)
    inc esi                                  ; (30)
    add edi, 2                               ; (31)
    loop .ll1                               ; (32)

    add edi, 2                               ; (33)
#endif                                        ; (34)

.ll2:                                        ; (35)
    _brk                                     ; (36)
#ifdef _SHOW_                                ; (37)
    sel inc byte [edi]                       ; (38)
#endif                                        ; (39)
    jmp .ll2                                ; (40)
...

```

```

dsc4: ; (72)
video: ; Video buffer ; (73)
    dw    25*80*2 - 1 ; limit low ; (74)
    dw    0x8000 ; base low ; (75)
    db    0xb ; base middle ; (76)
    db    1001_0010b ; P, dpl, S | Data, e, RW, a ; (77)
    db    0100_0000b ; g, B, 00 | limit high ; (78)
    db    0 ; (79)
    ...
start_long: ; (123)
    set_cr4_pae ; (124)
    set_cr3_pml4 ; (125)
    paging_on ; (126)

#ifdef _SHOW_ ; (127)
    set_seg_sel, video - gdt ; (128)
    mov    cl, '0' ; (129)
#endif ; (130)

waiting: ; (131)
.ll: ; (132)
#ifdef _SHOW_ ; (133)
    sel mov [0], cl ; (134)
    inc    cl ; (135)
#endif ; (136)
    chk_efer_lma ; (137)
    jnc    .ll ; (138)

long_mode: ; (139)
    _brk ; (140)
#ifdef _SHOW_ ; (141)
    sel mov [2], cl ; (142)
    inc    cl ; (143)
#endif ; (144)
    jmp    (code64 - gdt):entry64 ; (145)
end: ; (146)

```

В дескрипторе (73–79) определены расположение (75–76) и размер (74) видеобуфера. В строке (128) по этому дескриптору мы настраиваем селектор, заданный в (4), и сразу используем его для вывода на экран в (134) и (142), а также, после перехода в 64-битовый режим, в (29) и (38).

В 64-битовом режиме атрибуты и размер сегмента, заданные в (73–79), не должны приниматься в расчет. Попробуйте исправить число в (74) на двойку. (Меньше нельзя, т. к. (134) и (142) все-таки выполняются в защищенном режиме и в режиме совместимости, где предел сегмента контролируется.) И хотя операторы (29) и (38)

обращаются к видеобufferу по смещению больше двух, сбой в 64-битовом режиме не происходит.

Проверьте работу программы, исправив в (4) селектор `fs` на `gs`, а потом на `es`.

В этом варианте базовый адрес `fs/gs` ограничен величиной 4 Гбайт, а его установка возможна только в режиме совместимости и в защищенном режиме. В листинге 21.6 приведены два других варианта, не использующих дескриптор сегмента данных и применимых в 64-битовом режиме.

Листинг 21.6. Два варианта перезаписи базового адреса сегмента `fs/gs` (`fsgs_2.a`)

```

...
#define FS_base 0xc000_0100                ; (3)
#define GS_base 0xc000_0101                ; (4)
...
use64                                     ; (13)
...
%if 1                                     ; (24)
    mov     ecx, FS_base                   ; (25)
    rdmsr   ; edx:eax                       ; (26)
    add     eax, 80*2                       ; (27)
    wrmsr   ; (28)
%else                                     ; (29)
    mov     rax, cr4                       ; cr4.sfgsbase ; (30)
    bts     rax, 16                        ; (31)
    mov     cr4, rax                       ; ?           ; (32)

    rdfsbase rax                          ; (33)
    add     rax, 80*2                      ; (34)
    wrfsbase rax                          ; (35)
%endif                                    ; (36)
...

```

В этом примере мы читаем базовый адрес сегмента `fs`, прибавляем 160 и записываем обратно. В результате базовый адрес становится равным `0xb8160` и последующий вывод на экран "съезжает" на одну строку вниз.

Изначально в (24) выбран вариант (25–28), где доступ к базовым адресам `fs/gs` выполняется через MSR с адресами `0xc000_0100` (`FS.base`) и `0xc000_0101` (`GS.base`). Номер регистра для команд чтения (26) и записи (28) задан в `ecx`, а данные — в паре `edx:eax`. Этот вариант подходит и для защищенного режима, и для 64-битового, но требует `CPL = 0`.

Чтобы выбрать альтернативный вариант (30–36), исправьте в (24) единицу на ноль. В этом варианте мы сначала устанавливаем бит 16 в регистре `CR4`, чтобы получить доступ к четырем командам `{rd|wr}{fs|gs}base` для чтения-записи 64-битового адреса через регистр `rax`. Эти команды есть только в 64-битовом режиме¹³, но зато, согласно документации, они работают при любом `CPL`. Однако на команде (33) про-


```

        dw      0                ; base low                ; (52)
        db      0                ; base middle       ; (53)
        db      1001_0010b       ; P, dpl, S | Data, e, RW, a ; (54)
        db      1100_1111b       ; G, B, 00 | limit high (max) ; (55)
        db      0                ;                    ; (56)
dsc3:                                     ; (57)
code64:                                     ; OS's 64-bit code   ; (58)
        dw      0                ; (limit low)       ; (59)
        dw      0                ; (base low)        ; (60)
        db      0                ; (base middle)     ; (61)
        db      1001_1010b       ; P, dpl, S | 1, c, XR, (a)   ; (62)
        db      0010_0000b       ; 0, d, L, 0 | (limit high)   ; (63)
        db      0                ; (base high)       ; (64)

dsc4:                                     ; OS's stack        ; (65)
flat2:                                     ; Flat data         ; (66)
        dw      -1               ; limit low (max)    ; (67)
        dw      0                ; base low          ; (68)
        db      0                ; base middle       ; (69)
        db      1001_0010b       ; P, dpl, S | Data, e, RW, a ; (70)
        db      1100_1111b       ; G, B, 00 | limit high (max) ; (71)
        db      0                ;                    ; (72)

gdtr:                                     ; (73)
        dw      $ - gdt - 1      ; size(gdt)         ; (74)
        dd      gdt              ; + laddr(cs)       ; (75)

;-----                                     ; (76)
star_high:                               ; (77)
        dw      code64 - gdt      ; syscall_cs        ; (78)
        dw      code0  - gdt      ; sysret_cs         ; (79)
        ...

long_mode:                               ; (129)
.l12:  _brk                             ; (130)
        call    syscall_setup        ; (131)
.l13:  _brk                             ; (132)
#ifdef _SHOW_                             ; (133)
        store   0xb8002, byte [20]    ; (134)
        inc     byte [20]             ; (135)
#endif                                   ; (136)
        mov     edi, 30                ; video buffer offset ; (137)
        syscall                                ; !                 ; (138)
%if 0                                     ; (139)
        jmp     .l12                   ; (140)
%else                                     ; (141)
        jmp     .l13                   ; (142)
#endif                                   ; (143)

```

```

;----- Compatibility mode subroutines ; (144)

syscall_setup: ; (145)
    mov     ecx, star ; (146)
    clr     eax ; star(low) ; (147)
    mov     edx, [star_high] ; (148)
    wrmsr ; (149)

    mov     ecx, cstar ; (150)
    clr     edx ; high ; (151)
    mov     eax, subr_1 ; (152)
    wrmsr ; (153)

%if 0 ; (154)
    mov     ecx, lstar ; (155)
    clr     edx ; high ; (156)
    mov     eax, subr_1 ; (157)
    wrmsr ; (158)
%endif ; (159)
    mov     ecx, sfmask ; (160)
    clr     eax, edx ; (161)
    wrmsr ; (162)

    set_efer_sce ; enable syscalls ; (163)
    ret ; (164)
end: ; (165)

```

Начнем разбор листинга 21.7 с процедуры (145), которая вызывается из (131) — для подготовки к системному вызову и возврату. В [2, Figure 6-1] показана группа регистров MSR для настройки точки входа и точки выхода.

В регистре `STAR` мы обнуляем (147) младшие 32 бита, т. к. 32-битовый вариант вызова нам без надобности. В старшую часть записываем из (77–79) сведения о том, какой дескриптор кода использовать при входе, а какой — при выходе. Согласно (78–79) мы уйдем в 64-битовый код, а вернемся в режим совместимости. Однако в `STAR` каким-то образом должны быть определены также входное и выходное значения селектора стека [2, Figure 6-1]. Они не заданы явно, а получаются прибавлением числа 8 к значениям селектора кода, указанным в `STAR`. Это значит, что дескриптор стека должен следовать в GDT сразу за дескриптором кода¹⁶, и всего должно быть две такие пары: код (41) и стек (49) для возврата и код (57) и стек (65) для вызова.

Поскольку вызов идет из режима совместимости, мы пишем (152) точку входа в процедуру (13) в регистр `CSTAR`, а не в `LSTAR`. Затем инициализируем (160–162) регистр `SFMASK`, который используется при вызове для маскирования регистра флагов: если в каком-то бите `SFMASK` единица, то этот бит в регистре флагов будет сброшен. Устанавливаем (163) в регистре `EFER` нулевой бит (SCE — System-Call Extension), разрешая там самым использование системных вызовов.

После завершения подпрограммы подготовки следует вызов (138). При его выполнении в регистре `rcx` сохраняется адрес возврата, загружаются селекторы кода и

стека (согласно настройке в `STAR`), адрес перехода берется из `CSTAR`. Что бы ни было задано в селекторах, базовый адрес устанавливается равным нулю, предел — 4 Гбайт, `CPL = 0`, разрешено чтение кода, а также чтение и запись в стек, а сам стек — 64-битовый, адресуемый `rsp`. В регистре `r11` сохраняется текущее значение регистра флагов, после чего выполняется его маскирование с использованием `SFMASK`.

Войдя в процедуру (13), мы сохраняем (14–15) регистры, содержащие информацию для возврата, и восстанавливаем их (31–32) перед выходом (33). Как выяснится из следующего примера, команда возврата в том виде, как она записана в (33), переключает нас в режим совместимости.

После возврата мы выполняем команду (142), которая повторяет вызов (138), но уже без инициализации. Попробуйте заменить 0 на 1 в (139), и вы получите сбой на команде (149). Причина в том, что после возврата мы оказались на уровне `CPL = 3`, а (149) возможна лишь при `CPL = 0`. Таков побочный эффект команды (33): она принудительно устанавливает `CPL = 3`, команда (138) тоже устанавливает `CPL = 0`¹⁷.

Итак, системный вызов (138) в нашем примере может быть настроен лишь один раз, пока `CPL = 0`. Значит, точка входа в операционную систему всего одна, но этого может быть достаточно¹⁸.

Плохо лишь то, что в результате возврата `CPL` не совпадает с `DPL` (46). Команда (33) действует в обход условия выполнения кода `DPL = CPL`, но этот код работает лишь до первой проверки привилегий. Стоит добавить сюда, например, обработку исключений, и при первом же возврате в главную программу селекторы данных обнулятся, т. к. их исходные значения, сформированные при `CPL = 0`, непригодны для `CPL = 3`.

Оставим программу в листинге 21.7 с ее недостатками, а условие `CPL = DPL` для сегментов кода и стека учтем в следующем примере, который приведен в листинге 21.8.

Листинг 21.8. Системный вызов из 64-битового режима (sys_2.a)

```

...
;----- 64-bit mode                                     ; (10)

        bits 64                                           ; (11)

syscall_setup:                                           ; (12)
...
mov     ecx, lstar                                       ; (17)
clr     edx                                             ; (18)
mov     eax, subr_1                                     ; (19)
wrmsr                                       ; (20)
...
ret                                             ; (25)

```

```
subr_1:                                ; "system" subroutine                ; (26)
...
    o64 sysret                        ; return to 64-bit mode                ; (34)

entry64:                              ; CPL = 0                            ; (35)
...
    _brk                             ; (50)
    call    syscall_setup              ; (51)
.12:                                     ; (52)
    _brk                                ; CPL = 3                            ; (53)
    syscall                            ; (54)
    inc     byte [0]                   ; (55)
    jmp     .12                        ; (56)
...
```

Здесь вызов идет уже из 64-битового режима. В нем же выполняется и подготовка (12) к системному вызову¹⁹. Отличие от предыдущего примера лишь в том, что адрес точки входа помещается (17) в регистр `LSTAR`. Последующий системный вызов (54) в 64-битовом режиме выполняется точно так же, как в режиме совместимости. Но для возвращения в 64-битовый режим в команде (34) должен быть префикс расширения размерности операнда.

В табл. 21.1 показано расположение дескрипторов с учетом того, что при возврате в 64-битовый режим в селектор кода записывается не первый, а третий дескриптор²⁰ (а в селектор стека — по-прежнему второй). Данные и стек объединены.

Таблица 21.1. Порядок дескрипторов при выполнении системного вызова

Номер	Размерность	Содержание	DPL
1	32	данные и стек	0
2	32	код	0
3	32	данные и стек	3
4	64	код	3
5	64	код	0
6	32	данные и стек	0

Группа 5–6 служит для системного вызова, а группа 2–4 — для возврата. Так как для возврата в 64-битовый режим дескриптор 2 не требуется, он используется в 32-битовых режимах с `DPL = 0`. А дескриптор 1 для 32-битовых данных и стека можно было поместить и в конце, и в середине GDT между группами 2–4 и 5–6.

Теперь на основе примера из листинга 21.8 рассмотрим TSS и прерывания.

Сегмент задачи в 64-битовом режиме

Аппаратная мультизадачность в 64-битовом режиме упразднена, но один экземпляр TSS необходим для хранения указателей стека. У задачи может быть несколько отдельных стеков: три для привилегий уровня 0–2 и еще семь для исключений и прерываний классов 1–7, о чем чуть позже. Также при помощи TSS настраиваются ограничения доступа к портам ввода-вывода. Структура TSS в 64-битовом режиме претерпела значительные изменения [2, Figure 12-8].

В листинге 21.9 представлена программа из листинга 18.8, адаптированная к 64-битовому режиму. Эта программа ограничивает права доступа к портам ввода-вывода для пользовательского уровня. В начале выполняется код из листинга 21.8, чтобы перейти в 64-битовый режим с CPL = 3.

Листинг 21.9. Настройка ограничений доступа к портам (tss64.a)

```

...
subr_1:                                ; CPL = 0                                ; (27)
...
sysretq                                ; (32)

io_chk:                                ; CPL = 3                                ; (33)
    mov     ecx, ports                  ; (34)
    clr     dx                          ; (35)
#ifdef _SHOW_                          ; (36)
    mov     edi, 0xb8000 + 160          ; (37)
#endif                                ; (38)
.ll:                                    ; (39)
    _brk                                    ; (40)
#ifdef _SHOW_                          ; (41)
    mov     byte [rdi], '?'            ; (42)
#endif                                ; (43)
    in      al, dx                      ; (44)
    inc     dl                          ; (45)
#ifdef _SHOW_                          ; (46)
    mov     byte [rdi], '*'            ; (47)
    add     edi, 2                      ; (48)
    test    dl, 7                      ; (49)
    jnz     .l2                        ; (50)
    add     edi, 2                      ; (51)
.ll:                                    ; (52)
#endif                                ; (53)
    loop    .ll                        ; (54)
    ret                                    ; (55)
; (56)
entry64:                                ; CPL = 0                                ; (57)
    _brk                                    ; (58)
    mov     rbx, tss0                  ; (59)

```

```

mov     qword [_rsp0 + rbx], 0xf100                ; (60)
mov     qword [_rsp1 + rbx], 0xe100                ; (61)
mov     qword [_rsp2 + rbx], 0xd100                ; (62)
mov     word [_iomapb + rbx], iopb - tss0          ; !      ; (63)

mov     eax, ts_work - gdt                        ; (64)
ltr     ax                                       ; (65)

call    syscall_setup                            ; (66)
syscall                                ; (67)
call    io_chk                                ; CPL = 0      ; (68)
.l2:    jmp     .l2                            ; (69)
...

dcs7:                                       ; (125)
ts_work:                                   ; TSS, level 0      ; (126)
    dw     tss0_sz                            ; limit low        ; (127)
    dw     tss0                                ; base low         ; (128)
    db     0                                  ; base middle      ; (129)
    db     1000_1001b                          ; P, dpl, s | free TSS ; (130)
    db     0100_0000b                          ; g, 32 bit, 00 | limit high (0) ; (131)
    db     0                                  ; (132)
    dq     0                                  ; 64-bit extension ; (133)
    ...
tss0    resb    tss64_size                      ; (138)
iopb    db     0000_0000b                        ; 0-7              ; (139)
        db     1000_0000b                        ; 8-15             ; (140)
%if 0                                       ; (141)
        db     -1                                ; (142)
%endif                                       ; (143)
tss0_sz equ    $ - tss0                      ; (144)
...

```

В `nasm.inc` добавлено определение структуры `tss64` согласно [2, Figure 12-8]. Память под эту структуру зарезервирована в (138), к ней примыкает (139–140) массив `IOPB`. Адрес (138) используется в (128), а размер (144) массива `IOPB` учитывается при задании предела (127)²¹.

В строках (60–62) формируем в структуре `TSS` значения указателей стека для уровней привилегий 0–2. Эти указатели используются для переключения стека при изменении `CPL`²². Команда (63) записывает в `TSS` смещение массива `IOPB` от начала `TSS`. Командами (64–65) записываем дескриптор (126) в регистр текущей задачи.

Затем следует подготовка (66) к системному вызову и сам вызов (67). Макрос (32) для возврата в 64-битовый режим определен в `nasm.inc`. После возврата из процедуры (27) при `CPL = 3` вызываем (68) подпрограмму (33) проверки портов. Число портов используется командой (34), и начинаем тест с порта с нулевым номером (35). Перед доступом (44) к порту выводим (42) знак вопроса в позиции экрана, со-

ответствующей номеру порта. После успешного выполнения (44) в этой же позиции пишем знак "*". Если все порты прочитаны, на экране останется последовательность литер "*" с чередующимися цветами. В случае сбоя в конце прерванной последовательности останется знак вопроса.

Сейчас числа в (139–140) и (34) таковы, что сбой при выполнении подпрограммы (33) обеспечен. Исправьте эти числа так, чтобы сбоя не было или чтобы он произошел, например, при чтении порта не с номером 15, а 30 или, скажем, 41.

Обработка прерываний в 64-битовом режиме

Принципиальных отличий от 32-битового защищенного режима нет, разница — в размере вентиля. Далее приведены три примера обработки исключений (листинги 21.10–21.12)²³.

Листинг 21.10. Обработка прерываний 0 и 1 (trap_1.a)

```

...
#define scr_ofs 160 ; (8)
#define max_int 1 ; (9)
#define int_limit 10 ; (10)
...
subr_1: ; CPL = 0 ; (30)
    _brk ; (31)
    int 0 ; (32)
    int 1 ; (33)
    ...
    sysretq ; (37)

#ifdef _SHOW_ ; (38)
fill_scr: ; (39)
    mov edi, 0xb8000 + scr_ofs ; (40)
    mov ecx, max_int + 1 ; (41)
    mov ax, '0' + (7 << 8) ; (42)
    cld ; (43)
    rep stosw ; (44)
    ret ; (45)
#endif ; (46)

entry64: ; CPL = 0 ; (47)
    _brk ; (48)
#ifdef _SHOW_ ; (49)
    call fill_scr ; (50)
#endif ; (51)
...
mov word [_iomapb + rbx], iopb - tss0 ; ! ; (56)
...

```

```

        syscall                                ; (60)
        int     0                            ; CPL = 3      ; (61)
        int     1                            ; (62)
        int     1                            ; (63)
%if 0                                         ; (64)
        clr     eax                          ; (65)
        div     al                            ; int 0         ; (66)
%endif                                       ; (67)
        ...
; ----- isr64                             ; (74)

isr0:   push    rax                          ; (75)
        clr     rax                          ; (76)
        jmp     isr                          ; (77)

isr1:   push    rax                          ; (78)
        mov     eax, 1                       ; (79)
        jmp     isr                          ; (80)

limit   db      int_limit                   ; (81)

isr:     _brk                               ; (82)
%ifdef _SHOW_                               ; (83)
        shl     rax, 1                      ; (84)
        add     rax, 0xb8000 + scr_ofs      ; (85)
        inc     byte [rax]                  ; (86)
%endif                                       ; (87)
        dec     byte [limit]                ; (88)
stop:    jz      stop                       ; (89)

        pop     rax                          ; (90)
        iretq                                ; !           ; (91)
        ...

code0:                                     ; 32-bit user's code ; (106)
        dw      end - 1                     ; limit low        ; (107)
        dw      0                          ; base low         ; (108)
        db      0                          ; base middle      ; (109)
        db      1001_1010b                 ; P, dpl, S | 1, c, XR, a ; (110)
        db      0100_0000b                 ; g, 32 bit, 00 | limit high ; (111)
        db      0                          ; base high        ; (112)

flat3:                                     ; User stack (DPL = 3) ; (113)
        dw      -1                         ; limit low (max)   ; (114)
        dw      0                          ; base low         ; (115)
        db      0                          ; base middle      ; (116)
        db      1111_0010b                 ; P, DPL, S | Data, e, RW, a ! ; (117)
        db      1100_1111b                 ; G, B, 00 | limit high (max) ; (118)
        db      0                          ;                  ; (119)

```



```

code64u:                                ; 64-bit user code                ; (120)
    dw    0                            ; (limit low)                    ; (121)
    dw    0                            ; (base low)                     ; (122)
    db    0                            ; (base middle)                  ; (123)
    db    1111_1010b                   ; P, DPL, S | 1, c, XR, (a) !   ; (124)
    db    0010_0000b                   ; 0, d, L, 0 | (limit           ; (125)
    db    0                            ; (base high)                    ; (126)

code64s:                                ; 64-bit supervisor code        ; (127)
    dw    0                            ; (limit low)                    ; (128)
    dw    0                            ; (base low)                     ; (129)
    db    0                            ; (base middle)                  ; (130)
    db    1001_1010b                   ; P, dpl, S | 1, c, XR, (a) !   ; (131)
    db    0010_0000b                   ; 0, d, L, 0 | (limit high)     ; (132)
    db    0                            ; (base high)                    ; (133)

flat64s:                                ; Supervisor's stack            ; (134)
    dw    -1                           ; limit low (max)                ; (135)
    dw    0                            ; base low                       ; (136)
    db    0                            ; base middle                     ; (137)
    db    1001_0010b                   ; P, dpl, S | Data, e, RW, a    ; (138)
    db    1100_1111b                   ; G, B, 00 | limit high (max)   ; (139)
    db    0                            ;                                ; (140)
    ...
;----- IDT                            ; (153)
idt:                                    ; (154)
i0:                                    ; (155)
    dw    isr0                         ; low adr                        ; (156)
    dw    code64s - gdt                 ; selector                       ; (157)
    db    0                            ; (rsrv)                         ; (158)
    db    1110_1111b                   ; P, DPL, 0 | Trap             !   ; (159)
    dw    0                            ; high adr                       ; (160)
    dq    0                            ; 64-bit extension              !   ; (161)
i1:                                    ; (162)
    dw    isr1                         ; low adr                        ; (163)
    dw    code64s - gdt                 ; selector                       ; (164)
    db    0                            ; (rsrv)                         ; (165)
    db    1110_1111b                   ; P, DPL, 0 | Trap             !   ; (166)
    dw    0                            ; high adr                       ; (167)
    dq    0                            ; 64-bit extension              !   ; (168)

idtr  dw    $ - idt - 1                 ; (169)
      dq    idt                         ; !                               ; (170)

;----- TSS                            ; (171)

tss0  resb    tss64_size                 ; (172)
iopb:                                     ; (173)
iopb_sz equ    $ - iopb                  ; 0                               ; (174)

```

```

;-----; (175)
star_high:; (176)
    dw    code64s - gdt    ; syscall_cs    ; (177)
    dw    code0 - gdt     ; sysret_cs     ; (178)

;-----; (179)
start:; (180)
    ...
    lgdt  [gdtr]           ; (186)
    lidt  [idtr]           ; !           ; (187)
    ...

```

Мы еще не адаптировали модуль `_output.asm` к 64-битовому режиму, поэтому вывод на экран счетчиков прерываний выполняется иначе. Сначала, после входа в 64-битовый режим, вызывается (50) процедура (39), которая выводит строку нулей, количество которых равно числу прерываний²⁴. Исключения начинаются после вызова (60), при выполнении "системной" процедуры (30–37), и продолжают после возврата — в (61–63) и, если захотите, многократно в (66) до предела, заданного в (10). В строках (32–33) `CPL` равен нулю, а начиная с (61) — трем.

Входы (75) и (78) в процедуры обработки прерываний сделаны так же, как в примерах для защищенного режима, но в точке сборки (82) вывод на экран выполняется по-другому: мы увеличиваем значение литеры в позиции экрана, соответствующей номеру прерывания. Там была цифра "0", а станет "1" и т. д. Чтобы не допустить бесконечного вызова `fault`-исключений при выполнении (66), общее число прерываний ограничено (88–89) с использованием счетчика (81).

Путь прерывания по-прежнему проходит через регистр `IDTR`, который инициализируется в (187), и таблицу `IDT` (153–168). Единственное отличие от защищенного режима — дополнительное поле (161) и (168) в вентилях, необходимое для задания 64-битового адреса. Заметим, что в вентилях `DPL` = 3, т. е. они доступны для кода любого уровня привилегий. Если задать меньшее число, то вызовы исключений (61–63) и (66) уже не пройдут, т. к. `CPL` > `DPL`.

Помимо исключений, этот пример демонстрирует:

- полный запрет доступа к портам для `CPL` = 3;
- настройку дескрипторов и их расположение при использовании системного вызова.

Чтобы совсем запретить доступ к портам при `CPL` = 3, нужно задать смещение массива `IOPB` в `TSS` (56), расположив его (173) на вершине `TSS`, причем массив должен быть *пустым*.

Что касается дескрипторов в `GDT`, то значения `DPL` в них равны значениям `CPL` исполняемого кода: либо 0 — для системной процедуры, либо 3 — для главной программы. Поэтому для 64-битового кода подготовлены два дескриптора с разными значениями `DPL`. Соответственно исправлено (177) значение для регистра `STAR`. Исходный же вариант из листинга 21.8 в сочетании с вентилями прерываний не работает, при его корректировке были получены следующие ошибки:

- `interrupt(long mode): soft_int && gate.dpl < CPL`
- `check_cs(0x001b): non-conforming code seg descriptor dpl != cpl, dpl=0, cpl=3`
- `iret64: SS.dpl != CS selector RPL`

Первая намекает на то, что DPL для вентиля лучше сделать максимальным, тогда он будет универсальным. Вторая — это реакция на попытку выполнить код при несовпадении DPL и CPL, что допустимо только для конформных сегментов. Последняя напоминает о самом жестком правиле в системе привилегий: для сегмента стека должно быть DPL = CPL = RPL.

В листинге 21.11 — вариант рассмотренной программы, расширенный до 40 прерываний при помощи макрокоманд с циклами, аналогично листингу 15.2.

Листинг 21.11. Обработка прерываний 0–n (trap_n.a)

```

...
; ----- isr64 ; (75)

%macro set_isr 1 ; (76)
isr%1: _brk ; (77)
    push    rax ; (78)
    mov     eax, %1 ; (79)
    jmp     isr ; (80)
%endmacro ; (81)

%macro set_isrs 1 ; (82)
%assign i 0 ; (83)
%rep %1 ; (84)
    set_isr i ; (85)
    %assign i i+1 ; (86)
%endrep ; (87)
%endmacro ; (88)

    set_isrs max_int + 1 ; (89)
...
; ----- IDT ; (162)

%macro igate 1 ; (163)
i%1: ; (164)
    dw     isr%1 ; low adr ; (165)
    dw     code64s - gdt ; selector ; (166)
    db     0 ; (rsrv) ; (167)
    db     1110_1110b ; P, DPL, 0 | Intr ! ; (168)
    dw     0 ; high adr ; (169)
    dq     0 ; 64-bit extension ; (170)
%endmacro ; (171)

```

```

%macro igates 1 ; (172)
%assign i 0 ; (173)
%rep %1 ; (174)
    igate i ; (175)
    %assign i i+1 ; (176)
%endrep ; (177)
%endmacro ; (178)

idt: ; (179)
    igates max_int + 1 ; (180)
    ...

```

В завершение рассмотрим пример обработки исключений и прерываний, когда в вентиле задано ненулевое значение поля `IST` [2, Figure 4-24]. Число 1–7 в поле `IST` — это индекс для считывания нового указателя стека из массива, заданного в `TSS`.

Листинг 21.12. Обработка прерываний с использованием `IST` (`ist.a`)

```

...
subr_1: ; (30)
    _brk ; (31)
    int 3 ; ist = 3 ; (32)
    int 7 ; ist = 7 ; (33)
    int 8 ; ist = 0, cpl = 0 ; (34)
    ...
    sysretq ; (38)
    ...
entry64: ; (48)
    ...
    mov rbx, tss0 ; (53)
    mov qword [_rsp0 + rbx], 0xf100 ; (54)
    mov qword [_rsp1 + rbx], 0xe100 ; (55)
    mov qword [_rsp2 + rbx], 0xd100 ; (56)
    mov qword [_ist1 + rbx], 0x1100 ; ! ; (57)
    mov qword [_ist2 + rbx], 0x2100 ; (58)
    mov qword [_ist3 + rbx], 0x3100 ; (59)
    mov qword [_ist4 + rbx], 0x4100 ; (60)
    mov qword [_ist5 + rbx], 0x5100 ; (61)
    mov qword [_ist6 + rbx], 0x6100 ; (62)
    mov qword [_ist7 + rbx], 0x7100 ; (63)
    mov word [_iomapb + rbx], iopb - tss0 ; (64)
    ...
    syscall ; (68)

    int 0 ; ist = 0, cpl = 3 ; (69)
    int 1 ; ist = 1 ; (70)

```

Программы из листингов 21.11 и 21.12 исправно работают в Bochs. Но попытка выполнить их вне симулятора²⁵ приводит к срабатыванию общей защиты, что явствует из вывода на экран, приведенного в табл. 21.2. Счетчик срабатывания общей защиты выделен, и видно, что сбой происходит в конце обработки исключения (69), вызванного с уровня $CPL = 3$. Исключения (32–34), вызванные с нулевого уровня, проходят без сбоя.

Таблица 21.2. Вывод на экран в опытах с обработкой исключений

[illegible]

Подозрение на регистры `fs` и `gs`, которые в симуляторе равны 0, а в РС могут быть ненулевыми, не оправдалось: инициализация этих регистров ничего не дает.

Ошибка неочевидна, и при ее исследовании может понадобиться вывод на экран шестнадцатеричных чисел. В листинге 21.13 приведен вариант программы из листинга 21.11 с использованием модуля `_output.asm`.

Листинг 21.13. Вывод счетчиков исключений и кодов ошибки (trap_n2.a)

```

...
#define trap_limit 100                                ; (9)
...
;----- 64-bit mode                                  ; (12)

        bits 64                                       ; (13)

#ifdef _SHOW_                                          ; (14)
#include "_output.asm"                                ; (15)
#endif                                                ; (16)
...
; ----- trap64                                       ; (56)

%macro   set_trap 1                                    ; (57)
trap%1:  _brk                                          ; (58)
        enter    0, 0                                ; (59)
        push     rdx                                  ; (60)
        mov      edx, %1 * 2                          ; (61)
        jmp      trap                                  ; (62)
%endmacro                                           ; (63)

%macro   set_traps 1                                   ; (64)
i_cnt:   resd    %1                                    ; (65)
i_err:   resd    %1                                    ; (66)
%assign i 0                                           ; (67)
%rep %1                                              ; (68)
        set_trap i                                    ; (69)
        %assign i i+1                                ; (70)
%endrep                                              ; (71)
%endmacro                                           ; (72)

        align    4                                    ; (73)
        set_traps trap_no                             ; (74)

trap:                                             ; (75)
        push     rax                                  ; (76)
        lea      rax, [i_cnt + (rdx * 4)]             ; (77)

```

```

        inc     dword [rax]                                ; (78)
        mov     rax, [rax]                                ; counter ; (79)
.12:    _brk                                           ; (80)
        cmp     al, trap_limit                          ; (81)
        ja      .12                                     ; (82)
#ifdef _SHOW_                                           ; (83)
        call    wr_hex                                  ; (84)
#endif                                                  ; (85)
        mov     rax, [ebp + 8]                          ; err_code ; (86)
#ifdef _SHOW_                                           ; (87)
        inc     edx                                      ; (88)
        call    wr_hex                                  ; (89)
#endif                                                  ; (90)
        pop     rax                                      ; (91)
        pop     rdx                                      ; (92)
        leave                                       ; (93)
        iretq                                     ; ! ; (94)
        ...

```

С прерываниями связан массив счетчиков (65) и массив кодов ошибок (66). При каждом прерывании увеличивается (77–79) элемент в массиве счетчиков, его новое значение выводится на экран (84). Все счетчики проверяются (81) на предельное значение (9), при превышении которого программа уходит в бесконечный цикл (80–82), прекращая тем самым повтор fault-исключений. Для доступа к коду ошибки добавлено формирование (59) кадра стека, который используется в (86)²⁶. Вывод для каждого прерывания занимает две позиции на экране: сначала идет счетчик, а за ним — код ошибки.

Вывод, полученный на РС, приведен на рис. 21.3. Счетчик нулевого исключения равен двум, для исключения номер 1 — одному, а затем произошло стократное срабатывание общей защиты. Код ошибки для исключения 13 — это обычно номер селектора или сегмента, вызвавшего ошибку. Здесь число великовато для селектора, оно больше похоже на сегментный адрес реального режима. Но откуда ему взяться? Как уже выяснилось, регистры *fs* и *gs*, где могли остаться настройки реального режима, на ситуацию не влияют.

```

0000_0002 0000_0216 0000_0001 0000_01dc

                                0000_0064 0000_0574

```

Рис. 21.3. Вывод программы из листинга 21.13

В листинге 21.14 приведена еще одна модификация программы *trap_n*, выводящая в строке, номер которой равен номеру прерывания, не только счетчик и код ошибки, но и сегментные регистры.

Листинг 21.14. Вывод счетчиков, кодов ошибки и сегментных регистров (trap_n3.a)

```

...
subr_1:                                ; CPL = 0                                ; (31)
    _brk                                ; (32)
    int    0                            ; (33)
    int    1                            ; (34)
    sysretq                             ; (35)

entry64:                                ; CPL = 0                                ; (36)
    ...
    call    syscall_setup               ; (45)
    syscall                             ; (46)

    int     2                            ; CPL = 3                                ; (47)
    int     3                            ; (48)
%if 1                                     ; (49)
    clr     eax                         ; (50)
    div     al                           ; int 0                                ; (51)
%endif                                   ; (52)
.12:      jmp     .12                    ; (53)

; ----- trap64                         ; (54)

%macro    set_trap 1                     ; (55)
trap%1:   _brk                           ; (56)
    enter  0, 0                          ; (57)
    push   rdx                           ; (58)
    mov     rdx, %1                       ; trap no.                ; (59)
    jmp     trap                          ; (60)
%endmacro                                     ; (61)

...
trap:                                           ; (73)
    push    rax                           ; (74)
    lea     rax, [i_cnt + (rdx * 4)]       ; (75)
    inc     dword [rax]                   ; (76)
    mov     rax, [rax]                    ; counter                    ; (77)
.12:      _brk                             ; (78)
    cmp     al, trap_limit                 ; (79)
    ja      .12                           ; (80)
%ifdef _SHOW_                                ; (81)
    shl     rdx, 3                        ; pos := trap_no * 8        ; (82)
    call    wr_hex                         ; (83)

    mov     rax, [ebp + 8]                 ; err_code                    ; (84)
    inc     rdx                           ; (85)
    call    wr_hex                         ; (86)

```



```

        clr     eax                                ; (87)
        inc     rdx                                ; (88)
        mov     ax, cs                             ; (89)
        call    wr_hex                             ; (90)

        inc     edx                                ; (91)
        mov     ax, ds                             ; (92)
        call    wr_hex                             ; (93)

        inc     edx                                ; (94)
        mov     ax, es                             ; (95)
        call    wr_hex                             ; (96)

        inc     edx                                ; (97)
        mov     ax, fs                             ; (98)
        call    wr_hex                             ; (99)

        inc     edx                                ; (100)
        mov     ax, gs                             ; (101)
        call    wr_hex                             ; (102)

        inc     edx                                ; (103)
        mov     ax, ss                             ; (104)
        call    wr_hex                             ; (105)
%endif                                     ; (106)
        pop     rax                                ; (107)
        pop     rdx                                ; (108)
        leave                                       ; (109)
        iretq                                     ; (110)

```

При входе в прерывание (59) значение `rdx` равно номеру прерывания. В этом качестве оно используется для выбора (75) элемента в массиве счетчиков. Затем, когда элемент увеличен (76) и прочитан (77), в регистре `rdx` формируется (82) номер начальной позиции для вывода. Для прерывания 0 — это позиция 0, для прерывания 1 — позиция 8, т. е. начало следующей строки и т. д. Выводим (83) счетчик, затем считываем (84) и выводим в следующей позиции (85–86) код ошибки, а далее — регистры `cs`, `ds`, `es`, `fs`, `gs` и `ss`.

Результат все тот же: прерывания (33–34) при `CPL = 0` проходят, а после первого же прерывания (47) при `CPL = 3` срабатывает общая защита, и в (48) мы уже не попадаем. Значения сегментных регистров отличаются от кода ошибки тринадцатого исключения, так что эти регистры не виноваты. Кроме того, этот "код ошибки" у программ `trap_n2` и `trap_n3` разный и больше напоминает адрес возврата. Обращает на себя внимание обнуление регистра `ss` при выполнении (47), обсуждаемое в [2, 4.11.3]²⁷.

Примечания

1. Размер вентилей для доступа к 64-битовому коду удвоен. Это касается и режима совместимости: для переключения на 64-битовый код требуется 16-байтовый вентиль.
2. Хотя аппаратная многозадачность больше не поддерживается, один TSS необходим для хранения указателей стека. Интересно, что в 64-битовом режиме сохранились также LDT.
3. На это ограничение мы натолкнулись в *главе 20*, но обошли его за счет косвенного перехода.
4. Защита данных теперь — за счет ограничений при страничном преобразовании.
5. Вместо `r0–r7` используются `rax–rdi`, а указатель инструкций по-прежнему программно недоступен.
6. Это искажение не похоже на ошибку `nasm`, т. к. здесь он выдает предупреждение. Если это целенаправленная коррекция, то она необъяснима и потому опасна.
7. Относительная адресация применяется в командах переходов, начиная с `i8086`. Но для доступа к данным — только в 64-битовом режиме.
8. `Bochs`, конечно, их не показывает, я их дописал сам.
9. Поэтому данные, расположенные на расстоянии, превышающем диапазон 32-разрядных знаковых чисел, недоступны.
10. Выравнивание включено только ради удобства наблюдения в отладчике, а программа будет работать и без него.
11. Стек в 64-битовом режиме реализован только для 64-битовых операндов.
12. Слишком большое число задавать не стоит, потому что промежуток между массивами целиком перезаписывается (заполняется кодом `0x55`).
13. Само собой, т. к. используется 64-разрядный регистр.
14. Так выглядит диагностическое сообщение для процессора **turion64_tyler** (см. параметр **cpu** в файле `bochsrc`). Для процессора **corei7_sandy_bridge_2600k** в сообщении изменилась только маска **allowMask**. Неприятная особенность этого варианта в том, что его, похоже, нельзя заранее проверить с помощью команды `CPUID`.
15. Используется несколько 64-разрядных регистров общего назначения, а также MSR.
16. Впервые от нас потребовался некий *порядок* расположения дескрипторов.
17. В обход системы привилегий, не проверяя CPL и DPL. Вот оно, "quick-and-dirty" во всей красе!
18. Весь сервис операционной системы можно запрограммировать как одну процедуру, которая начинается с мультиветвления по номеру функции, заданному в регистре, а входные параметры и результаты также передаются в регистрах.

19. Хотя можно было и в режиме совместимости.
20. Напомню, что номер первого дескриптора задан в поле `SYSRET CS and SS` регистра `STAR`.
21. Интересно, что в нестандартном дескрипторе, каковым является `TSS`, указаны *адрес и размер* — как для 32-битового защищенного режима.
22. Системные вызовы не обращаются к сегменту `TSS`, для них главное — скорость. Перечисленные подробности `TSS` используются в работе с вентилями.
23. Обработка внешних прерываний выполняется аналогично, но с учетом контроллера прерываний, рассмотренного в *главе 15*.
24. Два в первом примере, сорок — во втором.
25. Загружая PC с USB-Flash с установленной FreeDOS.
26. Некоторые исключения не генерируют код ошибки, в частности все исключения, вызванные напрямую. Для них выводится не код ошибки, а то, что находится вместо него на вершине стека, т. е. адрес возврата.
27. Подраздел "Nested Returns to 64-Bit Mode Procedures" обещает, что все будет хорошо, но общая защита все же срабатывает. Неужели для разных уровней `CPL` выход из обработки прерываний в чем-то различный?

Послесловие

Чтобы выразить все сразу,
Кулаком я бью по тазу.

(Олег Григорьев)

...Ну а в пивной — там все на электричестве.
Нажал на кнопку — раз, вино в любом количестве.
Нажал еще — картошка с колбасой.
И не прошло пяти минут, как ты уже косой.

<...>

Нам электричество ночную тьму разбудит,
Нам электричество пахать и сеять будет.
И нам с братишечкой не надо и вставать —
Все будем водку пить и кнопки нажимать.

("Электричество", со слов Ю.Д. Кочеткова)

В Бранденбургском концерте № 3 Иоганн Себастьян Бах пошутил: средняя часть (Adagio) состоит всего из двух аккордов. В исполнении камерного оркестра I Musici эта каденция длится 14 секунд, будто улица Джона Леннона в Санкт-Петербурге.

Мое послесловие тоже будет кратким.

Но о чем оно должно быть?

Будь то учебник астрологии, климатологии или социологии, я бы побаловался прогнозами. Но у нас техника... А студенческая песенка хрущевских времен — чем не прогноз?

Если подводить итоги, то я, конечно, скажу: какие вы молодцы, что все проделали, и поняли, и почти расколдовали последний пример, который в симуляторе идет, а в РС вызывает исключение общей защиты. Осталось всего ничего (до вершины или до дна — кому как видится). Добавив в программу вывод счетчиков исключений с кодами ошибок, я снабдил вас кое-чем необходимым.

Спасибо всем, кто трудился над этой книгой, и тем, кто сейчас держит ее в руках и размышляет. Тому, кто начал с послесловия,— спасибо, что открыл. Издательству — что рискнуло связаться с немодной тематикой и потратило бесценную типографскую краску.


```
DB6_BUFFER:
010B MOV SI,011F
010E INT 3

AX 07AA i c 1: =e1,ax,, 10101010
BX 0000 IP 0100 2: =d1,ax,, 1962
CX 0000 CS 6309 3:
DX 6309 SS 6309 4:
SI 011F DS 6309 5: r5,b_1,, AB ab
DI 0000 ES 6309 6: r5,si,, AB ab
BP 0000 SP FFFE 0:

lea si,b_1
```

ПРИЛОЖЕНИЯ

- Приложение 1.** Компиляция Bochs
- Приложение 2.** Инструментальные программы FreeDOS
- Приложение 3.** Дополнительные опыты с FPU
- Приложение 4.** Ошибки в a86/a386
- Приложение 5.** Описание электронного архива

ПРИЛОЖЕНИЕ 1

Компиляция Bochs

Я целый день с винтом боролся
И победить его не мог,
Он что-то знал себе такое,
Чего никто уж знать не мог...
(Дмитрий А. Пригов)

Сборка симулятора Bochs из исходных файлов может понадобиться при изменении настроек в файле `conf`. Основной вариант компиляции — в Windows с использованием Visual Studio, а точнее, утилиты `nmake`, запускаемой из командной строки. Однако для подготовки файла `makefile` (для утилиты `nmake`) требуется Linux.

При любых изменениях в `conf`-файле следует "почистить" проект командой `make dist-clean` и собрать его заново.

Можно собрать Bochs целиком в Linux и для Linux, но проверка разных вариантов GUI показала, что графического отладчика нет, а текстовый работает только в вариантах `sdl2` и `x11`. Также выяснилось, что размер окна Bochs не увеличивается при всех вариантах компиляции, что опция `autoscale` бесполезна, а опция `fullscreen`, если не запрещена, то всего лишь отключает панель с кнопками **Power**, **reset** и т. п.

Подготовка к компиляции для Windows и Linux

Со страницы <https://sourceforge.net/projects/bochs/files/bochs/2.7> скачайте файлы `bochs-2.7-msvc-src.zip` и `bochs-2.7.tar.gz`.

Установите Linux Mint Cinnamon¹ и после его запуска добавьте пакет `g++` при помощи пиктограммы **Software Manager** или командой `sudo apt install g++`. (Linux понадобится в любом варианте, т. к. подготовка проекта к компиляции в Windows выполняется в его среде.)

Рассмотрим сначала основной вариант — для Windows.

Компиляция для Windows

Раскройте архив bochs-2.7-msvc-src.zip и отредактируйте файл .conf.win64-msvc так, как показано в листинге П1.1. Заметим, что файлы .conf.win64-msvc и .conf.win32-msvc различаются только первой строкой — target.

Листинг П1.1. Изменения в файле .conf.win64-msvc

```
./configure --target=x86_64-windows \  
            --enable-cpu-level=6 \  
            --enable-x86-64 \  
            --disable-readline \  
            --enable-debugger \  
            --enable-debugger-gui \  
            --enable-long-phy-address \  
            --enable-cdrom=no \  
            --enable-pci=no \  
            --enable-show-ips=no \  
            --enable-plugins=no
```

Затем выполните сценарий .conf.win64-msvc:

```
bash .conf.win64-msvc
```

Изменения в проекте в результате выполнения .conf.win64-msvc нужно сохранить:

```
make win32_snap
```

Почему не win64_snap? Наверное, потому, что цель одна и та же, независимо от платформы: сохранить дерево проекта. Результат — попросту архив проекта с тем же именем (bochs-2.7-msvc-src.zip), он появляется над каталогом bochs-2.7.

Переносим этот обновленный архив в Windows. Устанавливаем Visual Studio (VS) с сайта <https://visualstudio.microsoft.com/downloads/>, соглашаясь со всеми предложениями. Из предлагаемых средств (в окне **Need help choosing to install**) можно выбрать опцию **Desktop development with C++** или ничего. В любом случае, когда мы откроем проект bochs, VS обнаружит недостающий инструментарий и предложит его установить.

После перезагрузки следует запустить VS, нажав мышью на файл bochs.sln в каталоге bochs-2.7/vs2019. Открыв проект, VS выскажет две претензии, с которыми надо согласиться:

- ☐ Для компиляции проекта предлагается установить дополнительные файлы, в ответ нажимаем кнопку **Install**.
- ☐ Проект предназначен для VS версии 2019 г., а у нас новая версия², поэтому предлагается обновление проекта.

Затем выходим из VS и в стартовом меню Windows в группе **Visual Studio** находим пункт **x64 Native Command Prompt**³. (Для Win32 — **x86-Native Command Prompt**.)

Открывается текстовая консоль с переменными окружения, настроенными для работы с утилитами VS. Переходим в дерево проекта и вызываем `nmake`, например, так:

```
❏ d:
❏ cd bochs-2.7
❏ nmake
```

После трансляции в текущем каталоге (в примере `d:\bochs-2.7`) появится результат — исполняемый модуль `bochs.exe`. При его запуске появляется окно графического отладчика.

Компиляция для Linux

Рассмотрим сборку Bochs на примере того же Linux Mint 20.

Предварительно должен быть установлен пакет `g++`. Это можно сделать, либо вызвав команду **Software Manager**, либо, при наличии сетевого соединения, командой `sudo apt install g++`.

Варианты для Linux различаются библиотеками GUI: `x11` (по умолчанию), `term` (на основе псевдографического интерфейса `curses`), `sdl/sdl2` и `wxWidgets`⁴. Вариант GUI выбираем в строке `display_library` файла `conf`, например `display_library: sdl2`. (Опции `fullscreen` и `autoscale` в строке `display_library` в большинстве вариантов бесполезны или даже запрещены.)

Во всех вариантах, кроме `--no-gui`, требуется установить дополнительные библиотеки:

- ❏ `sudo apt-get install libx11-dev` — для интерфейса `x11`;
- ❏ `sudo apt-get install libncurses5-dev` — для интерфейса `curses`;
- ❏ `sudo apt-get install libgtk-3-dev` — библиотека GTK3 для графического отладчика;
- ❏ `SDL2-2.0.20.tar.gz` — для интерфейса `sdl2` (libsdl.org);
- ❏ `wxWidgets-3.1.6.tar.bz2` — для интерфейса `sdl2` (wxwidgets.org/downloads).

Библиотеки для `x11`, `curses` и GTK3 установит сама команда `apt`, но для `sdl/sdl2` и `wxWidgets` требуется ручная доводка. Для каждой библиотеки необходимо распаковать архив, зайти в дерево проекта и выполнить стандартную последовательность команд:

```
./configure
make
sudo make install
```

Для `wxWidgets` требуется выполнить еще одну команду в завершение: `sudo ldconfig`. Если этого не сделать, то при запуске Bochs потребует библиотеку `libwx-gtk3u-core.3.1.so.6`. Вообще `wxWidgets` — весьма объемистая библиотека, и ее сборка сравнима по времени с установкой VS. А полученный Bochs с виду ничем не отличается от вариантов `x11` и `sld2`, разве что работает медленнее.

При старте Bochs считывает файл `.bochsrc` (параметры запуска) из дерева проекта. Не рекомендуется давать `.bochsrc` альтернативное имя: `bochsrc` или `bochsrc.txt`. Bochs поймет, но вот при выполнении команды `sudo make install` ищется именно и только `.bochsrc`. До первого запуска Bochs следует исправить следующие строки в `.bochsrc`:

```
romimage: file="bios//BIOS-bochs-latest"
vgaromimage: file="bios//VGABIOS-lgpl-latest"
```

Для версии 2.7 были проверены все варианты GUI кроме `sdl`, а также некоторые для версии 2.6.11. Во всех случаях графический отладчик не появляется. Текстовый отладчик устойчиво работает в вариантах `x11` и `sdl2` (`--enable-debug`, `--enable-debug-gui=no`).

Вариант `term` — рекордсмен скорости и смотрится неплохо, но он запускается, только если в `.bochsrc` отключен вывод сообщений на консоль:

```
#log: -
log: test.log
```

Этот вариант мог бы пригодиться в первой части книги — при освоении `a86`, когда сообщения Bochs не представляют интереса (в этом варианте мы их и не увидим) и нам достаточно экрана FreeDOS.

Примечания

1. Установка Mint поверх Windows 10, вероятнее всего, приведет к тому, что загрузчик Windows будет стерт, сразу или потом, несмотря на уверения в дружбе с Windows 10.
2. Этот вопрос не возникает, если установлена версия Community или какая-либо из старых версий, например 2017 г. В версии Professional 2022 вопрос возникает, причём каждый раз после изменения `conf`-файла (в результате чего VS снова встречается с файлами версии 2019 г.).
3. Названия могут различаться в зависимости от версий VS.
4. Есть ещё вариант вообще без GUI (`--no-gui`), что предполагает использование встроенного отладчика.

ПРИЛОЖЕНИЕ 2

Инструментальные программы FreeDOS

...Духи в граненом хрустале;
Гребенки, пилочки стальные,
Прямые ножницы, кривые...
(А. С. Пушкин. Евгений Онегин, гл. 1)

Искусственные челюсти
Невыразимой прелести.
(Вадим Шефнер. Лачуга должника)

Здесь приведена краткая справка по файловому менеджеру VC и текстовому редактору edit. Сведения по ассемблеру a86/a386 и отладчику d86/d386 содержатся в документации разработчика, здесь приведены только форматы отображения данных в отладчике.

Файловый менеджер

Файловый менеджер вызывается в конце стартового файла autoexec.bat. Выйти из него можно клавишей <F10>, а перезапустить — командной строкой `vc <Enter>`.

Включение и выключение панелей:

- ☐ левой — <Ctrl>+<F1>;
- ☐ правой — <Ctrl>+<F2>;
- ☐ обеих (если нужно увидеть вывод в консоль) — <Esc> или <Ctrl>+<O>.

Позиция текущего файла отмечается маркером. Клавиши для перемещения маркера:

- ☐ между панелями — <Tab>;
- ☐ в пределах окна — клавишами-стрелками и клавишей <Home>.

Действия над файлами и каталогами:

- ☐ копирование текущего файла — <F5>;
- ☐ перемещение или переименование текущего файла — <F6>;
- ☐ создание каталога — <F7>;

- ❑ удаление файла или каталога — <F8>;
- ❑ редактирование — <F4>;
- ❑ смена устройства (A:/, C:/ и т. д.) — <Alt>+<F1> (в левой панели) или <Alt>+<F2> (в правой);
- ❑ поиск файла — <Alt>+<F7>;
- ❑ обработка файла — <Enter>.

Обработка файла означает вызов команды в зависимости от расширения файла. Она настраивается клавишей <F9> в меню **Options/Extension file editor** или же прямым редактированием файла `vs.ext`. Для работы с `a86/d86` в `vs.ext` должны быть строки:

- ❑ 8: `a86 !.!` — вызов ассемблера `a86` для файлов с расширением `"8"`;
- ❑ `sym: d86 !.com` — при нажатии на файл с расширением `"sym"` будет вызван отладчик `d86` с одноименным `com`-файлом.

Встроенный редактор, вызываемый клавишей <F4>, удобен для мелких исправлений. Что касается копирования блоков текста, то максимум, что он позволяет, — это удалить строку (<Ctrl>+<Y>) в одном месте и тут же вставить ее (<Ctrl>+<U>) в другом.

Выход из редактора — клавишей <Esc>.

Текстовый редактор `edit`

Этот текстовый редактор вызывается из командной строки по имени — `edit`. В командной строке перечисляются файлы для редактирования. Для перехода между файлами в окне редактора используйте клавиши <Alt>+<1>, <Alt>+<2> и т. д. Поддерживаются операции с блоками текста:

- ❑ выделение блока — клавишами-стрелками при нажатой клавише <Shift>;
- ❑ удаление, копирование и вставка — <Ctrl>+<X>, <Ctrl>+<C> и <Ctrl>+<V>.

Чтобы выбрать меню, нужно нажать <Alt> вместе с первой буквой в названии меню. Например, выбор меню `File` — <Alt>+<F>.

В диалоговых окнах клавиша <Tab> не работает. Элемент в диалоговом окне можно выбрать нажатием красной буквы (или первой буквы, если все они одного цвета). Например, чтобы при выходе из редактора не сохранять файл, надо нажать 'N' (без <Alt>) — по первой букве в названии кнопки **No**.

Редактор памяти `e32`

Редактор 32-битовых слов в шестнадцатеричном формате работает в пределах памяти, доступной DOS. Пример запуска: `e32 24fe 8`, где `24fe` — сегментный адрес блока памяти, а `8` — размер блока в килобайтах. Параметры указываются для защиты остальной памяти — выйти за пределы заданного блока `e32` не позволит.

В нижней строке экрана e32 выводит:

- ❑ слева — текущий линейный адрес (в скобках показано смещение от начала блока);
- ❑ в центре — буфер обмена емкостью один элемент (32-битовое слово).

Назначение клавиш, помимо клавиш-стрелок, следующее.

- ❑ <Home>/<End> — сначала переводят курсор в начало/конец текущей строки, при следующем нажатии — в начало/конец текущего экрана, а еще при одном нажатии — в начало/конец блока памяти.
- ❑ <Enter> — запись в буфер обмена (виден по центру нижней строки) текущего линейного *адреса*.
- ❑ <Ctrl>+<C> — запись в буфер обмена текущего *значения*.
- ❑ <Ctrl>+<V> — вставка из буфера обмена.
- ❑ <Ctrl>+<Enter> — переход по адресу, равному текущему значению; переход ограничен размером блока, указанным при запуске редактора.
- ❑ <Пробел> — обновление экрана.

В пакет e32 также входят сопутствующие утилиты:

- ❑ b2t (binary to text) — преобразование двоичного файла в текстовый, с указанием сегментного адреса блока памяти;
- ❑ f2m (bin-file to memory) — запись из файла в память;
- ❑ m2f (memory to bin-file) — копирование данных из памяти в файл, используется редко, т. к. e32 при завершении сохраняет блок памяти в файле \$\$\$tmp.

Чтобы узнать подробности вызова этих утилит, запустите их — и они сами сообщат, какие параметры следует указать.

Адрес во всех вызовах, включая e32, всегда сегментный.

Форматы отображения данных в d86

В табл. П2.1 приведены основные форматы для отображения чисел. Для литер рекомендуется формат *r* (Raw text). Когда последовательность литер заканчивается нулем, имеет смысл применить формат *s* (String, null).

Таблица П2.1. Базовые форматы для отображения чисел

Обозначение	Размерность	Формат отображения
b	Байт	Шестнадцатеричный
d	Слово	Десятичный
e	Байт	Двоичный
n	Байт	Десятичный
w	Слово	Шестнадцатеричный

ПРИЛОЖЕНИЕ 3

Дополнительные опыты с FPU

Вот, например: раз, два, три! Ничего не произошло! Вот я запечатлел момент, в котором ничего не произошло.
Я сказал об этом Заболоцкому. Тому это очень понравилось, и он целый день сидел и считал: раз, два, три! И отмечал, что ничего не произошло. За таким занятием застал Заболоцкого Шварц. И Шварц тоже заинтересовался этим оригинальным способом запечатлеть то, что происходит в нашу эпоху, потому что ведь из моментов складывается эпоха.
(Даниил Хармс. Однажды я пришел в Госиздат...)

Чем, как не ареной ужасов, была бы жизнь без арифметики?
(Адмирал Сидней Смит, письмо к юной леди от 22 июля 1835 г.)

Команда *fisttp*

В листинге ПЗ.1 приведен пример относительно новой команды *fisttp* (FPU Integer Truncate-Store-Pop) — запись в память целого числа с отбрасыванием дробной части¹. Поскольку *a86/a386* еще не "знают" о ней, пример написан на ассемблере *nasm*.

Листинг ПЗ.1. Инструкция *fisttp* (*fisttp.asm*)

```
org      0x100                ; (1)

jmp      start                ; (2)

dozen    dw      12            ; (3)
real     dd      3.7           ; (4)

start:                                     ; (5)
    finit                                     ; (6)
    fld     dword [real]          ; (7)
    fist    word [dozen]          ; (8)
    fisttp  word [dozen]          ; (9)

int       0x20                   ; (10)
```


Некоторые особенности `nasmm`, проявившиеся в этом примере:

- ☐ директива (1) установки счетчика адресов обязательна, т. к. по умолчанию он равен нулю;
- ☐ шестнадцатеричные числа в (1) и (10) могут обозначаться как в языке C;
- ☐ операнды в памяти (7–9) *всегда* записываются в квадратных скобках и часто требуют уточнения типа, т. к. в `nasmm` понятие типа отсутствует².

Для вызова `nasmm` нажмите <Enter> на файле с расширением `asm` или выполните команду: `nasmm fisttp.asm -f bin -o fisttp.com`. Вызовите `d86` с программой `fisttp.com` и пройдите ее по шагам. Незнакомую команду (9) `d86` дизассемблирует как может, но на выполнение кода это не влияет. В окне отображения данных видно, что результат — число три, как и ожидалось.

Формат BCD

В 64-битовом режиме команды BCD-коррекции упразднены [3]. Означает ли это отказ от формата BCD? Нет, просто центральный процессор не поддерживает BCD-арифметику. А в FPU этот формат по-прежнему используется. Заметим, что для десятичного ввода-вывода, а также умножения-деления на степень десяти формат BCD удобнее и быстрее двоичного³. Достаточно выделить тетраду в BCD-числе, и десятичная цифра готова. А чтобы умножить или разделить на 10 в степени n , нужно только сдвинуть BCD-число влево или вправо на $4n$ разрядов.

Посмотрим, как выглядит BCD-число в FPU. Загрузите в FPU 32-битовое целое (оно преобразуется в формат с плавающей точкой без потери точности) и сохраните его в памяти как BCD. Полученное значение 10-байтовое: в младших 9 байтах — 18 BCD-цифр (по тетраде на цифру), а в старшем байте — 0 с признаком знака в старшем бите. Чтобы вывести полученное число на экран, нужно прибавить к каждой значащей BCD-цифре код '0'.

Прерывания от i80x87

Математический сопроцессор `i80x87` сохраняет в регистре состояния `sw` особые ситуации, т. е. ошибки и предупреждения. Чтобы разрешить прерывание по какому-либо особому состоянию, необходимо в управляющем слове `sw` обнулить одноименный бит маскирования прерывания.

При обработке прерывания следует:

- ☐ прочесть состояние FPU в память;
- ☐ сбросить признаки особых ситуаций в слове состояния `sw`;
- ☐ найти в прочитанных данных состояния причину ошибки;
- ☐ ликвидировать последствия ошибки в программе.

Простейший пример обработки прерываний по особым ситуациям `i80x87` приведен в листинге ПЗ.2.

Листинг П3.2. Обработка прерывания от i80x87 (fpu_int.8)

```

include fpu.inc                                ; (1)

        jmp      start                          ; (2)

old_v2  dd      ?                              ; (3)
cw      dw      ?                              ; (4)
cw0     dw      ?                              ; (5)
cw1     dw      ?                              ; (6)
err_cnt dw      0                              ; (7)

start:                                       ; (8)
        mov     ax, 035 by 2                  ; (9)
        int     021                          ; (10)
        mov     w old_v2, bx                  ; (11)
        mov     w old_v2 + 2, es              ; (12)

        lea     dx, trap2                     ; (13)
        mov     ax, 025 by 2                  ; (14)
        int     021                          ; (15)

        finit                                       ; (16)
        fstcw   cw0                             ; (17)
        mov     cw, ax, cw0                    ; (18)
        and     cw, not 111111xb              ; (19)
        fldcw   cw                             ; (20)

        fldpi                                       ; (21)
        fmul    0                               ; (22)
        fadd    0                               ; (23)
        fldl    0                               ; (24)
        fadd                                          ; (25)
        fsqrt                                       ; (26)
        fstp    0                               ; (27)
        fstp    0                               ; (28)
        fstp    0                               ; (29)
        fist    d end                          ; (30)
        fwait                                       ; (31)

        mov     ax, 025 by 2                  ; (32)
        mov     dx, w old_v2                  ; (33)
        mov     ds, w old_v2 + 2              ; (34)
        int     021                          ; (35)
        mov     ds, cs                          ; (36)
        int     020                          ; (37)

```

```
trap2:                                ; (38)
    cs inc  err_cnt                    ; (39)
    fclex                               ; (40)
    cs fldcw cw                        ; (41)
    iret                               ; (42)
```

Команды (9–12) сохраняют вектор прерывания 2 (NMI, Non-Maskable Interrupt), а (13–15) направляют вектор на метку (38). В строках (17–20) разрешаем прерывания от FPU для всех особых ситуаций, а затем в (21–31) провоцируем эти ситуации. В (32–35) восстанавливаем вектор прерывания 2. Процедура обработки прерывания (38–42) минимальна⁴: она увеличивает (39) счетчик, сбрасывает (40) признаки особых ситуаций и вновь разрешает прерывания (41).

Можно пройти эту программу в отладчике сразу и целиком — командой `g <Enter>`. В `err_cnt` будет показано количество произошедших прерываний. А можно иначе: дойти до (21), нажимая клавишу `<F9>`, а потом двигаться по шагам до (31), наблюдая за увеличением `err_cnt`.

Интересно то, что прерывание по вектору 2, начиная с `i80386/387`, вызывается из обработчика аппаратного прерывания по входу `irq13` — так закодировано в BIOS. Получив управление, системный обработчик устанавливает `ds = 070` (характерный сегментный адрес для данных BIOS), что-то делает и напрямую вызывает `int 2`, не восстановив `ds`. Это серьезный недочет, т. к. в обработчике прерываний (39–42) нарушается доступ к переменным программы, и мы вынуждены добавлять префикс `cs`.

Чтобы обнаружить это, пришлось прибегнуть к помощи отладчика Bochs. Сначала добавьте между операторами (1) и (2) инструкцию `xchg bx, bx` (пустая операция, но для встроенного отладчика это так называемый `magic breakpoint`) и заново скомпилируйте `fpu_int.8`. Затем запустите сеанс Bochs-FreeDOS командой `__debug.bat`, после остановки в отладчике продолжите выполнение (кнопка **Continue**), и по окончании загрузки запустите `fpu_int.com`. Произойдет останов на точке `magic breakpoint`, и дальше можно идти по шагам до входа в прерывание, где вскоре и обнаружится запись числа `070` в регистр `ds`.

Итак, в листинге ПЗ.3 обрабатывается прерывание 2, как если бы запрос на прерывание был подключен ко входу NMI (Non-Maskable Interrupt). В IBM PC на базе `8086/87` и `80286/287` сигнал прерывания от FPU действительно шел на вход NMI (через схему логики NMI, вместе с сигналами сбоя при обращении к памяти и внешним устройствам).

Начиная с IBM PC на базе `80386/387`, сигнал от сопроцессора подключен не к NMI, а к контроллерам прерываний через вход `irq13` (пятый вход ведомого контроллера). В системах следующего поколения сигнал прерывания от `i80x87` тоже подключен к `irq13`, но с возможностью перенастройки на исключение 16.

Чтобы программы, написанные для IBM PC/XT, выполнялись также на PC/AT, в последовавшей версии BIOS добавили системную процедуру обработки прерывания от `irq13`, примерный текст которой показан⁵ в листинге ПЗ.3.

Листинг ПЗ.3. Системная процедура обработки прерывания от irq13

```

push    ax, dx
xor     al, al
out     0f0, al          ; Busy off
mov     al, 020
out     0a0, al          ; EOI to master PIC
out     020, al          ; EOI to slave PIC
pop     dx, ax
int     2                ; !
iret

```

Через порты 0f0...0ff CPU и FPU обмениваются данными. (Большая часть этих портов недоступна для команд in/out, но порт 0f0 доступен.) Когда FPU генерирует заявку на прерывание, он переходит в режим ожидания — пока в порт 0f0 не будет записан ноль. Выполнив эту запись, системная процедура сбрасывает заявки в контроллерах прерываний и напрямую вызывает обработку NMI (вектор два). Тем самым BIOS программно переводит прерывание от irq13 в NMI.

Начиная с 486DX, сопроцессор уже не является внешним устройством, так что запрос на прерывание от сопроцессора может быть замкнут внутри процессора. Для сохранения совместимости с программами, разработанными для 80386, в начальном состоянии процессора сигнал запроса выводится вовне — на irq13. При установке флага NE (Numeric Exception) в управляющем регистре CR0 запрос⁶ переключается с irq13 на линию внутреннего прерывания 16.

Флагу NE соответствует бит номер 5 регистра CR0. Его переключение в а386 выполняется командами:

```

mov     eax, cr0
xor     al, bit 5
mov     cr0, eax

```

Синхронизация процессора и сопроцессора

Незамаскированная особая ситуация, возникшая в FPU, вызывает запрос на прерывание сразу после завершения команды, которая привела к особой ситуации. Команда wait заставляет центральный процессор дожидаться завершения текущей операции i80x87. Эта команда гарантирует, что прерывание от FPU, если оно должно произойти, будет обработано до продолжения кода в CPU.

Вставка wait требуется лишь тогда, когда команды FPU и CPU чередуются. Это лишнее, если команды FPU идут сплошным потоком, а также в ситуации, когда процессор и сопроцессор синхронизируются автоматически при обмене данными через память, как в листинге ПЗ.4.

Листинг ПЗ.4. Синхронизация через память (sync.8)

```

        jmp      start                ; (1)

x       dw      -1                    ; (2)

start:  finit                     ; (3)
        fldpi                     ; (4)
        fist     x                  ; (5)
        mov     ax, x               ; (6)
        int     020                ; (7)

```

Процессор откладывает чтение (6) переменной (2) до тех пор, пока в нее не будет записан результат инструкции FPU (5). После выполнения этой пары команд в `ax` находится результат записи из FPU в `x`. Если бы CPU не стал дожидаться завершения (5), то в `ax` была бы записана `-1`.

Расширение MMX

С регистрами FPU аппаратно совмещены регистры арифметического расширения MMX [4]. Расширение MMX позволяет выполнять параллельные арифметические операции над группой одноименных байт, слов или двойных слов, расположенных в 64-битовых регистрах MMX. В случае с MMX наименьший элемент данных — это байт. Если выполняется байтовая MMX-операция, например сложение `paddb`, то каждый n -й байт приемника суммируется с n -м байтом источника ($n = 0 \dots 7$). Пары байтов изолированы друг от друга, так что переполнение при суммировании i -й пары не вызывает переноса в пару $i + 1$. (Также есть команды с насыщением, когда результат при переполнении устанавливается равным предельному значению диапазона.)

В листинге ПЗ.5 — простейший пример байтового сложения с использованием регистров и команд MMX. Для его трансляции требуется `a386`, а для наблюдения за регистрами MMX — `d386`.

Листинг ПЗ.5. Сложение массивов байтов с использованием MMX (mmx.8x)

```

        jmp      start                ; (1)

src     dq      0ff_fe, 02233445566   ; (2)
_cnt    equ     ($ - src) / type src   ; (3)

dst     dq      002_04, 0ffeeddccbb   ; (4)
_cnt    equ     ($ - dst) / type dst   ; (5)

start:                                     ; (6)
        mov     cx, _cnt                ; (7)
        mov     ebx, 0                  ; (8)

```

```
l1:                                     ; (9)
    movq    mm0, [src+ebx*8]           ; (10)
    movq    mm1, [dst+ebx*8]           ; (11)
    paddb   mm0, mm1                   ; (12)
    movq    dst[ebx*8], mm0            ; (13)
    inc     ebx                         ; (14)
    loop    l1                         ; (15)
    int     020                        ; (16)
```

Определены (2, 4) два массива по два 64-битовых элемента (т. е. 16 пар байтов). Результат сложения формируется на месте массива (4). Команды (10–13) — это команды MMX: запись (10–11) в регистры из памяти, сложение (13) пар байтов в регистрах и запись (13) результата в память. Кстати, в командах обращения к памяти применена индексная адресация со множителем 2/4/8, эта возможность впервые появилась в i80386. В качестве индекса можно использовать любой 32-битовый регистр.

На рис. ПЗ.1 показано окно MMX в отладчике d386 перед первым выполнением команды (12).

START:			MMX register set:		
0123 MOV CX,2					
0126 MOV EBX,0000_0000			high		low
L1:			----		----
012C MOVQ MM0,Q[8*EBX+0103]			MM0	0000_0000_0000	FFFE
0135 MOVQ MM1,Q[8*EBX+0113]			MM1	0000_0000_0000	0204
#	013E	PADDB MM0,MM1	MM2	0000_0000_0000_0000	
0141 MOVQ Q[8*EBX+0113],MM0			MM3	0000_0000_0000_0000	
...					
EAX	0000_0000	i	1: b8,src,,	FE FF 00 00 00 00 00 00	
EBX	0000_0000	IP 013E	2: "010B	66 55 44 33 22 00 00 00	
ECX	0009_0002	CS 64C9	3:		
EDX	0000_64C9	SS 64C9	4: b8,dst,,	04 02 00 00 00 00 00 00	
ESI	000E_0000	DS 64C9	5: "011B	BB CC DD EE FF 00 00 00	

Рис. ПЗ.1. Окно MMX перед первым сложением

При сложении (12) возникает переполнение, результат показан на рис. ПЗ.2.

На рис. ПЗ.3 показан окончательный результат выполнения программы.

На рис. ПЗ.4 показано, как выглядит окно FPU, если переключиться на него после выполнения программы в MMX.

Вникать в особенности расширений MMX, XMM и т. п. в отрыве от их применения, например для обработки аудио- и видеоданных, бессмысленно. Это материал для отдельной книги, а пока ограничимся рассмотренным примером.

Для сведения, на рис. ПЗ.5 показано первое окно XMM в отладчике d386. Второе окно (с регистрами XMM4...XMM7) появится при следующем нажатии <F10>.

START:	MMX register set:
0123 MOV CX,2	
0126 MOV EBX,0000_0000	high low
L1:	----
012C MOVQ MM0,Q[8*EBX+0103]	MM0 0000_0000_0000_0102
0135 MOVQ MM1,Q[8*EBX+0113]	MM1 0000_0000_0000_0204
013E PADDB MM0,MM1	MM2 0000_0000_0000_0000
0141 MOVQ Q[8*EBX+0113],MM0	MM3 0000_0000_0000_0000
# 014A INC EBX	MM4 0000_0000_0000_0000
014C LOOP L1	MM5 0000_0000_0000_0000

Рис. ПЗ.2. Окно MMX после первого сложения (без насыщения)

START:	MMX register set:
0123 MOV CX,2	
0126 MOV EBX,0000_0000	high low
L1:	----
012C MOVQ MM0,Q[8*EBX+0103]	MM0 0000_0021_2121_2121
0135 MOVQ MM1,Q[8*EBX+0113]	MM1 0000_00FF_EEDD_CCBB
013E PADDB MM0,MM1	MM2 0000_0000_0000_0000
0141 MOVQ Q[8*EBX+0113],MM0	MM3 0000_0000_0000_0000
014A INC EBX	MM4 0000_0000_0000_0000
014C LOOP L1	MM5 0000_0000_0000_0000
# 014E INT 020	MM6 0000_0000_0000_0000
...	
EAX 0000_0000 i 1: b8,src,, FE FF 00 00 00 00 00 00	
EBX 0000_0002 IP 014E 2: "010B 66 55 44 33 22 00 00 00	
ECX 0009_0000 CS 64C9 3:	
EDX 0000_64C9 SS 64C9 4: b8,dst,, 02 01 00 00 00 00 00 00	
ESI 000E_0000 DS 64C9 5: "011B 21 21 21 21 21 00 00 00	

Рис. ПЗ.3. Окно MMX после выполнения программы

	0: -NaN 0000 0021 2121 2121
0103 DB 0FE,0FF	1: -NaN 0000 00FF EEDD CCBB
0105 ADD B[BX+SI],AL	2: +0.
0107 ADD B[BX+SI],AL	3: +0.
0109 ADD B[BX+SI],AL	4: +0.
010B DB 066	5: +0.
010C PUSH BP	6: +0.
010D INC SP	7: +0.

Рис. ПЗ.4. Окно FPU после выполнения программы

# 0100 NOP		XMM0:	XMM2:
0101 NOP		+1.98116185 E+34	+7.31653403 E-31
0102 NOP		+7.03092556 E+28	+5.61504875 E-31
0103 NOP		+1.47634416 E+20	+1.78356012 E-37
0104 NOP		+4.63042613 E+27	-5.85508093 E-32
0105 NOP	XMM1:	XMM3:	
0106 NOP		+2.00746976 E-19	+4.26366039 E+35
0107 NOP	Den\2	+27942405962	+0.0017963661
0108 NOP		+1.62335063 E+19	+2.55625891 E+10
0109 NOP		+5.55861606 E-11	+6.78088383 E-32
010A NOP			
010B NOP	masked:		round near
010C NOP	occurred:		

Рис. ПЗ.5. Окно FPU после выполнения программы

Примечания

- 1. Независимо от режима округления, заданного в поле `rc` управляющего слова `cw`.
- 2. Это значит, что с именем не связана информация о размерности. Имя может быть определено директивой `db`, `dw` и т. п. или как метка, но все равно придется (в каждой команде, где это имя ссылается на память) явно указывать размерность.
- 3. Чтобы вывести двоичное число в десятичном формате, приходится многократно делить его на 10.
- 4. В прерывании для сброса причины прерывания годятся только Non-Wait инструкции (без ожидания, т. е. без неявной команды `wait`): `fnstenv`, `fninit`, `fnclex`. В `a86` команды `fstenv` и `fclex` транслируются в `fnstenv` и `fnclex`, если не установлен режим автоматической вставки `wait` (см. опцию `F` в документации по `a86`). Если команда с ожиданием была выполнена *до сброса* причины прерывания, то система "виснет".
- 5. К сожалению, там есть еще запись `070` в `ds`. Без восстановления `ds`, что не слишком вежливо со стороны разработчиков BIOS!
- 6. В IBM PC обработка внутреннего прерывания от FPU осложнена тем, что номер прерывания совпадает с номером программного прерывания для обращения к видеофункциям BIOS. Вновь разработанная процедура обработки прерывания по вектору `010` должна определять источник прерывания и в случае вызова функции BIOS передавать управление системной процедуре.

ПРИЛОЖЕНИЕ 4

Ошибки в a86/a386

Были они изящнейшей конструкции, но сочетание бедности его заказчиков и нехватки хорошей древесины со слабеющим зрением и надвигающейся старостью несколько снизили качество его работы.

(Фарли Моуэт. Шхуна, которая не хотела плавать)

В a86 v4.05 и a386 v4.04 обнаружен ряд ошибок, не препятствующих выполнению примеров и задач.

Существенных ошибок в d86 не выявлено, но есть неточность: если задать формат в строке отображения данных `w, bp`, то сегментный адрес берется из `ds` — несмотря на то, что `i80x86` при косвенной адресации через `bp` обращается к сегменту стека `ss`.

Отладчик d386 при листании окон клавишей <F10> может попытаться открыть окно несуществующего (неподдерживаемого) MMX или XMM — и "повиснет". Похоже, что d386 неправильно (слишком оптимистично) определяет наличие этих расширений. Также d386, дизассемблируя команду `a4 xlat`, не отображает префикс `a4`.

Ошибки в a86

1. Оператор `bit n` при `n = 0...15` работает исправно, но при `n > 15` возвращает ноль. Автор a86 не считает это ошибкой (ведь транслятор-то 16-битовый) и предлагает для 32-битовых данных использовать a386.
2. При трансляции директивы `db -129` предельная величина -129 преобразуется в предельную — -128, без уведомления.
3. Разбирая числа со множителем `k` (1024), a86 отказывается понимать недесятичные числа. Например, `010k` для него не `16k`, а именно `10k`.
4. Если процедура описана как дальняя, то при ее вызове из того же программного сегмента a86 заменяет инструкцию `call far <target>` на две команды: `push cs` и `call near <target>`. В `obj`-режиме эта замена выполняется корректно, но в `com`-режиме в машинном коде `call near` будет неправильный адрес перехода. Поэтому в версии v4.05 не следует вводить дальние процедуры в `com`-программах.

5. Выражение $-4*-2$ а86 вычисляет как -2 . Поэтому после знака операции $+$, $-$, $/$ или $*$ отрицательное число следует задавать в скобках.
6. Выражение `byte+3` не считается ошибкой и транслируется, как если бы между `byte` и `+3` был пробел, что означает байт по адресу 3.
7. Команда `dec b 0200:1` не считается ошибкой, хотя задавать адрес в формате "сегмент:смещение" можно лишь в командах перехода. Результат трансляции один и тот же и для `0020:1`, `020:1`, и для `2:1` — `dec b [-0c]`.
8. При трансляции пустого файла с ключом `+L` следует сообщение о сбое при генерировании `lst`-файла: "Failed synchronization between assembly and listing passes". То же в а386.

Ошибки в а386

1. Директива `org` в начале программы отрабатывается корректно только для значений 0 и 0100 (256). Любые другие значения воспринимаются так, как если бы было задано `org 0`. В результате на а386 неудобно проводить опыты с начальным загрузчиком, где требуется `org 07c00` — в начале `bin`-файла он запишет 07c00 нулевых байтов. Для этой цели лучше подходит ассемблер `nasm`.
2. В области действия директивы `use32` а386 некорректно транслирует встроенную макрокоманду `mov`. Например, `mov eax, ebx, edx, 0` преобразуется в три команды, две из которых неправильные: `mov edx, 0; mov bx, dx; mov ax, bx`.
3. При трансляции команды `jmp 8:pm`, необходимой после включения защищенного режима, а386 выдает ошибку. Приходится задавать эту команду в кодах:

```
db 0ea
dw offset pm, 8
```

ПРИЛОЖЕНИЕ 5

Описание электронного архива

Электронный архив, сопровождающий книгу, выложен на сервер издательства по адресу: <https://zip.bhv.ru/9785977517614.zip>. Ссылка на этот архив доступна и со страницы книги на сайте www.bhv.ru.

Содержимое электронного архива приведено в табл. П5.1.

Таблица П5.1. Содержимое электронного архива

Папки	Описание
arch	Каталоги 01–21 и арх3 с файлами к главам 1–21 и приложению 3; в основном это исходные тексты примеров. Примеры, которые компилируются в FreeDOS, нужно будет глава за главой переносить в образ диска (в каталог C:\work)
disk_C	Файлы, которые необходимо сразу скопировать в образ диска FreeDOS: autoexec.bat — стартовый командный файл, который должен находиться на диске C: (в корневом каталоге); tools — каталог, куда следует поместить указанные в главе 1 инструментальные программы и конфигурационные файлы; изначально там несколько авторских утилит и файлы для настройки файлового менеджера; work — пустой каталог, куда вы будете копировать примеры из папки arch
docs	01–04.pdf и 09–10.pdf — документация в открытом доступе, числовые имена взяты из списка источников. Основной источник информации — документ 2.pdf; вот пример ссылки на его таблицу — [2, Table 1-2], а чтобы найти ее в pdf-файле, нужно искать (<Ctrl>+<F>) строку Table 1-2

Список источников

1. AMD64 Architecture Programmer's Manual Volume 1: Application Programming. Advanced Micro Devices Inc., 2020. 385 p.
2. AMD64 Architecture Programmer's Manual, Volume 2: System Programming. Advanced Micro Devices Inc., 2021. 722 p.
3. AMD64 Architecture Programmer's Manual, Volume 3: General-Purpose and System Instructions. Advanced Micro Devices Inc., 2021. 640 p.
4. AMD64 Architecture Programmer's Manual, Volume 5: 64-Bit Media and x87 Floating-Point Instructions: 2013. 342 p.
5. Bartlett J. Learn to Program with Assembly: Foundational Learning for New Programmers. Apress, 2021. 344 p.
6. Berger H. Automating with STEP7 in STL and SCL: Programmable Controllers SIMATIC S7-300/400. Wiley, 2009. 543 p.
7. Carter P.A. PC Assembly Language. N.-Y.: Paul A. Carter, 2006. 195 p.
8. Duntemann J. Assembly Language Step-by-Step: Programming with Linux. 3rd ed. — Hoboken: Wiley, 2009. 648 p.
9. Intel 64 and IA-32 Architectures Software Developer's Manual. Volume 3 (3A, 3B, 3C & 3D): System Programming Guide. Intel Corporation, 2016. 1998 p.
10. Intel Processor Identification and the CPUID Instruction. Intel Corporation, 2009. 112 p.
11. Kusswurm D. Modern X86 Assembly Language Programming: Covers x86 64-bit, AVX, AVX2, and AVX-512. Apress, 2019. 604 p.
12. Lunt B. FYSOS: The System Core (FYSOS: Operating System Design Book 1). Kindle Edition, 2013. 320 p.
13. Nentwig J., Kreuder M., Morgenstern K. General and Inorganic Chemistry Made Easy (Chemistry Made Easy Series), 1st ed. Wiley-VCH, 1992. 727 p.
14. Ochem Q., Hoffmann G., Tice R., Rogers P. Ada for the Embedded C Developer. AdaCore, 2021. 253 p.
15. Sen-cuo Ro. I386/I486 Advanced Programming: Real Mode, Protected Mode, Virtual 8086 Mode. Automata Publishing Company, 1993. 227 p.
16. Shanley T. Protected Mode Software Architecture. Addison-Wesley Professional, 1996. 310 p.
17. Streib J. T. Guide to Assembly Language: A Concise Introduction, 2nd Edition. Springer Nature, 2020. 340 p.

Предметный указатель

В

Bochs

- ◇ конфигурация 15, 377
- ◇ образ диска 16
- ◇ отладочный вариант 18, 379, 380
 - magic breakpoint 151, 174, 387
 - отображение данных 152, 223, 250
- ◇ параметры запуска 15
- ◇ рабочий вариант 15

D

d86

- ◇ ошибки 393
- ◇ режим непосредственного выполнения 10, 69, 113
- ◇ форматы отображения данных 383

G

Global Descriptor Table, GDT 220

I

Interrupt Descriptor Table, IDT 253, 256, 362

P

Program Segment Prefix, PSP 29, 46, 68, 143, 149, 151, 153, 221, 275

- ◇ параметры командной строки 62, 149, 153, 159

A

Адрес

- ◇ видеобuffers 154
- ◇ виртуальный 174
- ◇ линейный 150
- ◇ номер параграфа 152
- ◇ сегмента линейный 150, 151, 153
- ◇ сегментный 152
- ◇ физический 174
- ◇ эффективный 150, 151

Адресация

- ◇ базово-индексная 66, 151
 - ◇ индексная со множителем 390
 - ◇ косвенная 63, 66, 71, 150, 151, 211
 - со смещением 64, 71
 - ◇ непосредственная 60, 70, 76
 - ◇ относительная 151, 157
 - ◇ прямая 61, 70, 76, 150, 151, 344
 - относительная 305, 342, 344, 345, 347, 348
 - ◇ регистровая 60, 70, 76
- Ассемблер
- ◇ a86 (особенности) 10, 21, 23, 32, 38, 50, 112, 126, 127, 175
 - ◇ nasm (особенности) 191, 224, 385

B

Вентиль 331, 341

- ◇ вызова 229, 253, 307, 311, 328, 329, 336, 339, 352
- ◇ прерываний 253, 259, 261, 265, 328, 341
 - поле IST 364, 365

Д

Дескриптор 220

- ◇ LDT 269
- ◇ TSS 276, 285, 311, 315
 - IOPB 317, 318, 358, 362
 - в 64-битовом режиме 277, 286, 341, 357
- ◇ атрибуты 341
 - C (Conforming) 311, 331
 - D (Data size) 331
 - L (Long) 331, 335, 336
- ◇ данных 237
- ◇ кодовый 221
- ◇ нулевой 221, 284

И

Исключение 185

- ◇ General Protection Fault (13) 188, 207, 208, 209, 221, 225, 232, 258, 272, 284, 369
- ◇ Page Fault (14) 293
- ◇ код ошибки 258

К

Каталог таблиц страниц 289, 292, 295

Л

Линия A20 242

П

Привилегии 226, 259

- ◇ CPL 226, 307
- ◇ DPL 226, 259, 307
- ◇ RPL 226, 307

Р

Регистры

- ◇ CR0 222, 225, 290, 327
 - бит NE 388
- ◇ CR2 293
- ◇ CR3 277, 289, 290, 296, 304, 327

- ◇ CR4 292, 306, 351
 - бит PAE 295, 327
 - бит PSE 292, 295
- ◇ GDTR 220, 222
- ◇ IDTR 253, 256
- ◇ LDTR 271
- ◇ MSR
 - EFER (Extended Feature Enable Register) 323, 354
 - LSTAR 354, 356
 - SFMASK 354
 - STAR 354
 - CSTAR 354
- ◇ MSR (Model Specific Register) 330, 351, 354
- ◇ r8–r15 329, 341, 343
- ◇ spl, bpl, sil, dil 343
- ◇ TR (Task Register) 279, 285, 286, 315, 320, 358
- ◇ сегментные 151, 153, 219
- ◇ селекторы 219, 284
 - теньевые 231, 232, 233, 272
- ◇ флагов 266
 - бит I 266
 - поле IOPL 266

Режим

- ◇ 64-битовый 304, 323, 333, 336, 340
- ◇ unreal 231, 241
- ◇ совместимости 323, 335, 336

С

Сегменты

- ◇ атрибуты 225
- ◇ защищенный режим 219
- ◇ конформные 227
- ◇ реальный режим 219

Системные вызовы

- ◇ SYSCALL/SYSRET 233
- ◇ SYSCALL/SYSRET 352
- ◇ SYSENTER/SYSEXIT 233

Страничное преобразование адресов 277

Т

Таблица страниц 289, 292